

Московский физико-технический институт
Физтех-школа прикладной математики и информатики

А л г о р и т м ы и с т р у к т у р ы д а н н ы х
1 СЕМЕСТР

Лектор: *Степанов Илья Данилович*



Автор: *Дмитриенко Еремей*
Telegram канал

осень 2024

Благодарности

Спасибо спонсорам за поддержку: Алексею Дмитриенко, Егору Демидову, Любове Васильевой, Илье Донову, Коту, Николаю Матвееву, Вадиму М. и Владиславу Х.

Содержание

Благодарности	1
1 Билет №1. RAM-модель и асимптотика	18
1.1 RAM-модель вычислений и измерение времени работы	18
1.2 Асимптотические обозначения O , Ω , Θ . Независимость от стартового индекса	19
1.3 Рекуррента $T(n) = 2T(n/2) + O(n)$	20
1.4 Бинарный поиск в отсортированном массиве	20
2 Билет №2. Асимптотические обозначения O, Ω, Θ. Независимость от стартового индекса	22
2.1 Зачем нужны асимптотические обозначения	22
2.2 Определения O , Ω , Θ	22
2.3 Связи между обозначениями и базовые свойства	22
2.4 Независимость от стартового индекса	23
2.5 Примеры использования O , Ω , Θ	24
3 Билет №3. Решение рекурренты $T(n) = 2T(\frac{n}{2}) + O(n)$	25
3.1 Откуда берется такая рекуррента	25
3.2 Корректная формулировка задачи	25
3.3 Решение методом дерева рекурсии	25
3.4 Что делать с округлениями	26
3.5 Итог	26
4 Билет №4. Проверка вхождения числа в отсортированный массив: бинарный поиск	27
4.1 Постановка задачи	27
4.2 Ключевая идея	27
4.3 Алгоритм	27
4.4 Доказательство корректности	27
4.5 Асимптотика	28
4.6 Крайние случаи и типичные ошибки	29
4.7 Пример	29
4.8 Псевдокод (идейно, близко к CLRS)	29
5 Билет №5. Задача сортировки: две постановки и их эквивалентность	30
5.1 Модель задачи и базовые определения	30
5.2 Две постановки задачи сортировки	30

5.3	Эквивалентность постановок	31
6	Билет №6. Доказательство формулы $\log(n!) = \Theta(n \log n)$	33
7	Билет №7. Нижняя оценка на число сравнений в сортировке сравнениями	34
7.1	Постановка задачи сортировки	34
7.2	Решающие деревья (decision tree model)	34
7.3	Нижняя оценка	35
8	Билет №8. Сортировка слиянием (Merge Sort)	37
8.1	Постановка задачи	37
8.2	Слияние двух отсортированных массивов	37
8.3	Сортировка слиянием	38
8.4	Асимптотика	39
8.5	Замечания и практические детали	40
9	Билет №9. Подсчёт числа инверсий в массиве	41
9.1	Наивное решение	41
9.2	Идея решения за $O(n \log n)$	41
9.3	Подсчёт пересекающих инверсий при слиянии	41
9.4	Полный алгоритм	42
9.5	Корректность	43
9.6	Асимптотика	43
9.7	Замечания и частые ошибки	43
10	Билет №10. Быстрая сортировка (Quick Sort)	44
11	Билет №11. Поиск k-й порядковой статистики со случайным пивотом (Quick Select)	48
11.1	Процедура разбиения <code>partition</code>	48
11.2	Алгоритм <code>QuickSelect</code>	48
11.3	Доказательство корректности	49
11.4	Асимптотика	49
12	Билет №12. Алгоритм поиска k-й порядковой статистики за $O(n)$	51
13	Билет №13. Алгоритм быстрой сортировки за $O(n \log n)$	55
13.1	Постановка задачи	55
13.2	Почему обычный <code>QuickSort</code> в худшем случае квадратичный	55

13.3	Идея: гарантировать «хороший» pivot	55
13.4	Алгоритм (детерминированный QuickSort)	55
13.5	Корректность	56
13.6	Время работы	56
13.7	Память	57
13.8	Практическое замечание	57
14	Билет №14. Стабильная сортировка подсчётом. Сортировка пар чисел	58
14.1	Стабильность	58
14.2	Стабильная сортировка подсчётом	58
14.2.1	Алгоритм (стабильный вариант)	58
14.2.2	Корректность	59
14.2.3	Асимптотика	60
14.3	Сортировка пар чисел	60
14.3.1	Алгоритм	60
14.3.2	Корректность	60
14.3.3	Асимптотика	60
14.4	Псевдокод (идейно, близко к CLRS)	61
15	Билет №15. Цифровая сортировка (LSD)	62
15.1	Постановка задачи	62
15.2	Алгоритм LSD Radix Sort	62
15.3	Доказательство корректности	63
15.4	Асимптотика	64
15.5	Практические замечания	64
16	Билет №16. Бинарная куча: определение и представление в массиве.	
	Требование кучи	65
16.1	Модельная задача: очередь с приоритетом	65
16.2	Определение бинарной кучи	65
16.3	Представление бинарной кучи в массиве	65
16.4	Требование кучи в терминах массива	66
16.5	Ключевые следствия и асимптотики	67
17	Билет №17. Операции siftUp и siftDown в бинарной куче	68
17.1	Контекст и соглашения	68
17.2	Операция siftUp (просеивание вверх)	68
17.3	Операция siftDown (просеивание вниз)	70

18 Билет №18. insert, getMin, extractMin, decreaseKey через siftUp и siftDown в бинарной куче	72
18.1 Почему getMin — это корень	72
18.2 Операция insert(x) через siftUp	72
18.3 Операция extractMin() через siftDown	73
18.4 Операция decreaseKey(v, newKey) через siftUp	73
18.5 Сводка: все операции через siftUp/siftDown	74
19 Билет №19. Построение кучи (heapify) за линейное время	75
19.1 Постановка задачи	75
19.2 Наивное решение и почему оно хуже	75
19.3 Алгоритм heapify (Флойда): “просеивание вниз” снизу вверх	75
19.4 Доказательство корректности	76
19.5 Асимптотика heapify: почему это $O(n)$	76
19.6 Практические замечания	77
20 Билет №20. Сортировка кучей (Heap Sort) и невозможность $O(1)$ для insert/extractMin в куче на сравнениях	78
20.1 Heap Sort: сортировка кучей за $O(1)$ дополнительной памяти	78
20.1.1 Вспомогательная процедура siftDown	78
20.1.2 Построение кучи heapify и идея Heap Sort	79
20.1.3 Корректность Heap Sort	79
20.1.4 Асимптотика и память	80
20.2 Несуществование кучи на сравнениях с insert и extractMin за $O(1)$	81
21 Билет №21. Удаление из кучи по указателю	82
21.1 Постановка задачи	82
21.2 Напоминание: базовые процедуры бинарной кучи	82
21.3 Способ 1: decreaseKey до $-\infty$ + extractMin	82
21.4 Способ 2: «поменять с последним и просеять один раз»	83
21.5 Замечания про «указатель» в реализации	84
22 Билет №22. Удаление из кучи по значению	85
23 Билет №23. Биномиальное дерево и биномиальная куча. Преимущество перед двоичной кучей	89
24 Билет №24. Операции merge, insert, getMin, extractMin и decreaseKey в биномиальной куче	92

24.1 Напоминание: что такое биномиальная куча	92
24.2 Базовая подоперация LINK (склейка двух деревьев одного ранга)	93
24.3 Операция merge (union): слияние двух биномиальных куч	93
24.4 Операция insert	94
24.5 Операция getMin	94
24.6 Операция extractMin	94
24.7 Операция decreaseKey	95
24.8 Итоговые асимптотики (binomial heap)	96
25 Билет №25. Амортизационный анализ и учётное время работы	97
25.1 Зачем нужен амортизационный анализ	97
25.2 Формальное определение “учётного времени”	97
25.3 Три классических способа доказывать амортизированные оценки	98
25.3.1 Агрегатный метод	98
25.3.2 Метод бухгалтерского учёта (banker’s method)	98
25.3.3 Метод потенциалов	98
25.4 Мини-пример (для интуиции)	99
26 Билет №26. Структура данных «вектор» (динамический массив)	100
26.1 Определение и инварианты	100
26.2 Реализация на массиве	100
26.3 Операции и асимптотика	101
26.4 Амортизированный анализ push_back при удвоении capacity	101
26.5 Удаление и (опционально) сжатие	102
26.6 Практические замечания про std::vector в C++	102
26.7 Когда вектор хорош, а когда нет	103
27 Билет №27. Односвязный и двусвязный списки. Добавление и удаление элементов	104
27.1 Что такое связанный список	104
27.2 Односвязный список	104
27.2.1 Определение и представление	104
27.2.2 Добавление элементов	105
27.2.3 Удаление элементов	106
27.2.4 Асимптотика	106
27.3 Двусвязный список	107
27.3.1 Определение и представление	107
27.3.2 Добавление	107

27.3.3 Удаление	108
27.3.4 Асимптотика	108
27.4 Сравнение односвязного и двусвязного списков	108
28 Билет №28. Фибоначчиева куча: merge, insert, getMin, extractMin. Оценка через $D(n)$	109
28.1 Постановка	109
28.2 Определение и представление фибоначчиевой кучи	109
28.3 Базовые операции: merge, insert, getMin	109
28.4 Операция extractMin: алгоритм и корректность	110
28.5 Асимптотика через $D(n)$	111
28.6 Итоговые оценки (через $D(n)$)	112
29 Билет №29. Фибоначчиева куча: реализация decreaseKey	113
29.1 Напоминание: устройство фибоначчиевой кучи	113
29.2 Операция decreaseKey: постановка	113
29.3 Идея: вырезание нарушителя и каскадные вырезания	114
29.4 Псевдокод	114
29.5 Доказательство корректности	115
29.6 Время работы и амортизационная оценка	115
30 Билет №30. Фибоначчиева куча: асимптотика функции $D(n)$ и доказательство асимптотики операций	117
30.1 Определение $D(n)$	117
30.2 Оценка $D(n)$ через числа Фибоначчи	117
30.3 Амортизированная асимптотика операций	118
30.4 Итоговая таблица асимптотик	120
31 Билет №31. Структура данных «стек»: реализация списком	122
31.1 Определение и интерфейс	122
31.2 Мотивация	122
31.3 Реализация стека односвязным списком	122
31.3.1 Структура узла	122
31.3.2 Операции	123
31.4 Корректность	123
31.5 Асимптотика	124
31.6 Псевдокод (идейно, близко к CLRS)	124

32 Билет №32. Проверка правильности скобочной последовательности с несколькими типами скобок	126
33 Билет №33. Поддержка минимума в стеке	129
33.1 Идея	129
33.2 Структура данных	129
33.3 Алгоритмы операций	129
33.4 Инвариант и корректность	130
33.5 Асимптотика	130
33.6 Псевдокод (идейно, близко к CLRS)	131
33.7 Замечание про альтернативы	131
34 Билет №34. Структура данных «очередь»: реализация списком	132
34.1 Формулировка	132
34.2 Идея реализации	132
34.3 Структура данных и инварианты	132
34.4 Алгоритмы операций	133
34.4.1 Операция <code>front</code>	133
34.4.2 Операция <code>push(x)</code>	133
34.4.3 Операция <code>pop()</code>	133
34.5 Доказательство корректности	133
34.6 Асимптотика	134
34.7 Практические замечания (C++)	134
35 Билет №35. Очередь на двух стеках. Амортизированная асимптотика. Минимум в очереди	137
35.1 Постановка и интерфейс	137
35.2 Идея: два стека <code>in</code> и <code>out</code>	137
35.3 Алгоритм операций	137
35.4 Корректность	137
35.5 Асимптотика: амортизированное $O(1)$	138
35.6 Минимум в стеке	139
35.7 Минимум в очереди на двух стеках	139
35.8 Итоговая асимптотика	139
35.9 Псевдокод: очередь на двух стеках с минимумом	139
36 Билет №36. Sparse Table: модельная задача, построение за $O(n \log n)$, ответ на запрос за $O(1)$	141

36.1	Определение разреженной таблицы	141
36.2	Построение за $O(n \log n)$	141
36.3	Запрос за $O(1)$	142
36.4	Асимптотика и память	142
36.5	Как получать $\lfloor \log_2 \rfloor$ за $O(1)$	142
36.6	Типичные ошибки	142
36.7	Псевдокод построения и запроса	143

37	Билет №37. Дерево отрезков: модельная задача. Обработка запросов и доказательство времени работы	144
37.1	Модельная задача	144
37.2	Определение дерева отрезков	144
37.3	Что храним: инвариант	144
37.4	Хранение в массиве	144
37.5	Построение	145
37.6	Запрос суммы на отрезке	145
37.7	Точечное обновление	146
37.8	Доказательство асимптотики запросов	146
37.9	Замечания	147
37.10	Псевдокод (дерево отрезков на суммах)	147

38	Билет №38. Дерево отрезков: двоичный спуск, поиск подотрезка с суммой $\geq x$	148
38.1	Модельная задача	148
38.2	Сведение к поиску по префиксным суммам	148
38.3	Дерево отрезков для сумм	148
38.4	Двоичный спуск (binary descent) по дереву	149
38.5	Как решить <code>find(1, x)</code>	149
38.6	Псевдокод (двоичный спуск по суммам)	150
38.7	Асимптотика	150
38.8	Типичные ошибки	150

39	Билет №39. Дерево отрезков: отложенные операции. Присваивание на отрезке. Операция push	152
39.1	Постановка задачи	152
39.2	Напоминание: дерево отрезков	152
39.3	Идея отложенных операций	152
39.4	Что храним в вершине	152

39.5	Операции <code>apply</code> и <code>push</code>	153
39.6	Псевдокод запросов	153
39.6.1	Присваивание на отрезке	153
39.6.2	Сумма на отрезке	153
39.7	Корректность	154
39.8	Асимптотика	154
40	Билет №40. Количество чисел на отрезке, значения которых не превосходят x. Fractional Cascading	156
40.1	Постановка модельной задачи	156
40.2	Базовое решение: Merge Sort Tree (сегментное дерево отсортированных списков)	156
40.3	Идея Fractional Cascading	156
40.4	Структура данных с “каскадированием”	156
40.5	Построение <code>toL</code> и <code>toR</code>	157
40.6	Алгоритм ответа на запрос	157
40.7	Доказательство корректности	158
40.8	Асимптотика	158
40.9	Практические замечания по реализации (C++)	158
41	Билет №41. Динамическое (неявное) дерево отрезков	161
41.1	Когда оно нужно	161
41.2	Модельная задача	161
41.3	Идея и структура	161
41.4	Операции	161
41.5	Корректность	162
41.6	Асимптотика	162
41.7	Псевдокод (динамическое дерево отрезков на сумме)	162
41.8	Практические замечания	163
42	Билет №42. Сжатие координат для дерева отрезков	164
42.1	Идея	164
42.2	Определение	164
42.3	Алгоритм (оффлайн)	164
42.4	Доказательство корректности	165
42.5	Важная тонкость: точки vs интервалы	165
42.6	Асимптотика	165
42.7	Псевдокод	166

42.8 Когда сжатие координат неприменимо	166
43 Билет №43. Персистентный массив	167
43.1 Постановка задачи	167
43.2 Идея персистентности	167
43.3 Представление массива как дерева	167
43.4 Персистентность через «копирование пути» (path copying)	167
43.5 Операции	168
43.6 Псевдокод	168
43.7 Доказательство корректности	169
43.8 Асимптотика	169
43.9 Связь с последующими билетами	170
44 Билет №44. Персистентный стек	171
45 Билет №45. Персистентное дерево отрезков	174
45.1 Напоминание: что такое персистентность	174
45.2 Постановка: обычное дерево отрезков	174
45.3 Ключевое наблюдение для персистентности	174
45.4 Как хранить версии	174
45.5 Структура вершины	175
45.6 Операции	175
45.6.1 Построение <code>build</code>	175
45.6.2 Запрос <code>query(root, l, r)</code>	175
45.6.3 Обновление <code>update(oldRoot, pos, newValue)</code>	175
45.7 Корректность	175
45.7.1 Корректность <code>query</code>	175
45.7.2 Корректность <code>update</code>	176
45.8 Оценка памяти	176
45.9 Псевдокод (копирование пути)	176
45.10 Где это используется (связь со следующими билетами)	177
46 Билет №46. Количество чисел на отрезке, значения которых не превосходят x: решение с персистентным деревом отрезков	179
47 Билет №47. Поиск k-й порядковой статистики на отрезке: персистентное дерево отрезков и параллельный спуск по двум версиям	183

48 Билет №48. Дерево поиска (BST): операции find, insert, erase; опцио- нально merge и split	187
48.1 Определение и инвариант	187
48.2 Высота и асимптотика	188
48.3 Операция find(x)	188
48.4 Операция insert(x)	189
48.5 Операция erase(x)	189
49 Билет №49. Наивное дерево поиска: обработка операций	191
49.1 Постановка задачи	191
49.2 Структура: наивное бинарное дерево поиска	191
49.3 Операция find(x)	191
49.4 Операция insert(x)	192
49.5 Операция erase(x)	192
49.6 Итог по асимптотике и ограничения “наивного” подхода	194
49.7 Псевдокод (идейно, близко к CLRS)	194
50 Билет №50. AVL-дерево: определение. Оценка глубины AVL-дерева на n вершинах	196
50.1 Определения	196
50.2 Оценка глубины AVL-дерева	196
51 Билет №51. Устранение дисбаланса в AVL-дереве	198
51.1 Что такое «дисбаланс» и где он возникает	198
51.2 Повороты как локальные преобразования	198
51.2.1 Правый поворот (Right rotation)	198
51.2.2 Левый поворот (Left rotation)	199
51.2.3 Важно: поворот сохраняет свойство BST	199
51.3 Четыре случая устранения дисбаланса	199
51.3.1 Случай 1: $bf(v) = -2$ (перевес влево)	199
51.3.2 Случай 2: $bf(v) = +2$ (перевес вправо)	200
51.4 Почему достаточно этих случаев	200
51.5 Пересчёт высот после поворотов	200
51.6 Корректность балансировки	202
51.7 Сколько раз приходится балансировать	203
51.8 Асимптотика	204
52 Билет №52. AVL-дерево: реализация операций find, insert и erase	205

52.1 Модель данных и инварианты	205
52.2 Вспомогательные процедуры	205
52.2.1 Псевдокод (идейно, близко к CLRS)	206
52.3 Операция <code>find</code>	207
52.4 Операция <code>insert</code>	208
52.5 Операция <code>erase</code>	208
52.6 Итоговые оценки	210
53 Билет №53. Splay-дерево: определение и практическая значимость	211
54 Билет №54. Splay-дерево: операции <code>zig</code>, <code>zig-zig</code> и <code>zig-zag</code>, операция <code>splay</code>	213
54.1 Контекст: что такое splay-дерево	213
54.2 Базовая операция: одиночный поворот (<code>rotation</code>)	213
54.3 Шаги <code>splay</code> : <code>zig</code> , <code>zig-zig</code> , <code>zig-zag</code>	214
54.3.1 Шаг <code>zig</code> (одинарный)	214
54.3.2 Шаг <code>zig-zig</code> (двойной, сонаправленный)	214
54.3.3 Шаг <code>zig-zag</code> (двойной, разнонаправленный)	215
54.4 Операция <code>splay</code>	215
54.5 Оценка времени	216
55 Билет №55. Метод потенциалов	217
56 Билет №56. Амортизированное время работы операции <code>splay</code> с помощью метода потенциалов	220
57 Билет №57. Splay-дерево: реализация <code>insert</code>, <code>erase</code> и <code>find</code>. Связь с <code>splay</code> (оценка времени — б/д)	224
57.1 Напоминание: что такое splay-дерево	224
57.2 Базовые примитивы: поворот и <code>splay</code>	224
57.3 Операция <code>find</code>	225
57.4 Операция <code>insert</code>	226
57.5 Операция <code>erase</code>	227
57.6 Связь операций с <code>splay</code>	228
57.7 Оценка времени работы (б/д)	228
57.8 Короткие практические замечания	229
58 Билет №58. Splay-дерево: <code>merge</code> и <code>split</code>, связь с <code>splay</code>, оценка времени (б/д)	230

59 Билет №59. Дерево поиска по неявному ключу (динамический массив):

вставка, удаление, изменение, сумма на отрезке	233
59.1 Постановка задачи	233
59.2 Идея: BST по <i>неявному</i> ключу	233
59.3 Базовые операции <code>merge</code> и <code>split</code> по позиции	233
59.3.1 <code>merge(A, B)</code>	234
59.3.2 <code>split(T, k)</code>	234
59.4 Реализация операций динамического массива	234
59.4.1 Вставка <code>insert(pos, x)</code>	234
59.4.2 Удаление <code>erase(pos)</code>	235
59.4.3 Изменение <code>set(pos, x)</code>	235
59.4.4 Сумма на отрезке <code>sum(l, r)</code>	235
59.5 Асимптотика	235
59.6 Ключевые инварианты корректности	235
59.7 Псевдокод (<code>treap</code> по неявному ключу)	236

60 Билет №60. Декартово дерево (`treap`): определение, глубина, линейное построение при отсортированных ключах

построение при отсортированных ключах	238
60.1 Определение	238
60.2 Теорема о глубине (высоте)	238
60.3 Построение за линейное время при отсортированных ключах	239

61 Билет №61. Декартово дерево: операции `merge` и `split`

61.1 Постановка	241
61.2 Напоминание: декартово дерево	241
61.3 Операция <code>merge</code>	241
61.3.1 Смысл и предусловие	241
61.3.2 Алгоритм	241
61.3.3 Корректность	242
61.3.4 Время работы	243
61.4 Операция <code>split</code>	243
61.4.1 Смысл	243
61.4.2 Алгоритм	243
61.4.3 Корректность	244
61.4.4 Время работы	245
61.5 Практические замечания	245

62 Билет №62. Декартово дерево: выражение insert и erase через merge и split	246
62.1 Постановка задачи	246
62.2 Напоминание: инварианты treap	246
62.3 Спецификация операций split и merge	246
62.4 Операция insert через split и merge	247
62.5 Операция erase через split и merge	248
62.6 Асимптотика	249
63 Билет №63. В-дерево: определение и практическая значимость. Операция find	250
63.1 Интуиция и мотивация	250
63.2 Определение В-дерева (минимальная степень t)	250
63.3 Практическая значимость	251
63.4 Операция find (поиск ключа)	251
63.5 Доказательство корректности find	252
63.6 Асимптотика find	252
64 Билет №64. Оценка глубины В-дерева на n ключах при фиксированном параметре t	253
65 Билет №65. Реализация операции insert в В-дереве	255
65.1 Постановка	255
65.2 Инварианты В-дерева (минимальная степень t)	255
65.3 Идея вставки	255
65.4 Вспомогательная процедура splitChild	256
65.5 Алгоритм insert	256
65.6 Корректность	257
65.7 Асимптотика	259
66 Билет №66. Реализация операции erase в В-дереве	260
67 Билет №67. Красно-чёрное дерево: определение	264
68 Билет №68. Оценка глубины красно-чёрного дерева на n ключах	266
68.1 Инварианты красно-чёрного дерева	266
68.2 Чёрная высота	266
68.3 Нижняя оценка на размер через чёрную высоту	267
68.4 Итоговая оценка высоты	267

69 Билет №69. Реализация операции insert в красно-чёрном дереве	269
69.1 Что такое красно-чёрное дерево (инварианты)	269
69.2 Вспомогательные операции: повороты	269
69.3 Идея вставки: BST-вставка + исправление (fixup)	269
69.3.1 Шаг 1: вставка как в BST	270
69.3.2 Шаг 2: RB-INSERT-FIXUP — разбор случаев	270
69.3.3 Псевдокод (классический вариант)	271
69.4 Доказательство корректности	272
69.5 Асимптотика	273
70 Билет №70. Реализация операции erase в красно-чёрном дереве	274
71 Билет №71. Сравнительный анализ различных реализаций дерева поиска: наивное, AVL-, splay-, B-, декартово и красно-чёрное дерево	278
71.1 Что такое «дерево поиска» и что мы от него хотим	278
71.2 Критерии сравнения	278
71.3 Наивное бинарное дерево поиска (BST)	279
71.4 AVL-дерево	279
71.5 Красно-чёрное дерево (RB-tree)	280
71.6 Splay-дерево	280
71.7 Декартово дерево (treap)	281
71.8 B-дерево	281
71.9 Сводная таблица	282
71.10 Итог: когда что выбирать	283
72 Билет №72. Хеш-функции. Коллизии. Универсальные и k-независимые семейства хеш-функций. Примеры	284
72.1 Хеширование и коллизии	284
72.2 Почему нужны семейства хеш-функций	284
72.3 Универсальные семейства	284
72.4 Пример универсального семейства (Carter–Wegman для целых чисел)	285
72.5 k -независимые (k -wise independent) семейства	286
72.6 Пример k -независимого семейства: многочлены степени $k - 1$ над полем	286
73 Билет №73. Хеш-таблицы с цепочками: find, insert, erase. Процедура rehash	288
73.1 Постановка и идея хеширования	288
73.2 Реализация операций find, insert, erase	288

73.3 Асимптотика операций (цепочки)	289
73.4 Процедура <code>rehash</code> (перестройка таблицы)	290
74 Билет №74. Совершенное хеширование	291
75 Билет №75. Хеш-таблицы с открытой адресацией. <code>find/insert/erase</code>. <code>rehash</code>. Теоремы о времени работы	295
75.1 Постановка задачи	295
75.2 Идея открытой адресации	295
75.3 Два важных варианта пробирования	295
75.4 Ключевой инвариант	296
75.5 Операции <code>find/insert/erase</code>	296
75.6 Процедура <code>rehash</code>	297
75.7 Теоремы о времени работы (линейное пробирование и двойное хеширование)	298
76 Билет №76. Фильтр Блума	299
76.1 Постановка задачи	299
76.2 Структура данных	299
76.3 Алгоритм	299
76.4 Корректность	299
76.5 Вероятность ложноположительного ответа	300
76.6 Оптимальный выбор параметров (k и m)	300
76.7 Асимптотика и практические замечания	301
77 Билет №77. Фильтр кукушки (б/д)	302
78 Билет №78. Skip List	305
78.1 Идея и устройство	305
78.2 Инварианты корректности	305
78.3 Операция <code>find</code>	306
78.4 Операция <code>insert</code>	306
78.5 Операция <code>erase</code>	307
78.6 Асимптотика (б/д)	308
78.7 Замечания	308

1 Билет №1. RAM-модель и асимптотика

Напоминание. В формулировках ниже под “временем работы” понимается число элементарных операций в **RAM-модели**. Во всех задачах, где это уместно, требуется:

- ▷ чётко сформулировать задачу;
- ▷ описать алгоритм;
- ▷ доказать корректность;
- ▷ оценить асимптотику.

1.1 RAM-модель вычислений и измерение времени работы

Определение 1.1 (RAM-модель (Random Access Machine)). **RAM-модель** — модель вычислений, в которой память представляет собой массив ячеек, доступ к любой ячейке по адресу осуществляется за $O(1)$. Алгоритм выполняется как последовательность *элементарных операций*; время работы — число таких операций.

Определение 1.2 (Элементарные операции (типичный набор)). В базовой версии RAM-модели за $O(1)$ считаются, например:

- ▷ операции над “машинными словами” фиксированного размера: $+$, $-$, $\cdot$, сравнение, логические операции, сдвиги;
- ▷ присваивание;
- ▷ условный/безусловный переход;
- ▷ обращение к массиву по индексу (чтение/запись) $a[i]$.

Замечание. В реальных компьютерах “случайный доступ” не идеален (кэш, локальность, задержки памяти), но для теории алгоритмов эта модель хорошо отделяет *порядок роста* времени от констант и особенностей железа.

Пример измерения времени работы: поиск минимума в массиве.

Задача. Дан массив $a[0 \dots n-1]$. Найти $\min_{0 \leq i < n} a[i]$.

Решение. Алгоритм. Храним текущий минимум mn .

- 1) Присвоить $mn \leftarrow a[0]$.
- 2) Для $i = 1, 2, \dots, n-1$:
 - ▷ если $a[i] < mn$, то $mn \leftarrow a[i]$.
- 3) Вывести mn .

Корректность.

Утверждение 1.1. После обработки индекса i (в конце итерации цикла) переменная mn равна $\min_{0 \leq j \leq i} a[j]$.

$\triangle$ *Доказательство.* База: при $i = 0$ имеем $mn = a[0] = \min_{0 \leq j \leq 0} a[j]$.

Переход: пусть после $i - 1$ верно $mn = \min_{0 \leq j \leq i-1} a[j]$. На шаге i мы сравниваем $a[i]$ и mn . Если $a[i] < mn$, то новый минимум на отрезке $[0..i]$ равен $a[i]$ и мы присваиваем $mn \leftarrow a[i]$. Иначе mn уже является минимумом на $[0..i]$ и не меняется. В обоих случаях после шага i получаем $mn = \min_{0 \leq j \leq i} a[j]$. ■

Асимптотика. Цикл выполняется $n - 1$ раз; в каждой итерации делается $O(1)$ операций (одно сравнение и, возможно, одно присваивание). Поэтому время $T(n) = \Theta(n)$.

1.2 Асимптотические обозначения O , Ω , Θ . Независимость от стартового индекса

Определение 1.3 (Большое O). Для функций $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ пишут $f(n) = O(g(n))$, если существуют константы $c > 0$ и $n_0 \in \mathbb{N}$ такие, что

$$\forall n \geq n_0 : f(n) \leq c g(n).$$

Определение 1.4 (Большое Ω). Пишут $f(n) = \Omega(g(n))$, если существуют $c > 0$ и n_0 такие, что

$$\forall n \geq n_0 : f(n) \geq c g(n).$$

Определение 1.5 (Большое Θ). Пишут $f(n) = \Theta(g(n))$, если одновременно $f(n) = O(g(n))$ и $f(n) = \Omega(g(n))$. Эквивалентно: существуют $c_1, c_2 > 0$ и n_0 , что

$$\forall n \geq n_0 : c_1 g(n) \leq f(n) \leq c_2 g(n).$$

Пример. $6n + 4 = \Theta(n)$, потому что для $n \geq 1$ верно $6n \leq 6n + 4 \leq 10n$.

Утверждение 1.2 (Независимость от стартового индекса (сдвиг аргумента)). Пусть $f(n) = O(g(n))$ и константа $k \in \mathbb{Z}$ фиксирована. Тогда (при определённости области) $f(n + k) = O(g(n))$ и $f(n) = O(g(n + k))$.

$\triangle$ *Доказательство.* Из $f(n) = O(g(n))$ существуют c, n_0 такие, что $f(n) \leq c g(n)$ при $n \geq n_0$. Тогда для $n \geq n_0 - k$ имеем $n + k \geq n_0$ и

$$f(n + k) \leq c g(n + k).$$

Поскольку k — константа, изменение порога n_0 не влияет на класс $O(\cdot)$. ■

Замечание. Практический смысл: “индексация с 0 или с 1” и любые константные поправки (например, $\lfloor n/2 \rfloor$ вместо $n/2$) не меняют асимптотику.

1.3 Рекуррента $T(n) = 2T(n/2) + O(n)$

Задача. Решить рекурренту

$$T(n) = 2T(n/2) + O(n).$$

Решение. Ответ: $T(n) = \Theta(n \log n)$.

Способ 1: дерево рекурсии. Развернём рекурсию для $n = 2^m$ (округления не меняют итоговую Θ -оценку). На 0-м уровне стоимость “вне рекурсии” равна $O(n)$. На 1-м уровне две подзадачи размера $n/2$, суммарная стоимость уровня: $2 \cdot O(n/2) = O(n)$. На 2-м уровне 4 подзадачи размера $n/4$, суммарно $4 \cdot O(n/4) = O(n)$. И так далее: каждый уровень даёт $O(n)$. Глубина дерева $m = \log_2 n$, значит

$$T(n) = \underbrace{O(n) + \dots + O(n)}_{\log n \text{ раз}} = O(n \log n).$$

Нижняя оценка. Если в рекурренте стоит $T(n) = 2T(n/2) + \Omega(n)$, аналогично получаем $T(n) = \Omega(n \log n)$. Вместе даёт $T(n) = \Theta(n \log n)$.

Способ 2: мастер-теорема. Здесь $a = 2$, $b = 2$, $f(n) = \Theta(n)$, а $n^{\log_b a} = n$. Это “второй случай” мастер-теоремы: $T(n) = \Theta(n \log n)$.

1.4 Бинарный поиск в отсортированном массиве

Задача. Дан отсортированный (неубывающий) массив $a[0 \dots n-1]$ и число x . Проверить, содержится ли x в массиве.

Решение. Алгоритм (поиск первого элемента $\geq x$). Поддерживаем полуинтервал $[l, r)$, где потенциально может находиться первый индекс p такой, что $a[p] \geq x$.

1) Инициализация: $l \leftarrow 0$, $r \leftarrow n$.

2) Пока $l < r$:

$$\triangleright m \leftarrow l + \left\lfloor \frac{r-l}{2} \right\rfloor;$$

$\triangleright$ если $a[m] < x$, то $l \leftarrow m + 1$, иначе $r \leftarrow m$.

3) В конце $p = l = r$. Тогда x содержится в массиве *тогда и только тогда*, когда $p < n$ и $a[p] = x$.

Утверждение 1.3 (Инвариант бинарного поиска). На каждой итерации выполняется:

- (i) $0 \leq l \leq r \leq n$;*
- (ii) если существует индекс p такой, что $a[p] \geq x$, то минимальный такой p лежит в интервале $[l, r)$.*

$\triangle$ *Доказательство.* (i) очевидно сохраняется по построению.

(ii) База: при $l = 0, r = n$ интервал покрывает все индексы.

Переход: пусть минимальный p с $a[p] \geq x$ существует. Если $a[m] < x$, то все индексы $\leq m$ не могут быть ответом (в неубывающем массиве там значения $< x$), поэтому $p \in [m+1, r)$ и присваивание $l \leftarrow m + 1$ корректно. Если $a[m] \geq x$, то ответ $p \leq m$, значит $p \in [l, m)$ и присваивание $r \leftarrow m$ сохраняет инвариант.

Цикл заканчивается при $l = r$, тогда $[l, r)$ пуст и минимальный p вынужденно равен l . ■

Асимптотика. На каждой итерации длина интервала $r - l$ уменьшается хотя бы в 2 раза. Значит, итераций $O(\log n)$, в каждой $O(1)$ действий, итого $\Theta(\log n)$.

Замечание. Формула $m = l + (r - l)/2$ предпочтительнее $m = (l + r)/2$ из-за риска переполнения при больших l, r .

2 Билет №2. Асимптотические обозначения O , Ω , Θ . Независимость от стартового индекса

2.1 Зачем нужны асимптотические обозначения

При анализе алгоритмов обычно рассматривают зависимость ресурсов от размера входа n : время работы $T(n)$ и/или память $M(n)$. Нас интересует порядок роста при $n \rightarrow \infty$, а также то, как меняется сложность при замене входа “в 10 раз больше”.

Асимптотические обозначения позволяют:

- ▷ игнорировать константы (“в 2 раза медленнее” обычно несущественно по сравнению с переходом от $n \log n$ к n^2);
- ▷ игнорировать вклад конечного числа первых значений (важно поведение при больших n);
- ▷ формально сравнивать функции роста.

2.2 Определения O , Ω , Θ

Далее $f, g : \mathbb{N} \rightarrow \mathbb{R}$ — функции, возникающие в анализе (обычно $f(n), g(n) \geq 0$ для всех n).

Определение 2.1 (Большое O). Говорят, что f ограничена сверху порядком g и пишут $f(n) = O(g(n))$, если существуют константы $c > 0$ и $n_0 \in \mathbb{N}$ такие, что

$$\forall n \geq n_0 : f(n) \leq c g(n).$$

Определение 2.2 (Большое Ω). Говорят, что f ограничена снизу порядком g и пишут $f(n) = \Omega(g(n))$, если существуют константы $c > 0$ и $n_0 \in \mathbb{N}$ такие, что

$$\forall n \geq n_0 : f(n) \geq c g(n).$$

Определение 2.3 (Большое Θ). Говорят, что f имеет тот же порядок роста, что и g и пишут $f(n) = \Theta(g(n))$, если существуют константы $c_1, c_2 > 0$ и $n_0 \in \mathbb{N}$ такие, что

$$\forall n \geq n_0 : c_1 g(n) \leq f(n) \leq c_2 g(n).$$

Замечание. Если функции могут быть отрицательными, то в определениях обычно используют модуль: $|f(n)| \leq c |g(n)|$ и т.д. В алгоритмах $f(n)$ почти всегда означает число операций/объём памяти, поэтому $f(n) \geq 0$ и модули не нужны. Также технически предполагают, что $g(n) > 0$ при всех $n \geq n_0$.

2.3 Связи между обозначениями и базовые свойства

Утверждение 2.1 (Эквивалентность Θ). $f(n) = \Theta(g(n))$ тогда и только тогда, когда одновременно выполнены $f(n) = O(g(n))$ и $f(n) = \Omega(g(n))$.

$\triangle$ *Доказательство.* $\Rightarrow$: из определения Θ сразу следуют обе оценки. $\Leftarrow$: если существуют c_2, n_2 такие, что $f(n) \leq c_2 g(n)$ при $n \geq n_2$, и существуют c_1, n_1 такие, что $f(n) \geq c_1 g(n)$ при $n \geq n_1$, то при $n \geq \max\{n_1, n_2\}$ верны обе оценки одновременно, то есть выполнено определение Θ . ■

Утверждение 2.2 (Транзитивность). Если $f = O(g)$ и $g = O(h)$, то $f = O(h)$. Аналогично для Ω и Θ .

$\triangle$ *Доказательство.* Пусть $f(n) \leq c_1 g(n)$ при $n \geq n_1$ и $g(n) \leq c_2 h(n)$ при $n \geq n_2$. Тогда при $n \geq \max\{n_1, n_2\}$ имеем $f(n) \leq c_1 c_2 h(n)$, то есть $f = O(h)$. Остальные случаи доказываются аналогично. ■

Замечание (Практическое чтение). $\triangleright f = O(g)$: “ f растёт не быстрее g (с точностью до константы)”.

$\triangleright f = \Omega(g)$: “ f растёт не медленнее g (с точностью до константы)”.

$\triangleright f = \Theta(g)$: “ f и g растут одинаково по порядку”.

Важно: $f = O(g)$ **не означает** “ $f \approx g$ ” и тем более не означает равенство. Например, $n = O(n^2)$, но это явно не один и тот же порядок роста.

2.4 Независимость от стартового индекса

Фраза “независимость от стартового индекса” означает, что все рассуждения про $O/\Omega/\Theta$ не зависят от того, нумеруем ли мы элементы “с нуля” или “с единицы”, и вообще не зависят от добавления/убавления константы в аргументе n .

Лемма 2.1 (Сдвиг аргумента на константу). Пусть k — фиксированная константа. Тогда для $n \rightarrow \infty$ выполнено

$$n + k = \Theta(n), \quad n - k = \Theta(n) \quad (\text{при } n > k).$$

$\triangle$ *Доказательство.* Пусть $n \geq 2|k|$. Тогда $\frac{1}{2}n \leq n + k \leq \frac{3}{2}n$. Это и есть определение $\Theta(n)$ для функции $n + k$ (с константами $c_1 = \frac{1}{2}$, $c_2 = \frac{3}{2}$). Аналогично $n - k$ (для $n > k$) ограничен сверху и снизу константами, умноженными на n . ■

Из леммы немедленно следует важный практический вывод:

- $\triangleright$ если в формуле встретилось $n - 1$, $n + 1$, $n + 100$ — для асимптотики это то же самое, что n ;
- $\triangleright$ поэтому нумерация массива $a_0, \dots, a_{n-1}$ и $a_1, \dots, a_n$ не влияет на итоговый класс сложности.

Пример (Одинаковая сложность у циклов с разной нумерацией). Циклы

```
for (i = 0; i < n; ++i)   и   for (i = 1; i <= n; ++i)
```

делают ровно n итераций и выполняют $\Theta(n)$ элементарных действий, хотя стартовый индекс различается.

а) Если внутри итерации выполняется $O(1)$ действий, то суммарно получаем $n \cdot O(1) =$

$\Theta(n)$.

б) Если границы отличаются на константу (например, $i < n-1$), то число итераций равно $n - 1 = \Theta(n)$.

2.5 Примеры использования O , Ω , Θ

Пример (Доминирование старшего члена).

$$f(n) = 3n^2 + 7n + 10.$$

Тогда $f(n) = \Theta(n^2)$, потому что при $n \geq 1$ выполнено

$$3n^2 \leq 3n^2 + 7n + 10 \leq 3n^2 + 7n^2 + 10n^2 = 20n^2.$$

а) Из правого неравенства получаем $f(n) = O(n^2)$.

б) Из левого неравенства получаем $f(n) = \Omega(n^2)$.

с) Вместе это даёт $f(n) = \Theta(n^2)$.

Пример (Сравнение $n \log n$ и n^2). Пусть $n \geq 2$. Тогда $\log n \leq n$, следовательно

$$n \log n \leq n \cdot n = n^2,$$

то есть $n \log n = O(n^2)$. При этом $n \log n \neq \Omega(n^2)$, поэтому $n \log n$ не равно $\Theta(n^2)$.

Пример (Основание логарифма не важно для Θ). Для любых оснований $a, b > 1$ выполнено

$$\log_a n = \Theta(\log_b n).$$

Действительно, по формуле перехода основания $\log_a n = \frac{\log_b n}{\log_b a}$, то есть отличается от $\log_b n$ ровно в константное число раз.

3 Билет №3. Решение рекурренты $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

3.1 Откуда берется такая рекуррента

Типичный источник рекуррентного соотношения вида

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

— алгоритмы «разделяй и властвуй». Например, **Merge Sort**: массив длины n делится на две половины, каждая сортируется рекурсивно, затем два отсортированных массива сливаются за линейное время. Поэтому

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n),$$

а для асимптотики (с точностью до констант и округлений) обычно пишут $T(n) = 2T(n/2) + \Theta(n)$.

3.2 Корректная формулировка задачи

Определение 3.1. Рекурренту $T(n) = 2T(n/2) + O(n)$ будем понимать так: существует функция $g(n) = O(n)$ и константа $T(1) = \Theta(1)$, что для всех $n \geq 2$

$$T(n) = 2T\left(\left\lceil \frac{n}{2} \right\rceil\right) + g(n).$$

(Округление вверх удобно: тогда аргумент строго меньше n .)

Замечание. Из условия $g(n) = O(n)$ следует оценка сверху $T(n) = O(n \log n)$. Чтобы получить точную $\Theta(n \log n)$, нужно дополнительно знать, что $g(n) = \Omega(n)$ (то есть $g(n) = \Theta(n)$), что и верно для Merge Sort (слияние «должно» просмотреть элементы, поэтому требует линейное время).

3.3 Решение методом дерева рекурсии

Сначала разберем случай $n = 2^k$ (без округлений), а затем прокомментируем общий случай.

Теорема 3.1 (Оценка сверху). Пусть $T(1) = O(1)$ и для всех $n \geq 2$

$$T(n) \leq 2T\left(\frac{n}{2}\right) + cn$$

для некоторой константы $c > 0$. Тогда $T(n) = O(n \log n)$.

Δ **Доказательство.** Рассмотрим дерево рекурсии для $n = 2^k$. На уровне i ($i = 0, 1, \dots, k$) находится 2^i подзадач размера $n/2^i$. Нерекурсивная работа в каждой вершине не превосходит $c \cdot (n/2^i)$, значит суммарная работа уровня i равна

$$2^i \cdot c \frac{n}{2^i} = cn.$$

Глубина дерева (число уровней, где еще есть разбиение) равна $k = \log_2 n$, следовательно суммарная работа по всем внутренним уровням не превосходит

$$\underbrace{cn + cn + \dots + cn}_{\log_2 n \text{ раз}} = cn \log_2 n.$$

Листьев $2^k = n$, каждый лист стоит $T(1) = O(1)$, значит суммарный вклад листьев равен $O(n)$. Итого

$$T(n) = O(n \log n) + O(n) = O(n \log n). \quad \blacksquare$$

Теорема 3.2 (Оценка снизу при линейном «склеивании»). Пусть $T(1) = \Omega(1)$ и для всех $n \geq 2$

$$T(n) \geq 2T\left(\frac{n}{2}\right) + c'n$$

для некоторой константы $c' > 0$. Тогда $T(n) = \Omega(n \log n)$.

$\triangle$ *Доказательство.* Аналогично предыдущему доказательству: на каждом уровне i суммарная нерекурсивная работа не меньше $c'n$, а уровней $\log_2 n$, значит вклад внутренних уровней не меньше $c'n \log_2 n$. Листья дают вклад $\Omega(n)$. Следовательно $T(n) = \Omega(n \log n)$. $\blacksquare$

Следствие. Если $T(1) = \Theta(1)$ и

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n),$$

то $T(n) = \Theta(n \log n)$.

3.4 Что делать с округлениями

Если рекурсия записана с $\lceil n/2 \rceil$ и/или $\lfloor n/2 \rfloor$, то глубина дерева по-прежнему $\Theta(\log n)$. На уровне i размеры подзадач не превосходят $\lceil n/2^i \rceil$, поэтому суммарная линейная работа уровня оценивается так:

$$2^i \cdot c \left\lceil \frac{n}{2^i} \right\rceil \leq 2^i \cdot c \left(\frac{n}{2^i} + 1 \right) = cn + c \cdot 2^i.$$

Суммируя по $i = 0, \dots, \lceil \log_2 n \rceil$, получаем

$$\sum_i (cn + c2^i) = cn \cdot \Theta(\log n) + c \cdot \Theta(2^{\log n}) = O(n \log n) + O(n) = O(n \log n).$$

Для нижней оценки при $g(n) = \Omega(n)$ рассуждение аналогично.

3.5 Итог

- ▷ Из $T(n) = 2T(n/2) + O(n)$ всегда следует $T(n) = O(n \log n)$.
- ▷ Если нерекурсивная часть на самом деле линейна и снизу, т.е. $T(n) = 2T(n/2) + \Theta(n)$ (как в Merge Sort), то $T(n) = \Theta(n \log n)$.

4 Билет №4. Проверка вхождения числа в отсортированный массив: бинарный поиск

4.1 Постановка задачи

Дан массив $a[0 \dots n - 1]$ длины n , отсортированный по **неубыванию**:

$$a[0] \leq a[1] \leq \dots \leq a[n - 1].$$

Дано число x . Требуется определить, существует ли индекс i такой, что $a[i] = x$.

Замечание. В RAM-модели чтение $a[i]$ по индексу и сравнение двух значений считаются элементарными операциями и выполняются за $O(1)$.

4.2 Ключевая идея

Отсортированность даёт возможность **отбрасывать половину кандидатов** после одного сравнения.

Пусть мы посмотрели на элемент $a[m]$:

- ▷ если $a[m] < x$, то все элементы слева от m (включая m) тоже строго меньше x и нам не подходят;
- ▷ если $a[m] \geq x$, то все элементы справа от m не меньше $a[m]$, поэтому первый элемент $\geq x$ (если он существует) лежит не правее m .

4.3 Алгоритм

Удобнее всего реализовать бинарный поиск как поиск `lower_bound` (первого элемента $\geq x$), а затем проверить равенство.

Полуинтервалы. Будем поддерживать полуинтервал индексов $[l, r)$: левая граница включена, правая — нет.

1. Инициализируем $l := 0, r := n$.

2. Пока $l < r$:

(a) $m := l + \left\lfloor \frac{r - l}{2} \right\rfloor$;

(b) если $a[m] < x$, то $l := m + 1$, иначе $r := m$.

3. После цикла $l = r$. Если $l < n$ и $a[l] = x$, то x присутствует в массиве, иначе отсутствует.

Замечание. Формула для середины $m = l + \lfloor (r - l)/2 \rfloor$ предпочтительнее, чем $m = \lfloor (l + r)/2 \rfloor$, потому что не вызывает переполнения при больших l, r .

4.4 Доказательство корректности

Докажем, что алгоритм действительно определяет наличие x .

Лемма 4.1 (Инварианты цикла). *На каждой итерации цикла выполняются следующие утверждения:*

1. $0 \leq l \leq r \leq n$;
2. для всех $i < l$ верно $a[i] < x$;
3. для всех $i \geq r$ верно $a[i] \geq x$.

$\triangle$ *Доказательство. База.* Изначально $l = 0$, $r = n$. Тогда (1) очевидно. Множества индексов $i < 0$ и $i \geq n$ пусты, значит (2) и (3) верны тривиально.

Переход. Пусть инварианты верны перед итерацией и $m = l + \lfloor (r - l)/2 \rfloor$.

Случай 1: $a[m] < x$. Из отсортированности следует, что для всех $i \leq m$ верно $a[i] \leq a[m] < x$. После присваивания $l := m + 1$ утверждение (2) верно для всех $i < l$. Утверждения (1) и (3) не нарушаются: l увеличилось, но остаётся $\leq r$, поскольку при $l < r$ всегда $m < r$.

Случай 2: $a[m] \geq x$. Из отсортированности следует, что для всех $i \geq m$ верно $a[i] \geq a[m] \geq x$. После присваивания $r := m$ утверждение (3) верно для всех $i \geq r$. Утверждения (1) и (2) сохраняются, так как r уменьшилось, но остаётся $\geq l$.

Следовательно, инварианты сохраняются на каждой итерации. ■

Лемма 4.2 (Терминация). *Цикл завершается за конечное число итераций.*

$\triangle$ *Доказательство.* На каждой итерации длина полуинтервала $r - l$ строго уменьшается:

- ▷ в случае $a[m] < x$ новая длина равна $r - (m + 1) < r - l$;
- ▷ в случае $a[m] \geq x$ новая длина равна $m - l < r - l$ (так как $m < r$ при $l < r$).

Поскольку $r - l$ — неотрицательное целое число, бесконечный цикл невозможен. ■

Теорема 4.1 (Корректность бинарного поиска). *После завершения цикла индекс $l = r$ равен минимальному i такому, что $a[i] \geq x$ (или $i = n$, если таких индексов нет). Следовательно,*

$$x \in \{a[0], \dots, a[n-1]\} \iff (l < n \wedge a[l] = x).$$

$\triangle$ *Доказательство.* По инварианту (2) все элементы с индексами $< l$ строго меньше x , значит индекс первого элемента $\geq x$ не может быть меньше l . По инварианту (3) все элементы с индексами $\geq r$ не меньше x , значит если элемент $\geq x$ существует, то такой индекс не превосходит r . В момент завершения $l = r$, поэтому минимальный индекс с $a[i] \geq x$ (если он существует) обязан равняться l ; если же таких элементов нет, то неизбежно $l = r = n$.

Проверка $a[l] = x$ (при $l < n$) эквивалентна существованию элемента, равного x . ■

4.5 Асимптотика

На каждой итерации мы выполняем $O(1)$ операций и уменьшаем длину отрезка поиска хотя бы вдвое с точностью до округления. Поэтому число итераций не превосходит $\lceil \log_2(n+1) \rceil$.

- ▷ Время: $O(\log n)$ сравнений и обращений к массиву.
- ▷ Память: $O(1)$ дополнительной памяти.

4.6 Крайние случаи и типичные ошибки

- ▷ **Пустой массив:** при $n = 0$ имеем $l = r = 0$, проверка $l < n$ сразу даёт ответ “нет”.
- ▷ **Повторы:** алгоритм находит *первую* позицию, где $a[i] \geq x$. Если в массиве есть несколько x , то получим индекс первого x .
- ▷ **Переполнение при вычислении середины:** не используйте $m = (l+r)/2$ на типах с возможным переполнением; используйте $m = l + (r - l)/2$.
- ▷ **Строгое/нестрогое сравнение:** для `lower_bound` критично, что ветвление устроено как

$$a[m] < x \Rightarrow l = m + 1, \quad a[m] \geq x \Rightarrow r = m.$$

Любая другая комбинация может “зациклить” или дать неверную границу.

4.7 Пример

Пусть $a = [1, 3, 3, 7, 10]$, $x = 3$. Начинаем с $l = 0$, $r = 5$.

$$m = 2, a[m] = 3 \geq 3 \Rightarrow r = 2; \quad m = 1, a[m] = 3 \geq 3 \Rightarrow r = 1; \quad m = 0, a[m] = 1 < 3 \Rightarrow l = 1.$$

Получили $l = r = 1$, и действительно $a[1] = 3$.

4.8 Псевдокод (идейно, близко к CLRS)

Псевдокод (идейно, близко к CLRS)

LowerBound(A, x):

1. $l \leftarrow 0, r \leftarrow |A|$ // поиск в полуинтервале $[l, r)$
2. **while** $l < r$:
 - (a) $m \leftarrow l + \left\lfloor \frac{r - l}{2} \right\rfloor$
 - (b) **if** $A[m] < x$ **then** $l \leftarrow m + 1$ **else** $r \leftarrow m$
3. **return** l // минимальный индекс i с $A[i] \geq x$ или $|A|$

ContainsSorted(A, x):

1. $i \leftarrow \text{LowerBound}(A, x)$
2. **return** $(i < |A|) \wedge (A[i] = x)$

5 Билет №5. Задача сортировки: две постановки и их эквивалентность

5.1 Модель задачи и базовые определения

Определение 5.1 (Полный порядок и сравнение). Пусть элементы массива принадлежат множеству X , на котором задан (**строгий**) **полный порядок** $<$ (эквивалентно: нестрогий порядок $\leq$), то есть для любых $a, b, c \in X$ выполняются:

- (1) **Трихотомия**: ровно одно из $a < b$, $a = b$, $b < a$ истинно.
- (2) **Транзитивность**: если $a < b$ и $b < c$, то $a < c$.

Доступ к операции сравнения (например, проверка $a < b$) считаем элементарным.

Определение 5.2 (Мультимножество элементов массива). Для массива $A = (a_1, \dots, a_n)$ **мультимножеством элементов** называется функция кратностей $m_A: X \rightarrow \mathbb{N}$, где $m_A(x)$ равно числу вхождений значения x в массив A .
Говорят, что два массива A и B **равны как мультимножества** (или что B является **перестановкой** A), если $m_A(x) = m_B(x)$ для всех $x \in X$.

Определение 5.3 (Перестановка индексов). **Перестановка** σ множества $\{1, 2, \dots, n\}$ — это биекция $\sigma: \{1, \dots, n\} \rightarrow \{1, \dots, n\}$. Эквивалентно: последовательность $(\sigma(1), \dots, \sigma(n))$ содержит каждый индекс из $\{1, \dots, n\}$ ровно один раз.

5.2 Две постановки задачи сортировки

Пусть дан массив $A = (a_1, \dots, a_n)$.

Задача (Постановка 1: «отсортированный массив»). *Требуется вывести массив $B = (b_1, \dots, b_n)$ такой, что:*

- (1) B **неубывает**: $b_1 \leq b_2 \leq \dots \leq b_n$;
- (2) A и B **равны как мультимножества**.

Задача (Постановка 2: «перестановка индексов»). *Требуется вывести перестановку σ индексов $\{1, \dots, n\}$ такую, что*

$$a_{\sigma(1)} \leq a_{\sigma(2)} \leq \dots \leq a_{\sigma(n)}.$$

(Если есть равные элементы, допускается любой порядок среди равных.)

Пример. Пусть $A = (3, -4, 7, -7, 5)$. Тогда ответом в постановке 1 является $B = (-7, -4, 3, 5, 7)$, а в постановке 2 можно вывести, например, $\sigma = (4, 2, 1, 5, 3)$.

5.3 Эквивалентность постановок

Идея эквивалентности: отсортированный массив и перестановка индексов содержат *одну и ту же информацию* (в каком порядке идут элементы), различается лишь формат ответа.

Теорема 5.1 (Эквивалентность двух постановок сортировки). Пусть $k(n)$ — функция, для которой $k(n) = \Omega(n)$ (то есть хотя бы линейная). Если существует алгоритм, решающий одну из постановок сортировки за $O(k(n))$, то существует алгоритм, решающий другую постановку также за $O(k(n))$.

Δ Доказательство. Докажем оба направления.

(2 $\Rightarrow$ 1). Пусть есть алгоритм $\mathcal{A}_2$, который по A строит перестановку σ за $O(k(n))$. Тогда отсортированный массив можно восстановить за $O(n)$:

$$b_i := a_{\sigma(i)} \quad (i = 1, \dots, n).$$

Очевидно, B неубывает по определению σ , и B содержит те же элементы, что и A (лишь переставленные). Итого время $O(k(n) + n) = O(k(n))$.

(1 $\Rightarrow$ 2). Пусть есть алгоритм $\mathcal{A}_1$, который по A строит отсортированный массив B за $O(k(n))$. Построим перестановку индексов.

Составим массив пар

$$P := ((a_1, 1), (a_2, 2), \dots, (a_n, n)).$$

Зададим на парах **лексикографический порядок**:

$$(x, i) < (y, j) \iff (x < y) \text{ или } (x = y \text{ и } i < j).$$

Тогда вторые компоненты (индексы) у всех пар различны, значит, все пары попарно различны.

Применим $\mathcal{A}_1$ к массиву P (мы сортируем уже не элементы из X , а элементы из $X \times \{1, \dots, n\}$, но сравнение пар по лексикографическому правилу по-прежнему элементарно). Получим отсортированный массив пар

$$P' = ((a_{\sigma(1)}, \sigma(1)), \dots, (a_{\sigma(n)}, \sigma(n))).$$

Теперь положим $\sigma(i)$ равным второй компоненте i -й пары массива P' . Тогда:

- ▷ σ — перестановка, так как все индексы $1, \dots, n$ встречаются ровно по одному разу среди вторых компонент исходного массива пар;
- ▷ последовательность $a_{\sigma(1)}, \dots, a_{\sigma(n)}$ неубывает, потому что пары отсортированы по первой компоненте, а при равенстве первой компоненты порядок фиксируется по индексу.

На построение P и извлечение σ уходит $O(n)$, сортировка занимает $O(k(n))$, итого $O(k(n) + n) = O(k(n))$.

Оба направления доказаны. ■

Замечание (Почему требуется $k(n) = \Omega(n)$). В любом «естественном» алгоритме сортировки время не может быть меньше $\Omega(n)$ хотя бы потому, что нужно прочитать вход из n элементов, а в постановке 1 также нужно выписать n элементов ответа.

Поэтому линейная добавка $+O(n)$ в редукциях выше не меняет асимптотику.

Замечание (Про равные элементы). Если в массиве есть равные значения, постановка 2 допускает несколько корректных ответов: можно по-разному упорядочить равные элементы. Конструкция с парами (a_i, i) снимает неоднозначность: индексы различны, а потому лексикографический порядок задаёт единственный порядок пар и автоматически задаёт корректную перестановку.

6 Билет №6. Доказательство формулы $\log(n!) = \Theta(n \log n)$

Напоминание. В этом билете $\log$ — логарифм по фиксированному основанию $b > 1$ (например, $b = 2$ или $b = e$). Смена основания меняет логарифм только в константное число раз:

$$\log_b x = \frac{\ln x}{\ln b}.$$

Поэтому утверждение вида $f(n) = \Theta(g(n))$ не зависит от выбора основания логарифма.

Теорема 6.1. Для $n \rightarrow \infty$ выполняется

$$\log(n!) = \Theta(n \log n).$$

$\triangle$ *Доказательство.* Докажем две оценки: сверху (O) и снизу (Ω).

1) Верхняя оценка $\log(n!) = O(n \log n)$. Для всех $k \in \{1, 2, \dots, n\}$ верно $k \leq n$, значит

$$n! = \prod_{k=1}^n k \leq \prod_{k=1}^n n = n^n.$$

Берём логарифм:

$$\log(n!) \leq \log(n^n) = n \log n.$$

То есть $\log(n!) = O(n \log n)$.

2) Нижняя оценка $\log(n!) = \Omega(n \log n)$. Рассмотрим «вторую половину» множителей факториала. Пусть $m = \lceil n/2 \rceil$. Тогда в произведении $n! = 1 \cdot 2 \cdots n$ есть как минимум $n - m + 1 \geq n/2$ множителей $m, m+1, \dots, n$, и каждый из них не меньше $n/2$. Следовательно,

$$n! \geq \prod_{k=m}^n k \geq \left(\frac{n}{2}\right)^{n/2}.$$

Берём логарифм:

$$\log(n!) \geq \frac{n}{2} \log\left(\frac{n}{2}\right) = \frac{n}{2}(\log n - \log 2) = \Omega(n \log n).$$

Итак, одновременно верны оценки $\log(n!) = O(n \log n)$ и $\log(n!) = \Omega(n \log n)$, значит $\log(n!) = \Theta(n \log n)$. ■

Замечание. Более точная асимптотика даётся формулой Стирлинга: $\ln(n!) = n \ln n - n + O(\ln n)$. Из неё тоже сразу следует $\log(n!) = \Theta(n \log n)$.

7 Билет №7. Нижняя оценка на число сравнений в сортировке сравнениями

7.1 Постановка задачи сортировки

Определение 7.1 (Задача сортировки). Дан массив (последовательность) $A[1..n]$ из элементов, на которых определено отношение строгого порядка $<$. Требуется переставить элементы так, чтобы

$$A[\sigma(1)] \leq A[\sigma(2)] \leq \dots \leq A[\sigma(n)],$$

где σ — перестановка индексов $\{1, \dots, n\}$. (Эквивалентно: вывести отсортированную версию массива.)

Определение 7.2 (Сортировка сравнениями). Алгоритм сортировки называется *сортировкой сравнениями*, если единственная операция, получающая информацию о взаимном порядке элементов, — сравнение двух элементов ($<$, $>$; при необходимости $=$). Время работы в этой модели измеряют числом сравнений.

Замечание. Для доказательства нижней оценки удобно считать, что все элементы попарно различны. Это не ослабляет оценку снизу: если алгоритм обязан уметь сортировать все входы (в том числе без равных элементов), то нижняя оценка для класса входов с попарно различными ключами автоматически является нижней оценкой в общей задаче.

7.2 Решающие деревья (decision tree model)

Определение 7.3 (Решающее дерево алгоритма сортировки). Рассмотрим детерминированный алгоритм сортировки сравнениями. Его работу можно представить в виде *двоичного решающего дерева*:

- ▷ каждая внутренняя вершина соответствует сравнению двух элементов (например, $A[i]$ и $A[j]$);
- ▷ два ребра из вершины соответствуют двум исходам сравнения (при попарно различных ключах: $A[i] < A[j]$ или $A[i] > A[j]$);
- ▷ лист соответствует завершению алгоритма и выдаче ответа (перестановки σ).

Путь от корня к листу соответствует последовательности сравнений и их исходов.

Утверждение 7.1 (Ключевой факт). Если все ключи попарно различны, то различным перестановкам входа должны соответствовать различные листья решающего дерева корректного алгоритма сортировки.

Δ *Доказательство.* Рассмотрим некоторый лист L решающего дерева. Путь в L фиксирует результаты всех сравнений, выполненных алгоритмом. Эти результаты задают набор

отношений вида $A[i] < A[j]$ и $A[i] > A[j]$.

Если для двух различных перестановок входных элементов алгоритм попал бы в один и тот же лист L , то он выдал бы один и тот же ответ (одну и ту же перестановку σ). Однако эти две перестановки задают разные истинные линейные порядки элементов, поэтому один и тот же выход не может быть корректным сразу для обеих. Следовательно, две разные перестановки входа не могут приводить в один и тот же лист: листья должны быть различны. ■

7.3 Нижняя оценка

Теорема 7.1 (Нижняя оценка на число сравнений). *Любой детерминированный алгоритм сортировки сравнениями в худшем случае выполняет*

$$\Omega(n \log n)$$

сравнений. Более точно: в худшем случае требуется не меньше, чем $\lceil \log_2(n!) \rceil$ сравнений.

△ *Доказательство.* Рассмотрим входы из n попарно различных элементов. Существует ровно $n!$ различных перестановок таких входов. По предыдущему утверждению, корректный алгоритм должен иметь в решающем дереве как минимум $n!$ листьев.

С другой стороны, двоичное дерево высоты h (то есть с максимальной длиной пути от корня до листа, равной h) имеет не более 2^h листьев. Значит,

$$2^h \geq n! \implies h \geq \log_2(n!).$$

Высота дерева — это количество сравнений в худшем случае. Следовательно, в худшем случае нужно как минимум $\log_2(n!)$ сравнений.

Осталось оценить $\log(n!)$ через $n \log n$. Для $n \geq 2$ верны неравенства

$$\left(\frac{n}{2}\right)^{n/2} \leq n! \leq n^n.$$

Действительно, верхняя оценка очевидна ($n! \leq n \cdot n \cdots n = n^n$), а нижняя получается из того, что среди сомножителей $1 \cdot 2 \cdot \dots \cdot n$ есть хотя бы $n/2$ чисел, не меньших $n/2$. Берём логарифм (основание неважно, отличается лишь на константу):

$$\frac{n}{2} \log \frac{n}{2} \leq \log(n!) \leq n \log n,$$

откуда $\log(n!) = \Theta(n \log n)$ и, значит, $\log_2(n!) = \Theta(n \log n)$. Итак, худший случай требует $\Omega(n \log n)$ сравнений. ■

Замечание (Средний случай). Из того же дерева решений следует, что *среднее* (по всем перестановкам равновероятно) число сравнений также не меньше $\log_2(n!)$, поскольку средняя глубина двоичного дерева с L листьями не меньше $\log_2 L$.
а.к.а. «в среднем» сортировка сравнениями тоже требует $\Omega(n \log n)$ сравнений.

Замечание (Оптимальность и когда можно быстрее). ▷ Сортировка слиянием (Merge Sort) делает $O(n \log n)$ сравнений, значит, оптимальна по порядку роста в модели сравнений.

- ▷ Быстрее, чем $\Omega(n \log n)$, можно лишь если выйти за рамки модели сравнений и использовать дополнительные свойства ключей (например, сортировка подсчётом при малом диапазоне значений, цифровая сортировка и т.п.).

8 Билет №8. Сортировка слиянием (Merge Sort)

8.1 Постановка задачи

Задача (Задача сортировки). Дан массив (или последовательность) $A[0..n-1]$ элементов из некоторого множества X , на котором задан **тотальный порядок** $\leq$ (то есть любые два элемента сравнимы). Требуется переставить элементы массива так, чтобы получилась неубывающая последовательность:

$$A[0] \leq A[1] \leq \dots \leq A[n-1],$$

и при этом множество элементов (с учётом кратностей) сохранилось.

Замечание. Алгоритм сортировки называется **стабильным**, если для любых равных по ключу элементов их относительный порядок в исходном массиве сохраняется в отсортированном.

8.2 Слияние двух отсортированных массивов

Ключевая подзадача Merge Sort — **слияние** двух уже отсортированных последовательностей.

Задача (Слияние). Даны два массива L и R , отсортированные по неубыванию. Требуется за время $O(|L| + |R|)$ построить массив B , являющийся результатом слияния: B отсортирован, и как мультимножество

$$B = L \uplus R.$$

Алгоритм слияния (двухуказательный). Держим два указателя i и j на текущие первые ещё не взятые элементы в L и R . На каждом шаге переносим в ответ меньший из $L[i]$ и $R[j]$ (если один массив закончился — дописываем остаток другого).

Псевдокод (идейно, близко к CLRS)

```

Merge(L, R):                                     // L и R отсортированы по неубыванию
1.  $i \leftarrow 0, j \leftarrow 0, B \leftarrow$  пустой массив
2. while  $i < |L|$  and  $j < |R|$  do
    (a) if  $L[i] \leq R[j]$  then                     (“ $\leq$ ” важно для стабильности)
        i. дописать  $L[i]$  в  $B$ ;  $i \leftarrow i + 1$ 
    (b) else
        i. дописать  $R[j]$  в  $B$ ;  $j \leftarrow j + 1$ 
3. дописать в  $B$  элементы  $L[i..|L| - 1]$ 
4. дописать в  $B$  элементы  $R[j..|R| - 1]$ 
5. return  $B$ 
  
```

Лемма 8.1 (Корректность слияния). Алгоритм *merge* возвращает отсортированный массив B , причём B содержит все элементы L и R ровно по одному разу. Кроме того, при выборе из L при равенстве ($L[i] \leq R[j]$) слияние является **стабильным**.

△ *Доказательство*. Докажем два факта.

(1) **B отсортирован**. Рассмотрим момент времени внутри цикла `while`. Пусть уже записана некоторая последовательность B . Инвариант: B отсортирован и состоит из наименьших $(i + j)$ элементов мультимножества $L \uplus R$.

База: при $i = j = 0$ массив B пуст, утверждение верно.

Переход: на очередном шаге сравниваем первые ещё не взятые элементы $L[i]$ и $R[j]$. Из отсортированности L и R следует, что любой ещё не взятый элемент в L не меньше $L[i]$, а в R — не меньше $R[j]$. Значит, $\min(L[i], R[j])$ является наименьшим среди всех оставшихся элементов $L \uplus R$. Мы добавляем его в конец B , поэтому отсортированность B сохраняется, а инвариант выполняется для $(i + j) + 1$.

Когда один из массивов заканчивается, оставшийся хвост второго массива уже отсортирован и все его элементы не меньше последнего добавленного в B (иначе противоречили бы выбору минимума на предыдущем шаге), поэтому дописывание хвоста сохраняет неубывание.

(2) **Мультимножество сохраняется**. Каждый шаг цикла увеличивает ровно один из индексов i или j и добавляет ровно один элемент в B , а в конце мы дописываем оставшиеся элементы (без изменений). Следовательно, каждый элемент L и R попадает в B ровно один раз.

Стабильность. Если $L[i] = R[j]$ и мы выбираем $L[i]$ (условие $L[i] \leq R[j]$), то элементы из левой половины в ответе идут раньше равных элементов из правой половины. Рекурсивно это означает сохранение относительного порядка равных элементов по всему алгоритму Merge Sort. ■

Утверждение 8.1 (Асимптотика слияния). Слияние работает за $\Theta(|L| + |R|)$ сравнений и операций записи.

△ *Доказательство*. Каждая итерация цикла `while` увеличивает i или j , поэтому итераций не больше $|L| + |R|$. Каждая итерация делает $O(1)$ работы. Дописывание хвоста также занимает $O(|L| + |R|)$. Снизу: чтобы выписать все элементы, нужно сделать хотя бы $|L| + |R|$ операций записи. ■

8.3 Сортировка слиянием

Идея (Divide & Conquer).

1. Разделить массив на две половины (примерно поровну).
2. Рекурсивно отсортировать каждую половину.
3. Слить две отсортированные половины за линейное время.

Классическая реализация (top-down).

Псевдокод (идейно, близко к CLRS)

MergeSort(A, l, r): // сортирует полуинтервал $A[l..r)$

1. **if** $r - l \leq 1$ **then return**
2. $m \leftarrow \lfloor \frac{l+r}{2} \rfloor$
3. **MERGE**Sort(A, l, m)
4. **MERGE**Sort(A, m, r)
5. $A[l..r) \leftarrow \text{MERGE}(A[l..m), A[m..r))$

Теорема 8.1 (Корректность Merge Sort). *Алгоритм `merge_sort` возвращает массив, отсортированный по неубыванию и являющийся перестановкой исходного массива. Если слияние стабильно (при равенстве выбираем элемент из левой части), то Merge Sort стабилен.*

$\triangle$ *Доказательство.* Индукция по $n = r - l$.

База: $n \leq 1$. Массив длины 0 или 1 уже отсортирован, алгоритм ничего не меняет.

Переход: пусть утверждение верно для всех массивов меньшей длины. Для $n > 1$ алгоритм делит массив на две части $A[l..m)$ и $A[m..r)$ меньшей длины, по предположению индукции рекурсивные вызовы их корректно сортируют. Затем вызывается `merge`, который по лемме о слиянии выдаёт отсортированную последовательность, содержащую все элементы обеих частей, то есть все элементы исходного $A[l..r)$. Значит, после присваивания $A[l..r)$ отсортирован и является перестановкой исходного. Стабильность следует из стабильности `merge` и того, что равные элементы могут «перемешаться» только при слиянии двух половин. ■

8.4 Асимптотика

Пусть $T(n)$ — время сортировки массива длины n (в RAM-модели, сравнения и записи считаем $O(1)$).

Утверждение 8.2 (Время работы). *Merge Sort работает за $\Theta(n \log n)$ в худшем, среднем и лучшем случаях.*

$\triangle$ *Доказательство.* На каждом шаге рекурсии выполняются два рекурсивных вызова на подмассивах размера примерно $n/2$, а затем линейное слияние за $\Theta(n)$. Получаем рекурренту

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n).$$

В стандартной записи (для простоты, при $n = 2^k$):

$$T(n) = 2T(n/2) + \Theta(n).$$

Решение такой рекуррентной — $\Theta(n \log n)$ (например, по мастер-теореме или дереву рекурсии). ■

Замечание (Число сравнений (полезно знать)). Если при слиянии двух массивов длины a и b делать не более $a + b - 1$ сравнений, то для $n = 2^k$ Merge Sort делает в худшем случае ровно

$$C(n) = n \log_2 n - n + 1$$

сравнений (решение рекуррентности $C(n) = 2C(n/2) + (n - 1)$, $C(1) = 0$). Для произвольного n справедлива верхняя оценка

$$C(n) \leq n \lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1.$$

Утверждение 8.3 (Память). $\triangleright$ Для массива классическая реализация требует дополнительный буфер $\Theta(n)$ для слияния.

- $\triangleright$ Глубина рекурсии $\Theta(\log n)$, значит дополнительная память под стек вызовов $\Theta(\log n)$.
- $\triangleright$ Для **связных списков** Merge Sort можно реализовать со слиянием без дополнительного массива, переставляя указатели (тогда доп.память, кроме стека рекурсии, $O(1)$).

8.5 Замечания и практические детали

Замечание (Когда Merge Sort особенно полезен). $\triangleright$ Требуется **стабильная** сортировка.

- $\triangleright$ Данные удобно читать **последовательно** (например, списки, внешняя память) — слияние естественно.
- $\triangleright$ Нужна предсказуемая асимптотика $\Theta(n \log n)$ (в отличие от Quick Sort без гарантий).

9 Билет №9. Подсчёт числа инверсий в массиве

Определение 9.1 (Инверсия). Пусть дан массив $a[0..n-1]$. *Инверсией* называется пара индексов (i, j) такая, что

$$0 \leq i < j < n \quad \text{и} \quad a_i > a_j.$$

Элементы, равные друг другу ($a_i = a_j$), инверсий не образуют.

Задача. Дан массив a длины n . Требуется найти количество инверсий в нём.

9.1 Наивное решение

Переберём все пары (i, j) при $i < j$ и посчитаем, сколько раз выполняется $a_i > a_j$. Это работает за $\Theta(n^2)$ сравнений, что слишком медленно при больших n .

9.2 Идея решения за $O(n \log n)$

Будем считать инверсии одновременно с сортировкой слиянием (Merge Sort).

Разобьём массив на две половины:

$$a[0..n-1] = L \parallel R,$$

где $L = a[0..m-1]$, $R = a[m..n-1]$, $m = \lfloor n/2 \rfloor$.

Лемма 9.1 (Разбиение инверсий на три типа). Любая инверсия (i, j) в исходном массиве относится ровно к одному из трёх типов:

1. обе позиции в левой половине ($i, j < m$);
2. обе позиции в правой половине ($i, j \geq m$);
3. инверсия пересекает разрез: $i < m$ и $j \geq m$.

Следовательно,

$$\text{inv}(a) = \text{inv}(L) + \text{inv}(R) + \text{inv}_\times(L, R),$$

где $\text{inv}_\times(L, R)$ — число инверсий, пересекающих разрез.

$\triangle$ *Доказательство.* Поскольку L и R образуют разбиение индексов $[0..n-1]$ на два непересекающихся множества, для любой пары $i < j$ возможен ровно один из случаев: оба индекса в L , оба в R , либо один в L и другой в R . Это и даёт формулу. ■

Таким образом, остаётся научиться быстро считать $\text{inv}_\times(L, R)$, а заодно отсортировать L и R , чтобы рекурсивно применять тот же приём.

9.3 Подсчёт пересекающих инверсий при слиянии

Предположим, что половины уже отсортированы по неубыванию:

$$L : \ell_0 \leq \ell_1 \leq \dots \leq \ell_{p-1}, \quad R : r_0 \leq r_1 \leq \dots \leq r_{q-1}.$$

Будем сливать их в один отсортированный массив стандартным “merge’ом” и одновременно считать пересекающие инверсии.

Лемма 9.2 (Как считать пересекающие инверсии). Пусть в процессе слияния указатели стоят на ℓ_i и r_j .

- ▷ Если $\ell_i \leq r_j$, то можно безопасно положить ℓ_i в выход: пересекающих инверсий, в которых участвует ℓ_i и ещё не взятые элементы R , нет.
- ▷ Если $r_j < \ell_i$, то элемент r_j образует инверсию со всеми оставшимися элементами левой половины $\ell_i, \ell_{i+1}, \dots, \ell_{p-1}$; таких элементов $p-i$, поэтому надо прибавить $p-i$ к ответу.

△ *Доказательство.* Рассмотрим второй случай: $r_j < \ell_i$. Поскольку L отсортирован, для любого $t \geq i$ выполнено $\ell_t \geq \ell_i > r_j$, значит каждая пара (ℓ_t, r_j) является инверсией, пересекающей разрез. Таких пар ровно $p-i$.

Почему не нужно учитывать элементы $\ell_0, \dots, \ell_{i-1}$? Они уже были извлечены в выход раньше, а значит в момент их извлечения выполнялось условие $\ell_t \leq r_j$ (ведь указатель j ещё указывал на текущий минимальный неиспользованный элемент правой половины). Следовательно, для $t < i$ верно $\ell_t \leq r_j$, и инверсий с r_j они не образуют. ■

9.4 Полный алгоритм

Рекурсивно считаем инверсии в половинах и затем считаем пересекающие инверсии во время слияния.

Псевдокод (идейно, близко к CLRS)

```

CountInv( $l, r$ ): // число инверсий на подмассиве  $a[l..r]$ 
1. if  $r - l \leq 1$  then return 0
2.  $m \leftarrow \lfloor \frac{l+r}{2} \rfloor$ 
3.  $ans \leftarrow \text{COUNTINV}(l, m) + \text{COUNTINV}(m, r)$ 
4.  $ans \leftarrow ans + \text{MERGEANDCOUNT}(l, m, r)$  // слияние + пересекающие инверсии
5. return  $ans$ 

MergeAndCount( $l, m, r$ ): // сливает  $a[l..m]$  и  $a[m..r]$ , возвращает  $\text{inv}_x$ 
1.  $i \leftarrow l, j \leftarrow m, tmp \leftarrow$  пустой массив,  $add \leftarrow 0$ 
2. while  $i < m$  and  $j < r$  do
    (a) if  $a[i] \leq a[j]$  then
        i. дописать  $a[i]$  в  $tmp$ ;  $i \leftarrow i + 1$ 
    (b) else
        i. дописать  $a[j]$  в  $tmp$ ;  $j \leftarrow j + 1$ 

```

```

        ii.  $add \leftarrow add + (m - i)$  // все оставшиеся слева больше текущего  $a[j]$ 
3. дописать в  $tmp$  остаток  $a[i..m)$  и  $a[j..r)$ 
4. скопировать  $tmp$  обратно в  $a[l..r)$ 
5. return  $add$ 

```

9.5 Корректность

Теорема 9.1. Алгоритм $countInv(0, n)$ возвращает точное число инверсий массива a .

$\triangle$ *Доказательство.* Индукция по длине рассматриваемого подмассива $n = r - l$.

База: при $n \leq 1$ инверсий нет, алгоритм возвращает 0.

Переход: пусть утверждение верно для всех массивов длины меньше n . Алгоритм делит массив на L и R , рекурсивно считает $inv(L)$ и $inv(R)$, а по предположению индукции эти значения посчитаны верно. Далее алгоритм выполняет слияние и, по лемме выше, корректно считает $inv_{\times}(L, R)$. По лемме о разбиении инверсий сумма трёх слагаемых равна $inv(a)$. Значит, итоговый ответ верен. ■

9.6 Асимптотика

Пусть $T(n)$ — время работы на массиве длины n . Рекурсия даёт

$$T(n) = 2T(n/2) + O(n),$$

потому что слияние и подсчёт пересекающихся инверсий работают за линейное время. Отсюда $T(n) = O(n \log n)$.

Дополнительная память: $O(n)$ на временный буфер для слияния.

9.7 Замечания и частые ошибки

- ▷ Ответ может быть до $\binom{n}{2}$, поэтому при $n \approx 2 \cdot 10^5$ нужен тип `long long`.
- ▷ В условии `if (a[i] <= a[j])` важен знак “ $\leq$ ”: так мы не считаем пары с равными значениями как инверсии (инверсия требует строгого “ $>$ ”).

Замечание (Альтернатива: дерево Фенвика). Если значения можно сжать по координатам, число инверсий можно считать проходом справа налево, поддерживая частоты и запрашивая количество элементов строго меньше текущего. Это тоже даёт $O(n \log n)$, но билет обычно ожидает решение через слияние.

10 Билет №10. Быстрая сортировка (Quick Sort)

Постановка задачи

Дан массив $a[0 \dots n-1]$ элементов, сравнимых отношением “ $\leq$ ”. Требуется переставить элементы так, чтобы они шли в неубывающем порядке.

Определение 10.1 (Отсортированный массив). Массив a отсортирован по неубыванию, если

$$a[0] \leq a[1] \leq \dots \leq a[n-1].$$

Определение 10.2 (Корректность сортировки). Результат сортировки корректен, если:

1. выходной массив отсортирован по неубыванию;
2. выходной массив является перестановкой входного (элементы не теряются и не добавляются).

Идея быстрой сортировки

Quick Sort — рекурсивная сортировка вида «разделяй и властвуй». Для отрезка $a[l \dots r]$ выбирается *пивот* (опорное значение) x . Далее выполняется *разбиение* (partition): элементы переставляются так, чтобы все элементы, меньшие x , оказались слева, а все элементы, большие x — справа. После этого рекурсивно сортируются получившиеся части.

Ключевой факт: разбиение можно сделать за линейное время по длине отрезка и (обычно) *на месте*, без выделения дополнительного массива.

Ключевая подзадача: разбиение (partition)

Вариантов partition много. Для корректной и быстрой работы при большом количестве одинаковых элементов удобно использовать *трехпутевое разбиение* (Dutch National Flag).

Определение 10.3 (Трехпутевое разбиение). Пусть дан отрезок $a[l \dots r]$ и опорное значение x . Трехпутевое разбиение возвращает индексы (lt, gt) такие, что после процедуры:

$$\begin{aligned} a[l \dots lt-1] &< x, \\ a[lt \dots gt] &= x, \\ a[gt+1 \dots r] &> x, \end{aligned}$$

а элементы на $[l, r]$ образуют перестановку исходных (элементы только меняются местами).

Псевдокод partition3 и Quick Sort

Ниже — типичная реализация трехпутевого разбиения и quick sort.

Псевдокод (идейно, близко к CLRS)**Partition3**(a, l, r, x): // трёхпутевое разбиение (Dutch National Flag)1. $lt \leftarrow l, i \leftarrow l, gt \leftarrow r$ 2. **while** $i \leq gt$ **do**(a) **if** $a[i] < x$ **then**i. SWAP($a[lt], a[i]$)ii. $lt \leftarrow lt + 1; i \leftarrow i + 1$ (b) **else if** $a[i] > x$ **then**i. SWAP($a[i], a[gt]$)ii. $gt \leftarrow gt - 1$ // i не увеличиваем: в $a[i]$ пришёл новый элемент(c) **else** ($a[i] = x$)i. $i \leftarrow i + 1$ 3. **return** (lt, gt)**QuickSort**(a, l, r):1. **if** $l \geq r$ **then return**2. $x \leftarrow \text{CHOOSEPIVOT}(a, l, r)$ // например, случайный элемент3. $(lt, gt) \leftarrow \text{PARTITION3}(a, l, r, x)$ 4. QUICKSORT($a, l, lt - 1$)5. QUICKSORT($a, gt + 1, r$)**Корректность разбиения****Лемма 10.1** (Корректность partition3). После выполнения $\text{partition3}(a, l, r, x)$ выполняются условия:

$$a[l \dots lt - 1] < x, \quad a[lt \dots gt] = x, \quad a[gt + 1 \dots r] > x,$$

и элементы на $[l, r]$ являются перестановкой исходных. $\triangle$ Доказательство. Поддерживаем инвариант цикла (на каждой итерации):

$$\begin{aligned} a[l \dots lt - 1] &< x, \\ a[lt \dots i - 1] &= x, \\ a[i \dots gt] &\text{еще не классифицированы,} \\ a[gt + 1 \dots r] &> x. \end{aligned}$$

Изначально $lt = i = l, gt = r$, все зоны, кроме «не классифицировано», пусты, инвариант

верен.

Рассмотрим одну итерацию:

- ▷ Если $a[i] < x$, то обмен `swap(a[lt], a[i])` переносит элемент $< x$ в левую зону, после чего lt и i увеличиваются. Остальные зоны не портятся.
- ▷ Если $a[i] > x$, то обмен `swap(a[i], a[gt])` переносит элемент $> x$ в правую зону, затем gt уменьшается. Индекс i не увеличиваем, потому что в $a[i]$ пришел новый элемент, который еще не классифицирован.
- ▷ Если $a[i] = x$, то просто увеличиваем i , расширяя среднюю зону равных x .

Во всех случаях инвариант сохраняется.

Цикл заканчивается при $i > gt$, то есть «не классифицированных» элементов не остается. Тогда из инварианта напрямую следует требуемое разбиение.

Перестановочность очевидна: алгоритм делает только обмены элементов местами. ■

Корректность быстрой сортировки

Теорема 10.1 (Корректность Quick Sort). Процедура `quick_sort(a, l, r)` сортирует подмассив $a[l \dots r]$ по неубыванию.

△ *Доказательство.* Индукция по длине $n = r - l + 1$.

База: $n \leq 1$. Тогда $l \geq r$ и функция сразу завершает работу, подмассив уже отсортирован.

Переход: пусть утверждение верно для всех отрезков меньшей длины. Рассмотрим $a[l \dots r]$ длины $n \geq 2$. Выбирается пивот x и выполняется `partition3`, получая индексы (lt, gt) . По лемме о разбиении:

$$a[l \dots lt - 1] < x, \quad a[lt \dots gt] = x, \quad a[gt + 1 \dots r] > x.$$

Алгоритм рекурсивно сортирует два подотрезка $[l, lt - 1]$ и $[gt + 1, r]$. Они имеют длину строго меньше n (элементы, равные x , в рекурсию не уходят). По предположению индукции оба подотрезка после рекурсии отсортированы.

Теперь весь отрезок $[l, r]$ отсортирован: слева стоят элементы $< x$, затем блок элементов $= x$, затем элементы $> x$. ■

Асимптотика (факты, б/д)

Разбиение `partition3` делает один линейный проход по $[l, r]$, значит работает за $\Theta(r - l + 1)$.

Время работы `quick sort` на n элементах удовлетворяет рекурренте

$$T(n) = T(n_1) + T(n_2) + \Theta(n),$$

где n_1 и n_2 — размеры частей слева и справа от пивота (при трехпутевом разбиении элементы, равные x , исключаются из рекурсии).

- ▷ **Худший случай:** разбиение каждый раз получается крайне неравномерным, например $n_1 = 0$, $n_2 = n - 1$. Тогда $T(n) = T(n - 1) + \Theta(n) = \Theta(n^2)$.

- ▷ **Лучший (сбалансированный) случай:** $n_1 \approx n_2 \approx n/2$, тогда $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.
- ▷ **Ожидаемое время:** если пивот выбирается случайно (равновероятно среди элементов отрезка), то математическое ожидание времени работы равно $\Theta(n \log n)$.

Память и практические замечания

- ▷ **Память.** Классический quick sort делает разбиение «на месте» и требует дополнительную память только на стек рекурсии. В среднем глубина рекурсии $O(\log n)$, в худшем случае — $O(n)$.
- ▷ **Как гарантировать маленький стек.** Практический прием: всегда рекурсировать в меньшую часть, а большую обрабатывать в цикле (эквивалент «удалению хвостовой рекурсии»). Тогда глубина стека становится $O(\log n)$ даже при неудачных разбиениях.
- ▷ **Стабильность.** Быстрая сортировка в стандартной in-place реализации *не стабильна*: равные элементы могут поменять относительный порядок.
- ▷ **Много равных элементов.** Трехпутевое разбиение принципиально улучшает работу на массивах с большим количеством повторов: равные пивоту элементы исключаются из рекурсии.

Замечание. В стандартной библиотеке C++ алгоритм `std::sort` обычно реализован как *introsort*: быстрая сортировка с переключением на `heap sort` при слишком большой глубине рекурсии, что дает гарантированное $O(n \log n)$ в худшем случае. (Конкретная реализация зависит от стандартной библиотеки, но идея *introsort* является общепринятой.)

11 Билет №11. Поиск k -й порядковой статистики со случайным пивотом (Quick Select)

Определение 11.1 (Порядковая статистика). Пусть дан массив A из n элементов и рассмотрим отсортированный (**неубывающий**) массив $A^\uparrow$, полученный из A перестановкой элементов. k -я *порядковая статистика* (k -й по величине элемент) — это элемент

$$A^\uparrow[k], \quad 1 \leq k \leq n.$$

Если в массиве есть равные элементы, то значение $A^\uparrow[k]$ определено однозначно, но “искомый элемент исходного массива” может быть не единственным.

Задача. Дан массив A длины n и целое k ($1 \leq k \leq n$). Найти значение k -й порядковой статистики массива A .

11.1 Процедура разбиения partition

Пусть выбран некоторый *пивот* (pivot) x — значение одного из элементов массива. Процедура **partition** переставляет элементы массива так, чтобы получились три подряд идущих блока:

$$\underbrace{L}_{<x} \quad \underbrace{M}_{=x} \quad \underbrace{R}_{>x},$$

то есть:

- ▷ каждый элемент блока L строго меньше x ;
- ▷ каждый элемент блока M равен x ;
- ▷ каждый элемент блока R строго больше x .

Также удобно знать размеры блоков $|L|$ и $|M|$ (или границы блоков).

Замечание. Практически полезно делать **трёхпутевое** разбиение (“голландский флаг”), чтобы корректно и быстро обрабатывать массивы с большим числом повторов.

11.2 Алгоритм QuickSelect

Идея: как в Quick Sort, выбираем пивот и делаем **partition**, но рекурсивно идём **только в ту часть**, где может лежать нужная порядковая статистика.

Алгоритм (в терминах размеров блоков $|L|, |M|$):

1. Если $n = 1$, вернуть $A[1]$.
2. Выбрать **случайный** элемент массива и положить x равным его значению.
3. Выполнить **partition** относительно x , получив блоки L, M, R .
4. Если $k \leq |L|$, рекурсивно найти k -ю порядковую статистику в массиве L .
5. Иначе если $k \leq |L| + |M|$, вернуть x (любой элемент из M).

6. Иначе рекурсивно найти $(k - |L| - |M|)$ -ю порядковую статистику в массиве R .

Замечание. Алгоритм легко переписать итеративно: после `partition` просто “сужать” рассматриваемый подмассив и обновлять k . Это убирает риск переполнения стека из-за глубокой рекурсии в плохом случае.

11.3 Доказательство корректности

Утверждение 11.1 (Свойство `partition`). После выполнения `partition` относительно x массив представим как конкатенацию блоков L, M, R со свойствами: $\forall a \in L (a < x)$, $\forall a \in M (a = x)$, $\forall a \in R (a > x)$.

Теорема 11.1 (Корректность `QuickSelect`). Алгоритм `QuickSelect` возвращает значение k -й порядковой статистики исходного массива.

△ *Доказательство.* Докажем по индукции по n .

База: $n = 1$. Тогда $k = 1$ и алгоритм возвращает единственный элемент массива — это и есть 1-я порядковая статистика.

Переход: пусть $n > 1$, и алгоритм выбрал пивот x и выполнил `partition`, получив блоки L, M, R . Рассмотрим отсортированный массив $A^\uparrow$. Из свойства `partition` следует:

- ▷ все элементы L будут стоять в $A^\uparrow$ раньше всех элементов M и R ;
- ▷ все элементы M (равные x) будут идти подряд сразу после элементов L ;
- ▷ все элементы R будут стоять после блоков L и M .

1. Если $k \leq |L|$, то элемент $A^\uparrow[k]$ целиком лежит среди элементов L . Алгоритм рекурсивно ищет k -ю порядковую статистику в L . По предположению индукции рекурсивный вызов вернёт правильный ответ.
2. Если $|L| < k \leq |L| + |M|$, то позиции $|L| + 1, \dots, |L| + |M|$ в $A^\uparrow$ заняты значением x . Значит, $A^\uparrow[k] = x$, и алгоритм корректно возвращает x .
3. Если $k > |L| + |M|$, то первые $|L| + |M|$ элементов $A^\uparrow$ принадлежат $L \cup M$. Поэтому $A^\uparrow[k]$ лежит в R и является $(k - |L| - |M|)$ -й порядковой статистикой массива R . Алгоритм делает именно такой рекурсивный вызов и по предположению индукции получает правильный ответ.

Во всех случаях алгоритм возвращает корректное значение. ■

11.4 Асимптотика

- ▷ **Ожидаемое время:** $\mathbb{E}T(n) = O(n)$ при равновероятном выборе пивота. Интуиция: `partition` стоит $O(n)$, а в среднем размер рекурсивного подмассива существенно уменьшается.
- ▷ **Худший случай:** $T(n) = \Theta(n^2)$ (например, если каждый раз пивот оказывается минимумом или максимумом).
- ▷ **Память:** при in-place `partition` дополнительная память $O(1)$, глубина рекурсии в среднем $O(\log n)$, в худшем случае $O(n)$.

Замечание. В программе экзамена для `QuickSelect` асимптотика помечена как «б/д», то есть строгого доказательства на экзамене обычно не требуют.

12 Билет №12. Алгоритм поиска k -й порядковой статистики за $O(n)$

Определение 12.1. Пусть дан массив A из n элементов из *линейно упорядоченного* множества (например, целые числа). k -я *порядковая статистика* (нумерация с 1) — это элемент, который оказался бы на позиции k после сортировки A по неубыванию. Эквивалентно: это такой элемент x , что строго меньших x элементов в массиве меньше k , а элементов, не превосходящих x , не меньше k .

Постановка задачи

Даны массив A длины n и число k ($1 \leq k \leq n$). Требуется найти k -ю порядковую статистику массива.

Контекст и идея

Классический `QuickSelect` делает `partition` относительно *пивота* и рекурсивно уходит только в ту часть, где лежит ответ. В среднем (при случайном пивоте) это $O(n)$, но в худшем случае (плохие пивоты) возможно $\Theta(n^2)$.

Чтобы гарантировать *худший случай* $O(n)$, достаточно каждый раз выбирать **хороший пивот**: такой элемент x , что хотя бы постоянная доля элементов массива лежит *строго* слева от него, и хотя бы постоянная доля — *строго* справа. Тогда размер задачи после `partition` уменьшается хотя бы в фиксированное число раз, а значит суммарное время остаётся линейным.

Подпроцедура трёхчастного разбиения

Для работы с повторами удобно использовать трёхчастное разбиение:

`partition3(A, x)`: переставить элементы так, чтобы шли все $< x$, затем все $= x$, затем все $> x$.

Пусть после разбиения получились подмассивы L (меньшие), E (равные), R (большие), и их размеры равны $|L| = \ell$, $|E| = m$, $|R| = r$. Тогда:

- ▷ каждый элемент из L строго меньше x ;
- ▷ каждый элемент из E равен x ;
- ▷ каждый элемент из R строго больше x ;
- ▷ $\ell + m + r = n$.

Алгоритм BFPRT («median of medians»)

Алгоритм детерминированно выбирает хороший пивот за линейное время.

1. Разбить массив A на группы по 5 элементов (последняя группа может иметь размер < 5).

2. В каждой группе отсортировать элементы (это константный размер, поэтому стоит $O(1)$ на группу) и выписать *медиану* группы.
3. Пусть M — массив медиан всех групп. Выбрать пивот x как медиану массива M (рекурсивно тем же алгоритмом).
4. Выполнить `partition3(A, x)` и получить L, E, R .
5. Если $k \leq |L|$, ответ лежит в L : вернуть `Select(L, k)`. Если $|L| < k \leq |L| + |E|$, ответ равен x . Иначе ответ лежит в R : вернуть `Select(R, k - |L| - |E|)`.

Псевдокод

Псевдокод (идейно, близко к CLRS)

```

Select( $A, k$ ):                                     //  $k = 1..|A|$ , ищем  $k$ -ю порядковую статистику
1. if  $|A| \leq 50$  then
    (a) SORT( $A$ )
    (b) return  $A[k]$ 
2. разбить  $A$  на группы по 5 элементов (последняя может быть меньше)
3.  $M \leftarrow$  пустой массив                               // медианы групп
4. for each группа  $G$  do
    (a) SORT( $G$ )
    (b) дописать MEDIAN( $G$ ) в  $M$ 
5.  $x \leftarrow \text{SELECT}\left(M, \left\lceil \frac{|M|}{2} \right\rceil\right)$            // медиана медиан
6.  $(L, E, R) \leftarrow \text{PARTITION3}(A, x)$                  //  $L < x, E = x, R > x$ 
7. if  $k \leq |L|$  then return SELECT( $L, k$ )
8. if  $k \leq |L| + |E|$  then return  $x$ 
9. return SELECT( $R, k - |L| - |E|$ )

```

Корректность

Утверждение 12.1. Алгоритм *Select* всегда возвращает k -ю порядковую статистику входного массива.

$\triangle$ *Доказательство.* Доказательство по индукции по $n = |A|$.

База. При $n \leq 50$ алгоритм сортирует массив и возвращает элемент на позиции k , что по определению и есть k -я порядковая статистика.

Переход. Пусть $n > 50$. Алгоритм выбирает пивот x (медиану массива медиан), затем выполняет трёхчастное разбиение A на L, E, R . По свойствам `partition3` верно: все элементы L строго меньше x , все элементы E равны x , все элементы R строго больше x .

- ▷ Если $k \leq |L|$, то после сортировки всего массива первые $|L|$ элементов будут ровно элементами L (в некотором порядке), значит k -я порядковая статистика всего массива совпадает с k -й порядковой статистикой массива L . Алгоритм рекурсивно вызывает `Select(L, k)`, что корректно по предположению индукции.
- ▷ Если $|L| < k \leq |L| + |E|$, то после сортировки на позициях $|L| + 1, \dots, |L| + |E|$ стоят элементы, равные x . Следовательно, искомым элемент равен x , и алгоритм возвращает x .
- ▷ Если $k > |L| + |E|$, то первые $|L| + |E|$ элементов отсортированного массива — это элементы L и E , а оставшаяся часть состоит из R . Тогда k -я порядковая статистика всего массива совпадает с $(k - |L| - |E|)$ -й порядковой статистикой в R . Алгоритм рекурсивно вызывает `Select(R, k - |L| - |E|)`, что корректно по предположению индукции.

Во всех случаях ответ корректен. ■

Ключевая лемма о качестве пивота

Лемма 12.1. Пусть $n = |A|$. Если разбить A на группы по 5 элементов и выбрать x как медиану медиан, то x гарантированно имеет ранг между $\frac{3n}{10}$ и $\frac{7n}{10}$. Иными словами, в массиве найдётся хотя бы $\frac{3n}{10}$ элементов $\leq x$ и хотя бы $\frac{3n}{10}$ элементов $\geq x$.

△ *Доказательство.* Разобьём A на группы по 5 и в каждой группе отсортируем элементы. В каждой полной группе медиана (3-й элемент) не меньше двух элементов группы и не больше двух элементов группы.

Пусть M — массив медиан групп, а x — медиана массива M . Тогда как минимум половина элементов массива M не превосходит x . Рассмотрим любую группу, чья медиана $\leq x$. В этой группе как минимум три элемента (первые три в отсортированной группе) $\leq$ её медианы, а значит и $\leq x$.

Число групп $\approx n/5$, медиан в M тоже $\approx n/5$, поэтому таких групп не меньше $|M|/2$. Следовательно, элементов $\leq x$ как минимум

$$3 \cdot \frac{|M|}{2} \geq 3 \cdot \frac{\lfloor n/5 \rfloor}{2} \geq \frac{3n}{10} - O(1).$$

Аналогично, как минимум половина медиан $\geq x$, и каждая соответствующая группа даёт как минимум три элемента $\geq x$. Отсюда получаем хотя бы $\frac{3n}{10} - O(1)$ элементов $\geq x$. Поглощая константу в асимптотике и учитывая неполную последнюю группу, получаем заявленную оценку ранга. ■

Асимптотика

Пусть $T(n)$ — худшее время работы `Select` на массиве длины n .

- ▷ Разбиение на группы и сортировка каждой пятёрки — $O(n)$ (групп $\lceil n/5 \rceil$, каждая сортируется за $O(1)$).
- ▷ Построение массива медиан M — $O(n)$.
- ▷ Рекурсивный поиск медианы массива M даёт слагаемое $T(\lceil n/5 \rceil)$.
- ▷ `partition3` — $O(n)$.
- ▷ По лемме выше размер рекурсивной подзадачи после разбиения не превосходит $\frac{7n}{10} + O(1)$, поэтому даёт слагаемое $T(\lceil 7n/10 \rceil)$.

Итого получаем рекурренту

$$T(n) \leq T\left(\left\lceil \frac{n}{5} \right\rceil\right) + T\left(\left\lceil \frac{7n}{10} \right\rceil\right) + cn \quad \text{для некоторой константы } c.$$

Утверждение 12.2. Из этой рекурренты следует $T(n) = O(n)$.

△ *Доказательство.* Докажем по индукции, что $T(n) \leq an$ для некоторой константы a и всех достаточно больших n . Пусть это верно для всех меньших аргументов. Тогда

$$T(n) \leq a\frac{n}{5} + a\frac{7n}{10} + cn = a\frac{9n}{10} + cn.$$

Выберем $a \geq 10c$. Тогда $a\frac{9}{10} + c \leq a$, и значит $T(n) \leq an$. База для малых n достигается увеличением a на константу. ■

Замечание. Почему группы именно по 5? Если взять группы по 3, получится гарантия ранга между $\frac{n}{3}$ и $\frac{2n}{3}$, и рекуррента станет $T(n) \leq T(n/3) + T(2n/3) + O(n)$, что даёт уже $\Theta(n \log n)$. Для групп по 5 сумма коэффициентов $\frac{1}{5} + \frac{7}{10} = 0.9 < 1$, поэтому получается линейное время.

Замечания по реализации

- ▷ Алгоритм можно реализовать *in-place* (как и `QuickSelect`), работая с подотрезками массива и делая `partition3` на месте. Тогда дополнительная память $O(1)$ (плюс глубина рекурсии).
- ▷ На практике детерминированный выбор пивота обычно медленнее рандомизированного `QuickSelect` из-за больших констант, но на экзамене важна именно гарантия худшего случая $O(n)$.

13 Билет №13. Алгоритм быстрой сортировки за $O(n \log n)$

13.1 Постановка задачи

Дан массив A длины n (элементы сравнимы; сравнение — $O(1)$ в RAM-модели). Требуется отсортировать A по неубыванию.

13.2 Почему обычный QuickSort в худшем случае квадратичный

Стандартный QuickSort:

1. выбирает опорный элемент (pivot) x ;
2. за линейное время разбивает массив на элементы $< x$, $= x$, $> x$;
3. рекурсивно сортирует левую и правую части.

Если каждый раз pivot оказывается минимумом/максимумом, одна часть имеет размер $n - 1$, а другая — 0. Тогда

$$T(n) = T(n - 1) + O(n) = O(n^2).$$

13.3 Идея: гарантировать «хороший» pivot

Хотим обеспечить, что на каждом шаге разбиение **не сильно перекошено**: существует константа $0 < \alpha < \frac{1}{2}$, такая что после разбиения обе рекурсивные части имеют размер

$$\leq (1 - \alpha)n.$$

Например, если pivot гарантированно лежит между 30-й и 70-й процентилями (то есть не хуже разбиения 30/70), то размеры частей $\leq 0.7n$.

Такой pivot можно выбирать **детерминированно** за $O(n)$ с помощью алгоритма выбора k -й порядковой статистики за линейное время (BFPRТ / median-of-medians) — это билет 12.

13.4 Алгоритм (детерминированный QuickSort)

Считаем, что есть подпрограмма $\text{Select}(A, k)$, которая возвращает k -ю порядковую статистику массива A за $O(|A|)$ **в худшем случае** (например, BFPRТ).

1. Если $|A| \leq 1$, вернуть A .
2. Выбрать опору $x := \text{Select}(A, \lfloor |A|/2 \rfloor)$ (медиану) *или* любую «достаточно хорошую» опору (BFPRТ гарантирует 30/70).
3. Выполнить трёхпутевое разбиение (partition) за $O(n)$:

$$A \longrightarrow L \parallel E \parallel R,$$

где $L = \{a \in A : a < x\}$, $E = \{a \in A : a = x\}$, $R = \{a \in A : a > x\}$.

4. Рекурсивно отсортировать L и R тем же алгоритмом.
5. Вернуть $\text{Sort}(L) \parallel E \parallel \text{Sort}(R)$.

Замечания по реализации.

- ▷ Разбиение удобно делать in-place; для корректной работы с равными элементами используйте трёхпутевое разбиение (аналог «Dutch national flag»).
- ▷ Если **Select** тоже реализована in-place, то доп. память $O(1)$, кроме стека рекурсии.

13.5 Корректность

Утверждение 13.1. Алгоритм возвращает массив, отсортированный по неубыванию и являющийся перестановкой исходного массива.

△ *Доказательство.* Докажем по индукции по $n = |A|$.

База: $n \leq 1$. Массив уже отсортирован.

Переход: пусть утверждение верно для всех размеров $< n$. После разбиения $A \rightarrow L \parallel E \parallel R$ выполняется:

- ▷ все элементы L строго меньше x ;
- ▷ все элементы E равны x ;
- ▷ все элементы R строго больше x ;
- ▷ L, E, R образуют разбиение мультимножества элементов A (ничего не теряем и не добавляем).

По предположению индукции рекурсивные вызовы возвращают отсортированные массивы $\text{Sort}(L)$ и $\text{Sort}(R)$, являющиеся перестановками L и R .

Тогда в конкатенации $\text{Sort}(L) \parallel E \parallel \text{Sort}(R)$: все элементы слева $\leq x$, затем идут элементы $= x$, затем элементы $\geq x$, значит вся последовательность отсортирована. Элементы сохранены (перестановка), так как объединение трёх частей даёт исходное A . ■

13.6 Время работы

Пусть $T(n)$ — худшее время.

На каждом шаге:

- ▷ поиск **pivot**: $O(n)$ (в худшем случае) — за счёт **Select**;
- ▷ **partition**: $O(n)$;
- ▷ рекурсии на двух частях.

Случай А: pivot — точная медиана. Тогда обе части имеют размер $\leq \lceil n/2 \rceil$:

$$T(n) \leq 2T(\lceil n/2 \rceil) + cn \Rightarrow T(n) = O(n \log n).$$

Случай В: pivot «достаточно хороший», например 30/70. BFPRT гарантирует, что pivot лежит между 30-й и 70-й перцентилями, то есть размеры рекурсивных частей не превосходят $0.7n$ (с точностью до округления):

$$T(n) \leq T(\leq 0.7n) + T(\leq 0.3n) + cn.$$

Покажем $T(n) = O(n \log n)$ через дерево рекурсии:

- ▷ На каждом уровне рекурсии подмассивы попарно не пересекаются и получаются разбиением исходного массива, значит суммарный размер всех подмассивов уровня $\leq n$. Следовательно, суммарная стоимость (**Select+partition**) на уровне — $O(n)$.
- ▷ Глубина рекурсии: на любом пути вниз размер уменьшается хотя бы в $1/0.7$ раз: $n, 0.7n, 0.7^2n, \dots$, поэтому глубина $O(\log n)$.

Итого $O(n)$ на уровень $\times O(\log n)$ уровней $= O(n \log n)$.

13.7 Память

При in-place разбиении доп. память — $O(1)$, кроме стека рекурсии. Так как разбиения гарантированно сбалансированы, глубина рекурсии $O(\log n)$, значит память под стек — $O(\log n)$.

13.8 Практическое замечание

Median-of-medians даёт сильную гарантию на худший случай, но с большой константой. На практике часто используют рандомизированный QuickSort (ожидаемо $O(n \log n)$) или гибриды (например, introsort: QuickSort с переключением на HeapSort при «плохой» глубине).

14 Билет №14. Стабильная сортировка подсчётом. Сортировка пар чисел

14.1 Стабильность

Определение 14.1 (Стабильная сортировка). Сортировка называется *стабильной*, если для любых двух элементов с равными ключами их относительный порядок в выходном массиве совпадает с относительным порядком во входном. Иными словами, равные элементы *не переупорядочиваются*.

Пример. Отсортируем по ключу пары (key, id) :

$[(2, \text{Carol}), (1, \text{David}), (2, \text{Alice}), (3, \text{Bob})]$.

Стабильная сортировка по key обязана оставить Carol раньше Alice, потому что их ключи равны и во входе Carol стояла раньше:

$[(1, \text{David}), (2, \text{Carol}), (2, \text{Alice}), (3, \text{Bob})]$.

14.2 Стабильная сортировка подсчётом

Задача. Дан массив A длины n . Каждый элемент — либо целое число, либо запись (record), у которой есть ключ $key(A[i])$. Предполагается, что ключи — целые числа из диапазона $\{0, 1, \dots, K\}$ (или из $[L; R]$, что сводится к $[0; K]$ сдвигом). Требуется отсортировать массив по неубыванию ключа, причём желательно стабильно.

Замечание. Сортировка подсчётом не использует сравнения элементов между собой: она опирается на то, что ключи лежат в небольшом диапазоне и по ним можно адресовать массив частот. Поэтому она не подпадает под нижнюю оценку $\Omega(n \log n)$ для сортировок сравнениями.

14.2.1 Алгоритм (стабильный вариант)

Будем считать, что индексация массивов в этом описании — с нуля.

1. Заводим массив счётчиков $cnt[0..K]$, изначально нули.
2. Проходим по входу и считаем частоты:

$$cnt[v] \leftarrow |\{i : key(A[i]) = v\}|.$$

3. Превращаем cnt в массив префиксных сумм:

$$cnt[v] \leftarrow cnt[v] + cnt[v - 1] \quad (v = 1..K).$$

Теперь $cnt[v]$ равно количеству элементов с ключом $\leq v$.

4. Заводим выходной массив $B[0..n - 1]$.

5. Проходим вход **справа налево**:

- (a) $v \leftarrow \text{key}(A[i])$.
- (b) $\text{cnt}[v] \leftarrow \text{cnt}[v] - 1$.
- (c) $B[\text{cnt}[v]] \leftarrow A[i]$.

6. При необходимости копируем B обратно в A .

Замечание. Если на шаге 5 идти *слева направо*, сортировка останется корректной по ключу, но станет *нестабильной*.

14.2.2 Корректность

Лемма 14.1 (Смысл префиксных сумм). *После шага 3 для каждого $v \in [0..K]$ величина $\text{cnt}[v]$ равна числу элементов входа с ключом $\leq v$.*

$\triangle$ *Доказательство.* До префиксных сумм $\text{cnt}[v]$ — это число элементов с ключом равно v . При замене $\text{cnt}[v] \leftarrow \text{cnt}[v] + \text{cnt}[v - 1]$ мы накапливаем сумму частот от 0 до v , т.е. получаем число элементов с ключом $\leq v$. ■

Лемма 14.2 (Правильные позиции для ключа v). *Пусть $P[v]$ — значение $\text{cnt}[v]$ сразу после шага 3 (до уменьшений на шаге 5), а $P[-1] = 0$. Тогда элементы с ключом v в выходном массиве B занимают ровно позиции*

$$[P[v - 1], P[v] - 1].$$

$\triangle$ *Доказательство.* На шаге 5 мы стартуем с $\text{cnt}[v] = P[v]$ и каждый раз делаем $\text{cnt}[v] \leftarrow \text{cnt}[v] - 1$, помещая очередной элемент в позицию $\text{cnt}[v]$. Значит, используются позиции $P[v] - 1, P[v] - 2, \dots$ ровно столько раз, сколько элементов с ключом v . Последняя использованная позиция равна $P[v] - |\{i : \text{key}(A[i]) = v\}| = P[v] - 1$. Следовательно, заняты ровно позиции от $P[v - 1]$ до $P[v] - 1$. ■

Лемма 14.3 (Стабильность). *Если $i < j$ и $\text{key}(A[i]) = \text{key}(A[j])$, то в выходном массиве элемент $A[i]$ окажется левее элемента $A[j]$.*

$\triangle$ *Доказательство.* Обход идёт справа налево, поэтому $A[j]$ обрабатывается раньше $A[i]$. Оба элемента имеют одинаковый ключ v , значит, при обработке $A[j]$ счётчик $\text{cnt}[v]$ уменьшается. Когда позже обрабатывается $A[i]$, значение $\text{cnt}[v]$ уже меньше, поэтому $A[i]$ попадает в более левую позицию. ■

Теорема 14.1. *Алгоритм сортировки подсчётом (выше) сортирует массив по неубыванию ключа и делает это стабильно.*

$\triangle$ *Доказательство.* По лемме о правильных позициях элементы каждого ключа v попадают в свой блок, причём блоки идут по возрастанию v , значит массив отсортирован по ключу. Стабильность даёт предыдущая лемма. ■

14.2.3 Асимптотика

- ▷ Время: $O(n + K)$.
- ▷ Память: $O(n + K)$ (буфер B и массив счётчиков cnt).

Замечание (Когда метод применим). Сортировка подсчётом выгодна, когда K не слишком велик относительно n (например, $K = O(n)$). Если диапазон ключей огромный, то метод становится невыгодным по памяти/времени.

14.3 Сортировка пар чисел

Задача. Дан массив пар (a_i, b_i) , $i = 1..n$, где $a_i \in [0..A]$, $b_i \in [0..B]$. Требуется отсортировать пары лексикографически.

Определение 14.2 (Лексикографический порядок на парах). $(a, b) < (a', b')$ тогда и только тогда, когда $a < a'$, либо $a = a'$ и $b < b'$.

14.3.1 Алгоритм

1. Стабильно отсортировать пары по второму компоненту b (ключ b , диапазон $0..B$).
2. Стабильно отсортировать результат по первому компоненту a (ключ a , диапазон $0..A$).

14.3.2 Корректность

Теорема 14.2. После двух проходов выше пары отсортированы лексикографически по (a, b) .

△ *Доказательство.* После первого прохода массив упорядочен по b . Второй проход сортирует по a : для разных a порядок становится корректным автоматически. Рассмотрим группу элементов с фиксированным $a = a_0$. Стабильность второй сортировки гарантирует, что относительный порядок элементов внутри группы a_0 сохраняется таким, каким он был до второго прохода, то есть после первого. А после первого прохода внутри группы элементы упорядочены по b . Значит, внутри каждой группы равных a элементы идут по возрастанию b , что и даёт лексикографический порядок. ■

14.3.3 Асимптотика

Два запуска сортировки подсчётом дают

$$O(n + B) + O(n + A) = O(n + A + B)$$

по времени и $O(n + \max\{A, B\})$ по памяти (если переиспользовать буферы).

Замечание. Приём обобщается: чтобы отсортировать по ключам $(k_1, k_2, \dots, k_t)$ лексикографически, достаточно стабильно сортировать *с конца*: сначала по k_t , затем по k_{t-1} и так далее до k_1 . Эта идея лежит в основе поразрядных сортировок.

14.4 Псевдокод (идейно, близко к CLRS)

Псевдокод (идейно, близко к CLRS)

```

StableCountingSort( $A, K, key$ ): // ключи  $key(A[i]) \in \{0, 1, \dots, K\}$ 

1.  $n \leftarrow |A|$ 
2. создать массив  $cnt[0..K]$  и заполнить нулями
3. for  $i = 0..n - 1$ :
   (a)  $v \leftarrow key(A[i])$ 
   (b)  $cnt[v] \leftarrow cnt[v] + 1$ 
4. for  $v = 1..K$ :  $cnt[v] \leftarrow cnt[v] + cnt[v - 1]$  // префиксные суммы
5. создать массив  $B[0..n - 1]$ 
6. for  $i = n - 1..0$  (справа налево):
   (a)  $v \leftarrow key(A[i])$ 
   (b)  $cnt[v] \leftarrow cnt[v] - 1$ 
   (c)  $B[cnt[v]] \leftarrow A[i]$ 
7. return  $B$ 

LexSortPairs( $P, A, B$ ): // пары  $(a, b)$ ,  $a \in [0..A]$ ,  $b \in [0..B]$ 

1.  $P \leftarrow \text{StableCountingSort}(P, B, second)$ 
2.  $P \leftarrow \text{StableCountingSort}(P, A, first)$ 
3. return  $P$ 

```

Замечание (Практические замечания). $\triangleright$ Если ключи лежат в диапазоне $[L..R]$, используйте сдвиг $idx = key - L$, чтобы получить диапазон $[0..K]$.

$\triangleright$ Если диапазон K слишком большой, массив cnt может не помещаться в память — тогда сортировка подсчётом не подходит.

$\triangleright$ Для лексикографической сортировки важно, что оба прохода *стабильны*, иначе порядок по “младшему” ключу потеряется.

15 Билет №15. Цифровая сортировка (LSD)

15.1 Постановка задачи

Пусть дан массив ключей $A[1..n]$. Каждый ключ представим в виде последовательности из d “цифр” (разрядов) в системе счисления с основанием B :

$$x = \sum_{i=0}^{d-1} \text{digit}(x,i) B^i, \quad \text{digit}(x,i) \in \{0, 1, \dots, B-1\}.$$

Здесь $\text{digit}(x,i)$ — i -й разряд *с конца* (*least significant digit*, LSD), то есть $i = 0$ — младший разряд.

Задача. Отсортировать A по неубыванию ключей.

Замечание. LSD-подход наиболее естественен при фиксированной длине ключа d . Если ключи переменной длины (например, строки), то часто:

- ▷ либо дописывают “ведущие нули” / минимальные фиктивные символы до общей длины,
- ▷ либо группируют по длине, сортируют каждую группу и склеивают группы по возрастанию длины.

Ниже считаем длину фиксированной: все ключи имеют ровно d разрядов.

15.2 Алгоритм LSD Radix Sort

Идея: выполнять устойчивую сортировку по одному разряду, начиная с младшего и двигаясь к старшему. Ключевой момент — **на каждом шаге нужна стабильная сортировка** по текущему разряду.

Определение 15.1. Сортировка называется *стабильной*, если для любых двух равных ключей их относительный порядок в выходном массиве совпадает с порядком во входном.

Схема алгоритма. Для $i = 0, 1, \dots, d-1$:

1. стабильно отсортировать массив A по значению $\text{digit}(\cdot, i)$.

Типичная реализация устойчивой сортировки разряда: подсчётом. Поскольку $\text{digit}(x,i) \in \{0, \dots, B-1\}$, удобно использовать стабильную сортировку подсчётом (Counting Sort) за $O(n + B)$.

Псевдокод (идейно, близко к CLRS)

RadixLSD(A, d, B): // d разрядов, основание B

1. **for** $i = 0..d-1$ **do**

```

    (a)  $A \leftarrow \text{STABLECOUNTINGSORTBYDIGIT}(A, i, B)$ 
2. return  $A$ 

StableCountingSortByDigit( $A, i, B$ ):    // сортировка по разряду  $i$  (LSD нумерация)
1.  $n \leftarrow |A|$ 
2. создать массив  $\text{cnt}[0..B-1]$  и заполнить нулями
3. for each  $x \in A$  do  $\text{cnt}[\text{DIGIT}(x, i)] \leftarrow \text{cnt}[\text{DIGIT}(x, i)] + 1$ 
4. for  $v = 1..B-1$  do  $\text{cnt}[v] \leftarrow \text{cnt}[v] + \text{cnt}[v-1]$            // префиксные суммы
5. создать массив  $\text{out}[1..n]$ 
6. for  $j = n$  downto 1 do                                           (справа налево даёт стабильность)
    (a)  $v \leftarrow \text{DIGIT}(A[j], i)$ 
    (b)  $\text{cnt}[v] \leftarrow \text{cnt}[v] - 1$ 
    (c)  $\text{out}[\text{cnt}[v] + 1] \leftarrow A[j]$                                // +1 из-за 1-индексации массива  $\text{out}$ 
7. return  $\text{out}$ 

```

Замечание. В строке $\text{out}[\text{cnt}[v]+1] = A[j]$ индексация зависит от соглашений (0/1-индексация). Важно лишь, что мы размещаем элементы в выходной массив по убыванию j (справа налево), уменьшая счётчик: это обеспечивает *стабильность* подсчёта.

15.3 Доказательство корректности

Обозначим через $x^{(i)}$ последовательность из $(i+1)$ младших разрядов числа x :

$$x^{(i)} = (\text{digit}(x, i), \text{digit}(x, i-1), \dots, \text{digit}(x, 0)).$$

Порядок по $x^{(i)}$ — лексикографический (сначала сравниваем $\text{digit}(x, i)$, при равенстве — $\text{digit}(x, i-1)$ и т.д.).

Лемма 15.1. После завершения итерации i (то есть после стабильной сортировки по разряду i) массив A отсортирован по ключу $x^{(i)}$.

$\triangle$ *Доказательство.* Индукция по i .

База: $i = 0$. После одной стабильной сортировки по $\text{digit}(\cdot, 0)$ массив упорядочен по младшему разряду, то есть по $x^{(0)}$.

Переход: пусть утверждение верно для $i-1$. Рассмотрим i -ю итерацию: мы стабильно сортируем A по $\text{digit}(\cdot, i)$. После этого элементы разбиваются на блоки с одинаковым $\text{digit}(\cdot, i)$, причём блоки идут в порядке возрастания этого разряда. Внутри каждого блока стабильность сохраняет прежний относительный порядок. Но до сортировки по i -му разряду массив уже был отсортирован по $x^{(i-1)}$, то есть внутри каждого блока элементы упорядочены по последовательности младших разрядов $(i-1, i-2, \dots, 0)$. Следовательно, после

текущей итерации массив отсортирован по лексикографическому ключу $(i, i-1, \dots, 0)$, то есть по $x^{(i)}$. ■

Теорема 15.1. После выполнения всех d итераций алгоритм *LSD Radix Sort* сортирует массив A по неубыванию ключей.

△ *Доказательство.* По лемме после итерации $d-1$ массив отсортирован по $x^{(d-1)}$, то есть по всем разрядам ключа. Это и означает сортировку исходных ключей. ■

Следствие (Стабильность всего алгоритма). Если внутренняя сортировка по разряду стабильна, то *LSD Radix Sort* в целом стабилен.

15.4 Асимптотика

При использовании стабильной сортировки подсчётом по каждому разряду:

- ▷ $O(B)$ на обнуление массива счётчиков и вычисление префиксных сумм;
- ▷ $O(n)$ на проход по массиву и построение выходного массива.

Итого один разряд: $O(n + B)$, а всего разрядов d :

$$T(n) = O(d(n + B)).$$

Память: $O(n + B)$ (выходной массив размера n + массив счётчиков размера B).

Замечание. Radix Sort — *несравнительная* сортировка: она не опирается на сравнения “<” между ключами, поэтому на неё не распространяется нижняя оценка $\Omega(n \log n)$ для сортировок сравнениями. Цена “линейности” — зависимость от числа разрядов d и размера алфавита B .

15.5 Практические замечания

- ▷ Для целых w -битных чисел удобно брать основание $B = 2^r$ (разряд = r бит), тогда $d = \lceil w/r \rceil$, а каждый разряд извлекается битовыми операциями.
- ▷ Для знаковых чисел обычно сводят задачу к беззнаковой (например, добавляя константу, или рассматривая старший бит отдельно); на экзамене часто предполагают неотрицательные ключи.
- ▷ LSD-подход особенно естественен для фиксированной длины ключа. Для переменной длины либо дополняют ключи фиктивными ведущими символами, либо сортируют группы по длине.

16 Билет №16. Бинарная куча: определение и представление в массиве. Требование кучи

16.1 Модельная задача: очередь с приоритетом

Одна из базовых задач, для которой используется куча, — реализовать *очередь с приоритетом* (min-priority queue): хранить мультимножество ключей S и поддерживать операции

- ▷ `insert(x)` — добавить элемент x в S ;
- ▷ `getMin()` — вернуть минимальный элемент из S (не удаляя);
- ▷ `extractMin()` — удалить минимальный элемент из S и вернуть его;
- ▷ `decreaseKey(p,  $\Delta$ )` — уменьшить ключ элемента по *указателю* p на величину $\Delta > 0$.

В последней операции важно, что мы уменьшаем *конкретный* элемент (нам дают “где он лежит”), а не “найди элемент со значением x ”.

16.2 Определение бинарной кучи

Определение 16.1 (Полное бинарное дерево). Бинарное дерево называется *полным*, если все уровни, кроме, возможно, последнего, заполнены полностью, а вершины последнего уровня заполняются слева направо без “дыр”.

Определение 16.2 (Бинарная (min-)куча). *Бинарная (min-)куча* — это бинарное дерево, удовлетворяющее двум условиям:

1. **Свойство формы (shape property):** дерево является полным.
2. **Свойство кучи (heap property):** ключ в каждой вершине не больше ключей в её детях.

То есть для любой вершины v и каждого её ребёнка u выполнено $key(v) \leq key(u)$.

Замечание. Если во втором пункте поставить знак $\geq$, получим *max-кучу*. В этом курсе (и в билетах 16–20) обычно рассматривается min-куча, потому что базовые операции называются `getMin/extractMin`.

16.3 Представление бинарной кучи в массиве

Ключевое наблюдение: так как форма кучи — полное бинарное дерево, его можно хранить *неявно* в массиве, записав вершины по уровням слева направо.

Определение 16.3 (Уровневое (неявное) хранение). Пусть куча содержит n элементов. Запишем вершины в массив $a[1..n]$ в порядке обхода по уровням (BFS):

- ▷ $a[1]$ — ключ в корне;

▷ затем идут вершины 2-го уровня слева направо, затем 3-го и т.д.

Такое представление не хранит указатели на детей: связи “родитель–дети” восстанавливаются арифметикой по индексам.

Утверждение 16.1 (Индексы родителя и детей в 1-индексации). Пусть вершина записана в $a[v]$.

▷ Её левый ребёнок (если существует) имеет индекс $2v$.

▷ Её правый ребёнок (если существует) имеет индекс $2v + 1$.

▷ Её родитель (для $v > 1$) имеет индекс $\lfloor \frac{v}{2} \rfloor$.

Ребёнок существует тогда и только тогда, когда соответствующий индекс не превосходит n .

Δ Доказательство. Нумерация по уровням совпадает с нумерацией вершин полного бинарного дерева “слева направо”. На уровне ℓ (где корень имеет уровень 0) находится ровно 2^ℓ вершин, а их индексы — подряд.

Пусть вершина v расположена на уровне ℓ и является k -й слева на этом уровне (нумерация k с нуля). Тогда её индекс равен

$$v = 2^\ell + k.$$

Её левый ребёнок расположен на следующем уровне $\ell + 1$ и имеет номер $2k$ среди вершин этого уровня, значит индекс равен

$$2^{\ell+1} + 2k = 2(2^\ell + k) = 2v.$$

Аналогично правый ребёнок имеет номер $2k + 1$ и индекс

$$2^{\ell+1} + (2k + 1) = 2v + 1.$$

Наконец, для $v > 1$ вершина является либо левым ребёнком p (тогда $v = 2p$), либо правым (тогда $v = 2p + 1$). В обоих случаях $p = \lfloor v/2 \rfloor$. ■

16.4 Требование кучи в терминах массива

В представлении $a[1..n]$ условие *heap property* записывается без слов:

Определение 16.4 (Требование (инвариант) min-кучи). Массив $a[1..n]$ является корректной min-кучей тогда и только тогда, когда для всех индексов v выполнено:

$$\begin{aligned} 2v \leq n &\implies a[v] \leq a[2v], \\ 2v + 1 \leq n &\implies a[v] \leq a[2v + 1]. \end{aligned}$$

16.5 Ключевые следствия и асимптотики

Утверждение 16.2 (Минимум лежит в корне). *В min-куче минимальный элемент множества S находится в корне, то есть равен $a[1]$.*

△ *Доказательство.* Рассмотрим произвольную вершину u . По требованию кучи ключ в родителе не больше ключа в ребёнке. Значит, двигаясь от корня к u по пути, мы получаем неубывающую последовательность ключей. Следовательно, ключ в корне не больше ключа в любой вершине, то есть является глобальным минимумом. ■

Утверждение 16.3 (Высота бинарной кучи). *Пусть куча содержит $n \geq 1$ элементов и хранится как полное бинарное дерево. Тогда её высота (число рёбер на максимальном пути от корня до листа) равна*

$$h = \lfloor \log_2 n \rfloor,$$

а число уровней равно $h + 1$. В частности, $h = O(\log n)$.

△ *Доказательство.* В полном бинарном дереве с высотой h полностью заполнены уровни $0, 1, \dots, h - 1$, поэтому число вершин не меньше

$$1 + 2 + 4 + \dots + 2^{h-1} = 2^h - 1.$$

С другой стороны, на уровне h может быть не более 2^h вершин, значит общее число вершин не превосходит

$$(2^h - 1) + 2^h = 2^{h+1} - 1.$$

Итак, $2^h \leq n < 2^{h+1}$, откуда $h \leq \log_2 n < h + 1$, то есть $h = \lfloor \log_2 n \rfloor$. ■

Следствие (Асимптотики хранения и доступа). Для бинарной кучи из n элементов:

- ▷ память $O(n)$ (массив из n ключей);
- ▷ `getMin` работает за $O(1)$ (это $a[1]$);
- ▷ любые процедуры, которые “поднимают” или “опускают” элемент вдоль одного пути (например, `siftUp/siftDown` в следующих билетах) работают за $O(h) = O(\log n)$.

Пример (Пример min-кучи). Пусть $a = [\underline{1}, 3, 4, 7, 9, 10]$ (подчёркнут корень). Тогда (в 1-индексации)

- ▷ дети корня $a[1]$ — это $a[2] = 3$ и $a[3] = 4$;
- ▷ дети вершины $a[2]$ — это $a[4] = 7$ и $a[5] = 9$;
- ▷ у вершины $a[3]$ есть только левый ребёнок $a[6] = 10$.

Неравенства $a[v] \leq a[2v]$ и $a[v] \leq a[2v + 1]$ (когда дети существуют) легко проверяются.

17 Билет №17. Операции siftUp и siftDown в бинарной куче

17.1 Контекст и соглашения

Далее рассматриваем **минимальную бинарную кучу** (*min-heap*):

- ▷ структура — **полное** бинарное дерево (все уровни, кроме последнего, заполнены полностью; последний заполняется слева направо);
- ▷ **требование кучи (heap-order)**: значение в вершине не превосходит значений в её детях; то есть минимальный элемент всегда находится в корне.

Хранение — в массиве $a[1 \dots n]$ (**индексация с 1**). Тогда

$$\text{parent}(v) = \left\lfloor \frac{v}{2} \right\rfloor, \quad \text{left}(v) = 2v, \quad \text{right}(v) = 2v + 1.$$

(Для индексации с 0: $\text{parent}(v) = \lfloor (v-1)/2 \rfloor$, $\text{left}(v) = 2v+1$, $\text{right}(v) = 2v+2$.)

Определение 17.1 (Требование кучи). Массив $a[1 \dots n]$ задаёт *корректную минимальную бинарную кучу*, если для всех вершин v таких, что соответствующий ребёнок существует,

$$a[v] \leq a[\text{left}(v)] \quad \text{и} \quad a[v] \leq a[\text{right}(v)].$$

Замечание. Для *max-heap* во всех формулах достаточно заменить сравнение “ $<$ ”/“ $\leq$ ” на “ $>$ ”/“ $\geq$ ”.

17.2 Операция siftUp (просеивание вверх)

Типичная ситуация применения.

- ▷ **insert**: новый элемент поставили в конец массива ($v = n$), то есть в новый лист.
- ▷ **decreaseKey**: в некоторой вершине v уменьшили ключ (значение), поэтому он мог стать меньше родителя.

В обоих случаях (до последнего изменения) структура была корректной кучей, а потенциальное нарушение могло появиться только на пути от v к корню.

Псевдокод $\text{siftUp}(v)$.

Псевдокод (идейно, близко к CLRS)

SiftUp(a, v): // $a[1..n]$ — min-heap

1. **while** $v > 1$ **and** $a[v] < a[\text{parent}(v)]$:

- (a) $p \leftarrow \text{parent}(v)$;
- (b) $\text{swap}(a[v], a[p])$;
- (c) $v \leftarrow p$.

Утверждение 17.1 (Корректность `siftUp`). Пусть после выполнения `insert` (в новый лист) или `decreaseKey` (уменьшения ключа в вершине v) единственное возможное нарушение требования кучи связано с тем, что $a[v]$ стало меньше родителя. Тогда после завершения `siftUp(v)` массив a задаёт корректную минимальную бинарную кучу.

△ *Доказательство.* Обозначим через v текущую позицию “поднимаемого” элемента внутри алгоритма. Докажем по индукции по числу итераций **инвариант**:

- ▷ все рёбра дерева, кроме, возможно, ребра $(\text{parent}(v), v)$, удовлетворяют требованию кучи;
- ▷ в поддереве вершины v требование кучи выполнено полностью.

База. Сразу после `insert` вершина v — лист, поэтому второе условие тривиально. После `decreaseKey` ключ в v уменьшился, значит, для её детей неравенство $a[v] \leq a[\text{child}]$ сохранится (становится только легче). Все остальные рёбра не менялись, поэтому возможное нарушение действительно только между v и её родителем.

Переход. Если условие цикла ложно, то либо $v = 1$, либо $a[\text{parent}(v)] \leq a[v]$. Тогда по инварианту нарушений не остаётся, и куча корректна.

Если условие цикла истинно, положим $p = \text{parent}(v)$. Пусть до обмена $a[v] = x$ и $a[p] = y$, причём $x < y$. Выполним `swap` в v и p . Покажем, что после обмена инвариант сохраняется для нового $v := p$.

1. **Рёбра из p вниз становятся корректными.** После обмена в p стоит x , а одним из детей p является прежняя вершина v , в которой теперь стоит y . Поэтому $x \leq y$. Если у p есть второй ребёнок s (“сестра” прежней вершины v), то до обмена выполнялось $y \leq a[s]$ (ребро (p, s) не нарушалось). Тогда из $x < y \leq a[s]$ следует $x \leq a[s]$. Значит, в p требование кучи выполнено для обоих детей.
2. **Поддерево прежней вершины v остаётся кучей.** После обмена в этой вершине стоит y . Заметим, что до последнего изменения структуры (до `insert/decreaseKey`) куча была корректной, поэтому y не превосходило *всех* своих потомков (в частности, всех вершин поддерева прежней v). Операции `insert` и `decreaseKey` не изменяют значения потомков прежней v . Значит, y по-прежнему не превосходит значений её детей, то есть рёбра вниз из прежней v корректны, а всё поддерево остаётся кучей.

Таким образом, после обмена возможное нарушение может остаться только между новым $v = p$ и его родителем (то есть мы “подняли” потенциальную проблему на уровень выше), а все остальные рёбра корректны. Инвариант доказан.

Завершение. Индекс v на каждой итерации заменяется на $\text{parent}(v)$, то есть строго уменьшается и не может уменьшаться бесконечно. Алгоритм обязательно остановится (либо в корне, либо в первой вершине, где $a[\text{parent}(v)] \leq a[v]$). По инварианту в момент остановки нарушений нет, значит, полученная структура является корректной минимальной кучей. ■

Асимптотика `siftUp`. На каждой итерации мы поднимаемся на один уровень вверх; высота кучи равна $\lfloor \log_2 n \rfloor$. Значит, `siftUp` работает за $O(\log n)$ времени и использует $O(1)$ дополнительной памяти.

17.3 Операция siftDown (просеивание вниз)

Типичная ситуация применения. После `extractMin` последний элемент массива переносят в корень. Тогда требование кучи может нарушиться в корне (или в текущей вершине v), но поддеревья детей остаются корректными кучами.

Псевдокод `siftDown(v)`.

Псевдокод (идейно, близко к CLRS)

SiftDown(a, v): // $a[1..n]$ — min-heap

1. **while** $\text{left}(v) \leq n$:
 - (a) $j \leftarrow \text{left}(v)$;
 - (b) **if** $\text{right}(v) \leq n$ **and** $a[\text{right}(v)] < a[j]$ **then** $j \leftarrow \text{right}(v)$;
 - (c) **if** $a[v] \leq a[j]$ **then break**;
 - (d) $\text{swap}(a[v], a[j])$;
 - (e) $v \leftarrow j$.

Утверждение 17.2 (Корректность `siftDown`). Пусть в массиве $a[1 \dots n]$ все рёбра удовлетворяют требованию кучи, кроме, возможно, рёбер, выходящих из некоторой вершины v к её детям, и при этом поддеревья детей вершины v являются корректными кучами. Тогда после завершения `siftDown(v)` массив a задаёт корректную минимальную бинарную кучу.

$\triangle$ *Доказательство.* Докажем инвариант цикла. Пусть v — текущая вершина в алгоритме.

Инвариант:

- ▷ все рёбра, не лежащие в поддереве текущей вершины v , корректны;
- ▷ оба поддерева детей вершины v (если дети существуют) — корректные min-heap;
- ▷ возможное нарушение требования кучи может быть только в рёбрах $(v, \text{left}(v))$ и $(v, \text{right}(v))$.

База. В стандартном применении (после переноса последнего элемента в корень при `extractMin`) поддеревья детей корня не меняются, значит, они остаются кучами; потенциальное нарушение действительно только между корнем и его детьми.

Переход. Если у v нет детей, то нарушений быть не может, алгоритм завершится. Пусть дети существуют. Алгоритм выбирает j — ребёнка с минимальным ключом среди детей вершины v .

1. Если $a[v] \leq a[j]$, то $a[v]$ не больше минимального из детей, значит, $a[v]$ не больше *обоих* детей. Тогда рёбра из v корректны, а по инварианту корректны и все остальные рёбра, следовательно, куча восстановлена и можно завершаться.
2. Если $a[v] > a[j]$, выполняется обмен $a[v] \leftrightarrow a[j]$. После обмена в вершине v стоит прежнее $a[j]$. Покажем, что теперь требование кучи в v выполнено:

- ▷ для ребёнка j : после обмена в j стоит прежнее $a[v]$, а мы находимся в случае $a[v] > a[j]$, значит новое значение в v не превосходит нового значения в j ;
- ▷ для второго ребёнка s (если он существует): по выбору j имеем $a[j] \leq a[s]$ до обмена, а значение в s не менялось, значит новое значение в v не превосходит $a[s]$.

Итак, вершина v стала корректной.

Старое значение $a[v]$ переместилось вниз в вершину j . Оно может нарушать требование кучи уже внизу, но поддеревья детей вершины j не менялись и остаются кучами. Поэтому инвариант сохраняется, если переобозначить текущую вершину как $v := j$.

Завершение. На каждой итерации v опускается на один уровень вниз, а глубина кучи конечна. Следовательно, алгоритм завершится, и в момент завершения по инварианту нарушений не остаётся. ■

Асимптотика `siftDown`. За одну итерацию мы спускаемся на один уровень вниз, а высота бинарной кучи равна $O(\log n)$. Следовательно, `siftDown` работает за $O(\log n)$ времени и использует $O(1)$ дополнительной памяти.

Замечание. Термины `siftUp`/`siftDown` также встречаются как *bubble up* / *bubble down*, *percolate up* / *percolate down*, *swim* / *sink*. Суть одна: локально восстановить требование кучи вдоль одного пути, поднимая слишком маленький элемент вверх или опуская слишком большой вниз.

18 Билет №18. insert, getMin, extractMin, decreaseKey через siftUp и siftDown в бинарной куче

В этом билете считаем, что у нас уже есть корректная **минимальная бинарная куча** (min-heap), представленная массивом. Пусть в куче n элементов, они хранятся в массиве $a[1 \dots n]$. Для индекса v :

$$\text{parent}(v) = \left\lfloor \frac{v}{2} \right\rfloor, \quad \text{left}(v) = 2v, \quad \text{right}(v) = 2v + 1.$$

Требование кучи (min-heap property): для любой вершины v , если сын существует, то $a[v] \leq a[\text{left}(v)]$ и $a[v] \leq a[\text{right}(v)]$.

Замечание. Если вы используете нумерацию с нуля ($a[0 \dots n-1]$), то формулы меняются: $\text{parent}(v) = \lfloor (v-1)/2 \rfloor$, $\text{left}(v) = 2v+1$, $\text{right}(v) = 2v+2$. Далее для определенности используем нумерацию с 1.

Напомним смысл базовых процедур:

- ▷ **siftUp**(v) — “всплытие” вершины v вверх, пока не восстановится требование кучи.
- ▷ **siftDown**(v) — “просеивание” вершины v вниз, пока не восстановится требование кучи.

Их корректность доказывается в билете 17.

18.1 Почему getMin — это корень

Утверждение 18.1. Минимальный элемент кучи равен $a[1]$.

△ *Доказательство.* По требованию кучи каждый родитель не больше каждого сына. Значит, по любому пути от корня вниз значения не убывают. Следовательно, $a[1]$ не превосходит ни одного элемента (все элементы достижимы из корня по ребрам вниз). ■

Отсюда:

- ▷ **getMin**() возвращает $a[1]$ за $O(1)$.

18.2 Операция insert(x) через siftUp

Идея. Новый элемент можно добавить только в “следующую свободную позицию” полного бинарного дерева, то есть в конец массива. После этого может нарушиться требование кучи только на пути от новой вершины к корню.

Утверждение 18.2. Алгоритм

1. положить $a[n+1] \leftarrow x$, увеличить $n \leftarrow n+1$;
2. выполнить **siftUp**(n)

корректно реализует **insert**(x).

$\triangle$ *Доказательство.* После шага 1 структура дерева остается полной: мы добавили ровно один новый лист. Все старые ребра (между существовавшими ранее вершинами) не менялись, значит требование кучи на них по-прежнему верно.

Нарушение возможно только на единственном ребре между новой вершиной $v = n$ и ее родителем $p = \text{parent}(v)$: может оказаться $a[v] < a[p]$. Процедура `siftUp` как раз повторяет обмен v с родителем, пока $a[\text{parent}(v)] \leq a[v]$ (или пока v не станет корнем).

По корректности `siftUp` после выполнения шага 2 требование кучи восстановлено во всем дереве, а множество элементов стало прежним $\cup \{x\}$. ■

Асимптотика. Высота бинарной кучи равна $\lfloor \log_2 n \rfloor$, поэтому `siftUp` работает за $O(\log n)$. Значит, `insert` работает за $O(\log n)$.

18.3 Операция `extractMin()` через `siftDown`

Идея. Минимум лежит в корне, но удалять корень напрямую нельзя, иначе сломается “полнота” дерева. Вместо этого переносим последний элемент (самый правый лист) в корень, уменьшаем размер, и восстанавливаем требование кучи процедурой `siftDown`.

Утверждение 18.3. *Алгоритм*

1. сохранить $\text{ans} \leftarrow a[1]$;
2. перенести последний элемент в корень: $a[1] \leftarrow a[n]$, уменьшить $n \leftarrow n - 1$;
3. если $n > 0$, выполнить `siftDown(1)`;
4. вернуть ans .

корректно реализует `extractMin()`.

$\triangle$ *Доказательство.* Шаг 1 действительно сохраняет удаляемое значение: по предыдущему утверждению минимум равен $a[1]$.

На шаге 2 мы удаляем ровно один элемент (последний лист), а его значение переносим в корень. Полнота дерева сохраняется: удаление последнего листа в массивном представлении как раз дает корректное “полное” дерево.

Какие неравенства могли нарушиться? Все ребра, не инцидентные корню, не изменились (их концы и значения прежние). Значит, нарушение возможно только на ребрах $(1, \text{left}(1))$ и $(1, \text{right}(1))$: новое значение в корне могло оказаться слишком большим.

Процедура `siftDown(1)` сравнивает корень с его детьми, меняет местами с меньшим ребенком, и повторяет процесс, пока требование кучи не восстановится. По корректности `siftDown` после шага 3 куча снова корректна.

Таким образом, мы вернули старый минимум и удалили его из структуры. ■

Асимптотика. `siftDown` опускает элемент не более чем на высоту дерева, то есть $O(\log n)$. Значит, `extractMin` работает за $O(\log n)$.

18.4 Операция `decreaseKey(v, newKey)` через `siftUp`

Постановка. В операции `decreaseKey` нам дают указатель/индекс v на конкретный элемент кучи. Это принципиально: по значению искать элемент в куче быстро нельзя.

Утверждение 18.4. Пусть $\text{newKey} \leq a[v]$. Алгоритм

1. присвоить $a[v] \leftarrow \text{newKey}$;

2. выполнить $\text{siftUp}(v)$

корректно реализует decreaseKey .

$\triangle$ *Доказательство.* Мы уменьшили значение в вершине v . Тогда требование кучи могло нарушиться только на ребре между v и его родителем: возможна ситуация $a[v] < a[\text{parent}(v)]$.

С детьми проблем возникнуть не может: раньше было $a[v] \leq a[\text{child}]$, а теперь $a[v]$ стало только меньше, значит неравенство с детьми сохраняется.

Поэтому достаточно восстановить требование кучи на пути вверх, что и делает $\text{siftUp}(v)$. ■

Асимптотика. decreaseKey работает за $O(\log n)$.

18.5 Сводка: все операции через $\text{siftUp}/\text{siftDown}$

Операция	Реализация через $\text{siftUp}/\text{siftDown}$	Время
getMin	вернуть $a[1]$	$O(1)$
$\text{insert}(x)$	добавить в конец и siftUp	$O(\log n)$
extractMin	заменить корень последним и siftDown	$O(\log n)$
decreaseKey	уменьшить ключ и siftUp	$O(\log n)$

Замечание. Если вам нужна операция $\text{changeKey}(v, \text{newKey})$ (изменение ключа в обе стороны), то достаточно сделать:

▷ если $\text{newKey} < a[v]$, то $\text{siftUp}(v)$;

▷ если $\text{newKey} > a[v]$, то $\text{siftDown}(v)$.

Это тот же принцип: нарушение возможно только “вверх” или только “вниз”.

19 Билет №19. Построение кучи (heapify) за линейное время

19.1 Постановка задачи

Дан массив $A[1 \dots n]$ (для определенности используем **1-индексацию**). Рассмотрим его как **полное бинарное дерево**: вершине с номером v соответствует $A[v]$, левый сын — $2v$, правый — $2v + 1$ (если эти индексы не превосходят n).

Определение 19.1. Массив A задает **минимальную двоичную кучу (min-heap)**, если для каждой вершины v , имеющей детей, выполнено *требование кучи*:

$$A[v] \leq A[2v], \quad A[v] \leq A[2v + 1].$$

Нужно за конечное время переставить элементы массива так, чтобы получилось корректное дерево-куча.

19.2 Наивное решение и почему оно хуже

Можно начать с пустой кучи и по одному вставлять элементы массива операцией `insert`, каждый раз восстанавливая свойство кучи с помощью `siftUp`. Так как вставка стоит $O(\log n)$, суммарно получаем $O(n \log n)$.

19.3 Алгоритм heapify (Флойда): “просеивание вниз” снизу вверх

Ключевая идея: листья уже являются корректными кучами (у них нет детей), а если оба поддерева вершины уже корректные кучи, то достаточно один раз “протащить” элемент вершины вниз.

Определение 19.2. `siftDown(v)` (просеивание вниз) — процедура, которая, пока в вершине v нарушено требование кучи, меняет местами $A[v]$ с меньшим из детей (левым или правым) и продолжает работу из той вершины, куда спустился элемент.

Алгоритм построения кучи (BuildHeap):

1. Представим исходный массив как полное бинарное дерево.
2. Для всех v от $\lfloor n/2 \rfloor$ до 1 (в порядке убывания) выполнить `siftDown(v)`.

Псевдокод (идейно, близко к CLRS)

BuildHeap(A, n): // построение min-кучи (1-индексация)

1. **for** $v = \lfloor \frac{n}{2} \rfloor$ **downto** 1 **do**
 (a) **SIFTDOWN**(A, v, n)

SiftDown(A, v, n): // просеивание вниз в min-heap

1. **while** $2v \leq n$ **do** // пока есть левый сын

```

(a)  $u \leftarrow 2v$  // левый сын
(b) if  $u + 1 \leq n$  and  $A[u + 1] < A[u]$  then  $u \leftarrow u + 1$  // правый сын меньше
(c) if  $A[v] \leq A[u]$  then break // свойство кучи выполнено
(d)  $\text{SWAP}(A[v], A[u])$ 
(e)  $v \leftarrow u$ 

```

Замечание: вершины с номерами $\lfloor n/2 \rfloor + 1, \dots, n$ — листья, поэтому начинать цикл правее бессмысленно.

19.4 Доказательство корректности

Утверждение 19.1. После выполнения *BuildHeap* массив A удовлетворяет требованию *min-кучи*.

Δ *Доказательство.* Докажем более сильное утверждение индукцией по v при проходе цикла сверху: после обработки вершины v (то есть после выполнения $\text{siftDown}(v)$) поддерево, корнем которого является v , является корректной *min-кучей*.

База. Если v — лист ($2v > n$), то в поддереве одна вершина, требование кучи выполнено тривиально.

Переход. Пусть утверждение верно для всех вершин с номерами строго больше v . Тогда поддеревья детей вершины v (это вершины $2v$ и, возможно, $2v + 1$) уже корректные кучи, поскольку их индексы больше v .

Остается “починить” возможное нарушение только в вершине v . Процедура $\text{siftDown}(v)$ делает именно это: пока $A[v]$ больше меньшего из детей, она меняет их местами, тем самым продвигая слишком большой элемент вниз. Важно, что при каждом обмене мы меняем местами элемент вершины и корень одного из уже корректных поддеревьев. После обмена:

- ▷ элемент, который поднялся вверх, не больше обоих своих детей (он был меньшим из детей), значит локальное требование кучи в этой вершине выполнено;
- ▷ “большой” элемент спускается в корень выбранного поддерева; если там возникло нарушение, siftDown продолжает работу именно в этой вершине.

Процесс заканчивается либо на листе, либо когда локальное требование кучи перестает нарушаться. Следовательно, после завершения $\text{siftDown}(v)$ в вершине v и на всём пути спуска требование кучи выполнено, а остальные вершины поддерева не портились (их поддеревья оставались кучами по предположению индукции). Значит, всё поддерево v стало корректной кучей.

В частности, после последнего шага цикла ($v = 1$) корректной кучей является поддерево корня, то есть вся структура. ■

19.5 Асимптотика *heapify*: почему это $O(n)$

Утверждение 19.2. Алгоритм *BuildHeap* работает за $O(n)$.

$\triangle$ *Доказательство.* Оценим суммарную стоимость всех вызовов `siftDown`. Время одного `siftDown(v)` пропорционально *высоте* вершины v , то есть числу ребер на максимальном пути от v до листа: за один шаг мы спускаемся максимум на один уровень.

В полном бинарном дереве (в представлении кучи):

- ▷ около $\frac{n}{2}$ вершин — листья, их высота 0 (или “стоимость” $O(1)$);
- ▷ около $\frac{n}{4}$ вершин имеют высоту 1 (могут спуститься не более чем на 1 уровень);
- ▷ около $\frac{n}{8}$ вершин имеют высоту 2;
- ▷ и т.д.

Точнее, число вершин высоты *ровно* k не превосходит $\frac{n}{2^{k+1}}$, поэтому достаточно грубой оценки

$$T(n) \leq \sum_{k \geq 0} \frac{n}{2^{k+1}} \cdot O(k).$$

Следовательно,

$$T(n) = O\left(n \sum_{k \geq 1} \frac{k}{2^k}\right).$$

Ряд $\sum_{k \geq 1} \frac{k}{2^k}$ сходится (можно пользоваться как фактом), а значит его сумма — константа. Следовательно, $T(n) = O(n)$. ■

19.6 Практические замечания

- ▷ Для **0-индексации** (массив $A[0 \dots n-1]$): дети вершины v — $2v+1$ и $2v+2$, а последний внутренний узел имеет индекс $\lfloor \frac{n-2}{2} \rfloor$. Поэтому цикл пишут так: `for (v = (n-2)/2; v >= 0; --v) siftDown(v)`.
- ▷ Аналогично строится **max-heap** (меняются знаки сравнений).

20 Билет №20. Сортировка кучей (Heap Sort) и невозможность $O(1)$ для insert/extractMin в куче на сравнениях

20.1 Heap Sort: сортировка кучей за $O(1)$ дополнительной памяти

Задача. Дан массив $A[1 \dots n]$ элементов, для которых определено сравнение “ $<$ ” (линейный порядок). Нужно отсортировать массив по неубыванию, используя только сравнения и $O(1)$ дополнительной памяти.

Определение 20.1 (Бинарная тах-куча). Пусть в массиве $A[1 \dots m]$ (где m — текущий размер кучи) задано полное бинарное дерево, где вершине с индексом v соответствуют дети $2v$ и $2v + 1$ (если эти индексы $\leq m$). Массив (дерево) называется *тах-кучей*, если для любого v выполняется

$$A[v] \geq A[2v] \quad \text{и} \quad A[v] \geq A[2v + 1] \quad (\text{если } 2v + 1 \leq m).$$

Утверждение 20.1. В тах-куче корень $A[1]$ является максимальным элементом среди $A[1 \dots m]$.

$\triangle$ *Доказательство.* По определению кучи значение в вершине не меньше значений в детях. Тогда по любому пути от корня вниз значения не возрастают. Значит, корень не меньше любого элемента в дереве, то есть является максимумом. ■

20.1.1 Вспомогательная процедура siftDown

Определение 20.2. $\text{siftDown}(v, m)$ (просеивание вниз) — процедура восстановления свойства тах-кучи, когда возможно нарушено неравенство только в вершине v , а оба поддерева её детей уже являются тах-кучами. Она спускает элемент из v вниз, меняя его местами с большим из детей, пока свойство кучи не восстановится.

Псевдокод (1-индексация):

Псевдокод (идейно, близко к CLRS)

SiftDown(v, m): // восстановление тах-кучи на $A[1..m]$ (1-индексация)

1. **while** $2v \leq m$ **do**

(a) $u \leftarrow 2v$ // левый сын

(b) **if** $u + 1 \leq m$ **and** $A[u + 1] > A[u]$ **then** $u \leftarrow u + 1$ // индекс большего сына

(c) **if** $A[v] \geq A[u]$ **then break** // свойство кучи уже выполнено

(d) **SWAP**($A[v], A[u]$)

(e) $v \leftarrow u$

20.1.2 Построение кучи `heapify` и идея `Heap Sort`

`Heap Sort` сортирует массив *in-place* (в той же памяти), интерпретируя исходный массив как кучу.

1. **Heapify**: построить *max-кучу* на $A[1 \dots n]$.
2. **Сортировка**: многократно извлекать максимум из кучи, помещая его в конец массива.

Псевдокод `heapify` (bottom-up):

Псевдокод (идейно, близко к CLRS)

```

Heapify( $n$ ):                                     // построение max-кучи на  $A[1..n]$ 
1. for  $v = \lfloor \frac{n}{2} \rfloor$  downto 1 do
    (a) SIFTDOWN( $v, n$ )
  
```

Псевдокод `Heap Sort`:

Псевдокод (идейно, близко к CLRS)

```

HeapSort( $n$ ):                                     // сортировка массива  $A[1..n]$  по неубыванию
1. HEAPIFY( $n$ )
2. for  $end = n$  downto 2 do
    (a) SWAP( $A[1], A[end]$ )                        // максимум уходит на своё место
    (b) SIFTDOWN(1,  $end - 1$ )                     // восстанавливаем кучу на префиксе
  
```

20.1.3 Корректность `Heap Sort`

Лемма 20.1. После выполнения *heapify* массив $A[1 \dots n]$ является *max-кучей*.

$\triangle$ *Доказательство.* Идём по вершинам $v = \lfloor n/2 \rfloor, \dots, 1$ (снизу вверх). Для листьев свойство кучи тривиально. Пусть мы уже обработали все вершины с индексом $> v$, тогда поддеревья детей $2v$ и $2v + 1$ уже являются *max-кучами*. Вызов `siftDown(v, n)` восстанавливает свойство кучи в вершине v , не нарушая его в детях (они остаются кучами, меняется только путь, по которому «просеивается» элемент). Значит, после обработки v поддерево вершины v — *max-куча*. В конце (при $v = 1$) *max-кучей* становится всё дерево. ■

Теорема 20.1 (Корректность `Heap Sort`). После выполнения `Heap Sort` массив A отсортирован по неубыванию.

$\triangle$ *Доказательство.* По лемме 20.1 после `heapify` префикс $A[1 \dots n]$ — *max-куча*. Рассмотрим цикл по `end` и докажем инвариант по убыванию `end`:

- ▷ (I1) префикс $A[1 \dots end]$ является *max-кучей*;

- ▷ (I2) суффикс $A[end + 1 \dots n]$ отсортирован по неубыванию и содержит $n - end$ наибольших элементов исходного массива.

База: перед первой итерацией $end = n$. Суффикс пуст, (I2) верно тривиально, (I1) верно по лемме 20.1.

Переход: пусть инвариант верен для текущего end ($end \geq 2$). По свойству *тах-кучи* корень $A[1]$ — максимум среди $A[1 \dots end]$. Команда `swap(A[1], A[end])` помещает этот максимум в позицию end . Так как в суффиксе уже лежат $n - end$ наибольших элементов и они отсортированы, новый элемент $A[end]$ становится следующим по величине и корректно расширяет отсортированный суффикс: получаем (I2) для $end - 1$.

После свапа свойство кучи может быть нарушено только в корне (поддерева детей по-прежнему кучи), поэтому `siftDown(1, end-1)` восстанавливает *тах-кучу* на $A[1 \dots end - 1]$, то есть (I1) для $end - 1$. Переход доказан.

Когда цикл заканчивается, имеем $end = 1$, а значит суффикс $A[2 \dots n]$ отсортирован и содержит $n - 1$ наибольших элементов, а оставшийся элемент $A[1]$ не больше $A[2]$. Следовательно, весь массив отсортирован. ■

20.1.4 Асимптотика и память

Утверждение 20.2. *Heap Sort работает за $O(n \log n)$ по времени в худшем случае, использует $O(1)$ дополнительной памяти и является нестабильной сортировкой.*

△ **Доказательство. Память.** Алгоритм хранит кучу прямо в массиве A и использует лишь константное число переменных (индексы, размер текущей кучи), значит доп. память $O(1)$.

Время.

- ▷ Построение кучи `heapify` работает за $O(n)$: вершины на высоте h (снизу) просеиваются вниз не более чем на h уровней, а таких вершин $\leq n/2^{h+1}$. Поэтому

$$\sum_{h \geq 0} \frac{n}{2^{h+1}} \cdot O(h) = O(n) \cdot \sum_{h \geq 0} \frac{h}{2^h} = O(n).$$

- ▷ Основной цикл выполняется $n - 1$ раз, каждый `siftDown` спускается не более чем на высоту кучи, то есть $O(\log n)$. Итого $O(n \log n)$.

Нестабильность. На каждой итерации выполняются свапы далёких элементов, поэтому относительный порядок равных ключей может измениться. ■

Замечание. Для сортировки по неубыванию обычно используют *тах-кучу* (в конец ставится максимум). Если построить *мін-кучу* и каждый раз уносить минимум в конец, получится сортировка по невозрастанию.

20.2 Несуществование кучи на сравнениях с `insert` и `extractMin` за $O(1)$

Определение 20.3. *Очередь с приоритетом (куча) поддерживает операции: `insert(x)` (вставить ключ x) и `extractMin()` (вернуть и удалить минимальный ключ). Говорят, что структура основана на сравнениях, если вся информация о порядке ключей получается только из операций сравнения ключей между собой.*

Теорема 20.2. *В модели сравнений не существует очереди с приоритетом, которая обрабатывает `insert` и `extractMin` за $O(1)$ (константу, не зависящую от n).*

Δ *Доказательство.* Предположим противное: существует структура данных H , которая в худшем случае выполняет `insert` и `extractMin` за $O(1)$ сравнений (а значит и за $O(1)$ времени в RAM-модели).

Тогда можно отсортировать произвольный массив из n попарно различных элементов так:

1. создать пустую кучу H ;
2. для каждого $A[i]$ выполнить `insert(A[i])`;
3. n раз выполнить `extractMin()`, записывая извлечённые элементы по порядку в новый массив.

Полученный массив будет отсортирован по неубыванию, поскольку каждый раз извлекается текущий минимум.

Число операций: n вставок и n извлечений, суммарно $O(n)$, и по предположению на каждую уходит $O(1)$ сравнений. Значит, сортировка работает за $O(n)$ сравнений.

Но любая сортировка сравнениями требует $\Omega(n \log n)$ сравнений в худшем случае (решающее дерево имеет как минимум $n!$ листьев, а каждое сравнение даёт не более двух исходов, поэтому глубина $\geq \log_2(n!) = \Omega(n \log n)$). Противоречие.

Следовательно, такой кучи H не существует. ■

Замечание. Из доказательства следует более общая мысль: в куче на сравнениях нельзя «ускорить всё сразу» — как минимум одна из операций `insert` или `extractMin` должна иметь стоимость $\Omega(\log n)$ (в худшем случае или амортизированно). Например, бинарная куча даёт $O(\log n)$ для обеих операций, а фибоначчиева куча — $O(1)$ амортизированно для `insert` и $O(\log n)$ амортизированно для `extractMin`.

21 Билет №21. Удаление из кучи по указателю

21.1 Постановка задачи

Пусть дана **бинарная min-куча** (далее просто *куча*) на n элементах, представленная массивом $A[1 \dots n]$ (нумерация с 1), где для каждой вершины v выполнено *требование кучи*:

$$A[v] \leq A[2v], \quad A[v] \leq A[2v + 1]$$

для существующих детей (то есть при $2v \leq n$ и/или $2v + 1 \leq n$).

Определение 21.1. Удаление из кучи по указателю (по хэндлу). В качестве *указателя* на элемент будем считать любой *хэндл*, позволяющий за $O(1)$ получить его текущую позицию v в массиве кучи (в простейшем виде это просто индекс v). Операция $delete(v)$ должна удалить элемент, находящийся в вершине v , и вернуть корректную кучу на оставшихся $n - 1$ элементах.

21.2 Напоминание: базовые процедуры бинарной кучи

В дальнейшем будем пользоваться двумя стандартными «просеиваниями».

Определение 21.2. $siftUp(v)$ (просеивание вверх) повторяет обмен $A[v]$ с родителем $A[\lfloor v/2 \rfloor]$ пока $v > 1$ и $A[v] < A[\lfloor v/2 \rfloor]$.
 $siftDown(v)$ (просеивание вниз) повторяет обмен $A[v]$ с меньшим из детей, пока существует ребенок с меньшим значением (то есть пока нарушено требование кучи в v).

Эти процедуры исправляют локальное нарушение требования кучи, возникающее после изменения одного ключа. Из предыдущих билетов считаем известным:

Утверждение 21.1. Если массив $A[1 \dots n]$ удовлетворяет требованию кучи всюду, кроме, возможно, вершины v , то после выполнения $siftUp(v)$ или $siftDown(v)$ (в зависимости от ситуации) требование кучи восстанавливается для всего массива.

Также считаем известными реализации:

- ▷ **decreaseKey**(v, Δ): уменьшить $A[v]$ на $\Delta > 0$ и вызвать $siftUp(v)$;
- ▷ **extractMin**: удалить корень $A[1]$ (минимум), заменив его последним элементом $A[n]$, уменьшив $n \leftarrow n - 1$ и вызвав $siftDown(1)$.

21.3 Способ 1: decreaseKey до $-\infty$ + extractMin

Идея из лекции: если у нас есть доступ к конкретной вершине v , то мы можем сделать её ключ «самым маленьким», поднять в корень и удалить как минимум.

Задача. Реализовать $delete(v)$ через уже имеющиеся операции **decreaseKey** и **extractMin**.

Решение. Алгоритм:

1. Выполнить **decreaseKey**(v, Δ) с таким Δ , чтобы новый ключ стал меньше любого допустимого ключа. Концептуально пишут: присвоить $A[v] \leftarrow -\infty$ и запустить **siftUp**(v).
2. Выполнить **extractMin**.

Утверждение 21.2. *Описанный алгоритм корректно удаляет элемент, находившийся в вершине v .*

Δ *Доказательство.* После шага 1 значение в вершине v становится строго меньше всех остальных ключей кучи. По определению **siftUp** элемент поднимается, пока не станет корнем или пока не перестанет быть меньше родителя. Но «перестать быть меньше родителя» он не может, ведь он меньше любого элемента, в том числе родителя. Значит, после **siftUp** он окажется в корне $A[1]$.

Далее шаг 2 (**extractMin**) удаляет корень, то есть именно этот элемент. Корректность **extractMin** гарантирует восстановление требования кучи для оставшихся элементов. ■

Утверждение 21.3. *Асимптотика $\text{delete}(v)$ в способе 1 равна $O(\log n)$.*

Δ *Доказательство.* **decreaseKey** делает один **siftUp**, который поднимается по высоте полного двоичного дерева, то есть выполняет $O(\log n)$ обменов. **extractMin** делает один **siftDown** также за $O(\log n)$. Сумма двух логарифмов остаётся $O(\log n)$. ■

Замечание. На практике способ 1 удобен в доказательствах и в «математической» модели. Если ключи — не числа, а произвольные объекты с компаратором, то « $-\infty$ » реализуют либо отдельным флагом, либо используют способ 2 (ниже), не требующий искусственного минимального значения.

21.4 Способ 2: «поменять с последним и просеять один раз»

Способ 1 использует две стандартные операции и потому делает два просеивания. Можно удалить за одно просеивание.

Решение. Алгоритм **delete**(v):

1. Поменять местами $A[v]$ и $A[n]$.
2. Уменьшить размер: удалить последний элемент (теперь это удаляемый) и сделать $n \leftarrow n - 1$.
3. Если $v > n$, то всё (мы удаляли последний элемент). Иначе восстановить требование кучи в позиции v :
 - ▷ если $v > 1$ и $A[v] < A[\lfloor v/2 \rfloor]$, выполнить **siftUp**(v);
 - ▷ иначе выполнить **siftDown**(v).

Утверждение 21.4. *Способ 2 корректен и работает за $O(\log n)$.*

△ *Доказательство.* После перестановки с последним элементом изменились значения только в вершине v и в листе n . Лист n затем удалён, поэтому потенциальное нарушение требования кучи остаётся только «вокруг» вершины v : она может стать меньше родителя (тогда нужно поднимать вверх), либо больше одного из детей (тогда нужно опускать вниз). В обоих случаях одно соответствующее просеивание восстанавливает требование кучи во всём дереве.

Каждое просеивание проходит не более высоты кучи, то есть $O(\log n)$ обменов. Остальные шаги — константные. ■

21.5 Замечания про «указатель» в реализации

Если в качестве указателя используется *индекс* v , то нужно помнить: во время **swap** (как в **siftUp/siftDown**, так и в **delete**) элементы меняются местами. Значит, внешний код не может хранить «вечный» индекс элемента.

Типичное решение — храним *хэндл* элемента (например, его уникальный `id`) и поддерживаем отображение `pos[id]` — текущая позиция этого элемента в массиве кучи. Тогда при каждом обмене двух позиций i и j мы делаем:

```
swap(A[i], A[j]); swap(pos[id(A[i])], pos[id(A[j])]);
```

и любой «указатель» (`id`) остаётся валиден.

Замечание. Именно наличие такого хэндла принципиально отличает билет 21 от билета 22: «по значению» удалить трудно, потому что куча не поддерживает быстрый поиск по ключу.

22 Билет №22. Удаление из кучи по значению

Задача. Дана куча (минимальная приоритетная очередь), поддерживающая операции *insert*, *getMin*, *extractMin* (*u*, возможно, *decreaseKey* по указателю). Требуется добавить операцию *erase(x)*: удалить один экземпляр элемента со значением (ключом) *x*, когда нам дано именно значение, а не указатель/позиция элемента в структуре.

Почему “в лоб” плохо

Куча хранит элементы так, что каждый узел не превосходит своих детей (для *min-heap*). Этого достаточно, чтобы быстро получать минимум, но недостаточно для поиска произвольного значения:

- ▷ если $x < \min$, то его точно нет;
- ▷ если $x > \min$, то он может быть где угодно.

В худшем случае, чтобы найти элемент со значением *x*, приходится просмотреть все *n* элементов, т.е. поиск занимает $\Theta(n)$. После этого удалить элемент уже можно (например, “удаление по указателю”), но доминирует именно поиск.

Идея: ленивое удаление (две кучи)

Будем хранить две кучи:

- ▷ *A* (active) — все элементы, которые когда-либо были *insert*-нуты;
- ▷ *D* (deleted) — все элементы, которые нам приказали удалить, но мы *пока* не искали их в *A*, а просто “пометили на удаление”.

Определение 22.1 (Нормализация). Определим процедуру *normalize()*:

пока $A \neq \emptyset$, $D \neq \emptyset$ и $\min(A) = \min(D)$: *extractMin(A)*, *extractMin(D)*.

Замечание. Ключевое предположение (как в типичных задачах): запрос *erase(x)* корректен, то есть в текущем *активном* мультимножестве действительно есть хотя бы один элемент со значением *x*. Иначе *D* могло бы содержать элемент, которого нет в *A*, и тогда минимумы *A* и *D* могли бы “разойтись” так, что мы никогда не почистим пометки.

Как обрабатывать операции

Пусть мы хотим поддерживать мультимножество (дубликаты допускаются, *erase(x)* удаляет один экземпляр).

- ▷ *insert(x)*: выполнить *A.insert(x)*.
- ▷ *erase(x)*: выполнить *D.insert(x)* (пометили один экземпляр *x* к удалению).
- ▷ *getMin()*: выполнить *normalize()*, затем вернуть *A.getMin()*.

▷ `extractMin()`: выполнить `normalize()`, затем вернуть `A.extractMin()`.

Интуиция: если минимальный элемент в A помечен к удалению, то в D он тоже обязан присутствовать и, будучи наименьшим среди помеченных, окажется в корне D . Тогда $\min(A) = \min(D)$, и мы можем физически удалить этот элемент из обеих куч. Повторяем, пока минимум A не станет “живым”.

Доказательство корректности

Обозначим через S текущее *активное* мультимножество элементов, которое мы хотим моделировать (после всех `insert` и `erase`). Формально удобно думать, что

$$S = A \ominus D,$$

где $\ominus$ — разность мультимножеств (вычитание кратностей).

Лемма 22.1. После вызова `normalize()` верно: либо $D = \emptyset$, либо $\min(D) > \min(A)$.

△ *Доказательство.* Если $\min(A) = \min(D)$, то тело цикла `normalize()` выполнится и удалит оба минимума. Цикл заканчивается, только когда равенство перестало быть верным или одна из куч опустела. Значит, после выхода либо $D = \emptyset$, либо $\min(A) \neq \min(D)$.

Осталось исключить случай $\min(D) < \min(A)$. Пусть $\min(D) = y$. Элемент y попал в D только из запроса `erase(y)`, а по предположению корректности запросов это значит, что в момент вызова `erase(y)` элемент y существовал среди активных, то есть (тем более) существовал в A . Следовательно, в A есть хотя бы один экземпляр y , а значит $\min(A) \leq y$. Противоречие с $y < \min(A)$. Значит, $\min(D) < \min(A)$ невозможен. ■

Теорема 22.1 (Корректность `getMin/extractMin`). После `normalize()` (если активные элементы существуют) значение $\min(A)$ равно минимуму активного мультимножества S . Следовательно, описанные реализации `getMin` и `extractMin` корректны.

△ *Доказательство.* Пусть после `normalize()` куча A не пуста и $m = \min(A)$. По лемме 22.1 либо $D = \emptyset$, либо $\min(D) > m$. В обоих случаях в D нет элемента со значением m , т.е. один экземпляр m не “вычтен” и остаётся активным: $m \in S$.

С другой стороны, любой активный элемент $s \in S$ обязательно содержится в A . Так как m — минимум в A , то $m \leq s$ для всех $s \in S$. Значит, m — минимум активного множества S .

Операция `extractMin` после `normalize()` физически удаляет этот минимум из A , т.е. корректно удаляет один минимальный активный элемент. ■

Асимптотика и амортизационный анализ

Пусть всего выполнено q запросов.

- ▷ `insert` и `erase` делают ровно одну вставку в кучу: $\mathcal{O}(\log n)$, где n — текущий размер соответствующей кучи.
- ▷ `normalize` может в одном вызове выполнить много итераций цикла и сделать много `extractMin` из A и D .

Утверждение 22.1 (Амортизированная оценка). Суммарное число итераций цикла в `normalize` за всю последовательность запросов равно $\mathcal{O}(q)$. Следовательно, суммарное время работы всех операций равно $\mathcal{O}(q \log q)$, а амортизированное время одной операции — $\mathcal{O}(\log q)$.

△ *Доказательство.* Каждая итерация `normalize` удаляет один элемент из A и один элемент из D . Любой конкретный экземпляр элемента:

- ▷ добавляется в A не более одного раза (в момент `insert`);
- ▷ добавляется в D не более одного раза (в момент `erase`, если его вообще удаляют);
- ▷ извлекается из A и/или D не более одного раза, потому что `extractMin` физически удаляет элемент.

Значит, всего итераций `normalize` (каждая уничтожает пару элементов) не больше, чем число вставок в D , то есть $\mathcal{O}(q)$. Каждая итерация стоит $\mathcal{O}(\log q)$ как две операции `extractMin`. Отсюда суммарно $\mathcal{O}(q \log q)$ и амортизированно $\mathcal{O}(\log q)$. ■

Замечание. Худшее время одного вызова `getMin` или `extractMin` может быть большим: например, после серии из k удалений минимумов он может выполнить $\Theta(k)$ итераций `normalize`, то есть $\Theta(k \log n)$. Но в среднем по последовательности запросов время остаётся логарифмическим.

Псевдокод / реализация

Ниже типичная реализация (`min-heap`). Дубликаты поддерживаются автоматически.

Псевдокод (идейно, близко к CLRS)

Будем хранить две `min`-кучи: A (active) и D (deleted). Предполагается, что запросы `Erase(x)` корректны (удаляемый экземпляр x существует среди активных).

Normalize():

1. **while** $A \neq \emptyset$ **and** $D \neq \emptyset$ **and** $\min(A) = \min(D)$ **do**
 - (a) `EXTRACTMIN(A)`
 - (b) `EXTRACTMIN(D)`

Insert(x): `INSERT(A, x)`.

Erase(x): `INSERT(D, x)` (помечаем один экземпляр x к удалению).

GetMin():

1. `NORMALIZE()`
2. **return** $\min(A)$

ExtractMin():

1. `NORMALIZE()`
2. **return** `EXTRACTMIN(A)`

Замечание. Эта техника часто называется *lazy deletion* (ленивое удаление): мы не удаляем элемент немедленно из основной кучи A , а откладываем удаление до момента, когда этот элемент “всплывёт” в минимум.

23 Билет №23. Биномиальное дерево и биномиальная куча. Преимущество перед двоичной кучей

1. Биномиальные деревья

Определение 23.1 (Биномиальное дерево B_k (ранга/порядка k)). Биномиальные деревья определяются рекурсивно:

- ▷ B_0 состоит из одной вершины.
- ▷ Для $k \geq 1$ дерево B_k получается сцеплением двух копий B_{k-1} : корень одного из них делается *левым* (или, в зависимости от соглашения, *правым*) ребёнком корня другого.

Эквивалентная (часто более удобная) формулировка структуры корня:

Утверждение 23.1 (Структура детей корня). Для $k \geq 1$ корень дерева B_k имеет ровно k детей. Если нумеровать детей корня слева направо, то i -й ребёнок (нумерация с 0) является корнем биномиального дерева B_{k-1-i} . Иными словами, поддеревья детей корня имеют ранги $k-1, k-2, \dots, 0$.

△ *Доказательство.* Докажем по индукции по k .

База $k = 1$: дерево B_1 состоит из двух вершин (корень и один ребёнок), т.е. у корня ровно один ребёнок, который является B_0 .

Переход: пусть утверждение верно для B_{k-1} . Построим B_k сцеплением двух копий B_{k-1} : возьмём деревья T и T' (оба B_{k-1}) и сделаем корень T' левым ребёнком корня T . Тогда у корня T появляется новый левый ребёнок ранга $k-1$ (это T'), а остальные дети корня T остаются детьми корня B_{k-1} , то есть имеют ранги $k-2, k-3, \dots, 0$ по предположению индукции. Получаем ровно k детей с рангами $k-1, k-2, \dots, 0$. ■

Лемма 23.1 (Ключевые свойства B_k). Для биномиального дерева B_k верно:

1. число вершин равно $|B_k| = 2^k$;
2. высота (длина максимального пути от корня до листа) равна k ;
3. степень (число детей) корня равна k ;
4. (усиление) число вершин на глубине i равно $\binom{k}{i}$ для $i = 0, 1, \dots, k$.

△ *Доказательство.* (1) По индукции по k . Для $k = 0$ имеем одну вершину: $|B_0| = 1 = 2^0$. Переход: B_k — сцепление двух копий B_{k-1} , значит $|B_k| = 2 \cdot |B_{k-1}| = 2 \cdot 2^{k-1} = 2^k$.

(2) По индукции. Для $k = 0$ высота 0. Переход: при сцеплении двух B_{k-1} добавляется новый корень, а максимальный путь идёт в самое глубокое поддерево ранга $k-1$, поэтому высота увеличивается на 1: получаем k .

(3) Следует из утверждения о структуре детей корня выше (у корня ранги $k-1, \dots, 0$, значит детей k).

(4) Также доказывается по индукции или через биномиальную формулу: B_k — объединение двух B_{k-1} , уровни суммируются по правилу Паскаля, поэтому на глубине i получа-

ется $\binom{k}{i}$. ■

2. Биномиальная куча

Определение 23.2 (Свойство кучи (min-heap order)). Пусть в вершинах дерева хранятся ключи (числа). Дерево удовлетворяет *свойству кучи*, если для любой вершины ключ в ней не превосходит ключей во всех её детях (и, следовательно, во всём её поддереве).

Определение 23.3 (Биномиальная куча). *Биномиальная куча* H — это набор (лес) биномиальных деревьев, удовлетворяющий:

1. каждое дерево в составе H — биномиальное дерево B_k для некоторого k и удовлетворяет свойству кучи;
2. в H нет двух деревьев одного и того же ранга (степени/порядка): для каждого k в куче присутствует не более одного дерева B_k .

Представление в памяти. Обычно хранят список (или массив) корней, упорядоченный по рангу:

$$\text{roots}[k] = \begin{cases} \text{указатель на корень дерева ранга } k, & \text{если оно есть;} \\ \text{nullptr}, & \text{если дерева ранга } k \text{ нет.} \end{cases}$$

Из леммы 23.1 следует, что дерево ранга k содержит 2^k вершин, поэтому ранги деревьев, присутствующих в куче, однозначно задаются двоичной записью $n = |H|$.

Утверждение 23.2 (Связь с двоичной записью размера). Пусть биномиальная куча содержит n элементов. Тогда:

1. число деревьев в куче не превосходит $\lfloor \log_2 n \rfloor + 1$;
2. множество рангов деревьев в куче совпадает с позициями единиц в двоичной записи n .

Δ *Доказательство.* Каждое дерево ранга k имеет размер ровно 2^k (лемма 23.1). Поскольку ранги различны, то размеры деревьев — различные степени двойки. Сумма различных степеней двойки разлагает n ровно так же, как двоичная запись числа. Отсюда (2). Из (2) следует (1), так как в двоичной записи числа длины $\lfloor \log_2 n \rfloor + 1$ может быть не более $\lfloor \log_2 n \rfloor + 1$ единиц. ■

3. Преимущество перед двоичной кучей

Ключевое преимущество биномиальной кучи — эффективная операция *слияния* (merge/union/meld) двух куч.

3.1. Почему binomial heap хорошо “сливается”

Идея: у биномиальной кучи корни деревьев отсортированы по рангу, а ранги уникальны. Слияние двух куч сводится к *слиянию двух отсортированных списков корней* и последующему “разрешению переносов”, полностью аналогичному сложению двоичных чисел:

- ▷ Если в результате объединения получились два дерева одного ранга k , их можно *сцепить* (link) в одно дерево ранга $k + 1$.
- ▷ Такой “перенос” может продолжиться дальше (как перенос бита при сложении), но на каждом ранге перенос происходит не более константного числа раз.

Утверждение 23.3 (Асимптотика числа деревьев). Пусть H_1 и H_2 содержат n_1 и n_2 элементов. Тогда число деревьев в каждой куче равно $O(\log n_1)$ и $O(\log n_2)$ соответственно, а в объединении — $O(\log(n_1 + n_2))$.

△ Доказательство. Следует из предложения 23.2. ■

Утверждение 23.1 (Следствие: слияние за логарифм). Слияние биномиальных куч занимает $O(\log(n_1 + n_2))$ по числу сцеплений деревьев.

△ Доказательство. Список корней имеет длину $O(\log n)$, значит, линейный проход по спискам корней (как при merge двух отсортированных массивов) занимает $O(\log(n_1 + n_2))$. Каждое сцепление двух деревьев одинакового ранга увеличивает ранг на 1, а максимальный возможный ранг в куче с $n_1 + n_2$ элементами равен $\lfloor \log_2(n_1 + n_2) \rfloor$. Значит, “переносы” могут пройти лишь по $O(\log(n_1 + n_2))$ позициям. ■

3.2. Почему binary heap сливать неудобно

Двоичная куча хранится в виде *одного* полного бинарного дерева (или массива), и при объединении двух куч нужно заново построить корректную кучу на объединённом наборе элементов. Стандартный способ: просто склеить массивы и сделать `heapify` за линейное время $O(n_1 + n_2)$ (либо вставлять элементы по одному за $O((n_1 + n_2) \log(n_1 + n_2))$, что ещё хуже). То есть слияние для двоичной кучи существенно дороже логарифма.

3.3. Итоговое сравнение (min-heap)

Ниже — типичные асимптотики (худшие случаи) для основных операций, если n — текущий размер:

Операция	Двоичная куча	Биномиальная куча
<code>getMin</code>	$O(1)$	$O(\log n)$ (или $O(1)$ при хранении указателя на мин. корень)
<code>insert</code>	$O(\log n)$	$O(\log n)$ (как merge с B_0)
<code>extractMin</code>	$O(\log n)$	$O(\log n)$
<code>decreaseKey</code>	$O(\log n)$	$O(\log n)$
<code>merge</code>	$O(n)$	$O(\log n)$

Отсюда **главное преимущество биномиальной кучи**: она является *сливаемой кучей* (*mergeable heap*) и позволяет эффективно объединять кучи за логарифм, что принципиально важно в ряде алгоритмов (например, в некоторых алгоритмах на графах и в задачах, где множество событий/задач естественно “склеивается”).

24 Билет №24. Операции merge, insert, getMin, extractMin и decreaseKey в биномиальной куче

24.1 Напоминание: что такое биномиальная куча

Определение 24.1 (Биномиальное дерево B_k). Биномиальные деревья определяются индуктивно.

- ▷ B_0 — одноэлементное дерево (одна вершина).
- ▷ B_{k+1} получается из двух деревьев B_k : корень одного из них делаем левым ребёнком корня другого (то есть добавляем один корень в список детей другого).

Ранг (*degree*) вершины — число её детей; ранг биномиального дерева — ранг его корня.

Лемма 24.1 (Структурные свойства B_k). Для биномиального дерева B_k верно:

1. $|B_k| = 2^k$.
2. Корень имеет ранг k .
3. Дети корня (слева направо) являются корнями биномиальных деревьев рангов $k-1, k-2, \dots, 0$.
4. Высота B_k равна k .

△ *Доказательство.* Индукция по k . База $k = 0$ очевидна. Переход: B_{k+1} строится из двух B_k , поэтому число вершин удваивается, т.е. $|B_{k+1}| = 2 \cdot 2^k = 2^{k+1}$. Ранг корня увеличивается на 1, так как к нему добавляется ещё один ребёнок. А список детей получается как «старые» дети рангов $k-1, \dots, 0$ плюс подвешенное второе дерево B_k ранга k . Высота увеличивается на 1 по той же причине. ■

Определение 24.2 (Биномиальная куча). Биномиальная куча H — это набор биномиальных деревьев, удовлетворяющий:

1. (Heap-order) В каждом дереве выполнено требование кучи (min-heap): ключ родителя не превосходит ключей детей.
2. (Уникальность рангов) Для каждого $k \geq 0$ в куче есть не более одного дерева ранга k .

Обычно корни деревьев хранятся в списке (root list), упорядоченном по неубыванию рангов.

Утверждение 24.1 (Сколько деревьев в куче). Если в куче n элементов, то число деревьев (корней) не превосходит $\lfloor \log_2 n \rfloor + 1$.

△ *Доказательство.* Дерево ранга k содержит 2^k вершин, а ранги попарно различны, значит суммарный размер есть сумма различных степеней двойки, откуда число слагаемых

$O(\log n)$. ■

24.2 Базовая подоперация LINK (склейка двух деревьев одного ранга)

Пусть даны два биномиальных дерева одного ранга k с корнями x и y . LINK превращает их в одно дерево ранга $k + 1$.

Определение 24.3 (LINK). Если $key(x) \leq key(y)$, делаем y левым ребёнком x (или добавляем в список детей x), иначе наоборот. Также обновляем указатель родителя у подвешиваемого корня и увеличиваем ранг нового корня на 1.

Утверждение 24.2 (Корректность LINK). *Результат LINK является биномиальным деревом ранга $k + 1$ и удовлетворяет требованию кучи.*

△ *Доказательство.* Структурно мы ровно повторяем индуктивное определение B_{k+1} . Требование кучи сохраняется, потому что корень с меньшим ключом становится родителем другого корня, а внутри поддеревьев порядок не менялся. ■

Утверждение 24.3 (Время работы LINK). LINK выполняется за $O(1)$.

24.3 Операция merge (union): слияние двух биномиальных куч

Задача. По двум корректным биномиальным кучам H_1 и H_2 построить кучу H , содержащую все элементы обоих.

Алгоритм.

1. *Слить списки корней по рангу.* Списки корней H_1 и H_2 уже отсортированы по рангу. Делаем стандартное «слияние» (как в merge-sort) и получаем общий список корней L , отсортированный по неубыванию ранга.
2. *Устранить повторяющиеся ранги.* Проходим по L слева направо. Если два соседних корня имеют один и тот же ранг k , то склеиваем соответствующие деревья операцией LINK и заменяем пару одним деревом ранга $k + 1$.

Здесь важен случай «три подряд одного ранга»: если подряд идут три корня ранга k , то первые два *не* склеиваем сразу, а пропускаем первый. Это полностью аналогично сложению двоичных чисел: наличие третьего дерева означает, что «перенос» уже существует и его надо обработать корректно.

Корректность.

- ▷ После шага 1 список корней отсортирован по рангу и содержит ровно все деревья из $H_1 \cup H_2$ (только в другом порядке).
- ▷ На шаге 2 мы устраняем все конфликтующие пары одинаковых рангов. Операция LINK заменяет два дерева ранга k одним деревом ранга $k + 1$, не теряя ни одного элемента. Условие «три подряд» гарантирует, что после прохода в списке останется не более одного дерева каждого ранга.

- ▷ Требование кучи сохраняется, поскольку единственная структурная модификация — LINK, а она сохраняет heap-order.

Значит, результат — корректная биномиальная куча.

Асимптотика. Число корней в каждой куче $O(\log n)$, поэтому:

- ▷ слияние двух отсортированных root-list работает за $O(\log n)$;
- ▷ проход с возможными LINK также $O(\log n)$ (каждое склеивание уменьшает число корней).

Итого merge работает за $O(\log(n_1 + n_2))$.

24.4 Операция insert

Задача. Вставить новый элемент x в кучу H .

Алгоритм. Создаём одноэлементную кучу H_x , состоящую из дерева B_0 с ключом x , и делаем

$$H \leftarrow \text{merge}(H, H_x).$$

Корректность. H_x — корректная биномиальная куча. Операция merge возвращает корректную кучу, содержащую объединение элементов. Значит, новый элемент действительно добавлен, и структура корректна.

Асимптотика. insert выражается через merge, поэтому время $O(\log n)$.

24.5 Операция getMin

Задача. Найти минимальный ключ среди всех элементов кучи.

Алгоритм. Минимальный элемент кучи обязательно находится в одном из *корней* (из heap-order в каждом дереве). Поэтому достаточно просканировать все корни и выбрать минимальный.

Корректность. Каждое дерево удовлетворяет heap-order, значит его корень — минимум в этом дереве. Глобальный минимум — минимум среди минимумов деревьев, т.е. среди корней.

Асимптотика. Корней $O(\log n)$, значит getMin работает за $O(\log n)$.

Замечание. Часто дополнительно хранят указатель на минимальный корень и поддерживают его при merge/insert/extractMin. Тогда getMin становится $O(1)$.

24.6 Операция extractMin

Задача. Удалить из кучи минимальный элемент и вернуть его.

Алгоритм.

1. Найти корень z с минимальным ключом (как в getMin).
2. Удалить z из root-list (дерево T_z ранга $k = \deg(z)$ исчезает из кучи).

3. Рассмотреть детей z . По лемме о структуре B_k дети z являются корнями деревьев рангов $k-1, k-2, \dots, 0$. Эти деревья образуют биномиальную кучу H_2 (у них попарно разные ранги).

В стандартном представлении (left-child/right-sibling) дети перечислены по убыванию рангов, поэтому список детей нужно *развернуть*, чтобы получить root-list, отсортированный по возрастанию рангов.

4. Пусть H_1 — «остаток» исходной кучи после удаления T_z . Ответом будет

$$H \leftarrow \text{merge}(H_1, H_2).$$

Корректность.

- ▷ После удаления корня z мы удалили ровно минимальный элемент, потому что z был минимумом среди корней.
- ▷ Куча H_1 корректна (мы просто удалили одно дерево, остальные деревья и уникальность рангов сохранились).
- ▷ Куча H_2 корректна по структурному свойству биномиального дерева: дети корня — корни деревьев рангов $0..k-1$, поэтому ранги попарно различны, и heap-order в каждом поддереве сохраняется.
- ▷ `merge` корректно объединяет H_1 и H_2 , значит итоговая куча корректна и содержит все элементы, кроме удалённого минимума.

Асимптотика.

- ▷ поиск минимума среди корней: $O(\log n)$;
- ▷ развернуть список детей: $O(\deg(z)) = O(\log n)$;
- ▷ `merge`: $O(\log n)$.

Итого `extractMin` работает за $O(\log n)$.

24.7 Операция decreaseKey

Задача. Уменьшить ключ в заданной вершине x до нового значения k (гарантируется $k \leq \text{key}(x)$). Подразумевается, что на вход подаётся указатель на вершину x (иначе поиск x внутри кучи может занять $\Theta(n)$).

Алгоритм.

1. Присвоить $\text{key}(x) \leftarrow k$.
2. Пока у x есть родитель p и $\text{key}(x) < \text{key}(p)$:
 - ▷ обменять ключи (или элементы) в x и p ;
 - ▷ перейти $x \leftarrow p$.

Это «просеивание вверх» (аналог `siftUp` в двоичной куче).

Корректность. После уменьшения ключа может нарушиться `heap-order` только на ребре (p, x) , потому что ключ уменьшили и он мог стать меньше родителя. Обмен ключей чинит это нарушение. При обмене вниз перемещается старый ключ родителя. Он был не больше ключа x до обмена, а ключ x был не больше ключей в поддереве x , значит после обмена этот ключ всё равно не превышает ключей детей x . Следовательно, возможное нарушение после обмена может остаться только выше, на новом ребре к родителю. Цикл заканчивается либо в корне, либо когда $key(parent) \leq key(x)$, т.е. `heap-order` восстановлен на всём пути.

Асимптотика. Цикл поднимается по цепочке родителей. Глубина любой вершины в дереве ранга k не превосходит k , а $k \leq \lfloor \log_2 n \rfloor$. Значит, `decreaseKey` работает за $O(\log n)$.

24.8 Итоговые асимптотики (binomial heap)

Операция	Время
<code>merge (union)</code>	$O(\log n)$
<code>insert</code>	$O(\log n)$
<code>getMin</code>	$O(\log n)$ (или $O(1)$ с указателем на минимум)
<code>extractMin</code>	$O(\log n)$
<code>decreaseKey</code>	$O(\log n)$

25 Билет №25. Амортизационный анализ и учётное время работы

25.1 Зачем нужен амортизационный анализ

При анализе структур данных и онлайн-алгоритмов нас часто интересует *серия* операций, а не одна операция. Бывает, что отдельная операция в худшем случае дорогая, но после неё внутреннее состояние структуры меняется так, что долгое время дорогие случаи не повторяются. Тогда оценка “худший случай на одну операцию” может быть слишком пессимистичной, и хочется оценить *суммарное* время на любую последовательность запросов.

Важно: амортизированный анализ — это не “среднее по вероятности”. Мы не предполагаем никаких распределений входа. Амортизированная оценка даёт *гарантию* на среднюю стоимость операции на *любой* последовательности запросов (в худшем случае по последовательности), а не только “в среднем по случайному входу”.

25.2 Формальное определение “учётного времени”

Рассмотрим структуру данных (или алгоритм), на которую поступает последовательность запросов. Пусть $q_1, q_2, \dots, q_n$ — некоторая последовательность из n операций. Обозначим через t_i *реальную* стоимость (время) выполнения i -й операции q_i (в RAM-модели: число элементарных действий).

Определение 25.1 (Учётные (амортизированные) стоимости для фиксированной последовательности). Числа $\hat{t}_1, \hat{t}_2, \dots, \hat{t}_n$ называются *учётными стоимостями* (или *амортизированными стоимостями*) данной последовательности, если для любого префикса длины k выполняется

$$\sum_{i=1}^k t_i \leq \sum_{i=1}^k \hat{t}_i.$$

Тогда, в частности, для всего префикса длины n имеем $\sum_{i=1}^n t_i \leq \sum_{i=1}^n \hat{t}_i$.

Смысл: мы “назначаем цену” каждой операции так, чтобы суммарно (и даже на каждом префиксе) учётная цена покрывала реальную. Тогда средняя учётная цена $\frac{1}{n} \sum \hat{t}_i$ верхне оценивает среднее реальное время $\frac{1}{n} \sum t_i$.

Определение 25.2 (Амортизированная асимптотика по типам операций). Пусть структура поддерживает m типов операций. Пусть каждому типу операции сопоставлена функция $a_j(n)$ — “учётная стоимость” операции j при общей длине последовательности n . Говорят, что операции имеют амортизированные (учётные) асимптотики $a_1(n), \dots, a_m(n)$, если существует константа $C > 0$ такая, что для любого n и любой последовательности $q_1, \dots, q_n$ выполняется

$$\sum_{i=1}^n t_i \leq C \sum_{i=1}^n a_{\text{type}(q_i)}(n).$$

Это формализация фразы “операция j работает за $O(a_j(n))$ амортизированно”: суммарное время на любую серию запросов ограничивается суммой этих учётных стоимостей.

25.3 Три классических способа доказывать амортизированные оценки

Существует три стандартных техники амортизированного анализа: *агрегатный метод*, *метод бухгалтерского учёта* (*banker's / accounting*) и *метод потенциалов*. Они эквивалентны по силе: выбор — вопрос удобства для конкретной задачи.

25.3.1 Агрегатный метод

Мы оцениваем суммарную стоимость $T(n)$ любой последовательности из n операций, а затем объявляем амортизированную стоимость одной операции равной $T(n)/n$. Этот метод обычно самый простой, но иногда даёт слишком грубую оценку.

25.3.2 Метод бухгалтерского учёта (banker's method)

Идея: “дешёвые” операции платят чуть больше своего реального времени и кладут “кредит” в банк. Этот кредит потом оплачивает редкие “дорогие” операции.

Определение 25.3 (Банковский счёт). Пусть каждой операции q_i мы назначаем учётную стоимость $\hat{t}_i$. Определим *баланс* после k операций как

$$B_k = \sum_{i=1}^k \hat{t}_i - \sum_{i=1}^k t_i.$$

Если $B_k \geq 0$ для всех k (банк никогда не уходит в минус), то $\hat{t}_i$ — корректные учётные стоимости, т.е. $\sum_{i=1}^k t_i \leq \sum_{i=1}^k \hat{t}_i$ для всех k .

Практически это реализуется так: мы кладем “монетки” в некоторые элементы/узлы структуры во время дешёвых операций, а при дорогой операции снимаем эти монетки, оплачивая реальную работу. Главное инвариантное условие: монеток нигде не становится отрицательно.

25.3.3 Метод потенциалов

Метод потенциалов — это бухгалтерский учёт, в котором “банк” задаётся *функцией от состояния структуры*.

Пусть D_i — состояние структуры после выполнения i операций (а D_0 — начальное состояние). Выберем функцию потенциала $\Phi(D) \geq 0$ такую, что $\Phi(D_0) = 0$.

Определение 25.4 (Амортизированная стоимость через потенциал). Определим учётную стоимость i -й операции как

$$\hat{t}_i \stackrel{\text{def}}{=} t_i + \Phi(D_i) - \Phi(D_{i-1}).$$

Утверждение 25.1 (Телескопическая сумма). Для любой последовательности операций выполняется

$$\sum_{i=1}^n \hat{t}_i = \sum_{i=1}^n t_i + \Phi(D_n) - \Phi(D_0) \geq \sum_{i=1}^n t_i.$$

$\triangle$ *Доказательство.* Сложим определения $\hat{t}_i$ по всем i . Слагаемые $+\Phi(D_i)$ и $-\Phi(D_i)$ сокращаются (телескопирование), остаётся $\sum t_i + \Phi(D_n) - \Phi(D_0)$. Так как $\Phi(D_n) \geq 0$ и $\Phi(D_0) = 0$, получаем требуемое. ■

Следовательно, если нам удалось выбрать потенциал так, что каждое $\hat{t}_i$ мало (например, $O(1)$ или $O(\log n)$), то и суммарное реальное время на любую последовательность не превосходит суммы этих малых $\hat{t}_i$.

25.4 Мини-пример (для интуиции)

Классический сюжет: есть операция, которая обычно делает $O(1)$ работы, но иногда выполняет “перестройку” за $O(k)$. Если можно доказать, что за всю жизнь структуры каждая единица перестройки оплачивается ограниченное число раз (например, каждый элемент переносится/копируется не более константного числа раз), то получаем амортизированную оценку $O(1)$ на операцию. Именно так доказывается амортизированная $O(1)$ для `push_back` в динамическом массиве (удвоение размера) и для очереди на двух стеках.

26 Билет №26. Структура данных «вектор» (динамический массив)

26.1 Определение и инварианты

Определение 26.1 (Динамический массив (*vector*)). **Динамический массив** — это структура данных, поддерживающая:

- ▷ доступ по индексу за $O(1)$;
- ▷ добавление в конец (`push_back`) за амортизированное $O(1)$;
- ▷ хранение элементов в непрерывном блоке памяти.

Как правило, динамический массив хранит три величины: `data` — указатель на начало выделенного блока, `size` — число реально хранимых элементов, `capacity` — размер выделенного блока (сколько элементов *можно* хранить без перевыделения памяти).

Инварианты.

- ▷ $0 \leq \text{size} \leq \text{capacity}$.
- ▷ В ячейках `data[0..size-1]` лежат актуальные элементы.
- ▷ Память под `data[0..capacity-1]` выделена одним непрерывным блоком.

26.2 Реализация на массиве

Базовая идея. Динамический массив — это *обычный массив*, который иногда *расширяется*:

- ▷ Пока `size < capacity`, добавление нового элемента в конец — это запись в `data[size]` и увеличение `size`.
- ▷ Когда `size == capacity`, нужно выделить новый блок памяти большего размера, скопировать туда все элементы и освободить старый блок.

Стратегия роста. Стандартная стратегия: при переполнении увеличивать `capacity` в константное число раз (чаще всего в 2 раза):

$$\text{new_capacity} = \max(1, 2 \cdot \text{capacity}).$$

Важно: для амортизированной оценки достаточно, чтобы коэффициент роста был строго больше 1 (например, $\times 1.5$ тоже подходит).

Псевдокод. Ниже — концептуальная реализация (без деталей конструкторов/деструкторов, исключений и т. п.).

Псевдокод (идейно, близко к CLRS)**PushBack**(V, x):

1. **if** $\text{size}(V) = \text{capacity}(V)$ **then**
 - (a) $\text{newCap} \leftarrow \max(1, 2 \cdot \text{capacity}(V))$
 - (b) $\text{newData} \leftarrow \text{ALLOCATE}(\text{newCap})$
 - (c) **for** $i \leftarrow 0$ **to** $\text{size}(V) - 1$ **do** $\text{newData}[i] \leftarrow V.\text{data}[i]$
 - (d) $\text{DEALLOCATE}(V.\text{data})$
 - (e) $V.\text{data} \leftarrow \text{newData}; \quad \text{capacity}(V) \leftarrow \text{newCap}$
2. $V.\text{data}[\text{size}(V)] \leftarrow x$
3. $\text{size}(V) \leftarrow \text{size}(V) + 1$

At(V, i): **return** $V.\text{data}[i]$.**26.3 Операции и асимптотика**

Считаем RAM-модель: чтение/запись одного элемента и присваивание указателя — $O(1)$.

Операция	Идея	Время
Доступ $a[i]$	обращение к адресу $\text{data}+i$	$O(1)$
push_back без перевыделения	запись в конец	$O(1)$
push_back с перевыделением	копирование всех элементов	$O(n)$ (редко)
push_back (в среднем)	с учётом редких копирований	$O(1)$ аморт.
pop_back	$-\text{size}$ (без сжатия)	$O(1)$
$\text{insert}(\text{pos}, x)$	сдвиг хвоста вправо	$O(n)$
$\text{erase}(\text{pos})$	сдвиг хвоста влево	$O(n)$
Поиск по значению	линейный просмотр	$O(n)$

26.4 Амортизированный анализ push_back при удвоении capacity

Модель стоимости. Пусть запись/копирование одного элемента стоит 1.

- ▷ Если места хватает: стоимость = 1 (записали новый элемент).
- ▷ Если места нет ($\text{size} = \text{capacity} = C$):
 - выделяем новый массив размера $2C$;
 - копируем C элементов (стоимость C);
 - записываем новый элемент (стоимость 1).

Итого: $C + 1$.

Агрегированный (суммарный) анализ. Рассмотрим последовательность из n операций `push_back`, начиная с пустого вектора. Перевыделение происходит при размерах $1, 2, 4, 8, \dots$. При каждом перевыделении копируется ровно текущее количество элементов:

$$1 + 2 + 4 + \dots + 2^{\lfloor \log_2 n \rfloor} < 2n.$$

Кроме копирования, мы записываем n новых элементов (ещё n операций). Следовательно, **суммарная** стоимость всех n добавлений:

$$\underbrace{n}_{\text{записи новых элементов}} + \underbrace{O(n)}_{\text{копирования при расширениях}} = O(n).$$

Отсюда **амортизированная** стоимость одной операции:

$$\frac{O(n)}{n} = O(1).$$

Полезные следствия.

- ▷ При удвоении всегда выполняется `capacity < 2 · size` после завершения вставки (кроме малых краевых случаев), то есть памяти “про запас” не больше чем **линейно**.
- ▷ Плохие операции ($O(n)$ из-за копирования) случаются редко: после расширения ещё минимум C вставок пройдёт без копирования.

26.5 Удаление и (опционально) сжатие

Без сжатия. Если при `pop_back` просто уменьшать `size`, то операция всегда $O(1)$. При этом `capacity` не уменьшается (память не освобождается автоматически).

Со сжатием (теоретическая модель динамического массива). Чтобы не держать слишком много лишней памяти, иногда добавляют правило: если `size` стало слишком маленьким (например, `size ≤ capacity/4`), то уменьшаем ёмкость в 2 раза. Тогда и серия удалений имеет амортизированное $O(1)$ (аналогично доказывается, что элементы копируются “не слишком часто” и нет “дребезга”).

26.6 Практические замечания про `std::vector` в C++

- ▷ `std::vector<T>` хранит элементы **континуально** (как массив): хорошая кэш-локальность, можно брать `data()` и передавать в C-API.
- ▷ `push_back/emplace_back` — амортизированное $O(1)$, но может произойти **перевыделение**, тогда:
 - все элементы перемещаются/копируются;
 - инвалидируются указатели/ссылки/итераторы на элементы (важно в задачах!).
- ▷ `reserve(k)` позволяет заранее увеличить `capacity` и избежать частых реаллокаций.

- ▷ `insert/erase` в середине требуют сдвига элементов и стоят $O(n)$.
- ▷ `shrink_to_fit()` — запрос на уменьшение ёмкости; стандарт не гарантирует, что память обязательно будет освобождена.

26.7 Когда вектор хорош, а когда нет

- ▷ **Хорош:** много обращений по индексу, проходы по массиву, добавления в конец.
- ▷ **Плох:** частые вставки/удаления в начале/середине (там лучше подходят списки/деки/деревья), а также ситуации, где критична стабильность адресов элементов.

Интуиция амортизации («монетки»). Можно мысленно «брать с каждой вставки фиксированную плату», например 3: 1 идёт на запись нового элемента, а 2 откладываются. Когда наступает редкое перевыделение памяти и нужно скопировать C старых элементов, копирование оплачивается отложенными монетками. Так как каждый элемент копируется лишь изредка (после роста ёмкости ещё долго хватает места), накопленного «кредита» всегда достаточно.

Типичная альтернатива. Если критичны частые вставки/удаления по указателю в середине последовательности, рассматривают связные списки. Цена — отсутствие $O(1)$ доступа по индексу и более слабая кэш-локальность.

27 Билет №27. Односвязный и двусвязный списки. Добавление и удаление элементов

27.1 Что такое связанный список

Определение 27.1. Связанный список — структура данных для хранения последовательности элементов, где каждый элемент хранится в отдельном узле (*node*) и содержит:

- ▷ значение (ключ, полезные данные);
- ▷ ссылки (*указатели*) на соседние узлы.

Интуитивно указатель — это “адрес” другой ячейки памяти. В списке узлы могут располагаться в памяти произвольно, поэтому доступ “по индексу” (как в массиве) отсутствует: чтобы дойти до k -го элемента, нужно пройти по ссылкам, начиная с головы.

Определение 27.2. Инвариант корректности списка:

- ▷ есть указатель **head** на первый узел (или **nullptr** для пустого списка);
- ▷ (опционально) есть указатель **tail** на последний узел (или **nullptr** для пустого списка);
- ▷ ссылки между узлами задают ровно ту последовательность, которую мы хотим хранить (нет “потерянных” узлов и нет циклов, если список не циклический).

В этом билете нас интересуют два классических варианта: **односвязный** и **двусвязный** список.

27.2 Односвязный список

27.2.1 Определение и представление

Определение 27.3. Односвязный список — связанный список, где каждый узел хранит указатель только на *следующий* узел.

Типичная структура узла (псевдокод):

Представление (псевдокод)

Node:

- ▷ **value** — хранимое значение;
- ▷ **next** — ссылка на следующий узел (или **NIL**).

List: **head** (начало), опционально **tail** (конец).

Если **tail** не хранить, “добавление в конец” требует пройти весь список.

27.2.2 Добавление элементов

Добавление в начало (`push_front`). Алгоритм для вставки значения x в голову:

1. создать новый узел u со значением x ;
2. положить $u \rightarrow \text{next} = \text{head}$;
3. сделать $\text{head} = u$;
4. если список был пуст, то также $\text{tail} = \text{head}$ (если tail поддерживается).

Утверждение 27.1. После *`push_front`* список остаётся корректным и новый элемент становится первым.

$\triangle$ *Доказательство.* Мы создаём один новый узел u и делаем его `next` равным старой голове. Значит, вся старая последовательность остаётся достижимой из u . Переназначив `head` на u , получаем, что u — первый элемент. Другие ссылки не менялись, значит порядок остальных элементов сохранён. В случае пустого списка корректно обновляем `tail`. ■

Добавление после данного узла (`insert_after`). Пусть дан указатель p на некоторый узел списка (не `nullptr`). Хотим вставить x после p .

1. создать новый узел u со значением x ;
2. положить $u \rightarrow \text{next} = p \rightarrow \text{next}$;
3. положить $p \rightarrow \text{next} = u$;
4. если p был хвостом, то обновить $\text{tail} = u$.

Утверждение 27.2. *`insert_after(p, x)` корректно вставляет новый узел сразу после p .*

$\triangle$ *Доказательство.* До операции $p \rightarrow \text{next}$ указывал на узел, следующий за p (или `nullptr`, если p был хвостом). После присваиваний новый узел u становится *единственным* узлом, на который указывает $p \rightarrow \text{next}$, а $u \rightarrow \text{next}$ указывает на прежнего преемника p . Поэтому порядок всех узлов, кроме вставленного, не меняется, а u стоит ровно между p и прежним $p \rightarrow \text{next}$. ■

Добавление в конец (`push_back`).

- ▷ Если `tail` поддерживается: вставляем после `tail` (как `insert_after`) и обновляем `tail`.
- ▷ Если `tail` не поддерживается: нужно пройти от `head` до последнего узла за $O(n)$.

27.2.3 Удаление элементов

Удаление из начала (`pop_front`). Если список непуст:

1. сохранить указатель на текущую голову `old = head`;
2. сделать `head = head->next`;
3. удалить `old` (освободить память);
4. если список стал пуст, то `tail = nullptr`.

Утверждение 27.3. *`pop_front` корректно удаляет первый элемент списка.*

△ *Доказательство.* После переназначения `head` на следующий узел, достижимая из `head` последовательность совпадает со старой, но без первого узла. Так как других ссылок мы не меняли, порядок оставшихся элементов сохраняется. Освобождение памяти удалённого узла не влияет на оставшиеся узлы, поскольку на него больше никто не ссылается из списка. ■

Удаление после данного узла (`erase_after`). Пусть дан `p`. Хотим удалить узел `q = p->next`. Если `q` существует:

1. сделать `p->next = q->next`;
2. если `q` был хвостом, обновить `tail = p`;
3. удалить `q`.

Это полностью симметрично вставке после узла.

Удаление произвольного узла по указателю. В односвязном списке, имея только указатель на узел `v`, удалить его за $O(1)$ нельзя в общем случае: чтобы “вырезать” `v` из цепочки, нужно изменить `next` у *предыдущего* узла, а без обратных ссылок предыдущий узел неизвестен.

Замечание. Есть трюк: скопировать содержимое следующего узла `v->next` в `v` и удалить `v->next`. Это даёт $O(1)$, но работает *только если* `v` не последний узел (и может быть недопустимо, если “идентичность” узла важна).

27.2.4 Асимптотика

Пусть в списке n элементов.

- ▷ Доступ к k -му элементу, поиск по значению, поиск предыдущего элемента: $\Theta(n)$.
- ▷ `push_front`, `pop_front`: $\Theta(1)$.
- ▷ `insert_after`, `erase_after` при наличии указателя на узел: $\Theta(1)$.
- ▷ `push_back`: $\Theta(1)$ при поддержке `tail`, иначе $\Theta(n)$.
- ▷ `pop_back`: в односвязном списке требует найти предпоследний элемент, поэтому $\Theta(n)$ даже при наличии `tail`.

27.3 Двусвязный список

27.3.1 Определение и представление

Определение 27.4. Двусвязный список — список, где каждый узел хранит указатели и на следующий, и на предыдущий узел.

Типичная структура узла (псевдокод):

Представление (псевдокод)

Node:

- ▷ `value` — хранимое значение;
- ▷ `next` — ссылка на следующий узел (или `NIL`);
- ▷ `prev` — ссылка на предыдущий узел (или `NIL`).

List: `head`, `tail`.

Наличие `prev` позволяет удалять и вставлять элемент “вокруг” данного узла за константное время.

27.3.2 Добавление

Вставка между двумя узлами. Пусть есть два соседних узла `a` и `b` такие, что `a->next == b` и `b->prev == a`. Хотим вставить новый узел `u` между ними.

1. `u->prev = a`, `u->next = b`;
2. `a->next = u`, `b->prev = u`.

Частные случаи:

- ▷ `push_front`: считаем, что слева от `head` стоит фиктивный “предыдущий” (или просто обрабатываем отдельно пустой список);
- ▷ `push_back`: аналогично справа от `tail`.

Удобный приём: фиктивные вершины (sentinel). Часто заводят два фиктивных узла `H` и `T` (голова и хвост-сторожи), которые *не содержат* пользовательских данных:

- ▷ `H->next` указывает на первый реальный элемент или на `T`,
- ▷ `T->prev` указывает на последний реальный элемент или на `H`.

Тогда вставка/удаление “в начало” и “в конец” сводится к общей операции “вставить между двумя узлами”, а большинство граничных случаев исчезает.

27.3.3 Удаление

Удаление данного узла по указателю (erase). Пусть дан указатель v на узел, который нужно удалить (и он не фиктивный сторож).

1. $a = v \rightarrow \text{prev}$, $b = v \rightarrow \text{next}$;
2. $a \rightarrow \text{next} = b$, $b \rightarrow \text{prev} = a$;
3. удалить v ;
4. при хранении `head/tail` без сторожей: обновить их, если удаляли крайний элемент.

Утверждение 27.4. В двусвязном списке операция $\text{erase}(v)$ корректна и работает за $\Theta(1)$.

$\triangle$ *Доказательство.* Пусть a и b — соседи узла v слева и справа. После переназначения $a \rightarrow \text{next} = b$ и $b \rightarrow \text{prev} = a$ узел v становится недостижим из `head` (и не достижим при обходе назад от `tail`), а последовательность остальных узлов сохраняет исходный порядок. Перекидывание двух указателей — константное число операций, следовательно время $\Theta(1)$. ■

27.3.4 Асимптотика

Пусть n — размер списка.

- ▷ Поиск по индексу/значению: $\Theta(n)$ (как и в односвязном).
- ▷ Вставка и удаление *по указателю на место* (или на узел): $\Theta(1)$.
- ▷ `push_front`, `push_back`, `pop_front`, `pop_back` при наличии `head/tail`: $\Theta(1)$.

27.4 Сравнение односвязного и двусвязного списков

- ▷ **Память:** односвязный хранит один указатель на узел, двусвязный — два (больше накладные расходы).
- ▷ **Удаление по указателю:** в односвязном обычно нужен указатель на предыдущий узел (или линейный поиск), в двусвязном удаление по указателю делается за $\Theta(1)$.
- ▷ **Обход:** односвязный — только вперёд; двусвязный — в обе стороны.
- ▷ **Крайние операции:** двусвязный список естественно поддерживает быстрый `pop_back`; односвязный — нет.

Замечание. В стандартной библиотеке C++ аналогами являются `std::forward_list` (односвязный список с операциями `insert_after/erase_after`) и `std::list` (двусвязный список, где `insert/erase` по итератору работают за $\Theta(1)$).

28 Билет №28. Фибоначчиева куча: merge, insert, getMin, extractMin. Оценка через $D(n)$

28.1 Постановка

Нужно поддерживать *сливаемую* приоритетную очередь над множеством (или мульти-множеством) ключей с операциями:

- ▷ $\text{merge}(H_1, H_2)$ — слить две кучи в одну;
- ▷ $\text{insert}(H, x)$ — добавить ключ x ;
- ▷ $\text{getMin}(H)$ — получить минимальный ключ;
- ▷ $\text{extractMin}(H)$ — удалить и вернуть минимальный ключ.

28.2 Определение и представление фибоначчиевой кучи

Определение 28.1 (Куча-упорядоченное дерево). Дерево называется *куча-упорядоченным* (min-heap ordered), если для любой вершины v и любого её ребёнка u выполнено

$$\text{key}(v) \leq \text{key}(u).$$

Тогда ключ в корне дерева минимален среди ключей этого дерева.

Определение 28.2 (Фибоначчиева куча). *Фибоначчиева куча* H — это набор (лес) куча-упорядоченных деревьев, где:

- ▷ корни всех деревьев объединены в один циклический двусвязный список (*root list*);
- ▷ у кучи хранится указатель $H.\text{min}$ на корень с минимальным ключом;
- ▷ для каждой вершины v хранится её *степень* $\text{deg}(v)$ — число детей.

(Для операций из этого билета достаточно этих полей; дополнительные поля **mark** понадобятся в `decreaseKey`.)

Определение 28.3 ($D(n)$). Обозначим через $D(n)$ *максимальную степень вершины* в фибоначчиевой куче из n элементов:

$$D(n) = \max\{\text{deg}(v) : v \text{ — вершина некоторой фибоначчиевой кучи размера } n\}.$$

Далее асимптотику операций будем выражать через $D(n)$ (оценка $D(n) = O(\log n)$ доказывается отдельно).

28.3 Базовые операции: merge, insert, getMin

Операция $\text{merge}(H_1, H_2)$.

1. Склеить (конкатенировать) два циклических списка корней $H_1.\text{root}$ и $H_2.\text{root}$ (переподвесив $O(1)$ указателей).

2. $H.\text{min} \leftarrow \min(H_1.\text{min}, H_2.\text{min})$.

3. $H.n \leftarrow H_1.n + H_2.n$.

Корректность. Куча-упорядоченность каждого дерева не меняется (мы лишь объединяем списки корней). Минимум среди всех элементов обязан лежать в одном из корней, а значит достаточно взять минимум из $H_1.\text{min}$ и $H_2.\text{min}$.

Время. $O(1)$.

Операция $\text{insert}(H, x)$. Создать одноэлементное дерево (вершину) v с ключом x и вставить корень этого дерева в список корней $H.\text{root}$ за $O(1)$. Если $x < \text{key}(H.\text{min})$, обновить $H.\text{min}$. Увеличить $H.n$.

Корректность. Новое дерево куча-упорядочено тривиально. Для всего леса инварианты сохраняются, а указатель $H.\text{min}$ обновляется до истинного минимума.

Время. $O(1)$.

Операция $\text{getMin}(H)$. Вернуть $\text{key}(H.\text{min})$.

Корректность. По определению $H.\text{min}$ всегда указывает на корень с минимальным ключом.

Время. $O(1)$.

28.4 Операция extractMin : алгоритм и корректность

Идея. Минимальный ключ всегда находится в корне одного из деревьев. После удаления этого корня его дети становятся корнями новых деревьев. Затем выполняется *уплотнение* (*consolidate*): мы многократно «сливаем» (*link*) корни одинаковой степени, чтобы в итоге у всех корней степени стали попарно различны.

Определение 28.4 (Операция *link*). Пусть x и y — корни двух куча-упорядоченных деревьев одинаковой степени d . $\text{link}(x, y)$ делает корнем дерево с меньшим ключом: если $\text{key}(x) \leq \text{key}(y)$, то подвешиваем y ребёнком к x (в список детей x), получая дерево степени $d + 1$.

Утверждение 28.1 (Корректность *link*). После *link* полученное дерево остаётся куча-упорядоченным.

$\triangle$ *Доказательство.* $\triangle$ До операции оба дерева куча-упорядочены. Мы подвешиваем корень y под корень x так, что $\text{key}(x) \leq \text{key}(y)$. Для всех старых рёбер свойство $\text{key}(\text{родитель}) \leq \text{key}(\text{ребёнок})$ не меняется, а для нового ребра (x, y) оно верно по выбору x . ■ ■

Алгоритм $\text{extractMin}(H)$.

1. $z \leftarrow H.\text{min}$.

2. Для каждого ребёнка x вершины z :

▷ удалить x из списка детей z ;

▷ добавить x в список корней $H.\text{root}$;

▷ $\text{parent}(x) \leftarrow \text{null}$.

3. Удалить z из списка корней. Уменьшить $H.n$.
4. Если список корней пуст, то $H.\text{min} \leftarrow \text{null}$ и завершить.
5. Иначе выполнить **consolidate** и обновить $H.\text{min}$.

Процедура **consolidate**.

1. Завести массив $A[0..D(n)]$, заполненный **null**.
2. Пройти по текущему списку корней. Для каждого корня w с $d = \text{deg}(w)$:
 - ▷ пока $A[d] \neq \text{null}$: пусть $y = A[d]$; выполнить **link**(w, y) так, чтобы корнем остался меньший; положить $A[d] \leftarrow \text{null}$ и увеличить d ;
 - ▷ положить $A[d] \leftarrow w$.
3. Пересобрать список корней из всех ненулевых $A[d]$ и за один проход выбрать новый минимум $H.\text{min}$.

Корректность **extractMin**. Проверим инварианты:

- ▷ *Куча-упорядоченность*. До удаления z каждое дерево куча-упорядочено. Перенос детей z в корни не меняет отношений внутри их поддеревьев, значит каждое из этих деревьев остаётся куча-упорядоченным. В **consolidate** деревья комбинируются только операцией **link**, которая сохраняет куча-упорядоченность.
- ▷ *Правильность минимума*. Минимум в куче обязан лежать в некотором корне. После завершения **consolidate** мы явно перебираем корни и ставим $H.\text{min}$ на минимальный из них, значит $H.\text{min}$ корректен.
- ▷ *Попарно различные степени корней*. **consolidate** устраняет конфликты степеней: как только два корня имеют одинаковую степень d , они связываются в дерево степени $d+1$. В массиве A в конце не остаётся двух различных корней с одинаковым индексом d , следовательно, степени корней в итоговом списке попарно различны.

28.5 Асимптотика через $D(n)$

Обозначим $t(H)$ — число деревьев (корней) в списке корней до выполнения **consolidate**.

- ▷ Перенос детей z в корни требует $O(\text{deg}(z)) \leq O(D(n))$ операций со списками.
- ▷ Цикл **consolidate** просматривает каждый корень $O(1)$ раз: внешний проход даёт вклад $O(t(H))$.
- ▷ Каждая операция **link** уменьшает число корней на 1 и выполняется за $O(1)$. Таких операций не больше $t(H)$.
- ▷ Размер массива A равен $D(n) + 1$, а финальная пересборка и поиск минимума занимают $O(D(n))$.

Утверждение 28.1 (Время работы `extractMin`). Фактическое время работы `extractMin` оценивается как

$$O(t(H) + D(n)).$$

- а) После `consolidate` число корней $t(H') \leq D(n) + 1$.
 б) Амортизированно (учётным временем) `extractMin` работает за $O(D(n))$.

$\triangle$ Доказательство. $\triangle$ (а) После `consolidate` степени всех корней попарно различны и лежат в диапазоне $0, 1, \dots, D(n)$. Следовательно, корней не больше $D(n) + 1$.

(б) Если использовать потенциал $\Phi(H) = t(H)$ (число деревьев в root list), то в `consolidate` каждая операция `link` уменьшает $t(H)$ на 1, то есть «оплачивается» падением потенциала. Остаются только неизбежные слагаемые $O(D(n))$ (перенос детей z и проходы по массиву A). Поэтому амортизированная стоимость равна $O(D(n))$. ■ ■

28.6 Итоговые оценки (через $D(n)$)

- ▷ `merge`: $O(1)$.
- ▷ `insert`: $O(1)$.
- ▷ `getMin`: $O(1)$.
- ▷ `extractMin`: фактически $O(t(H) + D(n))$, амортизированно $O(D(n))$.

Замечание. Ключевой следующий шаг (следующий билет) — доказать, что $D(n) = O(\log n)$, тогда `extractMin` становится $O(\log n)$.

29 Билет №29. Фибоначчиева куча: реализация decreaseKey

29.1 Напоминание: устройство фибоначчиевой кучи

Определение 29.1. **Фибоначчиева куча** (Fibonacci heap) — это набор корневых деревьев, удовлетворяющих *свойству кучи*: для любого ребра $p \rightarrow c$ выполняется $key(p) \leq key(c)$. Минимальный ключ среди всех элементов обязательно находится в одном из корней.

Куча хранится как *корневой список* (root list) из корней деревьев. Для быстрых вырезаний/склеек корневой список и список детей каждой вершины обычно реализуются как *кольцевые двусвязные списки*.

Поля структуры Node. Для каждой вершины x храним:

- ▷ $key(x)$ — ключ;
- ▷ $deg(x)$ — число детей (*степень*);
- ▷ $parent(x)$ — указатель на родителя (NIL у корней);
- ▷ $child(x)$ — указатель на *какого-нибудь* ребенка (NIL, если детей нет);
- ▷ $left(x), right(x)$ — соседи в списке (в корневом списке или списке детей);
- ▷ $mark(x) \in \{false, true\}$ — метка “потерял ребенка”.

Также храним указатель $H.min$ на корень с минимальным ключом (тогда `getMin` работает за $O(1)$).

Инвариант меток.

Определение 29.2. Вершина x (не корень) считается **помеченной** ($mark(x) = true$), если *после того, как она стала чьим-то ребенком, у нее уже вырезали одного ребенка*. Корни всегда считаются непомеченными.

Идея: *каждая некорневая вершина может “пережить” потерю не более одного ребенка*. Если она теряет второго, ее саму вырезают из родителя (*cascading cut*). Это ограничение и дает логарифмическую оценку на максимальную степень $D(n)$ (доказывается в билете 30).

29.2 Операция decreaseKey: постановка

Задача. Дан указатель на вершину x в фибоначчиевой куче H . Нужно уменьшить ее ключ: присвоить $key(x) := k$, где $k \leq key(x)$, сохранив корректность структуры.

Замечание. Если вместо указателя дано только значение ключа, то сначала нужно найти вершину. Фибоначчиева куча поиск по значению не поддерживает эффективно, поэтому в задачах `decreaseKey` почти всегда вызывают по *указателю*.

29.3 Идея: вырезание нарушителя и каскадные вырезания

Единственное место, где после уменьшения $key(x)$ может нарушиться свойство кучи — ребро между x и ее родителем $y = parent(x)$: могло стать $key(x) < key(y)$. Если нарушения нет, ничего больше делать не нужно. Если нарушение есть, “чинить” будем так:

1. **Cut**: вырезаем x из списка детей y и добавляем x в корневой список.
2. **Cascading cut**: вершина y только что потеряла ребенка. Если y не корень:
 - ▷ если y не была помечена, помечаем ее (первую потерю “прощаем”);
 - ▷ если y уже была помечена, то это вторая потеря — вырезаем y из ее родителя и продолжаем каскад вверх.

29.4 Псевдокод

Ниже приведен стандартный псевдокод (в стиле CLRS).

Псевдокод (идейно, близко к CLRS)

DecreaseKey(H, x, k):

1. **if** $k > key(x)$ **then error**
2. $key(x) \leftarrow k$
3. $y \leftarrow parent(x)$
4. **if** $y \neq \text{NIL}$ **and** $key(x) < key(y)$ **then**
 - (a) **CUT**(H, x, y)
 - (b) **CASCADINGCUT**(H, y)
5. **if** $key(x) < key(H.\text{min})$ **then** $H.\text{min} \leftarrow x$

Cut(H, x, y):

(вырезать x из детей y и поднять в корни)

1. **REMOVEFROMCHILDLIST**(y, x) (кольцевой двусвязный список)
2. $deg(y) \leftarrow deg(y) - 1$
3. **if** $deg(y) = 0$ **then** $child(y) \leftarrow \text{NIL}$
4. **else if** $child(y) = x$ **then** $child(y) \leftarrow right(x)$
5. **ADDTOROOTLIST**(H, x)
6. $parent(x) \leftarrow \text{NIL}$
7. $mark(x) \leftarrow \text{false}$

CascadingCut(H, y):

1. $z \leftarrow parent(y)$

```

2. if  $z \neq \text{NIL}$  then
    (a) if  $\text{mark}(y) = \text{false}$  then  $\text{mark}(y) \leftarrow \text{true}$ 
    (b) else
        i.  $\text{CUT}(H, y, z)$ 
        ii.  $\text{CASCADINGCUT}(H, z)$ 

```

Замечание. Операции `remove/add` для кольцевого двусвязного списка делаются за $O(1)$ перепривязкой константного числа указателей.

29.5 Доказательство корректности

Утверждение 29.1. После выполнения *DECREASE-KEY* куча H удовлетворяет свойству кучи, а инвариант меток сохраняется.

$\triangle$ *Доказательство.* 1) *Свойство кучи.* Мы меняем только $\text{key}(x)$, и только в сторону уменьшения. Следовательно, все ребра, не инцидентные x , как были корректны, так и остаются. Потенциальное нарушение возможно только на ребре (y, x) , где $y = \text{parent}(x)$.

Если $\text{key}(x) \geq \text{key}(y)$, ребро (y, x) тоже корректно, и ничего не ломается.

Если $\text{key}(x) < \text{key}(y)$, выполняем $\text{CUT}(H, x, y)$: ребро (y, x) исчезает, x становится корнем, а корни не имеют родителя, поэтому для x свойство кучи выполнено автоматически. У вершины y мы удалили ребенка x ; оставшиеся ребра $(y, \cdot)$ не менялись, значит они по-прежнему удовлетворяют $\text{key}(y) \leq \text{key}(\text{child})$.

Дальше *CASCADING-CUT* лишь многократно повторяет ту же операцию: удаляет ребро между вершиной и ее родителем, поднимая вершину в корневой список, не меняя ключей остальных вершин. Поэтому свойство кучи сохраняется на каждом шаге.

2) *Инвариант меток.* Рассмотрим некорневую вершину v . По определению $\text{mark}(v) = \text{true}$ означает, что v уже потеряла одного ребенка, оставаясь чьим-то ребенком. В *CASCADING-CUT* именно это и поддерживается:

- ▷ если v потеряла *первого* ребенка (была непомечена), то мы ставим $\text{mark}(v) = \text{true}$;
- ▷ если v теряет *второго* ребенка (уже была помечена), то мы вырезаем v из родителя, делая ее корнем и сбрасывая метку (в *CUT* выполняется $v.\text{mark} = \text{false}$).

Значит, некорневая вершина не может потерять двух детей и остаться некорневой. Корни по коду всегда делаются непомеченными, так что инвариант выполнен.

Наконец, обновление $H.\text{min}$ очевидно делает `getMin` корректным. ■

29.6 Время работы и амортизационная оценка

Пусть при выполнении *DECREASE-KEY* произошло $c \geq 0$ операций *CUT* (если нарушения не было, то $c = 0$; иначе $c \geq 1$). Тогда реальное время работы равно $O(1 + c)$, потому что каждый *CUT* — это константное число операций со списками.

Однако c может быть большим (длинная цепочка каскадных вырезаний). Для оценки используют потенциал:

$$\Phi(H) = t(H) + 2m(H), \quad (29.1)$$

где $t(H)$ — число деревьев (корней) в куче, а $m(H)$ — число помеченных вершин.

Утверждение 29.2. Амортизированное время операции *decreaseKey* равно $O(1)$.

$\triangle$ *Доказательство.* При каждом CUT число деревьев увеличивается на 1, то есть $\Delta t = +c$. Кроме того:

- ▷ первая вырезанная вершина x становится корнем и помеченной не бывает ($mark(x) = \text{false}$);
- ▷ все последующие вырезания происходят только у *помеченных* вершин, и при вырезании метка сбрасывается.

То есть на каждом “каскадном” шаге мы уменьшаем число помеченных на 1. В конце каскада либо мы дошли до корня, либо встретили непомеченную вершину и пометили ее, увеличив m на 1. Итого $\Delta m \leq -(c - 1) + 1 = 2 - c$.

Тогда

$$\Delta \Phi = \Delta t + 2\Delta m \leq c + 2(2 - c) = 4 - c.$$

Амортизированная стоимость равна

$$\underbrace{O(1 + c)}_{\text{реальное время}} + \underbrace{\Delta \Phi}_{\leq 4 - c} = O(1).$$

(константы поглощаются выбором единицы “оплаты” потенциалом). ■

Замечание. Полная таблица амортизированных асимптотик всех операций и оценка $D(n)$ (максимальной степени вершины) доказываются в билете 30.

30 Билет №30. Фибоначчиева куча: асимптотика функции $D(n)$ и доказательство асимптотики операций

30.1 Определение $D(n)$

Определение 30.1. *Фибоначчиева куча* (Fibonacci heap) — это структура данных для поддержки мультимножества ключей, представляющаяся *списком корней* нескольких куч-деревьев (min-heap trees) с поддержкой:

- ▷ операции `consolidate` после `extractMin`;
- ▷ механизма *отметок* (*mark*) и `cascading cut`, ограничивающего число “потерь” детей у вершин.

Для фибоначчиевой кучи на n вершинах обозначим через $D(n)$ *максимальную степень вершины*:

$$D(n) = \max\{\deg(v) \mid v \text{ — вершина кучи из } n \text{ элементов}\},$$

где $\deg(v)$ — число детей вершины v .

Ключевая цель: доказать $D(n) = O(\log n)$, после чего `extractMin` и `delete` будут работать за $O(\log n)$ (в амортизированном смысле), а `insert`, `merge`, `getMin`, `decreaseKey` — за $O(1)$.

30.2 Оценка $D(n)$ через числа Фибоначчи

Обозначим числа Фибоначчи: $F_0 = 0$, $F_1 = 1$, $F_k = F_{k-1} + F_{k-2}$ при $k \geq 2$.

Лемма 30.1 (Степени детей). *Пусть вершина x имеет детей $y_1, \dots, y_k$, упорядоченных по времени присоединения к x операцией `link` (от раннего к позднему). Тогда для всех i выполнено*

$$\deg(y_i) \geq i - 2.$$

$\triangle$ *Доказательство.* Когда y_i присоединялся к x , у x уже было ровно $i - 1$ ребенок, значит, обе склеиваемые корневые вершины имели степень $i - 1$, то есть в момент `link` было $\deg(y_i) = i - 1$.

Далее используется механизм отметок: вершина, оставаясь ребенком текущего родителя, может потерять не более одного ребенка — при первой потере она помечается, при второй вырезается (`cut`) и уходит в корневой список (`cascading cut`). Поэтому после присоединения к x вершина y_i могла потерять не более одного ребенка, значит

$$\deg(y_i) \geq (i - 1) - 1 = i - 2.$$

(для $i = 1$ правая часть равна -1 , а степень неотрицательна, так что неравенство также верно). ■

Лемма 30.2 (Минимальный размер поддерева). *Пусть x — вершина фибоначчиевой кучи со степенью $k = \deg(x)$. Тогда размер ее поддерева (число вершин в поддереве,*

включая x) не меньше F_{k+2} .

$\triangle$ *Доказательство.* Пусть $S(d)$ — минимально возможный размер поддерева вершины степени d (считая саму вершину). База: $S(0) = 1 = F_2$, $S(1) = 2 = F_3$.

Рассмотрим вершину степени $d \geq 2$ и упорядочим ее детей $u_1, \dots, u_d$ по времени присоединения. Тогда, по предыдущей лемме, для $i \geq 3$ выполнено $\deg(u_i) \geq i - 2$. Значит размер поддерева u_i не меньше $S(i - 2)$ (у вершины степени хотя бы $i - 2$ есть как минимум $i - 2$ детей, поэтому ее поддерево не может быть меньше минимума для степени $i - 2$).

Предположим по индукции по d , что для всех меньших степеней верно $S(j) \geq F_{j+2}$. Тогда для $i \geq 3$ имеем

$$|T(u_i)| \geq S(i - 2) \geq F_{(i-2)+2} = F_i,$$

а для $i = 1, 2$ всегда верно $|T(u_i)| \geq 1 = F_i$. Следовательно,

$$S(d) \geq 1 + \sum_{i=1}^d |T(u_i)| \geq 1 + \sum_{i=1}^d F_i.$$

Остается использовать тождество

$$\sum_{i=1}^d F_i = F_{d+2} - 1,$$

которое доказывается индукцией по d . Тогда

$$S(d) \geq 1 + (F_{d+2} - 1) = F_{d+2}.$$

В частности, для $d = k$ получаем $|T(x)| \geq F_{k+2}$. ■

Следствие (Оценка $D(n)$). Для кучи из n элементов

$$D(n) \leq \lfloor \log_{\varphi} n \rfloor = O(\log n),$$

где $\varphi = \frac{1+\sqrt{5}}{2}$.

$\triangle$ *Доказательство.* Пусть в куче есть вершина степени $k = D(n)$. По предыдущей лемме $n \geq F_{k+2}$. Из формулы Бине следует оценка $F_{k+2} \geq \varphi^k$ (для всех достаточно больших k ; для малых k проверяется напрямую). Тогда $n \geq \varphi^k$, откуда $k \leq \log_{\varphi} n$. ■

30.3 Амортизированная асимптотика операций

Для доказательства асимптотики всех операций используем *метод потенциалов*.

Определение 30.2 (Потенциал). Пусть $t(H)$ — число корней (деревьев) в корневом списке кучи H , а $m(H)$ — число отмеченных вершин. Определим потенциал

$$\Phi(H) = t(H) + 2m(H).$$

Утверждение 30.1 (Ключевой инвариант отметок). Если вершина v является ребенком (имеет родителя), то после того как она стала ребенком своего текущего родителя, она может потерять не более одного ребенка, оставаясь ребенком. После второй потери она будет вырезана и станет корнем.

Δ Доказательство. Это непосредственно следует из определения **cascading cut**: при первой потере ребенка вершина помечается, при второй — вырезается и переносится в корневой список (после чего процесс рекурсивно продолжается к родителю). ■

Амортизированную стоимость оцениваем как

$$\hat{T} = T_{\text{real}} + \Delta\Phi.$$

makeHeap, merge, getMin, insert

- ▷ **makeHeap**: реально $O(1)$, $\Phi = 0$.
- ▷ **merge (union)**: конкатенация корневых списков и обновление указателя на минимум. Реально $O(1)$, потенциал просто суммируется, амортизированно $O(1)$.
- ▷ **getMin**: указатель на минимум хранится явно, время $O(1)$.
- ▷ **insert**: добавляем новую вершину в корневой список. Реально $O(1)$. При этом t увеличивается на 1, m не меняется, значит $\Delta\Phi = +1$ и амортизированно $O(1)$.

Операция decreaseKey

Утверждение 30.2 (**decreaseKey**). Амортизированное время работы **decreaseKey** равно $O(1)$.

Δ Доказательство. Если после уменьшения ключа порядок кучи не нарушен (ключ не меньше ключа родителя), то операция делает $O(1)$ действий и потенциал не растет: $\hat{T} = O(1)$.

Иначе выполняются **cut** и **cascading cut**. Пусть выполнено $c \geq 1$ операций **cut**. Реальное время $T_{\text{real}} = O(1 + c)$, так как каждое **cut** занимает $O(1)$.

Оценим изменение потенциала. Каждый **cut** увеличивает число деревьев на 1: $\Delta t = +c$.

Рассмотрим отметки. Все вершины, которые вырезаются в ходе каскада *кроме, возможно, первой*, были отмечены (иначе каскад бы не продолжился), и при вырезании их отметка снимается. Значит, число отмеченных уменьшается как минимум на $c - 1$. При этом возможна одна новая отметка: каскад останавливается на первой вершине, которая была не отмечена и потеряла ребенка, после чего она помечается. Значит,

$$\Delta m \leq 1 - (c - 1) = 2 - c.$$

Тогда

$$\Delta\Phi = \Delta t + 2\Delta m \leq c + 2(2 - c) = 4 - c.$$

Следовательно,

$$\hat{T} = T_{\text{real}} + \Delta\Phi \leq (1 + c) + (4 - c) = O(1).$$

■

Операция extractMin

Утверждение 30.3 (`extractMin`). Амортизированное время работы *extractMin* равно $O(D(n)) = O(\log n)$.

Δ Доказательство. Пусть удаляемый минимум — вершина z . Реальное время состоит из:

1. удаления z из корневого списка;
2. переноса всех ее детей в корневой список (их $\deg(z) \leq D(n)$);
3. **consolidate**: многократного **link** корней одинаковых степеней.

Итак,

$$T_{\text{real}} = O(t_{\text{before}} + D(n)),$$

где t_{before} — число корней до консолидации: мы один раз обходим корневой список, делаем $O(t_{\text{before}})$ операций **link**, и используем массив (или словарь) размера $O(D(n))$ для учета степеней.

Оценим амортизацию. После **consolidate** число корней становится не больше $D(n) + 1$ (по одному дереву каждого ранга $0..D(n)$). При **extractMin** новые отметки не появляются, а у поднятых в корень детей отметка сбрасывается, поэтому $m_{\text{after}} \leq m_{\text{before}}$. Следовательно,

$$\Delta\Phi = \Phi_{\text{after}} - \Phi_{\text{before}} \leq (D(n) + 1) - t_{\text{before}}.$$

Тогда

$$\hat{T} = T_{\text{real}} + \Delta\Phi \leq (t_{\text{before}} + D(n)) + (D(n) + 1 - t_{\text{before}}) = O(D(n)) = O(\log n).$$

■

Операция delete

Утверждение 30.4 (`delete`). *delete*(x) реализуется как *decreaseKey*($x, -\infty$) и затем *extractMin*. Амортизированное время $O(\log n)$.

Δ Доказательство. Первая часть занимает $O(1)$ амортизированно (*decreaseKey*), вторая — $O(\log n)$ амортизированно (*extractMin*). Сумма — $O(\log n)$. ■

30.4 Итоговая таблица асимптотик

Операция	Реальное время	Амортизированное время
<code>makeHeap</code>	$O(1)$	$O(1)$
<code>merge (union)</code>	$O(1)$	$O(1)$
<code>getMin</code>	$O(1)$	$O(1)$
<code>insert</code>	$O(1)$	$O(1)$
<code>decreaseKey</code>	$O(1) + O(\#cuts)$	$O(1)$
<code>extractMin</code>	$O(t_{\text{before}} + D(n))$	$O(D(n)) = O(\log n)$
<code>delete</code>	—	$O(\log n)$

Здесь t_{before} — число корневых деревьев перед `consolidate`, а $D(n) = O(\log n)$ по доказанному выше.

31 Билет №31. Структура данных «стек»: реализация списком

31.1 Определение и интерфейс

Определение 31.1 (Стек). *Стек* (stack) — структура данных для хранения последовательности элементов, поддерживающая доступ только к **последнему добавленному** элементу. Принцип работы: **LIFO** (Last In — First Out), «последним пришёл — первым вышел».

Типовой минимальный интерфейс стека:

- ▷ `push(x)` — добавить элемент x на вершину стека;
- ▷ `pop()` — удалить элемент с вершины стека;
- ▷ `top()` — вернуть (посмотреть) элемент на вершине стека, не удаляя его;
- ▷ `empty()` — проверка на пустоту (часто используется как предикат корректности вызовов `pop/top`).

Замечание. В алгоритмических задачах обычно гарантируется, что `pop()` и `top()` не вызываются на пустом стеке. Если гарантий нет, то поведение нужно явно определить: например, бросать исключение, возвращать код ошибки или использовать `optional`.

31.2 Мотивация

Стек естественным образом моделирует, например, **стек вызовов функций**: при входе в функцию её «кадр» добавляется наверх, при выходе — снимается с вершины. Именно поэтому стек удобно реализовывать так, чтобы операции на вершине работали за константу.

31.3 Реализация стека односвязным списком

Идея: будем хранить элементы в **односвязном списке**, причём **голова списка является вершиной стека**.

31.3.1 Структура узла

Каждый узел содержит:

- ▷ значение `value`;
- ▷ указатель `next` на следующий узел (тот, что «ниже» в стеке).

Кроме того, стек хранит указатель `head` на верхний элемент (голову списка). Опционально можно хранить `size` для операции `size()`.

31.3.2 Операции

push(x). Создать новый узел v со значением x . Пусть раньше вершиной стека была **head**. Тогда:

$$v.\text{next} \leftarrow \text{head}, \quad \text{head} \leftarrow v.$$

top(). Вернуть $\text{head} \rightarrow \text{value}$.

pop(). Сохранить **head** во временный указатель, сдвинуть голову на следующий элемент и удалить старый узел:

$$\text{old} \leftarrow \text{head}, \quad \text{head} \leftarrow \text{head} \rightarrow \text{next}, \quad \text{delete old}.$$

empty(). Стек пуст тогда и только тогда, когда $\text{head} == \text{nullptr}$.

31.4 Корректность

Утверждение 31.1 (Инвариант представления). *После любой последовательности корректных операций стек представлен списком, начинающимся в **head**, причём:*

1. *элементы списка идут в **обратном** порядке добавления (последний добавленный лежит в **head**);*
2. *множество элементов списка в точности совпадает с множеством элементов стека;*
3. *вершина стека равна первому элементу списка (**head**).*

△ *Доказательство.* Докажем по индукции по числу выполненных операций.

База. Изначально стек пуст, $\text{head} == \text{nullptr}$. Список пуст, все пункты инварианта выполнены.

Переход. Предположим, что инвариант верен до очередной операции.

- ▷ **push(x):** создаётся новый узел со значением x , который становится новой головой и указывает на старую голову. Следовательно, x становится первым элементом списка, а прежняя последовательность элементов не изменяется. Значит, множество элементов увеличилось ровно на x , порядок добавления соблюден, а вершина стека совпадает с **head**.
- ▷ **top():** структура не изменяется, возвращается значение из **head**, которое по инварианту является вершиной стека.
- ▷ **pop():** удаляется ровно первый узел списка (**head**) и **head** сдвигается на **next**. По инварианту этот первый узел и есть вершина стека, поэтому удаляется именно тот элемент, который должен удалять **pop()**. Остальные элементы и их порядок не меняются.

Во всех случаях инвариант сохраняется. ■

31.5 Асимптотика

Пусть n — текущее число элементов в стеке.

- ▷ `push`, `pop`, `top`, `empty` работают за $\mathcal{O}(1)$ времени, так как изменяют/читают константное число указателей и значений.
- ▷ Память: $\mathcal{O}(n)$ узлов списка (по одному на элемент стека).

31.6 Псевдокод (идейно, близко к CLRS)

Псевдокод (идейно, близко к CLRS)

StackInit(S):

1. $S.\text{head} \leftarrow \text{NIL}$
2. $S.\text{size} \leftarrow 0$

Push(S, x):

1. $v \leftarrow \text{new_node}()$
2. $v.\text{value} \leftarrow x$
3. $v.\text{next} \leftarrow S.\text{head}$
4. $S.\text{head} \leftarrow v$
5. $S.\text{size} \leftarrow S.\text{size} + 1$

Top(S):

1. **require** $S.\text{head} \neq \text{NIL}$
2. **return** $S.\text{head}.\text{value}$

Pop(S):

1. **require** $S.\text{head} \neq \text{NIL}$
2. $\text{old} \leftarrow S.\text{head}$
3. $S.\text{head} \leftarrow \text{old}.\text{next}$
4. **delete**(old)
5. $S.\text{size} \leftarrow S.\text{size} - 1$

Empty(S):

1. **return** ($S.\text{head} = \text{NIL}$)

Замечание. Это именно «стек на списке»: все операции реализуются изменением головы односвязного списка. На практике стек часто реализуют и на динамическом массиве, но в этом билете по условию рассматривается списковая реализация.

32 Билет №32. Проверка правильности скобочной последовательности с несколькими типами скобок

Задача. Дана строка s длины n , состоящая из символов нескольких типов скобок (например, $()$, $[]$, $\{\}$, $\langle\rangle$). Требуется определить, является ли s правильной скобочной последовательностью (ПСП).

Определение 32.1 (ПСП для нескольких типов скобок). Множество правильных скобочных последовательностей (ПСП) над фиксированным набором парных скобок определяется индуктивно:

1. пустая строка — ПСП;
2. если t — ПСП и $\langle\cdot\rangle$ — одна из пар скобок (например, $()$, $[]$, $\{\}$), то строка $\langle t \rangle$ — ПСП;
3. если t_1 и t_2 — ПСП, то их конкатенация $t_1 t_2$ — ПСП;
4. никаких других строк не считаем ПСП.

Замечание. Эквивалентная (и очень полезная) формулировка: строка является ПСП тогда и только тогда, когда можно разбить её скобки на пары *одного типа* так, что

1. в каждой паре открывающая стоит левее закрывающей;
2. каждой скобке соответствует ровно одна парная;
3. пары *не пересекаются* (т.е. не бывает картинок вида $i < j < \pi(i) < \pi(j)$).

а.к.а. «все скобки можно сопоставить во вложенном порядке».

Алгоритм (стек)

Решение. Пройдём строку слева направо, поддерживая стек (LIFO) открывающих скобок, которые ещё не получили пару.

- ▷ Если текущий символ — открывающая скобка, кладём её в стек.
- ▷ Если текущий символ — закрывающая скобка:
 - если стек пуст, то закрывать нечего — ответ NO;
 - иначе смотрим верх стека: он обязан быть открывающей скобкой *того же типа*; если типы не совпали — NO;
 - если совпали — снимаем верх стека (пара найдена).

После обработки всей строки ответ YES тогда и только тогда, когда стек пуст.

Инвариант и корректность

Утверждение 32.1 (Инвариант стека). *После обработки префикса $s[1..i]$ стек содержит в точности те и только те открывающие скобки этого префикса, которые ещё не сопоставлены с закрывающими внутри $s[1..i]$. Причём порядок в стеке соответствует их порядку в строке: верх стека — последняя (самая правая) из ещё не закрытых открывающих скобок.*

△ *Доказательство.* Докажем по индукции по i .

База: $i = 0$. Префикс пуст, несопоставленных открывающих нет, стек пуст.

Переход: рассмотрим символ $s[i]$.

- ▷ Если $s[i]$ — открывающая скобка, то она становится новой «последней незакрытой», и алгоритм кладёт её на вершину стека. Инвариант сохраняется.
- ▷ Если $s[i]$ — закрывающая скобка, то по определению вложенности она (если вообще может быть сопоставлена) обязана закрывать *последнюю* незакрытую открывающую скобку (иначе возникло бы пересечение пар). По предположению индукции эта скобка находится на вершине стека.

Если стек пуст или типы не совпали, то сопоставление невозможно — алгоритм корректно завершает работу с ответом NO. Если типы совпали, алгоритм снимает вершину стека, тем самым делая пару сопоставленной внутри префикса. Инвариант сохраняется.

■

Теорема 32.1 (Корректность алгоритма). *Алгоритм со стеком выводит YES тогда и только тогда, когда входная строка является ПСП.*

△ *Доказательство.* ($\Rightarrow$) Пусть алгоритм вывел YES. Тогда он ни разу не столкнулся с ошибкой, а в конце стек пуст. Рассмотрим процесс сопоставления: каждый раз, когда встречалась закрывающая скобка, алгоритм сопоставлял её с вершиной стека и удалял эту открывающую. Так мы получаем разбиение всех скобок на пары одинаковых типов. По инварианту (вершина стека — последняя незакрытая) пары не могут пересекаться: каждая закрывающая закрывает «самую внутреннюю» доступную открывающую. Следовательно, существует непересекающееся паросочетание, значит строка — ПСП (см. замечание после определения).

($\Leftarrow$) Пусть строка s — ПСП. Тогда существует непересекающееся сопоставление пар скобок. Рассмотрим любой шаг алгоритма на закрывающей скобке. Она должна закрывать некоторую открывающую; из-за непересечения пар эта открывающая является последней незакрытой среди уже встреченных. По инварианту она находится на вершине стека и имеет тот же тип. Значит алгоритм никогда не ошибётся (не увидит пустой стек и не встретит несоответствие типов). В конце все открывающие будут закрыты, поэтому стек опустеет, и алгоритм выведет YES. ■

Асимптотика

Утверждение 32.2. Алгоритм работает за $O(n)$ по времени и использует $O(n)$ памяти в худшем случае.

△ Доказательство. Каждый символ обрабатывается за $O(1)$: максимум одна операция `push` или `pop` и несколько сравнений. Всего n символов, значит время $O(n)$. Размер стека не превосходит числа открывающих скобок в строке, т.е. $O(n)$. ■

Псевдокод (идейно, близко к CLRS)

Псевдокод (идейно, близко к CLRS)

MatchingOpen(c):

- ▷ $) \mapsto ($
- ▷ $] \mapsto [$
- ▷ $\} \mapsto \{$
- ▷ $> \mapsto <$

IsCorrectBrackets(s):

1. создать пустой стек S
2. **for each** символ c в s слева направо:
 - (a) **if** c — открывающая скобка, то **push**(S, c)
 - (b) **else if** c — закрывающая скобка:
 - i. **if** S пуст, то **return NO**
 - ii. **if** $\text{top}(S) \neq \text{MatchingOpen}(c)$, то **return NO**
 - iii. **pop**(S)
 - (c) **else return NO** // если по условию допустимы только скобки
3. **if** S пуст, то **return YES** **else return NO**

33 Билет №33. Поддержка минимума в стеке

Задача. Реализовать структуру данных *стек*, поддерживающую операции *push(x)*, *pop()*, *top()* и дополнительную операцию *getMin()* — вернуть минимальный элемент среди всех элементов, находящихся в стеке в данный момент. Требуется обеспечить время работы каждой операции $O(1)$.

33.1 Идея

Обычный стек (LIFO) не хранит никаких агрегатов, поэтому минимум пришлось бы искать линейным перебором за $O(n)$. Чтобы отвечать за $O(1)$, будем хранить вместе со стеком дополнительную информацию о минимуме.

Самый простой и надежный способ: поддерживать **второй стек минимумов**, синхронизированный по размеру с основным.

33.2 Структура данных

Поддерживаем два стека:

- ▷ S — основной стек значений;
- ▷ M — стек минимумов.

Интуитивно M хранит минимум на каждом «префиксе» стека S : на той же глубине, где в S лежит очередной элемент, в M лежит минимум среди всех элементов S ниже или на этой глубине.

33.3 Алгоритмы операций

Пусть $\min(a, b)$ — минимум двух чисел.

1. $\text{push}(x)$

- ▷ Положить x в S .
- ▷ Если M пуст, положить в M значение x .
- ▷ Иначе положить в M значение $\min(x, \text{top}(M))$.

2. $\text{pop}()$

- ▷ Удалить верхний элемент из S .
- ▷ Удалить верхний элемент из M .

3. $\text{top}()$

- ▷ Вернуть $\text{top}(S)$.

4. $\text{getMin}()$

- ▷ Вернуть $\text{top}(M)$.

Замечание. Операции `pop()`, `top()`, `getMin()` считаются корректными только если стек непуст. Если нужно обрабатывать пустой стек, добавьте проверку и договоритесь о поведении (например, бросать исключение или возвращать специальное значение).

33.4 Инвариант и корректность

Утверждение 33.1. После любой последовательности корректных операций выполняются свойства:

1. размеры стеков равны: $|S| = |M|$;
2. если $|S| > 0$, то $\text{top}(M) = \min(S)$, то есть верхушка M равна минимуму среди всех элементов стека S .

△ *Доказательство.* Докажем по индукции по числу выполненных операций.

База. Изначально оба стека пусты. Тогда $|S| = |M| = 0$, а второе свойство неприменимо.

Переход. Предположим, что инвариант верен перед очередной операцией.

- ▷ `push(x)`: в S добавляется один элемент, и ровно один элемент добавляется в M , значит $|S| = |M|$ сохраняется. Новый верхний элемент M равен $\min(x, \text{top}(M)_{\text{до}})$. По предположению индукции $\text{top}(M)_{\text{до}} = \min(S_{\text{до}})$, поэтому $\text{top}(M)_{\text{после}} = \min(x, \min(S_{\text{до}})) = \min(S_{\text{после}})$.
- ▷ `pop()`: операция удаляет по одному элементу из S и M , значит $|S| = |M|$ сохраняется. После удаления верхнего элемента, по определению построения M следующий сверху элемент в M и есть минимум для укороченного стека S , то есть $\text{top}(M) = \min(S)$.
- ▷ `top()` и `getMin()` не меняют структуру, инвариант сохраняется.

Таким образом, инвариант сохраняется после любой операции. Следовательно, `getMin()` всегда возвращает минимум в стеке. ■

33.5 Асимптотика

Каждая операция делает константное число действий со стеком (вставка/удаление/чтение верхушки) и одно сравнение, поэтому:

- ▷ время работы каждой операции — $O(1)$;
- ▷ дополнительная память — $O(n)$, где n — текущее число элементов в стеке.

Замечание. Если базовый стек реализован связным списком, то `push/pop/top` работают за $O(1)$ в худшем случае. Если использовать динамический массив (например, `std::vector`), то `push` и `pop` обычно имеют $O(1)$ амортизированное время из-за редких реаллокаций.

33.6 Псевдокод (идейно, близко к CLRS)

Псевдокод (идейно, близко к CLRS)

Init:

1. создать два пустых стека: основной S и стек минимумов M

Push(x):

1. push(S , x)
2. if M пуст then push(M , x)
3. else push(M , min(x , top(M)))

Pop():

1. require S непуст
2. pop(S)
3. pop(M)

Top():

1. require S непуст
2. return top(S)

GetMin():

1. require M непуст
2. return top(M)

33.7 Замечание про альтернативы

Иногда требуется экономить память до $O(1)$ «сверх стека». Существуют трюки с кодированием разности относительно текущего минимума, но они требуют аккуратной работы с переполнениями и обычно не нужны в стандартных задачах. В учебных целях и на экзамене наиболее ожидаемое решение — описанное выше с дополнительным стеком минимумов.

34 Билет №34. Структура данных «очередь»: реализация списком

34.1 Формулировка

Определение 34.1. *Очередь* (queue) — структура данных, поддерживающая упорядоченную последовательность элементов в режиме FIFO (*first in — first out*): элемент, добавленный раньше, извлекается раньше.

Стандартный набор операций:

- ▷ **push(*x*)** (enqueue) — добавить элемент *x* в *конец* очереди;
- ▷ **pop()** (dequeue) — удалить элемент *из начала* очереди;
- ▷ **front()** — посмотреть элемент *в начале* очереди (без удаления).

Дополнительно обычно поддерживают **empty()** и **size()**.

34.2 Идея реализации

Очередь удобно реализовать связным списком: элементы идут слева направо в порядке обслуживания,

удаление происходит с головы, *добавление* — в хвост.

Определение 34.2. **Односвязный список** — это вершины (узлы), где каждый узел хранит значение и указатель на следующий узел.

Чтобы push работал за $O(1)$, недостаточно хранить только голову: нужно хранить ещё и хвост.

Иначе пришлось бы каждый раз пробегать список до конца, что давало бы $O(n)$ на добавление.

34.3 Структура данных и инварианты

Будем хранить:

- ▷ **head** — указатель на первый узел (начало очереди);
- ▷ **tail** — указатель на последний узел (конец очереди);
- ▷ (опционально) **sz** — текущее количество элементов.

Узел (Node) для односвязного списка (псевдокод):

Представление (псевдокод)

Node:

- ▷ **val** — значение;
- ▷ **next** — ссылка на следующий узел (или NIL).

Утверждение 34.1 (Инварианты представления). *Во все моменты времени выполняется:*

1. Если очередь пуста, то $head = nullptr$ и $tail = nullptr$.
2. Если очередь непуста, то $head$ указывает на первый элемент, $tail$ — на последний, причём $tail \rightarrow next = nullptr$.
3. Порядок узлов при обходе по $next$ от $head$ к $tail$ совпадает с порядком элементов в очереди (FIFO).

34.4 Алгоритмы операций

34.4.1 Операция front

- ▷ Предусловие: очередь не пуста.
- ▷ Ответ: значение $head \rightarrow val$.

34.4.2 Операция push(x)

Создаём новый узел v со значением x и $v \rightarrow next = nullptr$.

- ▷ Если очередь пуста: $head = tail = v$.
- ▷ Иначе: $tail \rightarrow next = v$; $tail = v$.

(Если ведём sz , то $++sz$.)

34.4.3 Операция pop()

Удаляем первый узел.

- ▷ Предусловие: очередь не пуста.
- ▷ Пусть $v = head$. Тогда делаем $head = head \rightarrow next$ и удаляем v .
- ▷ Если после этого $head == nullptr$, то очередь стала пустой, поэтому нужно сделать $tail = nullptr$.

(Если ведём sz , то $-sz$.)

34.5 Доказательство корректности

Докажем, что описанная реализация корректно поддерживает FIFO и всегда удовлетворяет инвариантам.

Лемма 34.1 (Корректность push). *После выполнения $push(x)$ инварианты сохраняются, а новый элемент x становится последним в очереди.*

△ *Доказательство.* Рассмотрим случаи.

1) *Очередь пуста.* Тогда до операции `head = tail = nullptr`. Мы создаём единственный узел v и присваиваем `head = tail = v`. Получаем непустую очередь из одного элемента, причём `tail->next = nullptr`. Порядок FIFO очевиден.

2) *Очередь непуста.* Тогда `tail` указывает на последний узел и `tail->next = nullptr`. Мы делаем `tail->next = v` и `tail = v`. В результате новый узел становится последним, список остаётся связным, а `tail->next` снова равен `nullptr`. Относительный порядок старых элементов не меняется, а x добавлен строго *после* них, то есть FIFO сохраняется. ■

Лемма 34.2 (Корректность `pop`). *Если очередь непуста, то `pop()` удаляет ровно первый (самый ранний среди ещё не удалённых) элемент, и инварианты сохраняются.*

△ *Доказательство.* Удаляется узел, на который указывает `head`, то есть первый элемент очереди. Остальные элементы остаются в том же порядке, потому что мы лишь сдвигаем `head` на `head->next`.

Если после сдвига `head` стал `nullptr`, значит удалён был единственный элемент, и очередь должна стать пустой; установка `tail = nullptr` восстанавливает инвариант пустой очереди. В противном случае хвост не изменяется, а `tail->next` как был `nullptr`, так и остаётся `nullptr`. ■

Утверждение 34.2 (FIFO-свойство). *Для любой последовательности операций корректного типа (не делать `pop/front` на пустой очереди) элементы извлекаются в порядке добавления (FIFO).*

△ *Доказательство.* Единственная операция, меняющая порядок, — это `push`, но она всегда добавляет элемент *в конец*. Единственная операция, удаляющая элементы, — `pop`, и она всегда удаляет *из начала*. Следовательно, элемент может быть удалён только после того, как удалены все элементы, стоявшие перед ним, то есть в порядке FIFO. ■

34.6 Асимптотика

Утверждение 34.3. *Операции `push`, `pop`, `front`, `empty` выполняются за $O(1)$ времени. Память — $O(n)$, где n — текущий размер очереди.*

△ *Доказательство.* Каждая операция изменяет/читает константное число указателей (`head`, `tail`, `next`) и, при необходимости, одну вершину создаёт или удаляет. Следовательно, время $O(1)$. Память равна числу узлов списка, то есть $O(n)$. ■

34.7 Практические замечания (C++)

- ▷ В учебных задачах обычно гарантируется, что `pop` и `front` не вызываются на пустой очереди. В реальном коде это оформляют либо через `assert(!empty())`, либо через исключение.
- ▷ Чтобы не было утечек памяти, при разрушении очереди нужно удалить все узлы (например, циклом `while (!empty()) pop();`).

- ▷ Альтернатива без условных веток: использовать *фиктивный* (*sentinel*) узел в начале списка. Тогда пустая очередь — это `head == tail == dummy`, а операции становятся короче.

Замечание. Реализация очереди списком идеально даёт $O(1)$ на базовые операции. Но если к очереди добавить запросы вроде `getMin` (минимум среди всех элементов), то «в лоб» это уже не поддержать за $O(1)$ при `push/pop`; для этого используют другие идеи (например, очередь на двух стеках с поддержкой минимумов).

Псевдокод (каркас очереди)

QueueInit(Q):

1. $Q.head \leftarrow \text{NIL}$
2. $Q.tail \leftarrow \text{NIL}$
3. $Q.sz \leftarrow 0$

Empty(Q): return ($Q.head = \text{NIL}$).

Front(Q):

1. require $Q.head \neq \text{NIL}$
2. return $Q.head.val$

Push(Q, x):

1. $v \leftarrow \text{new_node}(); v.val \leftarrow x; v.next \leftarrow \text{NIL}$
2. if $Q.tail = \text{NIL}$ then
 - (a) $Q.head \leftarrow v; Q.tail \leftarrow v$else
 - (a) $Q.tail.next \leftarrow v$
 - (b) $Q.tail \leftarrow v$
3. $Q.sz \leftarrow Q.sz + 1$

Pop(Q):

1. require $Q.head \neq \text{NIL}$
2. $old \leftarrow Q.head$
3. $Q.head \leftarrow old.next$
4. delete(old)
5. if $Q.head = \text{NIL}$ then $Q.tail \leftarrow \text{NIL}$
6. $Q.sz \leftarrow Q.sz - 1$

Замечание (Фиктивный (sentinel) узел). Часто удобно завести «пустой» фиктивный узел перед первым элементом. Тогда условие «очередь пуста» сводится к `dummy.next == nullptr`, а `tail` можно держать равным `&dummy` в пустом состоянии. Это убирает часть ветвлений, но асимптотика и идеи остаются теми же.

35 Билет №35. Очередь на двух стеках. Амортизированная асимптотика. Минимум в очереди

35.1 Постановка и интерфейс

Определение 35.1. Очередь (queue) — абстрактный тип данных, поддерживающий операции:

- ▷ `push(x)` (`enqueue`): добавить элемент x в *конец* очереди;
- ▷ `pop()` (`dequeue`): удалить элемент из *начала* очереди;
- ▷ `front()`: посмотреть элемент в *начале* очереди.

Очередь работает по принципу FIFO: *First In — First Out*.

В билете требуется реализовать очередь, используя только операции стека (LIFO): `push`, `pop`, `top`.

35.2 Идея: два стека `in` и `out`

Храним два стека:

- ▷ `in` (или `A`) — сюда складываем все добавляемые элементы;
- ▷ `out` (или `D`) — отсюда извлекаем элементы из начала очереди.

Инвариант порядка. Если `out` не пуст, то его вершина — это `front()` очереди. Если `out` пуст, то порядок в очереди целиком задан элементами `in` (старые элементы лежат ближе к дну `in`).

35.3 Алгоритм операций

Опишем вспомогательную процедуру `shift()`, которая переносит элементы из `in` в `out` в обратном порядке:

`shift()`:

1. если `out` не пуст, ничего не делать;
2. иначе, пока `in` не пуст: `out.push(in.top())`, затем `in.pop()`.

Теперь операции очереди:

`push(x)`: `in.push(x)`.

`front()`: вызвать `shift()`, затем вернуть `out.top()`.

`pop()`: вызвать `shift()`, затем `out.pop()`.

35.4 Корректность

Утверждение 35.1. *Описанная структура корректно реализует FIFO-очередь.*

△ *Доказательство.* Рассмотрим произвольный момент времени.

- ▷ Если **out** не пуст, то элементы в **out** расположены в *правильном порядке извлечения*: вершина **out** — самый “старый” среди элементов, находящихся в **out**. Действительно, при переносе из **in** в **out** мы снимаем элементы с вершины **in** (от новых к старым) и кладём их на вершину **out**, поэтому в **out** порядок становится обратным: старые оказываются ближе к вершине.
- ▷ Элементы, которые ещё не переносились, находятся в **in**. Они добавлялись операцией **push** и потому лежат в **in** в порядке добавления: самый новый на вершине, самый старый ближе к дну.

Если **out** не пуст, то **front()** должен вернуть элемент, который был добавлен раньше всех среди текущих элементов очереди. Этот элемент либо уже лежит в **out** (и тогда он на вершине **out**), либо **out** пуст.

Если **out** пуст, то все элементы очереди лежат в **in**. Вызов **shift()** переносит все элементы из **in** в **out**, при этом самый старый элемент (лежащий внизу **in**) становится вершиной **out**. Следовательно, после **shift()** операции **front()** и **pop()** обращаются ровно к тому элементу, который должен быть в начале FIFO-очереди. ■

35.5 Асимптотика: амортизированное $O(1)$

Заметим, что отдельная операция **pop()** может вызвать перенос k элементов из **in** в **out** и занять $\Theta(k)$ времени. Тем не менее, *амортизированно* каждая операция работает за $O(1)$.

Утверждение 35.2. *Для любой последовательности из q операций **push/pop/front** суммарное число стековых операций (**push/pop** на стеках) равно $O(q)$. Следовательно, амортизированная стоимость одной операции равна $O(1)$.*

△ *Доказательство.* Рассмотрим жизненный цикл одного элемента x :

1. при **push**(x) он один раз кладётся в стек **in**;
2. возможно (и не более одного раза) он переносится из **in** в **out**: это одна операция **pop** из **in** и одна операция **push** в **out**;
3. при удалении из очереди он один раз снимается **pop** из **out**.

Итого, каждый элемент участвует в $O(1)$ элементарных стековых операциях (не более 4). Значит, суммарно по всем элементам, появившимся из q операций, получаем $O(q)$ стековых операций. Делим на q — получаем амортизированную стоимость $O(1)$. ■

Замечание. Альтернатива доказательству выше — метод потенциалов с потенциалом $\Phi = |\mathbf{in}|$: **push** увеличивает потенциал на 1, а перенос в **shift()** резко уменьшает потенциал и “оплачивает” перенос элементов.

35.6 Минимум в стеке

Теперь добавим поддержку запроса `getMin()` — минимум среди всех элементов структуры.

Определение 35.2. Стек с минимумом поддерживает операции `push`, `pop`, `top` и `getMin` за $O(1)$.

Один из стандартных способов:

- ▷ основной стек хранит значения;
- ▷ дополнительный стек `mn` хранит минимумы на префиксах: на вершине `mn` лежит минимум всех элементов основного стека.

Правило обновления. При `push(x)` кладём в `mn` значение `min(x, mn.top())` (если `mn` пуст, кладём `x`). При `pop()` удаляем верхний элемент и из основного стека, и из `mn`. Тогда `getMin()` возвращает `mn.top()`.

35.7 Минимум в очереди на двух стеках

Хотим поддерживать очередь с операциями `push`, `pop`, `front` и `getMin`.

Идея. Сделаем оба стека (`in` и `out`) стеками с минимумом. Тогда минимум во всей очереди равен минимуму среди элементов, лежащих в `in`, и элементов, лежащих в `out`:

$$\text{min(очередь)} = \text{min}(\text{min(in)}, \text{min(out)}).$$

Утверждение 35.3. Операция `getMin()` в такой очереди выполняется за $O(1)$ (в худшем случае).

△ *Доказательство.* Если один из стеков пуст, минимум равен минимуму другого. Если оба непусты, ответ равен минимуму двух чисел: `in.getMin()` и `out.getMin()`, что занимает $O(1)$. ■

Замечание про перенос. Процедура `shift()` (перекладывание из `in` в `out`) остаётся прежней, только теперь мы перекладываем элементы через операции стека с минимумом: снимаем с `in` и кладём в `out`. Это автоматически корректно поддерживает минимумы в обоих стеках.

35.8 Итоговая асимптотика

- ▷ `push` — $O(1)$ в худшем случае.
- ▷ `front`, `pop` — $O(1)$ амортизированно (в худшем случае может быть $\Theta(n)$ из-за `shift`).
- ▷ `getMin` — $O(1)$ в худшем случае.

35.9 Псевдокод: очередь на двух стеках с минимумом

Псевдокод (идейно, близко к CLRS)

MinStack (стек с минимумом).

1. **Состояние:** два стека S (значения) и MN (минимумы).
2. **Empty()**: вернуть (S пуст).
3. **Push(x)**:
 - (a) $S.push(x)$.
 - (b) если MN пуст, то $MN.push(x)$, иначе $MN.push(\min(x, MN.top()))$.
4. **Top()**: вернуть $S.top()$.
5. **Pop()**: $S.pop()$; $MN.pop()$.
6. **GetMin()**: вернуть $MN.top()$.

MinQueue (очередь на двух стеках с минимумом).

1. **Состояние:** два стека с минимумом in и out .
2. **Shift()**:
 - (a) если out не пуст, **return**.
 - (b) пока in не пуст:
 - i. $x \leftarrow in.Top()$; $in.Pop()$.
 - ii. $out.Push(x)$.
3. **Empty()**: вернуть ($in.Empty()$ и $out.Empty()$).
4. **Push(x)**: $in.Push(x)$.
5. **Front()**: **Shift()**; вернуть $out.Top()$.
6. **Pop()**: **Shift()**; $out.Pop()$.
7. **GetMin()**:
 - (a) если $in.Empty()$, вернуть $out.GetMin()$.
 - (b) если $out.Empty()$, вернуть $in.GetMin()$.
 - (c) вернуть $\min(in.GetMin(), out.GetMin())$.

Замечание. Очередь с минимумом — стандартный инструмент для задачи *минимум на скользящем окне*: двигаем окно длины k по массиву, добавляя справа и удаляя слева, и каждый раз спрашиваем `getMin`. Тогда весь алгоритм работает за $O(n)$.

36 Билет №36. Sparse Table: модельная задача, построение за $O(n \log n)$, ответ на запрос за $O(1)$

Задача. Дан *статический* массив $a[0 \dots n-1]$ (обновлений нет) и q запросов вида:

$$query(l, r) : \text{найти } \min\{a[l], a[l+1], \dots, a[r-1]\} \quad (0 \leq l < r \leq n).$$

Нужно после предподсчёта отвечать на запросы быстро.

Замечание. В качестве модельной задачи почти всегда рассматривают **RMQ** (Range Minimum Query). Однако Sparse Table работает не только для $\min$, а для любой операции $\circ$ со следующими свойствами:

▷ **ассоциативность:** $(x \circ y) \circ z = x \circ (y \circ z)$;

▷ **идемпотентность:** $x \circ x = x$.

Примеры: $\min$, $\max$, $\gcd$. Для суммы операция не идемпотентна, поэтому классический запрос за $O(1)$ через Sparse Table не работает.

36.1 Определение разреженной таблицы

Пусть $K = \lfloor \log_2 n \rfloor$. Построим таблицу

$$st[k][i] = \min(a[i \dots i + 2^k]), \quad 0 \leq k \leq K, 0 \leq i \leq n - 2^k.$$

То есть $st[k][i]$ хранит ответ на отрезке длины 2^k , начинающемся в i .

36.2 Построение за $O(n \log n)$

База:

$$st[0][i] = a[i].$$

Переход (динамика по длине):

$$st[k][i] = st[k-1][i] \circ st[k-1][i + 2^{k-1}].$$

Действительно, отрезок $[i, i + 2^k)$ распадается на два соседних отрезка длины 2^{k-1} :

$$[i, i + 2^{k-1}) \cup [i + 2^{k-1}, i + 2^k).$$

Для $\min$ это:

$$\min(a[i \dots i + 2^k]) = \min(\min(a[i \dots i + 2^{k-1}]), \min(a[i + 2^{k-1} \dots i + 2^k])).$$

36.3 Запрос за $O(1)$

Для запроса $[l, r)$ положим $\text{len} = r - l$ и $k = \lfloor \log_2(\text{len}) \rfloor$. Рассмотрим два отрезка длины 2^k :

$$[l, l + 2^k) \quad \text{и} \quad [r - 2^k, r).$$

Они **покрывают** весь $[l, r)$ (могут пересекаться). Тогда ответ:

$$\text{query}(l, r) = \text{st}[k][l] \circ \text{st}[k][r - 2^k].$$

Для $\min$:

$$\min(a[l \dots r]) = \min(\text{st}[k][l], \text{st}[k][r - 2^k]).$$

Утверждение 36.1 (Корректность ответа на запрос). Пусть операция $\circ$ ассоциативна и идемпотентна. Тогда формула

$$\text{ans} = \text{st}[k][l] \circ \text{st}[k][r - 2^k], \quad k = \lfloor \log_2(r - l) \rfloor,$$

возвращает $a[l] \circ a[l + 1] \circ \dots \circ a[r - 1]$.

$\triangle$ Доказательство. Обозначим $I = [l, r)$, $A = [l, l + 2^k)$, $B = [r - 2^k, r)$. По выбору k имеем $2^k \leq |I| < 2^{k+1}$, откуда $l + 2^k \geq r - 2^k$, то есть $A \cup B = I$ (покрытие, возможно с пересечением).

По определению таблицы $\text{st}[k][l]$ равен свёртке $\circ$ по всем элементам на A , а $\text{st}[k][r - 2^k]$ — по всем элементам на B . Тогда ans равен свёртке по мультимножеству элементов $A \cup B$. Если A и B пересекаются, элементы пересечения учитываются дважды. Но из идемпотентности $x \circ x = x$ следует, что двойной учёт не влияет на результат. Значит, ans совпадает со свёрткой по I , то есть является правильным ответом на запрос. ■

36.4 Асимптотика и память

- ▷ Построение: $\sum_{k=0}^K (n - 2^k + 1) = O(n \log n)$ операций.
- ▷ Запрос: $O(1)$ (при наличии $\lfloor \log_2 \rfloor$ за $O(1)$).
- ▷ Память: $O(n \log n)$.

36.5 Как получать $\lfloor \log_2 \rfloor$ за $O(1)$

Обычно заранее считают массив $\lg[i] = \lfloor \log_2 i \rfloor$ для $i \geq 1$:

$$\lg[1] = 0, \quad \lg[i] = \lg[\lfloor i/2 \rfloor] + 1 \quad (i \geq 2).$$

Тогда $k = \lg[r - l]$.

36.6 Типичные ошибки

- ▷ **Сумма и другие неидемпотентные операции.** Для суммы формула с двумя отрезками неверна из-за перекрытия. (Сумму на статическом массиве за $O(1)$ проще делать префиксами.)

- ▷ **Границы индексов.** Таблица $\text{st}[k][i]$ определена только при $i + 2^k \leq n$.
- ▷ **Смешивание $[l, r]$ и $[l, r)$.** Удобнее фиксировать полуинтервал $[l, r)$ и везде использовать $\text{len} = r - l$.

36.7 Псевдокод построения и запроса

Псевдокод (идейно, близко к CLRS)

BuildSparseTable($a[0 \dots n - 1]$):

1. $K \leftarrow \lfloor \log_2 n \rfloor$.
2. **Предподсчёт логарифмов:** $\text{lg}[1] \leftarrow 0$; для $i = 2..n$ положить $\text{lg}[i] \leftarrow \text{lg}[\lfloor i/2 \rfloor] + 1$.
3. Для $i = 0..n - 1$ положить $\text{st}[0][i] \leftarrow a[i]$.
4. Для $k = 1..K$:
 - (a) $\text{len} \leftarrow 2^k$, $\text{half} \leftarrow 2^{k-1}$.
 - (b) Для $i = 0..n - \text{len}$ положить

$$\text{st}[k][i] \leftarrow \min(\text{st}[k-1][i], \text{st}[k-1][i + \text{half}]).$$

Query(l, r) (RMQ на полуинтервале $[l, r)$):

1. $\text{len} \leftarrow r - l$, $k \leftarrow \text{lg}[\text{len}]$.
2. Вернуть

$$\min(\text{st}[k][l], \text{st}[k][r - 2^k]).$$

Замечание. Для произвольной операции $\circ$ с ассоциативностью и идемпотентностью замените $\min$ на $\circ$ и используйте ту же схему.

Замечание. Если вместо $\min$ нужна другая операция $\circ$ (например, gcd), замените $\min$ на неё и убедитесь, что операция идемпотентна.

37 Билет №37. Дерево отрезков: модельная задача. Обработка запросов и доказательство времени работы

37.1 Модельная задача

Дан массив $a[0 \dots n - 1]$. Нужно обрабатывать запросы:

- ▷ `update(pos, x)`: присвоить $a[\text{pos}] \leftarrow x$ (точечное обновление);
- ▷ `sum(l, r)`: найти сумму $\sum_{i=l}^r a[i]$ (запрос на отрезке).

Дерево отрезков решает эту задачу за $O(\log n)$ на запрос при $O(n)$ памяти.

37.2 Определение дерева отрезков

Определение 37.1. *Дерево отрезков* для массива $a[0 \dots n - 1]$ — это полное двоичное дерево, каждой вершине v которого сопоставлен подотрезок $[t_l, t_r]$ индексов.

- ▷ Корень соответствует $[0, n - 1]$.
- ▷ Если вершине соответствует $[t_l, t_r]$ и $t_l \neq t_r$, то положим $t_m = \lfloor (t_l + t_r) / 2 \rfloor$. Тогда левый сын соответствует $[t_l, t_m]$, правый — $[t_m + 1, t_r]$.
- ▷ Листья соответствуют отрезкам длины 1, то есть $[i, i]$.

37.3 Что храним: инвариант

Для модельной задачи в каждой вершине v (которая отвечает отрезку $[t_l, t_r]$) будем хранить значение

$$t[v] = \sum_{i=t_l}^{t_r} a[i].$$

Утверждение 37.1 (Инвариант). *В любой момент времени для каждой вершины дерева отрезков выполняется равенство*

$$t[v] = \sum_{i=t_l}^{t_r} a[i],$$

где $[t_l, t_r]$ — отрезок, соответствующий вершине v .

37.4 Хранение в массиве

Дерево удобно хранить не указателями, а в одном массиве: корень имеет номер $v = 1$, левый сын — $2v$, правый — $2v + 1$. Практически безопасно выделять массив размера $4n$.

37.5 Построение

Решение. Функция $\text{build}(v, t_l, t_r)$ строит значения $t[v]$:

- ▷ если $t_l = t_r$, то $t[v] = a[t_l]$;
- ▷ иначе строим детей на $[t_l, t_m]$ и $[t_m + 1, t_r]$, затем $t[v] = t[2v] + t[2v + 1]$.

Утверждение 37.1 (Корректность построения). После выполнения $\text{build}(1, 0, n-1)$ инвариант выполнен во всех вершинах.

Δ Доказательство. Индукция по длине отрезка $[t_l, t_r]$. Если $t_l = t_r$, то вершина соответствует одному элементу и мы явно кладём $t[v] = a[t_l]$. Иначе отрезок разбивается на два непересекающихся подотрезка детей. По предположению индукции после построения детей верны равенства $t[2v] = \sum_{i=t_l}^{t_m} a[i]$ и $t[2v + 1] = \sum_{i=t_m+1}^{t_r} a[i]$, а значит $t[v] = t[2v] + t[2v + 1] = \sum_{i=t_l}^{t_r} a[i]$. ■

Утверждение 37.2 (Время построения). build работает за $O(n)$.

Δ Доказательство. Каждая вершина дерева вычисляется ровно один раз за $O(1)$ (после вычисления детей). Число вершин в двоичном дереве отрезков линейно по n (например, не больше $4n$ при хранении массивом), следовательно суммарное время $O(n)$. ■

37.6 Запрос суммы на отрезке

Опишем рекурсивную функцию $\text{sum}(v, t_l, t_r, l, r)$ — сумма на пересечении $[l, r]$ с текущим отрезком вершины.

Решение. Алгоритм:

- ▷ если $l > r$, вернуть 0;
- ▷ если $[l, r] = [t_l, t_r]$, вернуть $t[v]$;
- ▷ иначе $t_m = \lfloor (t_l + t_r) / 2 \rfloor$ и

$$\text{sum}(v, t_l, t_r, l, r) = \text{sum}(2v, t_l, t_m, l, \min(r, t_m)) + \text{sum}(2v + 1, t_m + 1, t_r, \max(l, t_m + 1), r).$$

Утверждение 37.3 (Корректность запроса). Функция $\text{sum}(1, 0, n-1, l, r)$ возвращает $\sum_{i=l}^r a[i]$.

Δ Доказательство. Рассмотрим произвольный вызов $\text{sum}(v, t_l, t_r, l, r)$. Если $l > r$, то пересечение пусто и сумма равна 0. Если $[l, r] = [t_l, t_r]$, то по инварианту ответ равен $t[v]$. Иначе отрезок $[l, r]$ разбивается на две части: левую $[l, \min(r, t_m)]$ и правую $[\max(l, t_m + 1), r]$, которые лежат в отрезках детей и не пересекаются. Сумма по $[l, r]$ равна сумме по двум частям, и рекурсивные вызовы возвращают их корректно. ■

37.7 Точечное обновление

Опишем функцию `update(v, t_l, t_r, pos, x)`.

Решение. Алгоритм:

- ▷ если $t_l = t_r$ (лист), то присваиваем $t[v] \leftarrow x$;
- ▷ иначе $t_m = \lfloor (t_l + t_r) / 2 \rfloor$:
 - если $\text{pos} \leq t_m$, рекурсивно обновляем левого сына;
 - иначе — правого сына;
- ▷ после обновления пересчитываем $t[v] = t[2v] + t[2v + 1]$.

Утверждение 37.4 (Корректность обновления). *После выполнения `update(1, 0, n-1, pos, x)` инвариант остаётся верным во всех вершинах.*

$\triangle$ *Доказательство.* Меняется ровно один лист $[\text{pos}, \text{pos}]$, в нём мы явно кладём новое значение. Далее по пути вверх пересчитываем суммы как сумму двух детей. Вне этого пути вершины и соответствующие им отрезки не затронуты. Следовательно, в каждой вершине снова хранится сумма своего отрезка. ■

37.8 Доказательство асимптотики запросов

Утверждение 37.5 (Высота дерева). *Высота дерева отрезков равна $O(\log n)$.*

$\triangle$ *Доказательство.* На каждом шаге спуска длина отрезка уменьшается хотя бы в 2 раза. Через h уровней длина станет ≤ 1 , значит $2^h \geq n$ и $h \leq \lceil \log_2 n \rceil$. ■

Утверждение 37.6 (Время `update`). *`update` работает за $O(\log n)$.*

$\triangle$ *Доказательство.* Алгоритм спускается ровно по одному пути от корня к листу (по сравнению с t_m однозначно выбирается сын). Длина пути не превосходит высоту дерева, то есть $O(\log n)$. На каждом уровне делается $O(1)$ работы. ■

Лемма 37.1 (Сколько вершин посещает `sum`). *На каждом фиксированном уровне дерева существует не более двух вершин, чьи отрезки частично пересекают запросный отрезок $[l, r]$.*

$\triangle$ *Доказательство.* Отрезки вершин одного уровня образуют разбиение $[0, n-1]$ на непесекающиеся интервалы. Множество интервалов этого уровня, имеющих непустое пересечение с $[l, r]$, образует непрерывный блок. Только первый и последний интервалы блока могут пересекаться *частично* (содержать одну из границ l или r). Все интервалы строго внутри блока лежат целиком в $[l, r]$. Значит, частичных пересечений не больше двух. ■

Утверждение 37.7 (Время `sum`). *`sum` работает за $O(\log n)$.*

$\triangle$ *Доказательство.* Рекурсия ветвится только в вершинах с частичным пересечением. По лемме на каждом уровне таких вершин не больше двух, значит на следующий уровень породится не более четырёх рекурсивных вызовов. Уровней всего $O(\log n)$, и каждый вызов делает $O(1)$ действий, поэтому суммарное время $O(\log n)$. ■

37.9 Замечания

- ▷ Рассуждения полностью переносятся на любую ассоциативную операцию с нейтральным элементом ($\min$, $\max$, $\gcd$, и т.д.): в вершине хранится значение операции на её отрезке.
- ▷ Если n не степень двойки, дерево строится теми же формулами $t_m = \lfloor (t_l + t_r)/2 \rfloor$; высота остаётся $O(\log n)$.

37.10 Псевдокод (дерево отрезков на суммах)

Псевдокод (идейно, близко к CLRS)

Build(v, t_l, t_r):

1. если $t_l = t_r$, то $t[v] \leftarrow a[t_l]$ и **return**.
2. $t_m \leftarrow \lfloor (t_l + t_r)/2 \rfloor$.
3. **Build**($2v, t_l, t_m$); **Build**($2v + 1, t_m + 1, t_r$).
4. $t[v] \leftarrow t[2v] + t[2v + 1]$.

Sum(v, t_l, t_r, l, r) (сумма на $[l, r]$ внутри текущего отрезка):

1. если $l > r$, **return** 0.
2. если $l = t_l$ и $r = t_r$, **return** $t[v]$.
3. $t_m \leftarrow \lfloor (t_l + t_r)/2 \rfloor$.
4. **return**

$$\mathbf{Sum}(2v, t_l, t_m, l, \min(r, t_m)) + \mathbf{Sum}(2v + 1, t_m + 1, t_r, \max(l, t_m + 1), r).$$

Update(v, t_l, t_r, pos, x) (присваивание $a[pos] \leftarrow x$):

1. если $t_l = t_r$, то $t[v] \leftarrow x$ и **return**.
2. $t_m \leftarrow \lfloor (t_l + t_r)/2 \rfloor$.
3. если $pos \leq t_m$, то **Update**($2v, t_l, t_m, pos, x$), иначе **Update**($2v + 1, t_m + 1, t_r, pos, x$).
4. $t[v] \leftarrow t[2v] + t[2v + 1]$.

38 Билет №38. Дерево отрезков: двоичный спуск, поиск подотрезка с суммой $\geq x$

38.1 Модельная задача

Дан массив $a[0 \dots n-1]$. В классической постановке для этого билета предполагается неотрицательность элементов: $a_i \geq 0$.

Нужно обрабатывать запросы:

- ▷ `set(i, v)`: присвоить $a_i := v$ (или `add(i, Δ)`);
- ▷ `find(l, x)`: найти минимальный индекс $r \geq l$ такой, что

$$\sum_{i=l}^r a_i \geq x.$$

Если такого r нет, вернуть -1 .

38.2 Сведение к поиску по префиксным суммам

Определим **включающую** префиксную сумму

$$S(r) = \sum_{i=0}^r a_i \quad (0 \leq r \leq n-1).$$

Тогда для $l > 0$ имеем

$$\sum_{i=l}^r a_i = S(r) - S(l-1),$$

а для $l = 0$ просто $\sum_{i=0}^r a_i = S(r)$.

Значит, запрос `find(l, x)` равносильен поиску минимального r такого, что

$$S(r) \geq S(l-1) + x \quad (S(-1) \stackrel{\text{def}}{=} 0).$$

Замечание. Условие $a_i \geq 0$ обеспечивает монотонность $S(r)$ по r . Без него «первое место, где сумма достигает порога» может не существовать или быть неустойчивым к разбиению на отрезки.

38.3 Дерево отрезков для сумм

Строим дерево отрезков над $[0, n)$, в каждой вершине храним сумму на соответствующем отрезке. Поддерживаем:

- ▷ точечное обновление за $O(\log n)$;
- ▷ сумму на отрезке $[l, r)$ за $O(\log n)$.

38.4 Двоичный спуск (binary descent) по дереву

Пусть нужно найти минимальный индекс r , для которого $S(r) \geq X$. Если $X > \sum_{i=0}^{n-1} a_i$, ответа нет.

Идея двоичного спуска:

- ▷ стоим в корне (отрезок $[0, n)$);
- ▷ знаем сумму левого сына S_{left} (на отрезке $[0, \text{mid})$);
- ▷ если $S_{\text{left}} \geq X$, то ответ лежит в левом поддереве;
- ▷ иначе ответ лежит в правом поддереве, а порог можно уменьшить: $X \leftarrow X - S_{\text{left}}$ (мы «пропускаем» весь левый отрезок целиком).

Повторяем, пока не дойдём до листа (конкретного элемента массива). Это и есть искомый индекс r .

Утверждение 38.1 (Корректность двоичного спуска для сумм). Пусть $a_i \geq 0$ и $1 \leq X \leq \sum_{i=0}^{n-1} a_i$. Алгоритм двоичного спуска по суммам возвращает минимальный $r \in [0, n)$, для которого $S(r) \geq X$.

△ *Доказательство.* Рассмотрим шаг спуска в вершине, соответствующей отрезку $[L, R)$, с серединой M . Пусть сумма на левом сыне (отрезок $[L, M)$) равна S_{left} .

Если $S_{\text{left}} \geq X$, то существует индекс внутри $[L, M)$, где включающая сумма нарастёт до X , и любой индекс в правой половине $[M, R)$ не может быть меньше минимального индекса в левой половине. Значит, минимальный ответ находится слева.

Если же $S_{\text{left}} < X$, то внутри левой половины суммарно «набирается» меньше X , поэтому ни один индекс из $[L, M)$ не может удовлетворить $S(\cdot) \geq X$; ответ обязан быть справа. Переходя вправо, мы вычитаем S_{left} , тем самым переводя порог в систему отсчёта правой половины.

Так на каждом шаге мы сохраняем инвариант «ответ лежит в текущем поддереве, а порог X корректен для него». В конце получаем лист, то есть единственный возможный индекс — это и есть минимальный ответ. ■

38.5 Как решить `find(l, x)`

1. Посчитаем $P = \sum_{i=0}^{l-1} a_i$ запросом суммы: $P = \text{sum}(0, l)$ (если $l = 0$, то $P = 0$).
2. Ищем минимальный r такой, что $S(r) \geq P + x$.
3. Если $P + x > \sum a_i$, то ответа нет.
4. Иначе найдём r двоичным спуском за $O(\log n)$.

Итого один запрос `find` работает за $O(\log n)$ (сумма + спуск).

38.6 Псевдокод (двоичный спуск по суммам)

Псевдокод (идейно, близко к CLRS)

LowerBoundPref(x) (первый индекс r , где $S(r) \geq x$):

1. **Предусловие:** $1 \leq x \leq \sum_{i=0}^{n-1} a_i$ и $a_i \geq 0$.
2. $v \leftarrow 1$ (корень).
3. **while** v не лист:
 - (a) $s \leftarrow \text{sum}(\text{left}(v))$ (сумма на левом сыне).
 - (b) **if** $s \geq x$ **then** $v \leftarrow \text{left}(v)$, **else** $x \leftarrow x - s$; $v \leftarrow \text{right}(v)$.
4. **return** индекс листа, соответствующий v .

FindSubsegment(l, x) (минимальный $r \geq l$ такой, что $\sum_{i=l}^r a_i \geq x$):

1. **if** $x \leq 0$ **then return** l .
2. $P \leftarrow \text{Sum}(0, l)$ (префикс до l).
3. $X \leftarrow P + x$ (глобальный порог для префикса).
4. **if** $X > \text{Sum}(0, n)$ **then return** -1 .
5. $r \leftarrow \text{LowerBoundPref}(X)$.
6. **return** r (если r попадает в исходный диапазон индексов; иначе -1).

Замечание. В реализации хранится дерево отрезков на суммах. Для двоичного спуска используется сумма левого ребёнка, а порог x при уходе вправо уменьшается на эту сумму.

38.7 Асимптотика

- ▷ Построение: $O(n)$.
- ▷ **set**: $O(\log n)$.
- ▷ **sum**(l, r): $O(\log n)$.
- ▷ **lower_bound_pref**: $O(\log n)$.
- ▷ **find**(l, x): $O(\log n)$.

38.8 Типичные ошибки

- ▷ Смешать включающие и «по длине» префиксы (где $S(r) = \sum_0^r$, а где $\sum_0^{r-1}$).
- ▷ Использовать алгоритм при отрицательных значениях a_i .

- ▷ Переполнение (используйте `long long` для сумм).
- ▷ Забыть проверить случай «ответа нет» ($X > t[1]$).

39 Билет №39. Дерево отрезков: отложенные операции. Присваивание на отрезке. Операция push

39.1 Постановка задачи

Дан массив $a[0 \dots n - 1]$ чисел. Нужно обрабатывать онлайн два типа запросов:

1. Присваивание на отрезке $[l, r]$:

$$\text{assign}(l, r, x) : \quad \forall i \in [l, r] \quad a[i] \leftarrow x.$$

2. Сумма на отрезке $[l, r]$:

$$\text{sum}(l, r) = \sum_{i=l}^r a[i].$$

Цель — обрабатывать каждый запрос за $O(\log n)$.

39.2 Напоминание: дерево отрезков

Определение 39.1. **Дерево отрезков** для массива длины n — бинарное дерево, вершина v которого соответствует некоторому отрезку индексов $[t_l, t_r]$. Корень соответствует всему массиву $[0, n - 1]$. Для внутренней вершины ($t_l < t_r$) определим $t_m = \lfloor (t_l + t_r) / 2 \rfloor$. Тогда дети вершины отвечают отрезкам $[t_l, t_m]$ и $[t_m + 1, t_r]$.

В этом билете в вершине хранится сумма на её отрезке:

$$\text{sum}[v] = \sum_{i=t_l}^{t_r} a[i].$$

Обычно дерево хранят в массиве: корень $v = 1$, дети $2v$ и $2v + 1$. Размер массива достаточно взять $4n$.

39.3 Идея отложенных операций

Присваивание на отрезке нельзя делать «в лоб» за $\Theta(r - l + 1)$. Вместо этого, если текущая вершина v целиком лежит в запросе $[l, r]$, мы не спускаемся в листья, а *помечаем* вершину как «весь её отрезок присвоен в x ». Детям эта информация будет передана только когда мы действительно захотим спуститься ниже.

39.4 Что храним в вершине

Для каждой вершины v (отрезок $[t_l, t_r]$) храним:

- ▷ $\text{sum}[v]$ — сумму на отрезке вершины (уже с учётом всех выполненных присваиваний);
- ▷ $\text{has}[v]$ — флаг, есть ли в вершине отложенное присваивание;
- ▷ $\text{set}[v]$ — значение x , в которое нужно присвоить весь отрезок вершины, если $\text{has}[v] = \text{true}$.

Замечание. Присваивание перезаписывает присваивание. Если на вершине уже висит отложенное присваивание в y , а мы делаем новое присваивание в x , то старое можно забыть: итог всё равно «присвоить в x ». Поэтому метка `set[v]` при новом присваивании просто заменяется.

39.5 Операции apply и push

Определение 39.2. `apply(v, tl, tr, x)` применяет присваивание ко всему отрезку вершины v :

$$\text{sum}[v] \leftarrow x \cdot (t_r - t_l + 1), \quad \text{set}[v] \leftarrow x, \quad \text{has}[v] \leftarrow \text{true}.$$

Определение 39.3. `push(v, tl, tr)` проталкивает отложенное присваивание из вершины v в её детей. Если `has[v]=false`, то `push` ничего не делает. Иначе (`has[v]=true`, `set[v]=x`):

▷ если $t_l = t_r$ (лист), просто сбрасываем `has[v]=false`;

▷ иначе $t_m = \lfloor (t_l + t_r) / 2 \rfloor$ и выполняем

$$\text{apply}(2v, t_l, t_m, x), \quad \text{apply}(2v+1, t_m+1, t_r, x),$$

затем сбрасываем `has[v]=false`.

39.6 Псевдокод запросов

Далее используем инклюзивные границы: $[l, r]$.

39.6.1 Присваивание на отрезке

```
assign(v, tl, tr, l, r, x):
    if l > r: return
    if l == tl and r == tr:
        apply(v, tl, tr, x)
        return
    push(v, tl, tr)
    tm = (tl + tr) / 2
    assign(2v, tl, tm, l, min(r, tm), x)
    assign(2v+1, tm+1, tr, max(l, tm+1), r, x)
    sum[v] = sum[2v] + sum[2v+1]
```

39.6.2 Сумма на отрезке

```
sum_query(v, tl, tr, l, r):
    if l > r: return 0
    if l == tl and r == tr:
        return sum[v]
    push(v, tl, tr)
```

```

tm = (tl + tr) / 2
return sum_query(2v, tl, tm, l, min(r, tm))
      + sum_query(2v+1, tm+1, tr, max(l, tm+1), r)

```

39.7 Корректность

Утверждение 39.1 (Инвариант). Для каждой вершины v (отрезок $[t_l, t_r]$) в любой момент времени верно:

1. $\text{sum}[v]$ равна сумме текущих значений массива на отрезке $[t_l, t_r]$.
2. Если $\text{has}[v]=\text{true}$ и $\text{set}[v]=x$, то все элементы на $[t_l, t_r]$ равны x .

Лемма 39.1 (Корректность push). Если инвариант верен до вызова $\text{push}(v, tl, tr)$, то после вызова он остаётся верен, а также у детей вершины v становится корректной информация, согласованная с присваиванием $\text{set}[v]$.

△ Доказательство. Если $\text{has}[v]=\text{false}$, то push ничего не меняет.

Пусть $\text{has}[v]=\text{true}$ и $\text{set}[v]=x$. По инварианту все элементы на $[t_l, t_r]$ равны x . Тогда на левом подотрезке $[t_l, t_m]$ все элементы тоже равны x , а сумма равна $x \cdot (t_m - t_l + 1)$. Команда `apply` устанавливает детям ровно эти значения (и метку присваивания). Аналогично для правого ребёнка. Значение $\text{sum}[v]$ уже корректно и не меняется; метка $\text{has}[v]$ после передачи детям становится ненужной и сбрасывается. ■

Теорема 39.1 (Корректность `assign` и `sum_query`). Описанные процедуры `assign` и `sum_query` всегда поддерживают инвариант и возвращают правильные ответы.

△ Доказательство. Докажем индукцией по рекурсии.

(1) *assign*. Рассмотрим вызов $\text{assign}(v, tl, tr, l, r, x)$. Если $[l, r] = [t_l, t_r]$, выполняется `apply`, после чего весь отрезок вершины равен x , а $\text{sum}[v]$ равна $x \cdot (t_r - t_l + 1)$, то есть сумме на отрезке. Инвариант выполнен.

Если отрезок покрыт частично, то перед спуском вызывается `push`, который корректно подготавливает детей (лемма). Далее рекурсивные вызовы модифицируют левую и/или правую часть; по индукционному предположению они корректны и поддерживают инвариант у детей. После этого присваивание $\text{sum}[v]=\text{sum}[2v]+\text{sum}[2v+1]$ делает $\text{sum}[v]$ равной сумме на объединении детей, то есть на всём $[t_l, t_r]$. Следовательно, инвариант сохраняется.

(2) *sum_query*. Если $[l, r] = [t_l, t_r]$, возвращаем $\text{sum}[v]$, которая корректна по инварианту. Иначе вызываем `push` и рекурсивно считаем суммы на пересечениях с детьми. Эти пересечения дизъюнкты и в сумме дают $[l, r]$, а рекурсивные ответы корректны по индукционному предположению. Значит, сумма двух ответов равна искомой сумме на $[l, r]$. ■

39.8 Асимптотика

Высота дерева отрезков равна $O(\log n)$. Каждый запрос `assign` или `sum_query` посещает $O(\log n)$ вершин и делает в каждой $O(1)$ действий (включая возможный `push`). Значит, время работы одного запроса $O(\log n)$.

Память: $O(n)$ (обычно массивы размера $4n$ для `sum`, `set`, `has`).

Замечание. Если вместо суммы нужен минимум/максимум, то структура почти не меняется: при `apply` для минимума/максимума просто кладём значение x (без умножения на длину).

40 Билет №40. Количество чисел на отрезке, значения которых не превосходят x . Fractional Cascading

40.1 Постановка модельной задачи

Дан статический массив $a[0 \dots n-1]$ (без обновлений). Нужно отвечать на запросы вида:

$$\text{count}(l, r, x) = |\{i \in [l, r] : a[i] \leq x\}|.$$

Цель: препроцессинг за $O(n \log n)$, ответ на запрос за $O(\log n)$.

40.2 Базовое решение: Merge Sort Tree (сегментное дерево отсортированных списков)

Построим обычное дерево отрезков по индексам массива. В каждой вершине v , отвечающей отрезку индексов $[t_L, t_R]$, храним вектор vec_v — все значения $a[i]$ для $i \in [t_L, t_R]$, отсортированные по возрастанию.

Построение. В листе ($t_L = t_R$) имеем $\text{vec}_v = \{a[t_L]\}$. Внутри: vec_v получается слиянием двух отсортированных векторов детей.

Ответ на запрос. Отрезок $[l, r]$ представляется как дизъюнктное объединение $O(\log n)$ вершин дерева отрезков. Для каждой такой вершины v мы находим

$$c_v = \#\{y \in \text{vec}_v : y \leq x\} = \text{upper_bound}(\text{vec}_v, x),$$

и суммируем ответы: $\text{count}(l, r, x) = \sum c_v$.

Асимптотика. Построение: $O(n \log n)$ по времени и памяти. Запрос: $O(\log n)$ вершин $\times$ бинарный поиск $O(\log n)$ в каждой $\Rightarrow O(\log^2 n)$.

40.3 Идея Fractional Cascading

В ответе выше мы делаем много *родственных* бинарных поисков по отсортированным спискам: в списках, соответствующих вершинам одного дерева отрезков. Техника **fractional cascading** позволяет сделать *один* бинарный поиск, а результаты для остальных списков получать за $O(1)$ на список, т.е. за $O(\log n + k)$ для k списков.

В нашем случае $k = O(\log n)$ (столько вершин покрывают запросный отрезок), поэтому получится $O(\log n)$ на запрос.

40.4 Структура данных с “каскадированием”

Для каждой внутренней вершины v с детьми L и R храним:

- ▷ отсортированный список vec_v ;
- ▷ два массива переходов toL_v и toR_v длины $|\text{vec}_v| + 1$.

Смысл числа p . Для любого узла v будем поддерживать

$$p_v(x) := \#\{y \in \text{vec}_v : y \leq x\} = \text{upper_bound}(\text{vec}_v, x).$$

Это число от 0 до $|\text{vec}_v|$. Если узел v целиком лежит в запросе, то вклад в ответ — ровно $p_v(x)$.

Переход к детям. Хотим уметь по $p_v(x)$ быстро получать $p_L(x)$ и $p_R(x)$.

Определим массивы:

$$\text{toL}_v[0] = 0, \quad \text{toL}_v[p] = \#\{y \in \text{vec}_L : y \leq \text{vec}_v[p-1]\} \quad (p \geq 1),$$

и аналогично для toR_v .

Тогда для любого x , если $p = p_v(x)$, то $p_L(x) = \text{toL}_v[p]$, $p_R(x) = \text{toR}_v[p]$. (Почему это верно — в доказательстве корректности ниже.)

40.5 Построение toL и toR

Пусть известны vec_L и vec_R , а также vec_v — их слияние.

Массив toL_v можно построить одним проходом двумя указателями:

- ▷ положить $\text{toL}_v[0] = 0$, завести $j = 0$ (позиция в vec_L);
- ▷ для $p = 1..|\text{vec}_v|$ увеличить j , пока $j < |\text{vec}_L|$ и $\text{vec}_L[j] \leq \text{vec}_v[p-1]$;
- ▷ записать $\text{toL}_v[p] = j$.

Поскольку указатель j только растёт, весь проход линейный по размерам векторов. Аналогично строится toR_v .

40.6 Алгоритм ответа на запрос

Опишем рекурсивную функцию $\text{solve}(v, [t_L, t_R], [l, r], x, p)$, где $p = p_v(x) = \text{upper_bound}(\text{vec}_v, x)$.

- ▷ Если $[t_L, t_R] \cap [l, r] = \emptyset$, вернуть 0.
- ▷ Если $[t_L, t_R] \subseteq [l, r]$, вернуть p .
- ▷ Иначе спуститься в детей:

$$p_L = \text{toL}_v[p], \quad p_R = \text{toR}_v[p],$$

и вернуть сумму ответов детей.

Старт: в корне $root$ вычислить $p_{root}(x)$ одним бинарным поиском:

$$p_{root}(x) = \text{upper_bound}(\text{vec}_{root}, x),$$

и запустить solve с этим p .

40.7 Доказательство корректности

Докажем инвариант: в каждом рекурсивном вызове *solve* параметр p равен $p_v(x)$, т.е. числу элементов на отрезке вершины v , не превосходящих x .

База. В корне p вычислен как $\text{upper_bound}(\text{vec}_{\text{root}}, x)$, по определению это и есть $p_{\text{root}}(x)$.

Переход к детям. Пусть в вершине v мы знаем $p = p_v(x)$. Если $p = 0$, то $x < \text{vec}_v[0]$, следовательно, ни в одном из поддеревьев нет элементов $\leq x$, и действительно $\text{toL}_v[0] = \text{toR}_v[0] = 0$.

Пусть теперь $p \geq 1$, и обозначим $y = \text{vec}_v[p - 1]$. По определению $p = \#\{z \in \text{vec}_v : z \leq x\}$, значит $y \leq x$ и y — максимальный элемент в vec_v , не превосходящий x . Следовательно, в vec_v нет элементов из интервала $(y, x]$.

Рассмотрим левое поддерево (правое аналогично). Любой элемент $z \in \text{vec}_L$, удовлетворяющий $z \leq x$, также удовлетворяет $z \leq y$: иначе было бы $z \in (y, x]$, но тогда (так как vec_v есть слияние vec_L и vec_R) этот z присутствовал бы в vec_v , что противоречит отсутствию элементов $(y, x]$. Значит,

$$p_L(x) = \#\{z \in \text{vec}_L : z \leq x\} = \#\{z \in \text{vec}_L : z \leq y\} = \text{toL}_v[p],$$

где последняя равенство — это определение $\text{toL}_v[p]$. Инвариант сохранён.

Корректность суммы. Алгоритм суммирует $p_v(x)$ по вершинам, которые образуют дизъюнктное покрытие $[l, r]$. Каждая вершина отвечает за свой подотрезок индексов, и эти подотрезки не пересекаются, поэтому суммирование даёт общее число элементов на $[l, r]$, не превосходящих x .

40.8 Асимптотика

Построение. Как и в merge sort tree, суммарный размер всех vec по всем вершинам — $O(n \log n)$, слияния на каждом уровне дают $O(n \log n)$ времени. Построение массивов toL, toR для каждой вершины линейно от $|\text{vec}_v|$, значит суммарно также $O(n \log n)$ времени и памяти.

Запрос. Один бинарный поиск в корне: $O(\log n)$. Дальше рекурсивный спуск посещает $O(\log n)$ вершин (как в обычном дереве отрезков), в каждой из которых делается $O(1)$ работы и вычисление p для детей через toL, toR . Итого: $O(\log n)$ на запрос.

40.9 Практические замечания по реализации (C++)

- ▷ Для подсчёта $\#\{y \leq x\}$ используйте `std::upper_bound` (работает и при повторениях).
- ▷ В узле храните vec и массивы toL, toR длины $|\text{vec}| + 1$.
- ▷ Типы значений: если элементы массива могут не помещаться в `int`, храните `long long`, а вектора переходов (toL, toR) можно хранить как `int`, т.к. это индексы.

Псевдокод.

Псевдокод (идейно, близко к CLRS)**Build**(v, tl, tr) (строит vec_v , toL_v , toR_v):

1. **if** $tl = tr$ **then**
 - (a) $vec_v \leftarrow [a[tl]]$.
 - (b) $toL_v \leftarrow [0,0]$, $toR_v \leftarrow [0,0]$.
 - (c) **return**.
2. $tm \leftarrow \lfloor (tl + tr)/2 \rfloor$.
3. **Build**(L, tl, tm); **Build**($R, tm + 1, tr$).
4. $vec_v \leftarrow \text{Merge}(vec_L, vec_R)$.
5. $m \leftarrow |vec_v|$; инициализировать $toL_v[0] = toR_v[0] = 0$.
6. $j_L \leftarrow 0$, $j_R \leftarrow 0$.
7. **for** $p = 1..m$:
 - (a) $y \leftarrow vec_v[p - 1]$.
 - (b) **while** $j_L < |vec_L|$ и $vec_L[j_L] \leq y$: $j_L \leftarrow j_L + 1$.
 - (c) **while** $j_R < |vec_R|$ и $vec_R[j_R] \leq y$: $j_R \leftarrow j_R + 1$.
 - (d) $toL_v[p] \leftarrow j_L$, $toR_v[p] \leftarrow j_R$.

Query(v, tl, tr, l, r, p) (где $p = p_v(x)$ уже посчитан):

1. если $[tl, tr] \cap [l, r] = \emptyset$, **return** 0.
2. если $[tl, tr] \subseteq [l, r]$, **return** p .
3. $tm \leftarrow \lfloor (tl + tr)/2 \rfloor$.
4. $p_L \leftarrow toL_v[p]$, $p_R \leftarrow toR_v[p]$.
5. **return** **Query**(L, tl, tm, l, r, p_L) + **Query**($R, tm + 1, tr, l, r, p_R$).

Count(l, r, x):

1. $p_{root} \leftarrow \text{UpperBound}(vec_root, x)$ (число элементов $\leq x$ в корне).
2. **return** **Query**($root, 0, n - 1, l, r, p_{root}$).

Типовые ошибки.

- ▷ Неверный выбор `lower_bound/upper_bound`: для условия $\leq x$ удобнее `upper_bound`.
- ▷ Забыли про $p = 0$: обязательно задайте $toL[0] = toR[0] = 0$.

- ▷ Неправильная граница разбиения узла (индексы): договоритесь об индексации заранее (0- или 1-based).

41 Билет №41. Динамическое (неявное) дерево отрезков

41.1 Когда оно нужно

Обычное дерево отрезков строят на массиве длины n и хранят как массив размера $\Theta(n)$ (обычно $4n$), что даёт:

- ▷ $\Theta(n)$ памяти;
- ▷ $O(\log n)$ времени на обновление/запрос.

Но бывают постановки, где **координаты очень большие** (например, до 10^{18}), а значимы лишь **немногие точки**:

- ▷ изначально во всех позициях стоят нули;
- ▷ приходит сравнительно мало точечных обновлений;
- ▷ запросы нужно обрабатывать **онлайн** (по мере поступления).

Строить дерево на всём диапазоне невозможно по памяти. Тогда используют **динамическое (неявное, implicit/sparse) дерево отрезков**: вершины создаются только там, где они реально нужны.

41.2 Модельная задача

Фиксирован целочисленный диапазон $[L, R]$ (например, $L = 0, R = 10^{18}$). Изначально $a[i] = 0$ для всех $i \in [L, R]$. Нужно поддерживать:

- ▷ `update(pos, val)`: присвоить $a[\text{pos}] \leftarrow \text{val}$;
- ▷ `get_sum(l, r)`: вернуть $\sum_{i=l}^r a[i]$.

41.3 Идея и структура

Определение 41.1. Динамическое дерево отрезков — это дерево отрезков, в котором вершина создаётся по требованию (обычно при спуске в неё во время `update`). Отсутствующая вершина трактуется как «в её поддереве всё нули».

Структура вершины для суммы:

- ▷ агрегат (здесь `sum`);
- ▷ ссылки на детей (`left, right`).

Границы отрезка $[tl, tr]$ можно *не хранить* в самой вершине: они известны из параметров рекурсии.

41.4 Операции

Точечное обновление. Спускаемся как в обычном дереве отрезков. Если нужного ребёнка нет, создаём. После обновления листа пересчитываем сумму в предках.

Запрос суммы на отрезке. Рекурсивно разбиваем запрос. Если вершина отсутствует (`nullptr`), то её вклад равен 0. Важно: `get_sum` не должна создавать вершины.

41.5 Корректность

Утверждение 41.1 (Инвариант). *Для любой существующей вершины v , соответствующей отрезку $[tl, tr]$, поле $v.sum$ равно сумме текущих значений $a[i]$ на этом отрезке. Если вершины нет, сумма на её отрезке равна 0.*

$\triangle$ *Доказательство.* Изначально существует только корень с суммой 0, инвариант верен.

`update` создаёт вершины лишь на пути к `pos` и меняет значение в листе. При подъёме вверх каждая посещённая вершина пересчитывает сумму как сумму детей (отсутствующий ребёнок даёт вклад 0), значит инвариант сохраняется.

`get_sum` использует стандартное разбиение отрезка. Если вершины нет, то на всём её диапазоне значения не менялись (иначе вершина была бы создана на каком-то обновлении), следовательно там нули и вклад 0 корректен. ■

41.6 Асимптотика

Пусть $C = R - L + 1$. Высота дерева равна $O(\log C)$.

▷ `update`: $O(\log C)$ времени и до $O(\log C)$ новых вершин.

▷ `get_sum`: $O(\log C)$ времени.

После q точечных обновлений число созданных вершин не превосходит $O(q \log C)$.

41.7 Псевдокод (динамическое дерево отрезков на сумме)

Псевдокод (идейно, близко к CLRS)

Node:

1. поля: `sum`, `left`, `right`.
2. отсутствующий ребёнок обозначаем `NIL` (вклад суммы равен 0).

Update(v, tl, tr, pos, val) (точка: $a[pos] \leftarrow val$):

1. если $v = \text{NIL}$, то создать вершину: $v \leftarrow \text{new Node}(0)$.
2. если $tl = tr$, то $v.sum \leftarrow val$ и **return**.
3. $tm \leftarrow tl + \lfloor (tr - tl) / 2 \rfloor$.
4. если $pos \leq tm$, то **Update**($v.left, tl, tm, pos, val$), иначе **Update**($v.right, tm + 1, tr, pos, val$).
5. $v.sum \leftarrow (v.left ? v.left.sum : 0) + (v.right ? v.right.sum : 0)$.

GetSum(v, tl, tr, l, r) (сумма на $[l, r]$):

1. если $v = \text{NIL}$ или $[l, r] \cap [tl, tr] = \emptyset$, **return** 0.
2. если $[tl, tr] \subseteq [l, r]$, **return** $v.sum$.
3. $tm \leftarrow tl + \lfloor (tr - tl)/2 \rfloor$.
4. **return** $\text{GetSum}(v.left, tl, tm, l, r) + \text{GetSum}(v.right, tm + 1, tr, l, r)$.

41.8 Практические замечания

- ▷ Середину отрезка считайте как $tl + (tr - tl)/2$ (защита от переполнения).
- ▷ Частые **new** могут быть медленными — в контестах полезен пул вершин.
- ▷ Для других операций (**min/max**) меняются агрегат и нейтральный элемент.
- ▷ Если запросы известны заранее, часто выгоднее сжатие координат (следующий билет).

42 Билет №42. Сжатие координат для дерева отрезков

Задача. Часто требуется обрабатывать запросы к массиву/множеству точек на прямой, где координаты лежат в очень большом диапазоне (например, $[0, 10^{18}]$), но реально в запросах встречается лишь небольшое число различных координат. Построить обычное дерево отрезков на всём диапазоне невозможно по памяти, а динамическое дерево отрезков даёт лишний логарифмический фактор $\log C$ от величины диапазона C .

Задача. Предполагая, что все запросы известны заранее (оффлайн), свести работу на большом диапазоне координат к работе на компактном диапазоне индексов и затем применять обычное дерево отрезков.

42.1 Идея

Если нас интересует только *порядок* координат (сравнения $\leq$, разбиение на отрезки, двоичный спуск в дереве отрезков), то «пустые промежутки» между реально встречающимися координатами не несут информации. Поэтому можно заменить каждую координату на её *ранг* в отсортированном списке всех координат, встречающихся во входе.

Именно этот приём называется *сжатием координат*.

42.2 Определение

Определение 42.1 (Сжатие координат). Пусть дан набор чисел (координат) $X = \{x_1, \dots, x_m\}$. Построим отсортированный список различных значений

$$\text{coords} = \text{sort}(\text{unique}(X)).$$

Для любого $x \in X$ определим *сжатую координату*

$$\text{id}(x) = \min\{i \mid \text{coords}[i] = x\}.$$

То есть $\text{id}(x)$ — индекс x в массиве `coords` (обычно ищется `lower_bound`).

42.3 Алгоритм (оффлайн)

1. Считать все запросы и выписать **все координаты**, которые когда-либо встречаются.
 - ▷ Для запросов вида `update(pos)` добавить `pos`.
 - ▷ Для запросов вида `query(l, r)` добавить `l` и `r`.
2. Отсортировать координаты и удалить повторы, получив `coords`.
3. Для каждой координаты x из запроса заменить её на $\text{id}(x)$. Тогда любой запрос на отрезке по координатам превращается в запрос на отрезке по индексам.
4. Построить обычное дерево отрезков на массиве длины $M = |\text{coords}|$ и обрабатывать уже «сжатые» запросы.

42.4 Доказательство корректности

Лемма 42.1 (Сохранение порядка). Для любых $x, y \in X$ выполнено:

$$x < y \iff \text{id}(x) < \text{id}(y).$$

$\triangle$ *Доказательство.* Массив `coords` получен сортировкой уникальных координат, значит он строго возрастает. Индекс элемента в строго возрастающем массиве сохраняет порядок: меньший элемент стоит левее, больший — правее. ■

Утверждение 42.1 (Корректность замены координат). Пусть в исходной задаче все обращения к координатам происходят только через сравнения и отрезки (например, запросы на отрезке $[l, r]$ и двоичный спуск в дереве отрезков). Тогда после замены $x \mapsto \text{id}(x)$ ответы на все запросы сохраняются (с точностью до переобозначения оси).

$\triangle$ *Доказательство.* После сжатия сохраняется относительный порядок координат (лемма выше). Любой отрезок $[l, r]$ на оси координат задаётся именно порядком точек: это множество всех x , для которых $l \leq x \leq r$. Так как порядок сохранён, множество «интересующих нас» координат внутри $[l, r]$ ровно соответствует множеству индексов внутри $[\text{id}(l), \text{id}(r)]$. Следовательно, дерево отрезков на сжатой оси обрабатывает те же запросы над теми же элементами, только под другими номерами. ■

42.5 Важная тонкость: точки vs интервалы

Сжатие координат бывает двух типов, и важно не перепутать.

- (а) **Сжатие точек.** Если значения хранятся в *точках* (например, “в позиции *pos* прибавить”, “сумма по всем позициям с координатами из набора”), то достаточно сжать сами координаты точек. Тогда размер дерева $M = |\text{coords}|$.
- (б) **Сжатие интервалов (учёт расстояний).** Если операция/ответ зависит от *длины* промежутков на исходной оси (например, длина объединения отрезков), то нужно помнить реальные расстояния между соседними координатами. Обычно делают так:
 - ▷ собирают все *границы* (концы отрезков);
 - ▷ сортируют и уникализируют их: `coords`;
 - ▷ рассматривают элементарные интервалы $[\text{coords}[i], \text{coords}[i+1])$ для $i = 0..M-2$;
 - ▷ строят дерево отрезков уже на $M-1$ элементарных интервалах и в вершинах хранят, например, покрытую длину, используя веса $\text{coords}[i+1] - \text{coords}[i]$.

Для *целочисленных* отрезков $[l, r]$ (включительно) часто удобно дополнительно добавить координату $r+1$, чтобы корректно работать с полуинтервалами $[l, r+1)$.

42.6 Асимптотика

Пусть q — число запросов, а M — число различных координат, встречающихся во входе.

- ▷ Построение массива `coords`: $O(M \log M)$ (сортировка и удаление повторов).

- ▷ Замена координат в запросах:
 - через `lower_bound`: $O(q \log M)$,
 - либо за $O(q)$ после построения `unordered_map` из координаты в индекс.
- ▷ Дальнейшая обработка запросов деревом отрезков: $O(\log M)$ на запрос.
- ▷ Память дерева отрезков: $O(M)$.

Сравнение. Динамическое дерево отрезков обычно даёт $O(\log C)$ на запрос по диапозону координат $[0, C]$. После сжатия координат логарифм зависит от M (числа реально используемых координат): $O(\log M)$.

42.7 Псевдокод

Псевдокод (идейно, близко к CLRS)

CoordinateCompress(*queries*):

1. `coords` $\leftarrow []$ (пустой список).
2. **for** каждый запрос q из *queries*:
 - (a) если q содержит точку pos , добавить pos в `coords`.
 - (b) если q содержит границы l, r , добавить l и r в `coords`.
3. `coords` $\leftarrow \text{Sort}(\text{coords})$; затем удалить повторы (**Unique**).
4. определить функцию **Id**(x): индекс x в `coords` (через **LowerBound**).
5. **for** каждый запрос q :
 - (a) заменить все координаты x внутри q на **Id**(x).
6. **return** (`coords`, *queries'*) — список координат и переписанные запросы.

Замечание. Если нужно учитывать *длины* промежутков (задача на интервалах, а не на точках), то после сжатия храните веса $w_i = \text{coords}[i+1] - \text{coords}[i]$ и стройте дерево на элементарных интервалах $[\text{coords}[i], \text{coords}[i+1])$.

42.8 Когда сжатие координат неприменимо

- ▷ Если запросы приходят *онлайн* и заранее неизвестно, какие координаты встретятся (или могут появляться новые координаты), то сжатие сделать нельзя. Тогда используют динамическое дерево отрезков (билет 41).
- ▷ Если вы сжали точки, но задача на самом деле требует учитывать *длины* промежутков (п. (b) выше), то ответы будут неверными: “пропущенные” промежутки нельзя просто выбросить, их вклад в длину нужно учитывать.

43 Билет №43. Персистентный массив

43.1 Постановка задачи

Нужно поддерживать массив $a[0 \dots n-1]$ так, чтобы после каждой модификации *старая версия массива оставалась доступной*.

Типичный набор операций:

- ▷ $\text{get}(\text{root}, i)$: узнать значение $a[i]$ в версии, задаваемой указателем root .
- ▷ $\text{set}(\text{root}, i, x)$: получить **новую версию** массива, равную старой, но с заменой $a[i] \leftarrow x$.

43.2 Идея персистентности

Определение 43.1 (Персистентная структура данных). Структура данных называется *персистентной*, если она позволяет обращаться к предыдущим версиям после модификаций.

Замечание (Частичная и полная персистентность). ▷ **Частичная (partial)**: запросы чтения возможны к любой версии, но обновления делаются только из последней версии.

▷ **Полная (full)**: обновления можно делать из *любой* версии (ветвление истории). В данной реализации мы автоматически получаем **полную** персистентность: каждая операция set возвращает новый корень, и от любого корня можно строить новые версии.

43.3 Представление массива как дерева

Идея: представить массив в виде полного двоичного дерева (как дерево отрезков), где листья — элементы массива, а внутренние вершины соответствуют отрезкам индексов.

Определение 43.2 (Дерево массива). Пусть корень соответствует отрезку $[0, n-1]$. Вершина, отвечающая $[l, r]$:

- ▷ если $l = r$, это лист, в нём хранится значение $a[l]$;
- ▷ иначе $m = \lfloor (l+r)/2 \rfloor$, левый сын отвечает $[l, m]$, правый — $[m+1, r]$.

Дерево однозначно задаёт массив: значение $a[i]$ находится в листе пути к индексу i .

43.4 Персистентность через «копирование пути» (path copying)

Ключевой приём: узлы **никогда не изменяются после создания**. При обновлении $a[i] \leftarrow x$ мы создаём новые вершины *только на пути* от корня к листу i , а все остальные поддеревья *переиспользуем* (делим между версиями).

Старая версия остаётся прежней, новая отличается лишь $O(\log n)$ узлами.

43.5 Операции

Хранимое состояние. Для каждой версии массива храним указатель на корень дерева. Пусть $\text{roots}[v]$ — корень версии v .

Запрос get. Спускаемся по дереву как в обычном дереве отрезков: на каждом шаге выбираем левого/правого сына в зависимости от i .

$$\text{get}(\text{root}, i) : O(\log n)$$

Запрос set. Рекурсивно строим новый путь:

- ▷ если пришли в лист $[i, i]$, возвращаем новый лист со значением x ;
- ▷ иначе создаём новую вершину, у которой один сын — результат рекурсивного обновления, а второй сын — *старый* указатель (шарим поддереву).

$$\text{set}(\text{root}, i, x) : O(\log n) \text{ времени и } O(\log n) \text{ памяти (новых узлов)}$$

43.6 Псевдокод

Псевдокод (идейно, близко к CLRS)

Node: запись с полями `val` (только в листе), `left`, `right`.

Build(a, tl, tr):

1. **if** $tl = tr$ **then return** `NEWLEAF( $a[tl]$ )`
2. $tm \leftarrow \lfloor (tl + tr)/2 \rfloor$
3. $L \leftarrow \text{BUILD}(a, tl, tm)$
4. $R \leftarrow \text{BUILD}(a, tm + 1, tr)$
5. **return** `NEWNODE( $L, R$ )`

Get(v, tl, tr, i):

1. **if** $tl = tr$ **then return** $v.\text{val}$
2. $tm \leftarrow \lfloor (tl + tr)/2 \rfloor$
3. **if** $i \leq tm$ **then return** `GET( $v.\text{left}, tl, tm, i$ )`
4. **else return** `GET( $v.\text{right}, tm + 1, tr, i$ )`

Set(v, tl, tr, i, x): (копирование пути, остальные поддеревья переиспользуем)

1. **if** $tl = tr$ **then return** `NEWLEAF( $x$ )`
2. $tm \leftarrow \lfloor (tl + tr)/2 \rfloor$
3. **if** $i \leq tm$ **then**

```

    (a)  $newL \leftarrow \text{SET}(v.\text{left}, tl, tm, i, x)$ 
    (b) return  $\text{NEWNODE}(newL, v.\text{right})$     (правое поддереву шарим)

4. else

    (a)  $newR \leftarrow \text{SET}(v.\text{right}, tm + 1, tr, i, x)$ 
    (b) return  $\text{NEWNODE}(v.\text{left}, newR)$     (левое поддереву шарим)

```

Замечание (Практика). $\triangleright$ Чтобы не платить за **new** много раз, обычно делают **пул аллокаций** (вектор узлов и выдача следующего свободного).

$\triangleright$ Глубина рекурсии $O(\log n)$ безопасна для типичных ограничений, но при желании можно переписать итеративно.

43.7 Доказательство корректности

Утверждение 43.1 (Корректность **get**). Для любой версии с корнем $root$ процедура $\text{get}(root, i)$ возвращает значение $a[i]$ этой версии.

$\triangle$ *Доказательство.* По определению дерева массива лист, соответствующий i , единственный и лежит на пути спуска, выбирая левую/правую половину в зависимости от сравнения i с серединой отрезка. **get** проходит ровно по этому пути и в листе возвращает записанное там значение, то есть $a[i]$ данной версии. ■

Утверждение 43.2 (Корректность **set**). Пусть $root2 = \text{set}(root1, i, x)$. Тогда версия $root2$ соответствует массиву b , где

$$b[j] = \begin{cases} x, & j = i, \\ a[j], & j \neq i, \end{cases}$$

где a — массив версии $root1$.

$\triangle$ *Доказательство.* Алгоритм создаёт новые вершины только на пути к листу i .

- $\triangleright$ В листе $[i, i]$ создаётся новый узел со значением x , поэтому в новой версии $b[i] = x$.
- $\triangleright$ Для любого $j \neq i$ путь к листу j не пересекается с изменяемым путём, следовательно указатели на соответствующие поддеревья в новой версии совпадают со старыми (они «шарятся»), а значит значение в листе j не меняется: $b[j] = a[j]$.

Тем самым новая версия отличается от старой только в позиции i . ■

43.8 Асимптотика

Пусть n — длина массива, q — число операций **set** (создающих новые версии).

- $\triangleright$ Построение первой версии (**build**): $O(n)$ узлов, $O(n)$ времени.
- $\triangleright$ **get**: спуск по высоте дерева $O(\log n)$.

- ▷ **set**: создаётся по одному новому узлу на каждом уровне, всего $O(\log n)$ узлов и времени.
- ▷ Итоговая память: $O(n + q \log n)$ узлов.

43.9 Связь с последующими билетами

Персистентный массив — частный случай **персистентного дерева отрезков** (билет 45): если в каждой внутренней вершине дополнительно хранить агрегат (например, сумму на отрезке), то получаем версии структуры для диапазонных запросов при тех же идеях «копирования пути».

44 Билет №44. Персистентный стек

Постановка

Определение 44.1. Персистентная структура данных сохраняет доступ ко всем старым версиям после модификаций: операция не “переписывает” объект, а создаёт новую версию, разделяющую с предыдущими общие (неизменённые) части.

Для стека это удобно формулировать так.

Задача. Поддерживать множество версий стека. Каждая операция создаёт новую версию, ссылаясь на некоторую предыдущую версию v :

- ▷ $push(v, x)$: получить новую версию, где x добавлен на вершину стека версии v ;
- ▷ $pop(v)$: получить новую версию, равную стеку версии v без верхнего элемента (гарантируется, что стек не пуст);
- ▷ $top(v)$: узнать верхний элемент версии v .

Требуется описать структуру данных, доказать корректность и оценить асимптотику.

Идея

Обычный стек на связном списке уже почти персистентен: если не изменять существующие узлы, то при $push$ достаточно создать *один* новый узел и сделать его головой. Все предыдущие узлы остаются как были и автоматически образуют старые версии.

Структура данных

- ▷ Узел (элемент стека): $Node\{value, prev\}$, где $prev$ — ссылка на предыдущий узел (нижний элемент стека).
- ▷ Версия стека: указатель (или индекс) $head[v]$ на вершину (голову) списка; пустой стек соответствует $head[v] = null$.

Узлы после создания **не меняются**. Разные версии “делят” общие хвосты списков.

Операции

Пусть $head[v]$ — вершина версии v .

- ▷ $push(v, x)$: создать новый узел u со значениями $u.value = x$, $u.prev = head[v]$. Положить $head[new] = u$.
- ▷ $pop(v)$: положить $head[new] = head[v].prev$.
- ▷ $top(v)$: ответ $head[v].value$.

Корректность

Утверждение 44.1. Для любой версии v последовательность элементов стека (сверху вниз) равна последовательности значений, встречающихся при движении по указателям $prev$ от $head[v]$ до $null$.

△ *Доказательство.* Индукция по числу выполненных операций.

База. Версия 0 — пустая: $head[0]=null$, цепочка пуста.

Переход.

- ▷ $push(v, x)$ создаёт новый узел u и делает его головой новой версии. Тогда в новой версии сверху стоит x , а ниже идёт ровно цепочка версии v , потому что $u.prev = head[v]$. Старые узлы не менялись, значит версия v не испорчена.
- ▷ $pop(v)$ переносит голову на $head[v].prev$, то есть удаляет ровно верхний элемент цепочки и оставляет остальные в том же порядке.
- ▷ $top(v)$ возвращает значение текущей головы, что по определению и есть верх стека.

Инвариант сохранён. ■

Асимптотика

- ▷ $push$ — $O(1)$ времени и $O(1)$ памяти (создаётся 1 узел).
- ▷ pop — $O(1)$ времени и $O(1)$ дополнительной памяти.
- ▷ top — $O(1)$ времени.

Суммарная память $O(P)$, где P — число выполненных операций $push$ (в остальных операциях новые узлы не появляются).

Псевдокод (через индексы в массиве узлов)

Псевдокод (идейно, близко к CLRS)

Состояние:

1. массив узлов **nodes** (только добавление в конец);
2. массив **head[ver]** — индекс вершины стека в версии **ver**;
3. **NIL** обозначает отсутствие вершины (например, -1); $head[0] = \text{NIL}$.

Node: поля **value**, **prev** (индекс предыдущего узла или **NIL**).

Push(v, x): (создать версию **new** из версии **v**)

1. $u \leftarrow \text{nodes.append}(\text{Node}(\text{value}=x, \text{prev}=\text{head}[v]))$.
2. $\text{head}[\text{new}] \leftarrow u$; **return new**.

Pop(v): (стек версии v непуст)

```
1. head[new] ← nodes[ head[v] ].prev; return new.
```

Top(v):

```
1. return nodes[ head[v] ].value.
```

Замечания

- ▷ Если разрешены обновления только от последней версии — это **частичная персистентность**. Если `push/pop` могут ссылаться на любую версию v — это **полная персистентность**. Для стека описанная реализация поддерживает оба режима.
- ▷ В отличие от персистентного массива/дерева отрезков, здесь не нужен `path copying` по дереву: стек — это уже “цепочка”, и одна новая вершина полностью описывает изменение.

45 Билет №45. Персистентное дерево отрезков

45.1 Напоминание: что такое персистентность

Персистентной называется структура данных, которая хранит доступ не только к текущему состоянию, но и ко *всем (или многим) прошлым версиям* состояния. Персистентность удобно воспринимать как *контроль версий*: после каждого обновления появляется новая версия структуры, а старые версии остаются доступными для запросов.

В рамках этого билета нас интересует типичная *частичная персистентность*:¹ мы обрабатываем обновления последовательно, сохраняя корень каждой версии.

45.2 Постановка: обычное дерево отрезков

Пусть есть массив $a[0 \dots n-1]$. Классическое дерево отрезков строится для поддержки:

- ▷ `update(pos, ...)` — изменение в одной точке (например, присваивание или прибавление);²
- ▷ `query(l, r)` — запрос агрегата на отрезке (сумма/минимум/максимум/...).

В каждой вершине дерева хранится агрегат (например, сумма) на соответствующем подотрезке.

45.3 Ключевое наблюдение для персистентности

При **точечном обновлении** в дереве отрезков меняются значения **только на одном пути** от корня к листу *pos*. Все остальные поддеревья остаются неизменными.

Отсюда естественная идея (*path copying, копирование пути*):

- ▷ старую версию **не трогаем** (она «заморожена»);
- ▷ создаём **копии вершин** на пути от корня до *pos*;
- ▷ все поддеревья, которые не лежат на этом пути, **переиспользуем** (новые вершины просто указывают на старых детей).

Таким образом, новая версия отличается от старой ровно $O(\log n)$ новыми вершинами.

45.4 Как хранить версии

Пусть $root[t]$ — корень дерева отрезков после обработки первых t обновлений. Тогда:

- ▷ $root[0]$ — корень исходного дерева (построенного по массиву или по нулям);
- ▷ каждое обновление возвращает **новый корень** $root[t+1]$;
- ▷ запрос `query` можно выполнять на любой версии, просто начиная с соответствующего корня.

По сути, мы храним **массив корней** версий.

¹Изменять разрешено только последнюю версию, но запрашивать можно любую прошлую.

²Именно точечное обновление важно для стандартной персистентности; отложенные операции тоже можно персистентить, но это отдельная (сложнее) история.

45.5 Структура вершины

Вершина персистентного дерева отрезков обычно хранит:

- ▷ агрегат на своём отрезке (например, сумму);
- ▷ два указателя на детей (левый/правый).

Границы $[tl, tr]$ обычно **не хранят** в вершине, а передают параметрами рекурсии (это экономит память).

45.6 Операции

Ниже считаем, что агрегат — сумма; для любого *моноида* (ассоциативная операция + нейтральный элемент) всё аналогично.

45.6.1 Построение build

Строим обычное дерево отрезков по массиву a . Время и память — $O(n)$ вершин (в реализации часто $\approx 4n$).

45.6.2 Запрос query(root, l, r)

Полностью совпадает с обычным деревом отрезков: спускаемся в те подотрезки, которые пересекают запрос.

Асимптотика. $O(\log n)$ посещённых вершин (стандартная оценка для дерева отрезков).

45.6.3 Обновление update(oldRoot, pos, newValue)

Возвращает **новый корень** новой версии. Рекурсивно:

- ▷ если мы в листе $tl = tr = pos$, создаём новую вершину с новым значением;
- ▷ иначе копируем текущую вершину, но рекурсивно обновляем только того ребёнка, который содержит pos ;
- ▷ второй ребёнок переиспользуется (указатель берём из старой версии);
- ▷ агрегат пересчитываем из детей.

Асимптотика. Время $O(\log n)$ и память $O(\log n)$ на одно обновление, потому что создаётся по одной новой вершине на каждом уровне дерева.

45.7 Корректность

45.7.1 Корректность query

Это обычный запрос дерева отрезков, корректность доказывается стандартно: мы разбиваем отрезок запроса на дизъюнктное объединение отрезков вершин, суммируем их значения.

45.7.2 Корректность update

Докажем два утверждения.

Утверждение 45.1 (Персистентность (старые версии не меняются)). *При выполнении **update** мы не изменяем ни одну вершину старой версии, а только создаём новые вершины. Значит, все $root[t]$ остаются корректными корнями прежних версий.*

△ *Доказательство.* В реализации **update** все изменения делаются только через создание новых объектов-вершин. Старые указатели читаются, но не переписываются. Следовательно, граф достижимости вершин из *oldRoot* не меняется. ■

Утверждение 45.2 (Новая версия соответствует обновлённому массиву). *Пусть версия *oldRoot* соответствует некоторому массиву a , а обновление меняет $a[pos]$ на $newValue$. Тогда возвращаемый корень *newRoot* соответствует массиву a' , где $a'[pos] = newValue$, а все остальные элементы совпадают с a .*

△ *Доказательство.* Рассмотрим рекурсивный спуск по пути к листу *pos*. На каждом уровне создаётся новая вершина, отличающаяся от старой только в одном ребёнке: в том, куда рекурсивно спускаемся. Другой ребёнок переиспользуется, а значит его подотрезок совпадает со старой версией.

В листе *pos* значение заменяется на *newValue*. Поднимаясь обратно, мы пересчитываем суммы в новых вершинах как сумму сумм детей. Значит, для каждого подотрезка на пути сумма корректно обновляется, а вне пути все подотрезки целиком переиспользованы и совпадают со старой версией. Следовательно, новая версия соответствует ровно одному изменению в позиции *pos*. ■

45.8 Оценка памяти

Пусть исходное дерево занимает $O(n)$ вершин. Каждое обновление добавляет $O(\log n)$ новых вершин. Тогда после q обновлений память составляет

$$O(n + q \log n).$$

Это ключевое преимущество персистентности: вместо полного копирования дерева ($O(n)$ на обновление) мы копируем только путь.

45.9 Псевдокод (копирование пути)

Псевдокод (идейно, близко к CLRS)

Node: поля *sum*, *left*, *right*.

Build(*tl*,*tr*):

1. если $tl = tr$, **return** new Node($sum=a[tl]$).
2. $tm \leftarrow \lfloor (tl + tr)/2 \rfloor$.
3. $L \leftarrow \text{Build}(tl, tm)$, $R \leftarrow \text{Build}(tm + 1, tr)$.

4. **return** new Node(sum=L.sum+R.sum, left=L, right=R).

Query(v, tl, tr, l, r):

1. если $v = \text{NIL}$ или $[l, r] \cap [tl, tr] = \emptyset$, **return** 0.
2. если $[tl, tr] \subseteq [l, r]$, **return** $v.sum$.
3. $tm \leftarrow \lfloor (tl + tr)/2 \rfloor$.
4. **return** **Query**($v.left, tl, tm, l, r$) + **Query**($v.right, tm + 1, tr, l, r$).

Update($v, tl, tr, pos, newVal$): (вернуть новый корень)

1. если $tl = tr$, **return** new Node(sum=newVal).
2. $tm \leftarrow \lfloor (tl + tr)/2 \rfloor$.
3. **if** $pos \leq tm$ **then**
 - (a) $newL \leftarrow \text{Update}(v.left, tl, tm, pos, newVal)$.
 - (b) $newR \leftarrow v.right$.
- else**
 - (a) $newL \leftarrow v.left$.
 - (b) $newR \leftarrow \text{Update}(v.right, tm + 1, tr, pos, newVal)$.
4. **return** new Node(sum=newL.sum+newR.sum, left=newL, right=newR).

Использование версий:

1. $root[0] \leftarrow \text{Build}(0, n - 1)$.
2. для обновления: $root[t + 1] \leftarrow \text{Update}(root[t], 0, n - 1, pos, newVal)$.
3. для запроса на версии ver : $ans \leftarrow \text{Query}(root[ver], 0, n - 1, l, r)$.

Замечания по аккуратности реализации.

- ▷ Если n может быть 0, обработайте отдельно (пустое дерево).
- ▷ Если операция — не присваивание, а «прибавить Δ », то в листе делайте $newVal = old + delta$.
- ▷ Следите за типами: суммы обычно требуют `long long`.
- ▷ Для ускорения используйте пул памяти (иначе большое число **new** может замедлить).

45.10 Где это используется (связь со следующими билетами)

Персистентность особенно полезна, когда нужно быстро отвечать на запросы к **разным версиям** массива. Классический приём: строить версии по префиксам (или по «моментам

включения» элементов), а затем отвечать на оффлайн-запросы, сравнивая две версии. Именно эта техника лежит в основе решений билетов **46** и **47**.

46 Билет №46. Количество чисел на отрезке, значения которых не превосходят x : решение с персистентным деревом отрезков

Постановка задачи

Дан статический массив $a[1..n]$ (изменений нет). Для каждого запроса заданы (l, r, x) . Нужно найти

$$\text{cnt}(l, r, x) = |\{i \in [l, r] : a[i] \leq x\}|.$$

Ключевая идея: “включаем лампочки” и храним версии

Сопоставим каждому индексу i лампочку, которая светится тогда и только тогда, когда элемент $a[i]$ уже “разрешён” (то есть мы решили считать его среди $\leq x$). Если бы мы знали, какие именно индексы имеют $a[i] \leq x$, то ответ на запрос — это просто количество включённых лампочек на отрезке $[l, r]$, то есть сумма 1 по включённым позициям.

Сделаем это системно:

1. Отсортируем пары $(a[i], i)$ по значению $a[i]$ (по возрастанию).
2. Будем включать лампочки по одной в этом порядке: сначала индекс минимального элемента, затем следующего и т. д.
3. После включения t лампочек мы получаем версию структуры, соответствующую всем элементам, значения которых не превосходят t -го по порядку включения.

Чтобы уметь быстро считать **сумму на отрезке** включённых лампочек, будем хранить дерево отрезков по индексам $1..n$ (в листьях 0/1, во внутренних вершинах — суммы). А чтобы иметь доступ ко **всем прошлым версиям**, сделаем дерево **персистентным**: при включении новой лампочки меняется только один путь от корня к листу, поэтому мы *клоним* только вершины на этом пути, а остальные поддеревья переиспользуем.

Построение персистентного дерева отрезков

Пусть $root[t]$ — корень версии после включения первых t элементов в отсортированном порядке. Тогда:

- ▷ $root[0]$ — дерево из нулей (все лампочки выключены);
- ▷ для $t = 1..n$ пусть (val_t, pos_t) — t -я пара в отсортированном списке, тогда $root[t] = \text{update}(root[t-1], pos_t)$, где **update** включает лампочку в точке pos_t (то есть прибавляет 1 в листе pos_t).

Персистентный update. Функция $\text{update}(v, tl, tr, pos)$ возвращает *новый* корень, соответствующий дереву v , в котором в позиции pos значение увеличено на 1. При этом создаются новые вершины только на пути к pos (их $O(\log n)$), а остальные вершины используются повторно.

Запрос суммы. `get_sum(v, tl, tr, l, r)` возвращает сумму на отрезке $[l, r]$ в версии с корнем v .

Ответ на запрос

Пусть отсортированные значения (с учётом кратностей) равны

$$b[1] \leq b[2] \leq \dots \leq b[n],$$

где $b[t]$ — значение t -й включённой лампочки (то есть val_t из пары (val_t, pos_t)).

Для заданного x найдём число включённых лампочек

$$t = \max\{k : b[k] \leq x\} = \text{upper_bound}(b, x).$$

(Если $x < b[1]$, то $t = 0$; если $x \geq b[n]$, то $t = n$.)

Тогда ответ:

$$\text{cnt}(l, r, x) = \text{get_sum}(\text{root}[t], 1, n, l, r).$$

Корректность

Утверждение 46.1. Для каждого $t \in [0..n]$ в версии $\text{root}[t]$ лист i равен 1 тогда и только тогда, когда $a[i] \leq b[t]$ (где по соглашению $b[0] = -\infty$).

$\triangle$ *Доказательство.* Построение идёт по отсортированным парам $(a[i], i)$. Ровно индексы первых t пар получают значение 1, остальные остаются 0. Поскольку пары отсортированы по $a[i]$, множество “включённых” индексов после t шагов совпадает с множеством индексов элементов, значения которых не превосходят $b[t]$. ■

Утверждение 46.2. Для любого запроса (l, r, x) алгоритм возвращает $\text{cnt}(l, r, x)$.

$\triangle$ *Доказательство.* Пусть $t = \text{upper_bound}(b, x)$, т.е. $b[t] \leq x < b[t+1]$ (или граничные случаи $t = 0$ / $t = n$). Тогда по предыдущему утверждению в версии $\text{root}[t]$ включены в точности те индексы i , для которых $a[i] \leq x$. Следовательно, сумма на отрезке $[l, r]$ в этой версии равна числу таких индексов, то есть $\text{cnt}(l, r, x)$. ■

Асимптотика

- ▷ Сортировка пар $(a[i], i)$: $O(n \log n)$.
- ▷ Каждое включение (персистентный `update`) клонирует $O(\log n)$ вершин, значит построение всех n версий: $O(n \log n)$ времени и $O(n \log n)$ памяти.
- ▷ Один запрос:
 - поиск t через `upper_bound`: $O(\log n)$;
 - `get_sum` в дереве отрезков: $O(\log n)$.

Итого $O(\log n)$ на запрос.

Полезное расширение

Количество чисел, попадающих в диапазон значений $(a, b]$ на отрезке $[l, r]$, легко получить разностью:

$$|\{i \in [l, r] : a < a[i] \leq b\}| = \text{cnt}(l, r, b) - \text{cnt}(l, r, a).$$

Псевдокод (идейно, близко к CLRS)

Псевдокод (идейно, близко к CLRS)

Структура вершины. У каждой вершины храним поле `sum` (сумма на отрезке) и указатели `left`, `right`. Пустое поддерево обозначаем `NIL` и считаем, что $\text{Sum}(\text{NIL}) = 0$.

Sum(v):

1. **if** $v = \text{NIL}$ **then return** 0.
2. **return** $v.\text{sum}$.

Update(v, tl, tr, pos): *персистентное включение лампочки* (увеличить значение в точке pos на 1), вернуть *новый* корень.

1. **if** $v = \text{NIL}$ **then** считать, что $v.\text{left} = v.\text{right} = \text{NIL}$ и $v.\text{sum} = 0$.
2. Создать новую вершину u как “копию” v (те же дети).
3. **if** $tl = tr$ **then**
 - (a) $u.\text{sum} \leftarrow v.\text{sum} + 1$.
 - (b) **return** u .
4. $tm \leftarrow \left\lfloor \frac{tl + tr}{2} \right\rfloor$.
5. **if** $pos \leq tm$ **then** $u.\text{left} \leftarrow \text{Update}(v.\text{left}, tl, tm, pos)$ **else** $u.\text{right} \leftarrow \text{Update}(v.\text{right}, tm + 1, tr, pos)$.
6. $u.\text{sum} \leftarrow \text{Sum}(u.\text{left}) + \text{Sum}(u.\text{right})$.
7. **return** u .

Query(v, tl, tr, l, r): сумма на отрезке $[l, r]$ в версии с корнем v .

1. **if** $v = \text{NIL}$ **or** $[l, r] \cap [tl, tr] = \emptyset$ **then return** 0.
2. **if** $l \leq tl$ **and** $tr \leq r$ **then return** $v.\text{sum}$.
3. $tm \leftarrow \left\lfloor \frac{tl + tr}{2} \right\rfloor$.
4. **return** $\text{Query}(v.\text{left}, tl, tm, l, r) + \text{Query}(v.\text{right}, tm + 1, tr, l, r)$.

Построение версий $\text{root}[t]$:

1. Отсортировать пары $(a[i], i)$ по $a[i]$ по возрастанию, получить последовательность позиций $pos_1, \dots, pos_n$.
2. $root[0] \leftarrow \text{NIL}$.
3. **for** $t = 1..n$ **do** $root[t] \leftarrow \text{Update}(root[t-1], 1, n, pos_t)$.

47 Билет №47. Поиск k -й порядковой статистики на отрезке: персистентное дерево отрезков и параллельный спуск по двум версиям

Задача. Дан массив $a[1..n]$ (значения могут повторяться). Нужно отвечать на запросы вида (l, r, k) : найти k -й **наименьший** элемент мультимножества $\{a_l, a_{l+1}, \dots, a_r\}$ (с учётом повторов), где $1 \leq l \leq r \leq n$ и $1 \leq k \leq r - l + 1$.

Определение 47.1 (Порядковая статистика). Пусть S — мультимножество из m элементов. Его k -я порядковая статистика (нумерация $k = 1..m$) — это элемент, который стоял бы на позиции k после сортировки S по неубыванию.

Ключевая идея

Построим **персистентные** версии частотного дерева отрезков по *значениям* для всех префиксов массива. Тогда частоты на подотрезке $[l, r]$ получаются как разность двух префиксов: версия r минус версия $l - 1$. А k -ю порядковую статистику можно найти за $O(\log M)$, спускаясь по дереву и каждый раз решая, в какую половину диапазона значений попадает ответ.

Шаг 1. Сжатие координат

Так как значения a_i могут быть большими, делаем сжатие:

- ▷ возьмём отсортированный массив всех различных значений $vals$,
- ▷ заменим a_i на $c_i = \text{rank}(a_i) \in \{0, 1, \dots, M - 1\}$, где $M = |vals|$,
- ▷ обратно значение восстанавливаем как $vals[c_i]$.

Шаг 2. Персистентное дерево отрезков частот

Строим дерево отрезков на диапазоне индексов значений $[0..M - 1]$. Каждый узел хранит:

$sum(v)$ = сколько элементов префикса попало в диапазон значений этого узла.

Версия $root[i]$ соответствует префиксу $a[1..i]$:

$root[0]$ — пустая версия (езде 0), $root[i] = \text{update}(root[i - 1], c_i, +1)$.

Персистентность (копирование пути). При точечном обновлении в обычном дереве отрезков меняется только путь от корня до листа. В персистентной версии мы *клоним* узлы только на этом пути, а все остальные поддеревья переиспользуем (указатели просто копируются).

Псевдокод update

Ниже «персистентное» обновление: возвращает новый корень, не разрушая старую версию.

Псевдокод (идейно, близко к CLRS)

Node: запись с полями sum , $left$, $right$.

Update(v, tl, tr, pos): (увеличить частоту в точке pos на 1, вернуть новый корень)

1. **if** $tl = tr$ **then**

(a) **return** $\text{NEWNODE}(sum(v) + 1, \text{NIL}, \text{NIL})$

2. $tm \leftarrow \lfloor (tl + tr)/2 \rfloor$

3. **if** $pos \leq tm$ **then**

(a) $newL \leftarrow \text{UPDATE}(v.left, tl, tm, pos)$

(b) $newR \leftarrow v.right$ (переиспользуем)

4. **else**

(a) $newL \leftarrow v.left$ (переиспользуем)

(b) $newR \leftarrow \text{UPDATE}(v.right, tm + 1, tr, pos)$

5. **return** $\text{NEWNODE}(sum(v) + 1, newL, newR)$

Замечание. Чтобы не писать проверки на `nullptr`, удобно завести один глобальный узел `ZERO` с $sum=0$, и считать, что у «пустых» детей указатель ведёт в `ZERO`. Тогда формулы работают всегда.

Лемма: разность двух версий даёт частоты на отрезке

Лемма 47.1. Для любого диапазона значений $[L..R]$ число элементов подотрезка $a[l..r]$, попавших в $[L..R]$, равно

$$\#\{i \in [l, r] : c_i \in [L..R]\} = sum(root[r]_{L..R}) - sum(root[l-1]_{L..R}),$$

где $root[t]_{L..R}$ — узел версии $root[t]$, соответствующий диапазону $[L..R]$.

$\triangle$ *Доказательство.* $root[r]$ хранит частоты на префиксе $[1..r]$, а $root[l-1]$ — на префиксе $[1..l-1]$. Разность даёт ровно элементы с индексами $l..r$. ■

Шаг 3. Запрос (l, r, k) : параллельный спуск по двум версиям

Обозначим:

$$A = root[r], \quad B = root[l-1].$$

Мы одновременно спускаемся по обоим деревьям. Пусть текущие узлы A, B соответствуют диапазону значений $[tl..tr]$ и $tm = \lfloor (tl + tr)/2 \rfloor$.

Посчитаем сколько элементов подотрезка попадает в левую половину:

$$cntLeft = sum(A_{left}) - sum(B_{left}).$$

Тогда:

- ▷ если $k \leq cntLeft$, ответ лежит в $[tl..tm]$, переходим в (A_{left}, B_{left}) ;
- ▷ иначе ответ в $[tm + 1..tr]$, переходим в (A_{right}, B_{right}) и делаем $k \leftarrow k - cntLeft$.

Доходим до листа $tl = tr$ — это и есть индекс сжатого значения.

Псевдокод kth

Псевдокод (идейно, близко к CLRS)

Kth(A, B, tl, tr, k): ($A = root[r], B = root[l - 1]$)

1. **if** $tl = tr$ **then return** tl
2. $tm \leftarrow \lfloor (tl + tr) / 2 \rfloor$
3. $cntLeft \leftarrow sum(A.left) - sum(B.left)$
4. **if** $k \leq cntLeft$ **then**
 - (a) **return** **КТН**($A.left, B.left, tl, tm, k$)
5. **else**
 - (a) **return** **КТН**($A.right, B.right, tm + 1, tr, k - cntLeft$)

Корректность kth

Теорема 47.1. Алгоритм $kth(root[r], root[l - 1], 0, M - 1, k)$ возвращает индекс с такой, что $vals[c]$ равен k -й порядковой статистике на подотрезке $a[l..r]$.

△ *Доказательство.* Индукция по длине диапозона $[tl..tr]$.

База: $tl = tr$. В диапазоне только одно значение, оно и есть ответ.

Переход: $tl < tr$. По лемме $cntLeft$ — это количество элементов подотрезка со значениями в левой половине $[tl..tm]$. Если $k \leq cntLeft$, то k -я статистика не может быть в правой половине (там все значения $\geq$ любых из левой), значит ищем влево. Если $k > cntLeft$, то первые $cntLeft$ элементов в отсортированном подотрезке точно лежат в левой половине, значит нужная статистика в правой половине имеет номер $k - cntLeft$. Переходы совпадают с определением порядковой статистики. ■

Асимптотика

Пусть M — число различных значений ($M \leq n$).

- ▷ Сжатие координат: $O(n \log n)$.

- ▷ Построение версий: n обновлений по $O(\log M)$, итого $O(n \log M)$ времени.
- ▷ Память: на каждое обновление создаётся $O(\log M)$ новых узлов, итого $O(n \log M)$ узлов.
- ▷ Один запрос k -й статистики: спуск глубины $O(\log M)$, итого $O(\log M)$.

Типичные тонкости

- ▷ **Повторы значений** учитываются автоматически: в листе хранится частота.
- ▷ $root[0]$ должен быть корректной «пустой» версией, чтобы запросы с $l = 1$ работали без костылей.
- ▷ **Проверка k** : обязательно $1 \leq k \leq r - l + 1$.
- ▷ **Реализация в C++**: обычно выгоднее хранить узлы в массиве (пулы) и детей как индексы, чтобы не было дорогих `new` на миллионах узлов.

48 Билет №48. Дерево поиска (BST): операции find, insert, erase; опционально merge и split

48.1 Определение и инвариант

Определение 48.1 (Ключи и порядок). Пусть каждый элемент имеет *ключ* (например, целое число). На множестве ключей задан *строгий полный порядок* $<$, то есть:

- ▷ **сравнимость**: для любых a, b верно ровно одно из $a < b$, $a = b$, $a > b$;
- ▷ **транзитивность**: $a < b$ и $b < c \Rightarrow a < c$;
- ▷ **антисимметричность**: одновременно $a < b$ и $b < a$ быть не может.

Именно этот порядок используется во всех “поисковых” операциях.

Определение 48.2 (Бинарное дерево поиска (Binary Search Tree, BST)). *Бинарное дерево поиска* — это двоичное дерево, в каждой вершине которого хранится ключ, и выполняется *инвариант BST*:

для любой вершины v с ключом $key(v)$

$$\forall x \in \text{Left}(v): x < key(v), \quad \forall x \in \text{Right}(v): key(v) < x,$$

где $\text{Left}(v)$ и $\text{Right}(v)$ — множества ключей в левом и правом поддеревьях вершины v .

Замечание (Про равные ключи). В определении выше предполагается, что дубликаты не хранятся (структура данных “множество”). Если нужны дубликаты (“multiset”), есть два стандартных подхода:

- ▷ хранить в вершине счётчик кратности ключа;
- ▷ или договориться, куда отправлять равные ключи (например, “ $\leq$ влево, $>$ вправо”); тогда инвариант нужно переписать под эту договорённость.

Дальше для определённости считаем, что ключи различны.

Утверждение 48.1 (Эквивалентная формулировка инварианта через диапазоны). *BST-инвариант эквивалентен следующему: каждому узлу v можно сопоставить полуинтервал допустимых значений (L, R) (границы определяются предками), и тогда всегда $L < key(v) < R$.*

Утверждение 48.2 (Inorder-обход даёт ключи в отсортированном порядке). *Если выполнить симметричный (inorder) обход BST: “левое поддерево, затем вершина, затем правое поддерево”, то полученная последовательность ключей будет строго возрастающей.*

△ *Доказательство.* Докажем по структурной индукции по дереву.

- ▷ База: пустое дерево — пустая последовательность, она отсортирована.

- ▷ Переход: пусть корень содержит ключ k , левое поддерево даёт при inorder-обходе отсортированную последовательность L , правое — отсортированную R . По BST-инварианту все элементы L меньше k , а все элементы R больше k . Тогда конкатенация L, k, R строго возрастает.



48.2 Высота и асимптотика

Определение 48.3 (Высота). Высота (глубина) дерева h — длина (в рёбрах) самого длинного пути от корня до листа. Для пустого дерева удобно считать $h = -1$, для дерева из одной вершины $h = 0$.

Все базовые операции BST работают за $O(h)$, потому что сводятся к спуску по одному пути от корня вниз (или к небольшому числу таких спусков).

Замечание. В **наивном** BST высота в худшем случае может быть $h = n - 1$ (вырожденная “цепочка”), и тогда операции работают за $O(n)$. В **сбалансированных** деревьях поиска (AVL, красно-чёрные, treap, splay и т. д.) поддерживают $h = O(\log n)$, поэтому те же операции работают за $O(\log n)$.

48.3 Операция find(x)

Задача: проверить, есть ли ключ x в дереве (и при необходимости вернуть указатель на вершину).

Алгоритм:

1. Начать в корне v .
2. Пока $v \neq \text{null}$ и $\text{key}(v) \neq x$:
 - ▷ если $x < \text{key}(v)$, перейти влево: $v \leftarrow v.\text{left}$;
 - ▷ иначе ($x > \text{key}(v)$) перейти вправо: $v \leftarrow v.\text{right}$.
3. Если $v = \text{null}$, ключа нет. Иначе $\text{key}(v) = x$, ключ найден.

Утверждение 48.3 (Корректность find). Алгоритм $\text{find}(x)$ возвращает “не найдено” тогда и только тогда, когда x отсутствует в дереве.

△ *Доказательство.* Поддерживаем инвариант поиска: если ключ x есть в дереве, то он обязательно лежит в текущем поддереве вершины v .

- ▷ В начале v — корень, инвариант верен.
- ▷ На шаге сравниваем x и $\text{key}(v)$. Если $x < \text{key}(v)$, то по BST-инварианту все ключи в правом поддереве строго больше $\text{key}(v)$, значит не могут равняться x ; поэтому, если x существует, он должен быть в левом поддереве — переход влево сохраняет инвариант. Аналогично для случая $x > \text{key}(v)$.

Если алгоритм дошёл до вершины с ключом x , то он найден. Если дошёл до `null`, значит подходящего поддерева больше нет, а по инварианту других мест для x не осталось — значит, x отсутствует. ■

Асимптотика: один спуск по дереву $\Rightarrow O(h)$.

48.4 Операция `insert(x)`

Задача: добавить новый ключ x (предполагаем, что x отсутствует; иначе можно “ничего не делать” или обновить данные в вершине).

Алгоритм (вставка листом):

1. Идти как в `find(x)`, но запоминая последнюю непустую вершину p (будущего родителя).
2. Когда попытка перейти в нужного сына приводит в `null`, создать новую вершину z с ключом x и подвесить её к p :

$$x < key(p) \Rightarrow p.left \leftarrow z, \quad x > key(p) \Rightarrow p.right \leftarrow z.$$

3. Если дерево было пустым, z становится корнем.

Утверждение 48.4 (Корректность `insert`). После вставки нового ключа x дерево остаётся *BST*.

$\triangle$ *Доказательство.* Новая вершина z добавляется как лист, её поддеревья пусты, поэтому проверить *BST*-инвариант нужно только для ребра (p, z) и для согласованности со всеми предками p .

- ▷ По построению: если z повешен слева от p , то $key(z) = x < key(p)$; если справа — $key(p) < x$. Значит локально инвариант выполняется.
- ▷ Путь спуска при поиске места для вставки гарантирует, что x лежит в допустимом диапазоне для этой позиции (эквивалентная формулировка инварианта через диапазоны).

Следовательно, добавление листа сохраняет *BST*-инвариант. ■

Асимптотика: один спуск к месту вставки $\Rightarrow O(h)$.

48.5 Операция `erase(x)`

Задача: удалить ключ x из дерева (предполагаем, что он есть; иначе можно ничего не делать).

Шаг 1: найти вершину z с ключом x с помощью `find` за $O(h)$.

Дальше различают три случая.

Случай 1: у z нет детей (лист)

Просто “отцепляем” z от родителя: соответствующий указатель родителя делаем `null`. *BST*-инвариант очевидно сохраняется, так как мы удаляем лист.

Случай 2: y z ровно один ребёнок

Пусть единственный ребёнок — вершина s . Тогда “прокалываем” z : подвешиваем s напрямую к родителю z вместо z . Инвариант сохраняется, потому что все ключи поддерева s уже находились в допустимом диапазоне для позиции z .

Случай 3: y z два ребёнка

Это единственный нетривиальный случай.

1. Найдём *следующего по порядку* после z (inorder successor):

$$y = \min(z.\text{right}) \quad (\text{самая левая вершина в правом поддереве}).$$

2. Скопируем ключ (и данные) из y в z : теперь в вершине z лежит ключ $key(y)$.
3. Удалим вершину y как в случае 1 или 2.

Почему шаг 3 всегда сводится к случаю 1/2? Потому что y — минимум в правом поддереве, значит у y нет левого ребёнка (иначе в левом поддереве был бы меньший элемент).

Утверждение 48.5 (Корректность удаления в случае двух детей). *После описанной операции дерево остаётся BST и ключ x удалён.*

△ *Доказательство.* ▷ После замены $key(z) \leftarrow key(y)$ порядок относительно левого поддерева z сохраняется: все ключи слева были $< x$, а так как y лежит в правом поддереве z , то $x < key(y)$, следовательно все ключи слева всё ещё меньше нового ключа.

▷ Порядок относительно правого поддерева тоже сохраняется: y — минимальный в правом поддереве, значит все остальные ключи в правом поддереве не меньше $key(y)$. После удаления вершины y (лист/один ребёнок) структура BST сохраняется.

Удаляется именно одна вершина y , а ключ x исчезает из дерева (потому что единственная вершина с x была z , и её ключ мы заменили). ■

Асимптотика: find за $O(h)$ плюс поиск минимума в правом поддереве (ещё один спуск) и “прокол” $\Rightarrow O(h)$.

Замечание (Предшественник вместо преемника). Вместо inorder successor можно симметрично взять inorder predecessor $y = \max(z.\text{left})$. Тогда у y не будет правого ребёнка, и удаление снова сведётся к случаю 1/2.

49 Билет №49. Наивное дерево поиска: обработка операций

49.1 Постановка задачи

Нужно поддерживать *множество* ключей S (обычно — целые числа) под онлайн-запросами:

- ▷ **find**(x): проверить, содержится ли ключ x в S (и при необходимости вернуть указатель/ссылку на вершину);
- ▷ **insert**(x): добавить ключ x в S (если ключи уникальны, повторный **insert** можно игнорировать);
- ▷ **erase**(x): удалить ключ x из S (если ключа нет, запрос можно игнорировать).

49.2 Структура: наивное бинарное дерево поиска

Определение 49.1 (Бинарное дерево поиска (BST)). Бинарное дерево с ключами в вершинах называется *деревом поиска*, если для каждой вершины v с ключом $key(v)$ выполняется:

- ▷ все ключи в левом поддереве v строго меньше $key(v)$;
- ▷ все ключи в правом поддереве v строго больше $key(v)$.

Замечание (Про равные ключи). В билете считаем, что множество не содержит дубликатов. Если дубликаты нужны, есть два стандартных варианта:

- ▷ хранить в вершине **cnt** — количество одинаковых ключей и уменьшать/увеличивать **cnt**;
- ▷ договориться, что равные ключи всегда уходят, например, вправо (тогда инвариант становится «слева $<$, справа $\geq$ »).

Дальше для простоты считаем все ключи различными.

49.3 Операция find(x)

Алгоритм. Стартуем из корня. Пока текущая вершина существует и её ключ не равен x :

- ▷ если $x < key(v)$, переходим в левого сына;
- ▷ иначе ($x > key(v)$), переходим в правого сына.

Если пришли в пустоту (**nullptr**) — ключа нет. Если встретили x — ключ найден.

Утверждение 49.1 (Корректность find). Алгоритм $find(x)$ возвращает вершину с ключом x тогда и только тогда, когда $x \in S$.

△ *Доказательство.* Рассмотрим текущую вершину v .

- ▷ Если $x < \text{key}(v)$, то по инварианту BST все ключи правого поддерева строго больше $\text{key}(v)$, значит строго больше x ; следовательно, x не может находиться справа, и безопасно идти влево.
- ▷ Аналогично, если $x > \text{key}(v)$, то по инварианту все ключи левого поддерева строго меньше $\text{key}(v)$, значит строго меньше x ; следовательно, x не может находиться слева, и безопасно идти вправо.

Если алгоритм дошёл до вершины с ключом x , то x действительно присутствует. Если алгоритм дошёл до пустого сына, то по рассуждению выше x не мог находиться ни в одном из отброшенных поддеревьев, а единственный возможный путь завершился; значит, $x \notin S$. ■

Асимптотика. Пусть h — высота дерева (максимальная длина пути от корня до листа). Тогда `find` делает не более h шагов, то есть работает за $O(h)$. В худшем случае (вырожденное дерево) $h = O(n)$, где $n = |S|$.

49.4 Операция `insert(x)`

Алгоритм.

1. Идём по дереву как в `find` и находим место, куда должен встать x : это первый `nullptr` на пути.
2. Создаём новую вершину z с ключом x и подвешиваем её листом туда, где был `nullptr`.

Утверждение 49.2 (Корректность `insert`). После выполнения `insert(x)` множество ключей в дереве становится $S \cup \{x\}$, а свойство BST сохраняется.

△ *Доказательство.* Алгоритм спускается по тому же правилу, что и `find`: на каждом шаге выбирается единственное поддерево, в котором ключ x *может* находиться относительно текущего ключа.

Когда мы доходим до пустого указателя, мы нашли единственную позицию, где x может быть добавлен, не нарушая порядок: на всём пути от корня до позиции вставки выполнены сравнения, которые гарантируют:

- ▷ x больше всех ключей, через которые мы уходили вправо;
- ▷ x меньше всех ключей, через которые мы уходили влево.

Подвесив x листом в найденном месте, мы не меняем ключи в остальных вершинах, а для новой вершины инвариант BST очевиден (детей нет). Следовательно, свойство BST сохраняется. ■

Асимптотика. Спуск занимает $O(h)$, создание и подвешивание вершины — $O(1)$. Итого `insert` работает за $O(h)$, в худшем случае $O(n)$.

49.5 Операция `erase(x)`

Сначала находим вершину z с ключом x (если её нет — ничего не делаем). Далее возможны три случая.

Случай 1: y z нет левого сына

Тогда правое поддерево z (возможно пустое) можно *поднять* на место z .

Случай 2: y z нет правого сына

Аналогично, левое поддерево z можно поднять на место z .

Случай 3: y z два сына

Берём *следующего* по порядку элемента: $y = \min(z.\text{right})$ (минимум в правом поддереве, он же in-order successor).

- ▷ У y нет левого сына (иначе он не был бы минимумом справа).
- ▷ Переносим y на место z (и затем удаляем y из его старой позиции, где у него максимум один ребёнок).

Удобно пользоваться вспомогательной процедурой `transplant(u, v)`: заменить поддерево, корнем которого является u , на поддерево с корнем v (переподвесить у родителя и поправить `parent`).

Утверждение 49.3 (Корректность `erase`). После выполнения `erase(x)` множество ключей становится $S \setminus \{x\}$, а свойство *BST* сохраняется.

△ *Доказательство.* **Случаи 1–2.** Если y z нет одного из сыновей, то мы заменяем z его единственным поддеревом. Для каждого ключа в этом поддереве из инварианта *BST* следует, что он либо строго меньше всех ключей, лежащих правее предков z , либо строго больше всех ключей, лежащих левее предков z (в зависимости от того, где стоял z). Поэтому переподвешивание не нарушает порядка.

Случай 3. Пусть y — минимум в правом поддереве z . Тогда верно:

- ▷ все ключи в левом поддереве z строго меньше $key(z)$ и, следовательно, строго меньше $key(y)$ (так как y находится справа от z);
- ▷ все ключи в правом поддереве z не меньше $key(y)$ (так как y — минимальный среди них).

Заменяя ключ z на y (или, эквивалентно, переносим вершину y на место z), мы сохраняем корректное разделение: слева остаются элементы $< key(y)$, справа — элементы $> key(y)$.

После переноса y удаляется из старого места. Так как у y нет левого сына, его удаление сводится к случаю 1 (поднять правого сына, возможно пустого). Следовательно, после всех переподвешиваний инвариант *BST* сохраняется, а ключ x удалён. ■

Асимптотика. Поиск z занимает $O(h)$. Поиск минимума в правом поддереве (переходы по левым сыновьям) также занимает $O(h)$. Все `transplant` и переприсваивания указателей — $O(1)$. Итого `erase` работает за $O(h)$, в худшем случае $O(n)$.

49.6 Итог по асимптотике и ограничения “наивного” подхода

- ▷ Все операции `find`, `insert`, `erase` работают за $O(h)$, где h — высота дерева.
- ▷ В худшем случае (например, вставки в отсортированном порядке) дерево вырождается в список: $h = n$, и операции становятся $O(n)$.

Именно поэтому далее вводят самобалансирующиеся деревья поиска (AVL, splay, красно-чёрные и т. п.), гарантирующие $h = O(\log n)$.

49.7 Псевдокод (идейно, близко к CLRS)

Псевдокод (идейно, близко к CLRS)

Условности. `NIL` — пустой указатель. У вершины v есть поля `key`, `left`, `right`, `parent`. Структура T хранит `T.root`.

Find(T, x):

1. $v \leftarrow T.root$.
2. **while** $v \neq NIL$ **and** $v.key \neq x$ **do**
 - (a) **if** $x < v.key$ **then** $v \leftarrow v.left$ **else** $v \leftarrow v.right$.
3. **return** v (это либо вершина с ключом x , либо `NIL`).

Minimum(v):

1. **while** $v.left \neq NIL$ **do** $v \leftarrow v.left$.
2. **return** v .

Transplant(T, u, v): заменить поддерево с корнем u на поддерево с корнем v .

1. **if** $u.parent = NIL$ **then** $T.root \leftarrow v$.
2. **else if** $u = u.parent.left$ **then** $u.parent.left \leftarrow v$.
3. **else** $u.parent.right \leftarrow v$.
4. **if** $v \neq NIL$ **then** $v.parent \leftarrow u.parent$.

Insert(T, x):

1. $p \leftarrow NIL$, $cur \leftarrow T.root$.
2. **while** $cur \neq NIL$ **do**
 - (a) $p \leftarrow cur$.
 - (b) **if** $x < cur.key$ **then** $cur \leftarrow cur.left$.
 - (c) **else if** $x > cur.key$ **then** $cur \leftarrow cur.right$.
 - (d) **else return** (ключ уже есть).

3. Создать новую вершину z с $z.\text{key} = x$, $z.\text{left} = z.\text{right} = \text{NIL}$, $z.\text{parent} = p$.
4. **if** $p = \text{NIL}$ **then** $T.\text{root} \leftarrow z$ **else if** $x < p.\text{key}$ **then** $p.\text{left} \leftarrow z$ **else** $p.\text{right} \leftarrow z$.

Erase(T, x):

1. $z \leftarrow \text{Find}(T, x)$.
2. **if** $z = \text{NIL}$ **then return**.
3. **if** $z.\text{left} = \text{NIL}$ **then**
 - (a) $\text{Transplant}(T, z, z.\text{right})$.
 - (b) Удалить вершину z .
 - (c) **return**.
4. **if** $z.\text{right} = \text{NIL}$ **then**
 - (a) $\text{Transplant}(T, z, z.\text{left})$.
 - (b) Удалить вершину z .
 - (c) **return**.
5. // два сына
6. $y \leftarrow \text{Minimum}(z.\text{right})$ (in-order successor)
7. **if** $y.\text{parent} \neq z$ **then**
 - (a) $\text{Transplant}(T, y, y.\text{right})$.
 - (b) $y.\text{right} \leftarrow z.\text{right}$; $y.\text{right}.\text{parent} \leftarrow y$.
8. $\text{Transplant}(T, z, y)$.
9. $y.\text{left} \leftarrow z.\text{left}$; $y.\text{left}.\text{parent} \leftarrow y$.
10. Удалить вершину z .

Замечание. В учебных задачах на память обычно не обращают внимания, но в реальном коде не забудьте про освобождение памяти (или используйте `std::unique_ptr`).

50 Билет №50. AVL-дерево: определение. Оценка глубины AVL-дерева на n вершинах

50.1 Определения

Определение 50.1 (Высота (глубина) бинарного дерева). Высотой (глубиной) бинарного дерева будем называть максимальное число **вершин** на пути от корня до листа. Эквивалентно, если обозначить через $h(v)$ высоту поддерева вершины v , то удобно договориться, что

$$h(\emptyset) = 0, \quad h(v) = 1 + \max(h(\text{left}(v)), h(\text{right}(v))).$$

Тогда высота дерева равна $h(\text{root})$.

Определение 50.2 (AVL-дерево). *AVL-дерево* — это бинарное дерево поиска, для которого для **каждой** вершины v выполнено

$$|h(\text{left}(v)) - h(\text{right}(v))| \leq 1.$$

Величину $\text{bf}(v) = h(\text{right}(v)) - h(\text{left}(v)) \in \{-1, 0, 1\}$ называют *баланс-фактором* вершины.

Замечание. BST-свойство (дерева поиска): для любой вершины v все ключи в левом поддереве строго меньше ключа в v , а все ключи в правом поддереве строго больше. В этом билете нас интересует именно AVL-ограничение на высоты поддеревьев.

50.2 Оценка глубины AVL-дерева

Теорема 50.1. Если AVL-дерево содержит n вершин, то его высота h удовлетворяет

$$h = O(\log n).$$

Более точно,

$$h \leq \log_{\varphi}(n + 1), \quad \text{где } \varphi = \frac{1 + \sqrt{5}}{2}.$$

Δ *Доказательство.* Обозначим через $S(h)$ минимально возможное число вершин в AVL-дерева высоты h (при соглашении $h(\emptyset) = 0$). Очевидно, $S(0) = 0$ и $S(1) = 1$.

Пусть $h \geq 2$, и рассмотрим AVL-дерево минимального размера высоты h . У корня хотя бы одно поддерево имеет высоту $h - 1$ (иначе высота всего дерева была бы меньше h). Из условия AVL второе поддерево должно иметь высоту *не меньше* $h - 2$. Чтобы число вершин было минимальным, берём второе поддерево минимально возможной высоты $h - 2$. Значит,

$$S(h) = 1 + S(h - 1) + S(h - 2). \quad (50.1)$$

Теперь введём числа Фибоначчи: $F_0 = 0$, $F_1 = 1$, $F_k = F_{k-1} + F_{k-2}$. Покажем по индукции, что

$$S(h) = F_{h+2} - 1. \quad (50.2)$$

База: $h = 0$: $S(0) = 0$, а $F_2 - 1 = 1 - 1 = 0$; $h = 1$: $S(1) = 1$, а $F_3 - 1 = 2 - 1 = 1$. Переход: используя (50.1) и предположение индукции,

$$S(h) = 1 + (F_{h+1} - 1) + (F_h - 1) = F_{h+2} - 1.$$

Остаётся связать числа Фибоначчи с экспонентой. Докажем вспомогательную оценку: для всех $k \geq 2$ выполнено

$$F_k \geq \varphi^{k-2}, \quad \varphi = \frac{1 + \sqrt{5}}{2}. \quad (50.3)$$

Индукция по k . База: $k = 2$: $F_2 = 1 = \varphi^0$. Переход: пусть (50.3) верно для k и $k - 1$. Тогда

$$F_{k+1} = F_k + F_{k-1} \geq \varphi^{k-2} + \varphi^{k-3} = \varphi^{k-3}(\varphi + 1) = \varphi^{k-3}\varphi^2 = \varphi^{k-1},$$

поскольку $\varphi^2 = \varphi + 1$.

Теперь из (50.2) и (50.3) получаем: если AVL-дерево имеет высоту h , то

$$n \geq S(h) = F_{h+2} - 1 \geq \varphi^h - 1.$$

То есть $n + 1 \geq \varphi^h$, откуда

$$h \leq \log_{\varphi}(n + 1) = O(\log n).$$

■

Замечание. Иногда высоту считают числом *рёбер* на пути от корня до листа. Тогда оценки отличаются на константу и асимптотика остаётся той же: $h = O(\log n)$.

Замечание. Логарифм по основанию φ можно переписать через двоичный логарифм:

$$\log_{\varphi}(n + 1) = \frac{\log_2(n + 1)}{\log_2 \varphi} \approx 1.44 \log_2(n + 1).$$

То есть высота AVL-дерева не превосходит примерно $1.44 \log_2 n$ (с точностью до аддитивной константы).

51 Билет №51. Устранение дисбаланса в AVL-дереве

Определение 51.1. AVL-дерево — это бинарное дерево поиска, в котором для каждой вершины v выполнено

$$|h(v_{\text{left}}) - h(v_{\text{right}})| \leq 1,$$

где $h(\cdot)$ — высота поддерева (удобно считать $h(\text{nullptr}) = -1$, тогда высота листа равна 0).

51.1 Что такое «дисбаланс» и где он возникает

Будем хранить в каждой вершине:

- ▷ ключ $key(v)$ (для свойства бинарного дерева поиска);
- ▷ указатели на детей $v.left, v.right$;
- ▷ высоту $h(v) = 1 + \max\{h(v.left), h(v.right)\}$.

Определим **баланс-фактор** вершины (фиксируем знак один раз и везде используем его):

$$bf(v) = h(v.right) - h(v.left).$$

Тогда условие AVL эквивалентно $|bf(v)| \leq 1$ для всех v .

После операции **insert** или **erase** (вставка/удаление листа или перестановка вершины при удалении) изменяются высоты только на пути от места изменения к корню. Поэтому дисбаланс (если он возник) тоже появляется только на этом пути.

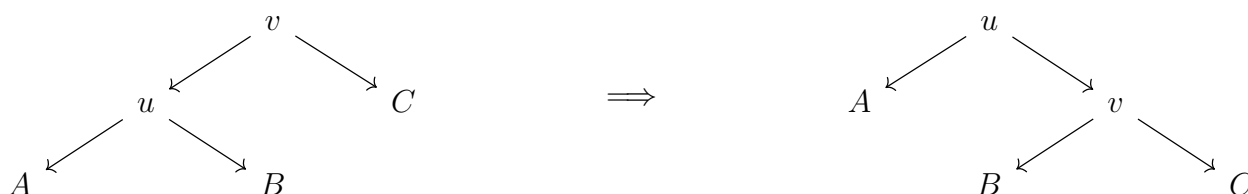
Определение 51.2. Вершина v называется **разбалансированной**, если $|bf(v)| = 2$.

Ключевой факт для балансировки: достаточно найти **первую** (ближайшую к месту изменения) разбалансированную вершину v на пути к корню и локально исправить её одним (одинарным) или двумя (двойным) поворотами.

51.2 Повороты как локальные преобразования

51.2.1 Правый поворот (Right rotation)

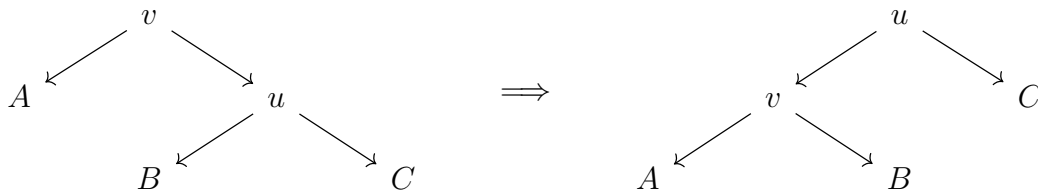
Пусть v — корень рассматриваемого поддерева, $u = v.left$. Обозначим поддеревья буквами, чтобы не потеряться:



То есть u поднимается вверх, v опускается вправо, а B переезжает из $u.right$ в $v.left$.

51.2.2 Левый поворот (Left rotation)

Полностью симметрично. Пусть v — корень поддерева, $u = v.right$:



51.2.3 Важно: поворот сохраняет свойство BST

Лемма 51.1. *И левый, и правый поворот сохраняют порядок ключей бинарного дерева поиска.*

△ *Доказательство.* Докажем для правого поворота; левый — симметрично.

До поворота верно:

- ▷ все ключи в A меньше $key(u)$;
- ▷ все ключи в B лежат между $key(u)$ и $key(v)$ (так как B — правое поддерево u , но при этом u — левый сын v);
- ▷ все ключи в C больше $key(v)$.

После поворота структура поддеревьев A, B, C не меняется, меняются только «склейки» между u и v . Поддерево B становится левым поддеревом v , и это корректно, потому что все ключи в B по-прежнему меньше $key(v)$ и больше $key(u)$. Следовательно, все неравенства для BST сохраняются. ■

51.3 Четыре случая устранения дисбаланса

Пусть v — первая разбалансированная вершина на пути к корню (то есть $|bf(v)| = 2$). Обозначим «тяжёлого» сына u той стороны, где высота больше.

51.3.1 Случай 1: $bf(v) = -2$ (перевес влево)

Здесь $h(v.left) = h(v.right) + 2$, то есть левое поддерево слишком высокое. Положим $u = v.left$.

1. LL-случай (лево-лево): $bf(u) \leq 0$.

Интуиция: рост высоты (или «перевес») идёт по левым рёбрам: v тяжёлый влево и u не тяжёлый вправо.

Исправление: **одинарный правый поворот** вокруг v .

2. LR-случай (лево-право): $bf(u) > 0$.

Интуиция: v тяжёлый влево, но его левый сын u тяжёлый вправо.

Исправление: **двойной поворот**: сначала **левый** поворот вокруг u , затем **правый** поворот вокруг v .

51.3.2 Случай 2: $bf(v) = +2$ (перевес вправо)

Полная симметрия предыдущего. Положим $u = v.right$.

1. **RR-случай (право-право):** $bf(u) \geq 0$.

Исправление: **одинарный левый поворот** вокруг v .

2. **RL-случай (право-лево):** $bf(u) < 0$.

Исправление: **двойной поворот**: сначала **правый** поворот вокруг u , затем **левый** поворот вокруг v .

51.4 Почему достаточно этих случаев

Лемма 51.2. Если $|bf(v)| = 2$, то единственный способ восстановить условие $|bf| \leq 1$ локально (не меняя множества ключей и сохраняя BST-порядок) — это применить один из четырёх сценариев **LL**, **LR**, **RR**, **RL**.

△ *Доказательство.* Если $|bf(v)| = 2$, «перевес» находится либо слева, либо справа. Чтобы уменьшить высоту тяжёлого поддерева и/или увеличить высоту лёгкого, надо поднять вверх одного из детей v (иначе высоты поддеревьев v не изменятся). Поднять можно либо левого ребёнка (даёт правый поворот), либо правого (даёт левый поворот). Если при этом внутренний ребёнок (например, $u.right$ в левом перевесе) «тяжелее», одного поворота недостаточно, и нужен предварительный поворот на сыне (двойной поворот). Других локальных преобразований, сохраняющих BST-порядок и меняющих высоты нужным образом, не требуется. ■

51.5 Пересчёт высот после поворотов

После любого поворота меняются только 2–3 вершины (те, которые участвовали в поворотах), поэтому высоты можно обновить за $O(1)$.

- ▷ **Одинарный поворот:** достаточно пересчитать высоты сначала у опустившейся вершины (бывшего корня поддерева), затем у поднимающейся.
- ▷ **Двойной поворот:** аналогично, но вершин три; пересчёт делается снизу вверх в порядке, соответствующем фактической перестановке.

Удобная реализация (псевдокод):

Псевдокод (идейно, близко к CLRS)

Считаем $HEIGHT(NIL) = -1$.

Height(v):

1. if $v = NIL$ then return -1 else return $v.h$

Recalc(v):

(пересчитать высоту)

1. $v.h \leftarrow 1 + \max(HEIGHT(v.left), HEIGHT(v.right))$

BF(v): return HEIGHT($v.right$) – HEIGHT($v.left$).

RotateRight(v):

1. $u \leftarrow v.left$
2. $B \leftarrow u.right$
3. $u.right \leftarrow v$
4. $v.left \leftarrow B$
5. RECALC(v); RECALC(u)
6. **return** u

RotateLeft(v):

1. $u \leftarrow v.right$
2. $B \leftarrow u.left$
3. $u.left \leftarrow v$
4. $v.right \leftarrow B$
5. RECALC(v); RECALC(u)
6. **return** u

Rebalance(v):

1. RECALC(v)
2. **if** BF(v) = -2 **then**
 - (a) **if** BF($v.left$) ≤ 0 **then return** ROTATERIGHT(v) (LL)
 - (b) $v.left \leftarrow$ ROTATELEFT($v.left$) (LR шаг 1)
 - (c) **return** ROTATERIGHT(v) (LR шаг 2)
3. **if** BF(v) = $+2$ **then**
 - (a) **if** BF($v.right$) ≥ 0 **then return** ROTATELEFT(v) (RR)
 - (b) $v.right \leftarrow$ ROTATERIGHT($v.right$) (RL шаг 1)
 - (c) **return** ROTATELEFT(v) (RL шаг 2)
4. **return** v

51.6 Корректность балансировки

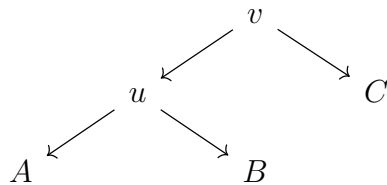
Теорема 51.1. Пусть поддереву с корнем v было бинарным деревом поиска, и нарушено условие AVL только в вершине v (то есть $|bf(v)| = 2$, а ниже по дереву все вершины AVL-корректны). Тогда применение $rebalance(v)$:

1. сохраняет множество ключей поддерева и BST-порядок;
2. восстанавливает условие $|bf| \leq 1$ во всех вершинах полученного поддерева;
3. обновляет высоты корректно.

△ *Доказательство.* (1) Порядок BST сохраняется каждым элементарным поворотом (лемма выше), а двойной поворот — композиция двух поворотов.

(2) Покажем, что после балансировки все баланс-факторы в затронутой «малой» области становятся в пределах $\{-1, 0, 1\}$. Рассмотрим подробно два «левые» случая $bf(v) = -2$; правые полностью симметричны.

LL (одинарный правый поворот). Пусть



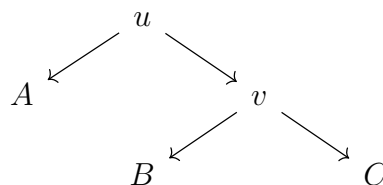
где $u = v.left$. Обозначим $a = h(A)$, $b = h(B)$, $c = h(C)$. Из условия $bf(v) = -2$ получаем

$$c - h(u) = -2 \quad \Rightarrow \quad h(u) = c + 2 \quad \Rightarrow \quad \max\{a, b\} = c + 1.$$

Из $bf(u) \leq 0$ следует $b - a \leq 0$, то есть $b \leq a$. Вместе с $|a - b| \leq 1$ (так как u AVL-корректна) имеем:

$$a = c + 1, \quad b \in \{c, c + 1\}.$$

После правого поворота получаем



Тогда

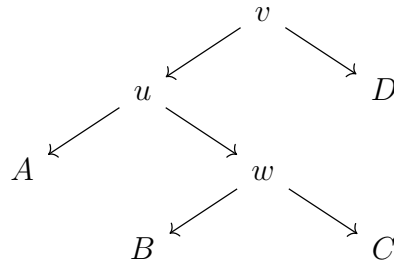
$$bf(v) = h(C) - h(B) = c - b \in \{0, -1\},$$

и, значит, $|bf(v)| \leq 1$. Кроме того,

$$h(v) = 1 + \max\{b, c\} = 1 + (c + 1) = c + 2, \quad bf(u) = h(v) - h(A) = (c + 2) - (c + 1) = 1,$$

то есть $|bf(u)| \leq 1$. Итак, LL-поворот действительно восстанавливает AVL-условие в вершинах u и v .

LR (двойной поворот). Пусть $u = v.left$, $w = u.right$ и схема такова:



Обозначим $a = h(A)$, $b = h(B)$, $c = h(C)$, $d = h(D)$. Из $bf(v) = -2$ имеем $h(u) = d + 2$. Условие $bf(u) > 0$ означает, что u «тяжёл» вправо, значит

$$h(w) = a + 1 \quad (\text{поскольку } |bf(u)| \leq 1).$$

Тогда

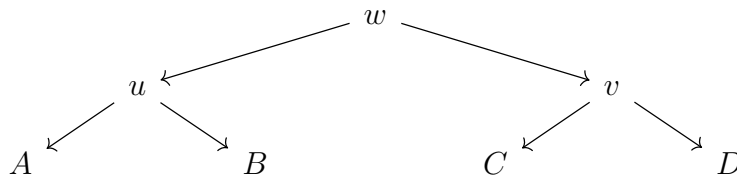
$$h(u) = 1 + \max\{a, h(w)\} = 1 + (a + 1) = a + 2.$$

Сравнивая с $h(u) = d + 2$, получаем $a = d$.

Вершина w AVL-корректна, значит $|b - c| \leq 1$ и

$$h(w) = 1 + \max\{b, c\} = a + 1 \quad \Rightarrow \quad \max\{b, c\} = a.$$

После двойного поворота (левого вокруг u , затем правого вокруг v) получаем:



Теперь

$$h(u) = 1 + \max\{a, b\} = a + 1, \quad bf(u) = h(B) - h(A) = b - a \in \{0, -1\},$$

так как $b \leq a$ (ведь $\max\{b, c\} = a$). Аналогично

$$h(v) = 1 + \max\{c, d\} = 1 + \max\{c, a\} = a + 1, \quad bf(v) = h(D) - h(C) = a - c \in \{0, 1\}.$$

И, наконец,

$$bf(w) = h(v) - h(u) = (a + 1) - (a + 1) = 0.$$

Значит, после LR-поворота все три вершины u, v, w удовлетворяют AVL-условию.

Симметричные рассуждения работают для случаев $bf(v) = +2$ (RR и RL).

(3) Высоты пересчитываются по определению $h = 1 + \max$, причём затронуты только вершины, участвовавшие в поворотах. ■

51.7 Сколько раз приходится балансировать

▷ **После вставки.** Поднимаемся от вставленного листа к корню, обновляя высоты.

Первая вершина v с $|bf(v)| = 2$ балансируется одним вызовом `rebalance`. После это-

го высота соответствующего поддеревья восстанавливается так, что выше по дереву высоты уже не меняются — можно остановиться.

- ▷ **После удаления.** Высота поддеревья может уменьшаться, поэтому дисбаланс может «всплывать» несколько раз на пути к корню. В этом случае идём до корня и в каждой вершине выполняем **rebalance** (если нужно).

51.8 Асимптотика

Высота AVL-дерева равна $O(\log n)$, поэтому путь от изменённой вершины до корня имеет длину $O(\log n)$.

- ▷ Один вызов **rebalance** делает не более двух поворотов и $O(1)$ пересчётов высот, то есть работает за $O(1)$.
- ▷ **insert**: $O(\log n)$ на поиск места + $O(\log n)$ на подъём с пересчётом высот и (максимум) одной балансировкой.
- ▷ **erase**: $O(\log n)$ на поиск/удаление + $O(\log n)$ на подъём с возможными балансировками.

Итого устранение дисбаланса в AVL-дереве укладывается в $O(\log n)$ на операцию изменения дерева.

52 Билет №52. AVL-дерево: реализация операций `find`, `insert` и `erase`

52.1 Модель данных и инварианты

AVL-дерево — это бинарное дерево поиска (BST) с дополнительным ограничением на баланс высот.

Определение 52.1 (BST-инвариант). Для любой вершины v со значением (ключом) $k(v)$ выполняется:

- ▷ все ключи в левом поддереве v строго меньше $k(v)$;
- ▷ все ключи в правом поддереве v строго больше $k(v)$.

(Если разрешены дубликаты, нужно отдельно фиксировать политику: хранить счётчик, либо относить равные влево/вправо. Ниже считаем, что ключи различны, а повторный `insert` игнорируется.)

Определение 52.2 (Высота). Высота (глубина) поддерева $h(v)$ — длина (в вершинах) максимального пути от v до листа. Договоримся: $h(\text{nullptr}) = 0$, тогда лист имеет высоту 1.

Определение 52.3 (AVL-инвариант). Для любой вершины v выполняется

$$|h(\text{right}(v)) - h(\text{left}(v))| \leq 1.$$

Эквивалентно, если определить *balance factor*

$$\text{bf}(v) = h(\text{right}(v)) - h(\text{left}(v)),$$

то для всех вершин $\text{bf}(v) \in \{-1, 0, 1\}$.

Что храним в вершине.

- ключ k ;
- указатели (или индексы) на детей `left`, `right`;
- высоту поддерева h .

52.2 Вспомогательные процедуры

Ниже `height(v)` возвращает $h(v)$, а для `nullptr` возвращает 0. Высоту пересчитываем формулой

$$h(v) = 1 + \max\{h(\text{left}(v)), h(\text{right}(v))\}.$$

Определение 52.4 (Повороты). *Правый поворот* вокруг вершины p (случай LL) и *левый поворот* вокруг вершины q (случай RR) — локальные операции, которые

- ▷ сохраняют BST-инвариант (in-order обход не меняется);

- ▷ меняют высоты и баланс факторов только в $O(1)$ вершинах;
- ▷ уменьшают модуль дисбаланса в проблемном поддереве.

Утверждение 52.1 (Повороты сохраняют порядок ключей). Пусть в некоторой вершине нарушился AVL-инвариант. Тогда после выполнения соответствующего (одинарного или двойного) поворота BST-инвариант сохраняется.

△ *Доказательство.* Любой поворот — это локальная перестановка трёх поддеревьев (обозначим их α, β, γ), причём единственный допустимый способ подвешивания поддеревьев после поворота выбирается так, чтобы слева оставались элементы $< k$, а справа — $> k$. Эквивалентно: in-order обход до и после поворота даёт одну и ту же возрастающую последовательность ключей. ■

52.2.1 Псевдокод (идейно, близко к CLRS)

Псевдокод (идейно, близко к CLRS)

Будем считать, что $\text{height}(\text{NIL}) = 0$, а поле h хранит высоту поддерева.

FixHeight(v): (пересчитать высоту)

1. $v.h \leftarrow 1 + \max(\text{height}(v.\text{left}), \text{height}(v.\text{right}))$

BF(v): return $\text{height}(v.\text{right}) - \text{height}(v.\text{left})$.

RotateRight(p): (LL-поворот)

1. $q \leftarrow p.\text{left}$

2. $p.\text{left} \leftarrow q.\text{right}$

3. $q.\text{right} \leftarrow p$

4. **FIXHEIGHT**(p); **FIXHEIGHT**(q)

5. **return** q (новый корень поддерева)

RotateLeft(q): (RR-поворот)

1. $p \leftarrow q.\text{right}$

2. $q.\text{right} \leftarrow p.\text{left}$

3. $p.\text{left} \leftarrow q$

4. **FIXHEIGHT**(q); **FIXHEIGHT**(p)

5. **return** p (новый корень поддерева)

Balance(v): (восстановить AVL-инвариант в корне поддерева v)

1. **FIXHEIGHT**(v)

```

2. if  $\text{BF}(v) = 2$  then
    (a) if  $\text{BF}(v.\text{right}) < 0$  then  $v.\text{right} \leftarrow \text{ROTATERIGHT}(v.\text{right})$  (RL)
    (b) return  $\text{ROTATELEFT}(v)$  (RR)
3. if  $\text{BF}(v) = -2$  then
    (a) if  $\text{BF}(v.\text{left}) > 0$  then  $v.\text{left} \leftarrow \text{ROTATELEFT}(v.\text{left})$  (LR)
    (b) return  $\text{ROTATERIGHT}(v)$  (LL)
4. return  $v$ 

```

Замечание. Функция $\text{balance}(v)$ реализует всю «перебалансировку»: пересчитывает высоту и, если $\text{bf}(v) = \pm 2$, делает нужный поворот(ы) (см. билет 51).

52.3 Операция find

Задача. По ключу x определить, присутствует ли он в дереве (и при необходимости вернуть указатель на вершину).

Алгоритм. Точно как в обычном BST:

- ▷ начинаем в корне;
- ▷ если $x < k(v)$ — идём влево, если $x > k(v)$ — вправо;
- ▷ если дошли до `nullptr` — элемента нет.

Псевдокод (идейно, близко к CLRS)

Find(v, x):

```

1. while  $v \neq \text{NIL}$  and  $\text{key}(v) \neq x$  do
    (a) if  $x < \text{key}(v)$  then  $v \leftarrow v.\text{left}$ 
    (b) else  $v \leftarrow v.\text{right}$ 
2. return  $v$  (NIL, если не нашли)

```

Утверждение 52.2 (Корректность find). *find* возвращает вершину с ключом x , если она существует, и `nullptr` иначе.

$\triangle$ *Доказательство.* Инвариант цикла: текущая вершина v является корнем поддерева, внутри которого *единственно возможно* нахождение ключа x . Если $x < k(v)$, то по BST-инварианту все ключи правого поддерева строго больше $k(v)$ и не могут равняться x ; значит, искать нужно только слева. Аналогично для $x > k(v)$. Если мы пришли в `nullptr`, то все потенциальные места для x были отброшены, следовательно x отсутствует. Если встретили $k(v) = x$, то нашли требуемую вершину. ■

Асимптотика. $O(h)$, где h — высота дерева; для AVL $h = O(\log n)$ (см. билет 50).

52.4 Операция insert

Задача. Добавить ключ x в множество ключей дерева, сохранив BST- и AVL-инварианты.

Идея.

1. Вставляем x как в обычное BST: спускаемся до `nullptr` и создаём новый лист.
2. Возвращаясь из рекурсии, пересчитываем высоты и вызываем `balance` на каждой вершине пути.

Псевдокод (идейно, близко к CLRS)

Insert(v, x):

1. if $v = \text{NIL}$ then return `NEWNODE`(x) ($h=1$)
2. if $x < \text{key}(v)$ then $v.\text{left} \leftarrow \text{INSERT}(v.\text{left}, x)$
3. else if $x > \text{key}(v)$ then $v.\text{right} \leftarrow \text{INSERT}(v.\text{right}, x)$
4. else return v (дубликат: ничего не делаем)
5. return `BALANCE`(v)

Утверждение 52.3 (Корректность insert). После `insert(root, x)` дерево остаётся корректным AVL-деревом и содержит ключ x (если его раньше не было).

$\triangle$ **Доказательство. BST-инвариант.** Новый ключ добавляется листом в единственное место, куда приводит стандартный BST-спуск; другие ключи не перемещаются, значит порядок сохраняется. Далее выполняются только повороты, а они сохраняют BST-инвариант.

AVL-инвариант. Вставка добавляет ровно одну вершину, поэтому высота может измениться только на пути от места вставки к корню. На каждой вершине этого пути `balance` пересчитывает высоту и при необходимости исправляет дисбаланс ± 2 одинарным/двойным поворотом. После этого корень рассматриваемого поддерева удовлетворяет AVL-инварианту. Поднимаясь до корня, восстанавливаем инвариант во всём дереве. ■

Асимптотика. Спуск и подъём идут по одной ветке длины h , на каждой вершине делаем $O(1)$ работы; итого $O(h) = O(\log n)$.

Замечание. Классический факт: при вставке в AVL достаточно перебалансировать *первую* сверху вершину, где дисбаланс стал ± 2 . Но даже балансировка на всём пути не меняет оценки $O(\log n)$.

52.5 Операция erase

Задача. Удалить ключ x из дерева, сохранив BST- и AVL-инварианты.

Структурное удаление (как в BST). После нахождения вершины v с ключом x :

- ▷ **0 детей:** удалить вершину и вернуть `nullptr`;
- ▷ **1 ребёнок:** удалить вершину и вернуть единственного ребёнка;
- ▷ **2 ребёнка:** заменить x на *следующий* ключ (минимум в правом поддереве), затем удалить этот минимум из правого поддерева.

После структурного изменения при подъёме вверх вызываем `balance`.

Псевдокод (идейно, близко к CLRS)

FindMin(v):

1. if $v.\text{left} = \text{NIL}$ then return v
2. else return FINDMIN($v.\text{left}$)

RemoveMin(v): (вырезает минимум и возвращает новый корень поддерева)

1. if $v.\text{left} = \text{NIL}$ then return $v.\text{right}$
2. $v.\text{left} \leftarrow \text{REMOVE_MIN}(v.\text{left})$
3. return BALANCE(v)

Erase(v, x):

1. if $v = \text{NIL}$ then return NIL
2. if $x < \text{key}(v)$ then $v.\text{left} \leftarrow \text{ERASE}(v.\text{left}, x)$
3. else if $x > \text{key}(v)$ then $v.\text{right} \leftarrow \text{ERASE}(v.\text{right}, x)$
4. else (нашли вершину с ключом x)
 - (a) $L \leftarrow v.\text{left}, \quad R \leftarrow v.\text{right}$
 - (b) DELETE(v)
 - (c) if $R = \text{NIL}$ then return L (0 или 1 ребёнок)
 - (d) $m \leftarrow \text{FINDMIN}(R)$ (*in-order successor*)
 - (e) $m.\text{right} \leftarrow \text{REMOVE_MIN}(R)$
 - (f) $m.\text{left} \leftarrow L$
 - (g) return BALANCE(m)
5. return BALANCE(v)

Утверждение 52.4 (Корректность erase). После `erase(root, x)` дерево остаётся корректным AVL-деревом и не содержит ключ x .

$\triangle$ *Доказательство. BST-инвариант.* Случаи 0/1 ребёнка сохраняют порядок очевидно. В случае 2 детей берём минимум m в правом поддереве: по определению $k(m)$ больше всех ключей левого поддерева и не меньше ключа x , а также не больше остальных ключей справа, поэтому замена ключа на $k(m)$ сохраняет BST-порядок. Удаление минимума из правого поддерева сводится к случаю 0/1 ребёнка. Повороты внутри `balance` BST-инвариант не нарушают.

AVL-инвариант. Удаление уменьшает высоту некоторых поддеревьев максимум на 1, поэтому потенциальный дисбаланс появляется только на пути к корню. Поднимаясь вверх, применяем `balance` и исправляем все локальные дисбалансы ± 2 ; следовательно AVL-инвариант восстановлен во всём дереве. ■

Асимптотика. В худшем случае балансировка нужна на каждом уровне, но уровней $h = O(\log n)$, а работа на уровне $O(1)$, значит `erase` работает за $O(\log n)$.

52.6 Итоговые оценки

Пусть n — число вершин AVL-дерева. Тогда (см. билет 50) $h = O(\log n)$, и

`find`, `insert`, `erase` работают за $O(\log n)$ в худшем случае.

Память: $O(n)$.

53 Билет №53. Splay-дерево: определение и практическая значимость

Определения

Определение 53.1 (Бинарное дерево поиска (BST)). Бинарное дерево поиска для множества ключей S — это бинарное дерево, в каждой вершине которого хранится ключ $key(v)$, и выполнено свойство порядка:

- ▷ все ключи в левом поддереве вершины v строго меньше $key(v)$;
- ▷ все ключи в правом поддереве вершины v строго больше $key(v)$.

Это условие гарантирует, что симметричный (inorder) обход перечисляет ключи в возрастающем порядке.

Определение 53.2 (Splay-дерево). *Splay-дерево* — это бинарное дерево поиска без явного условия балансировки (в отличие от AVL/красно-чёрного), но со следующим правилом самоадаптации: после каждого обращения к некоторой вершине x (например, после `find`, `insert`, `erase`) выполняется операция `splay(x)`, которая поднимает x в корень дерева.

Определение 53.3 (Операция `splay`). Операция `splay(x)` — это последовательность локальных поворотов (rotation) вдоль рёбер на пути от x к корню, в результате которой вершина x становится корнем. Стандартно используются три “шаблона” поворотов: `zig`, `zig-zig`, `zig-zag` (подробно — в билете 54).

Утверждение 53.1 (Корректность (сохранение BST)). *Каждый локальный поворот (rotation) сохраняет свойство бинарного дерева поиска, следовательно, после `splay` структура остаётся BST.*

△ *Доказательство.* Поворот меняет местами родителя и ребёнка и переподвешивает одно поддерево так, чтобы относительный порядок ключей не нарушился. Эквивалентно: симметричный обход (inorder) до и после поворота выдаёт один и тот же отсортированный список ключей, поэтому BST-инвариант сохраняется. ■

Интуиция работы

Идея splay-дерева — “подстраиваться” под последовательность запросов: вершины, к которым обращались недавно, оказываются ближе к корню. Поэтому если в запросах есть *локальность* (повторные обращения к одним и тем же ключам или к соседним ключам), то дерево само перестраивается так, чтобы такие запросы обрабатывались быстрее.

Практическая значимость

- ▷ **Простая реализация и экономия памяти.** В вершине нужно хранить только ключ и ссылки на детей/родителя. В отличие от AVL (высота/баланс) и красно-чёрного

дерева (цвет), нет дополнительных полей для балансировки.

- ▷ **Амортизированная эффективность без жёсткой балансировки.** Хотя отдельная операция может занять $\Theta(n)$ (дерево может стать “цепочкой”), для любой последовательности запросов суммарное время обычно оценивается как $O(m \log n)$, где m — число операций, n — текущее число элементов. Это и означает амортизированное $O(\log n)$ на операцию.
- ▷ **Адаптивность к паттернам доступа.** Splay-деревья обладают сильными адаптивными свойствами (“последние/частые/близкие” элементы поднимаются выше). На практике это полезно, когда данные запрашиваются неравномерно (есть “горячие” ключи).
- ▷ **Удобные split/merge и “дерево по неявному ключу”.** Splay-дерево естественно поддерживает операции разделения и слияния (амортизированно за $O(\log n)$), а также популярную технику “неявного ключа”: хранить последовательность, где ключ — это позиция в массиве. Тогда можно делать вставки/удаления по позиции, запросы на подотрезках (сумма, минимум и т.п.) при стандартных дополнениях в вершинах.
- ▷ **Строительный блок более сложных структур.** Splay-деревья используются как базовый компонент в динамических деревьях (например, link-cut trees) и в некоторых реализациях “горе” (редактируемые строки).

Замечание. Если в задаче требуется *жёсткая* гарантия “каждый запрос не дольше $O(\log n)$ ”, то обычно выбирают AVL/красно-чёрное дерево. Splay-дерево выбирают, когда важна простота, небольшие константы и/или адаптивность к последовательности запросов.

54 Билет №54. Splay-дерево: операции zig, zig-zig и zig-zag, операция splay

54.1 Контекст: что такое splay-дерево

Splay-дерево — это бинарное дерево поиска (BST), в котором после каждого обращения к вершине x выполняют операцию $\text{splay}(x)$: вершина x “всплывает” в корень серией локальных поворотов. Ключевое: **splay** не нарушает BST-инвариант, то есть множество ключей и их симметричный (in-order) порядок остаются теми же.

В дальнейшем будем пользоваться обозначениями:

- ▷ x — вершина, которую нужно поднять;
- ▷ $p = \text{parent}(x)$ — родитель x ;
- ▷ $g = \text{parent}(p)$ — дедушка x (если существует);
- ▷ $\alpha, \beta, \gamma, \delta$ — произвольные поддеревья, которые при поворотах “перекидываются”.

54.2 Базовая операция: одиночный поворот (rotation)

Все “шаги” zig/zig-zig/zig-zag состоят из *одиночных поворотов* вокруг ребра между родителем и ребёнком.

Определение 54.1 (Одиночный поворот). Пусть p — родитель вершины x .

- ▷ Если x — *левый* сын p , выполняется *правый* поворот ($\text{rotateRight}(p)$) вокруг ребра (p, x) .
- ▷ Если x — *правый* сын p , выполняется *левый* поворот ($\text{rotateLeft}(p)$) вокруг ребра (p, x) .

В обоих случаях вершина x поднимается на один уровень вверх, а p опускается на один уровень вниз.

Замечание (Как выглядит поворот на указателях). Для реализации удобно хранить у каждой вершины указатели `left`, `right`, `parent`. Ниже — схема для случая, когда x — левый сын p (правый случай симметричен).

Пусть B — правое поддерево x (оно станет левым поддеревом p после поворота), а $g = \text{parent}(p)$. Тогда правый поворот вокруг (p, x) состоит из константного числа присваиваний:

- ▷ $p.\text{left} \leftarrow B$; если $B \neq \emptyset$, то $B.\text{parent} \leftarrow p$;
- ▷ $x.\text{right} \leftarrow p$ и $p.\text{parent} \leftarrow x$;
- ▷ $x.\text{parent} \leftarrow g$;
- ▷ если $g = \emptyset$, то x становится корнем, иначе у g заменяется соответствующий сын:
 - если p был левым сыном g , то $g.\text{left} \leftarrow x$;
 - иначе $g.\text{right} \leftarrow x$.

Лемма 54.1 (Одиночный поворот сохраняет BST-инвариант). *После любого одиночного поворота симметричный обход (in-order) поддерева не меняется, а значит, дерево остаётся корректным BST.*

△ *Доказательство.* Достаточно рассмотреть случай, когда x — левый сын p (правый случай симметричен). До поворота ключи упорядочены как

$$\alpha < x < \beta < p < \gamma,$$

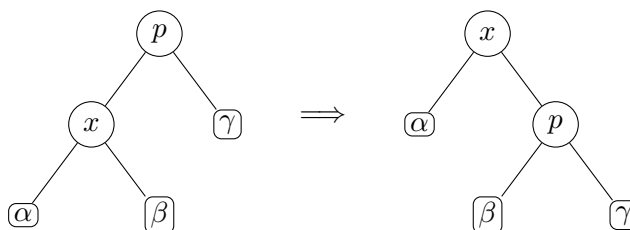
где “<” означает “все ключи в левом поддереве строго меньше всех ключей в правом”. После правого поворота x становится над p , а поддерево β переезжает в левое поддерево p . Порядок ключей остаётся тем же: α остаётся слева от x , β — между x и p , γ — справа от p . Следовательно, BST-свойство сохраняется. ■

54.3 Шаги splay: zig, zig-zig, zig-zag

54.3.1 Шаг zig (одинарный)

Условие применения: p — корень (то есть у x нет дедушки). Тогда выполняем *ровно один* одиночный поворот вокруг ребра (p, x) .

Ниже приведён случай, когда x — левый сын p (случай правого сына — зеркальный).



54.3.2 Шаг zig-zig (двойной, сонаправленный)

Условие применения: p не корень, и x и p находятся по одну сторону:

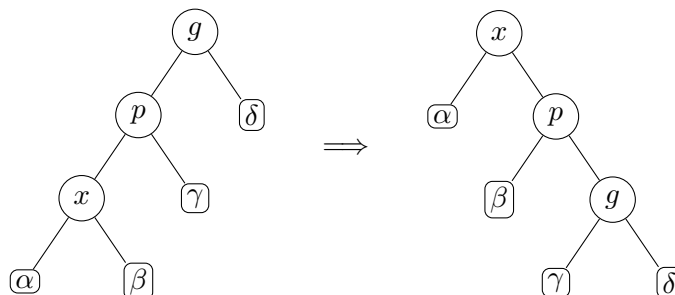
- ▷ либо x — левый сын p и p — левый сын g (left-left),
- ▷ либо x — правый сын p и p — правый сын g (right-right).

Действие: выполняются *два* одиночных поворота в одном направлении:

- ▷ для left-left: rotateRight(g), затем rotateRight(p);
- ▷ для right-right: rotateLeft(g), затем rotateLeft(p).

После этого x поднимается на два уровня (на место g).

Ниже — случай left-left (правый — зеркальный):



54.3.3 Шаг zig-zag (двойной, разнонаправленный)

Условие применения: p не корень, и x и p находятся по разные стороны:

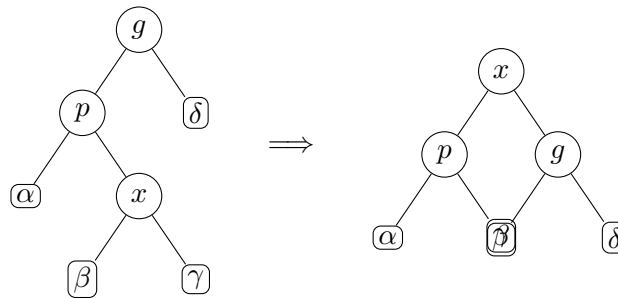
- ▷ либо x — правый сын p , а p — левый сын g (**left-right**),
- ▷ либо x — левый сын p , а p — правый сын g (**right-left**).

Действие: выполняются два одиночных поворота в разных направлениях:

- ▷ для **left-right**: `rotateLeft(p)`, затем `rotateRight(g)`;
- ▷ для **right-left**: `rotateRight(p)`, затем `rotateLeft(g)`.

После этого x также поднимается на два уровня (на место g).

Ниже — случай **left-right** (второй — зеркальный):



54.4 Операция splay

Определение 54.2 (Операция **splay**). `splay(x)` — процедура, которая поднимает вершину x в корень дерева, повторяя шаги **zig/zig-zig/zig-zag**.

Алгоритм (bottom-up).

1. Если x уже корень, остановиться.
2. Пусть $p = \text{parent}(x)$.
3. Если p — корень, выполнить **zig**(x) и остановиться.
4. Иначе пусть $g = \text{parent}(p)$.
5. Если x и p по одну сторону относительно своих родителей (**left-left** или **right-right**), выполнить **zig-zig**(x).
6. Иначе выполнить **zig-zag**(x).
7. Перейти к пункту 1.

Утверждение 54.1 (Корректность **splay**). После выполнения `splay(x)` вершина x становится корнем, а дерево остаётся корректным BST.

△ *Доказательство.* Каждый шаг **zig**, **zig-zig** или **zig-zag** является композицией одного или двух одиночных поворотов, а одиночный поворот сохраняет BST-инвариант (лемма выше). Значит, инвариант сохраняется после каждого шага.

На каждом шаге глубина вершины x строго уменьшается: на 1 в случае **zig** и на 2 в случаях **zig-zig** или **zig-zag**. Поэтому процесс не может продолжаться бесконечно и обязательно закончится, когда x окажется корнем. ■

54.5 Оценка времени

Каждый одиночный поворот меняет константное число указателей, то есть выполняется за $O(1)$. `splay` делает не более $O(h)$ поворотов, где h — исходная глубина вершины x (в худшем случае $h = \Theta(n)$).

Замечание. Ключевой факт про `splay`-дерево: хотя отдельный `splay` может занимать $\Theta(n)$ в худшем случае, в последовательности операций достигается $O(\log n)$ *амортизированно* (это анализируется отдельным билетом).

55 Билет №55. Метод потенциалов

Идея

В амортизированном анализе нас интересует не время *каждой* операции, а суммарное время на *любой* последовательности операций. Метод потенциалов позволяет назначить операциям *учётную* (амортизированную) стоимость, такую что сумма учётных стоимостей (почти) телескопически ограничивает сверху реальное время. Интуитивно потенциал — это «запас работы/монет», накопленный в структуре.

Определения

Рассмотрим последовательность запросов $op_1, \dots, op_m$ к структуре. Пусть D_i — состояние структуры *после* выполнения первых i операций (D_0 — начальное состояние). Через t_i обозначим *реальное* время выполнения операции op_i (число элементарных действий в RAM-модели).

Определение 55.1 (Потенциал). *Потенциалом* называется функция $\Phi(D)$, сопоставляющая каждому состоянию D неотрицательное число. Обычно требуют $\Phi(D) \geq 0$ для всех достижимых состояний и $\Phi(D_0) = 0$.

Определение 55.2 (Амортизированная стоимость). *Амортизированной* (учётной) стоимостью операции op_i относительно потенциала Φ называется

$$\hat{t}_i \stackrel{\text{def}}{=} t_i + \Phi(D_i) - \Phi(D_{i-1}). \quad (55.1)$$

Теорема 55.1 (Телескопическая оценка). *Пусть $\Phi(D) \geq 0$ для всех достижимых D . Тогда для любой последовательности операций длины m верно:*

$$\sum_{i=1}^m t_i \leq \sum_{i=1}^m \hat{t}_i + \Phi(D_0). \quad (55.2)$$

В частности, если $\Phi(D_0) = 0$, то $\sum_{i=1}^m t_i \leq \sum_{i=1}^m \hat{t}_i$.

△ *Доказательство.* Просуммируем определения амортизированных стоимостей:

$$\begin{aligned} \sum_{i=1}^m \hat{t}_i &= \sum_{i=1}^m (t_i + \Phi(D_i) - \Phi(D_{i-1})) \\ &= \sum_{i=1}^m t_i + (\Phi(D_1) - \Phi(D_0)) + (\Phi(D_2) - \Phi(D_1)) + \dots + (\Phi(D_m) - \Phi(D_{m-1})) \\ &= \sum_{i=1}^m t_i + \Phi(D_m) - \Phi(D_0). \end{aligned}$$

Отсюда $\sum_{i=1}^m t_i = \sum_{i=1}^m \hat{t}_i - \Phi(D_m) + \Phi(D_0)$. Так как $\Phi(D_m) \geq 0$, получаем $\sum_{i=1}^m t_i \leq \sum_{i=1}^m \hat{t}_i + \Phi(D_0)$. ■

Как использовать метод на практике

1. Выберите потенциал $\Phi(D)$ так, чтобы он:
 - ▷ был простым для оценки/вычисления;
 - ▷ отражал «запас работы» (например, степень «разбалансированности», число элементов в некотором буфере, и т.п.);
 - ▷ был снизу ограничен (обычно ≥ 0).
2. Для каждой операции оцените сверху $\hat{t}_i = t_i + \Delta\Phi$, где $\Delta\Phi = \Phi(D_i) - \Phi(D_{i-1})$.
3. Если получилось $\hat{t}_i \leq f(\cdot)$ для всех i (в зависимости от параметров задачи), то суммарное реальное время любой последовательности длины m не превосходит $\sum \hat{t}_i = O(m f(\cdot))$.

Связь с методом «монеток» (бухгалтерским учётом)

Метод потенциалов эквивалентен методу учёта: если мы «взимает» с операции $\hat{t}_i$ монет, а реально тратим t_i , то разность $\hat{t}_i - t_i = \Phi(D_i) - \Phi(D_{i-1})$ можно интерпретировать как изменение баланса счёта. Потенциал $\Phi(D)$ — это количество монет «на счёте» в состоянии D . Доказательство из теоремы выше соответствует тому, что счёт не уходит в минус (потенциал неотрицателен), а значит суммарно «взятого» хватает на суммарно «потраченного».

Пример 1: динамический массив (vector)

Рассмотрим операцию `push_back` в динамическом массиве, который хранит:

- ▷ n — текущее число элементов (size);
- ▷ m — вместимость массива (capacity), причём всегда $n \leq m$;
- ▷ при переполнении выполняется увеличение capacity в 2 раза и копирование всех m элементов.

Реальная стоимость одной операции:

- ▷ если $n < m$, то $t = 1$ (просто записали элемент);
- ▷ если $n = m$, то $t = m + 1$ (скопировали m элементов и записали новый).

Выберем потенциал

$$\Phi(n, m) = \max\{0, 2n - m\}. \quad (55.3)$$

Он неотрицателен и удобно «накапливает» запас перед дорогим расширением.

Утверждение 55.1. Для `push_back` при удвоении capacity амортизированная стоимость не превосходит 3.

$\triangle$ *Доказательство.* **Случай 1: без расширения** ($n < m$). Тогда $t = 1$. При увеличении $n \mapsto n+1$ потенциал растёт не более чем на 2: $\Phi(n+1, m) - \Phi(n, m) \leq 2$. Значит $\hat{t} \leq 1 + 2 = 3$.

Случай 2: расширение ($n = m$). До операции: $\Phi_{\text{old}} = \max\{0, 2m - m\} = m$. После расширения: $m' = 2m$, $n' = m + 1$, поэтому $\Phi_{\text{new}} = \max\{0, 2(m + 1) - 2m\} = 2$. Реальная стоимость $t = m + 1$, изменение потенциала $\Delta\Phi = 2 - m$. Тогда

$$\hat{t} = (m + 1) + (2 - m) = 3.$$

■

Следовательно, любая последовательность из q операций `push_back` выполняется за $O(q)$, то есть `push_back` имеет амортизированную стоимость $O(1)$.

Пример 2: очередь на двух стеках

Пусть очередь реализована двумя стеками:

- ▷ A (in-stack): сюда кладём элементы при `enqueue`;
- ▷ B (out-stack): отсюда достаём элементы при `dequeue`.

Операция `dequeue` работает так: если B непуст, то просто `pop` из B . Если B пуст, то переложить все элементы из A в B (последовательностью `pop` из A и `push` в B), после чего сделать `pop` из B .

Выберем потенциал

$$\Phi = 2|A|, \tag{55.4}$$

где $|A|$ — число элементов в стеке A .

Утверждение 55.2. Амортизированная стоимость `enqueue` не превосходит 3, а `dequeue` не превосходит 1.

$\triangle$ *Доказательство.* **enqueue:** реальная стоимость $t = 1$ (один `push` в A). Потенциал увеличился на 2, значит $\hat{t} = 1 + 2 = 3$.

dequeue, случай 1: B непуст. Тогда $t = 1$ (один `pop` из B), а $|A|$ не меняется, поэтому $\Delta\Phi = 0$, и $\hat{t} = 1$.

dequeue, случай 2: B пуст, $|A| = k$. Перекладка k элементов стоит $2k$ (каждый элемент: `pop` из A и `push` в B), и ещё 1 за итоговый `pop` из B : итого $t = 2k + 1$. После перекладки A пуст, значит потенциал падает на $2k$: $\Delta\Phi = -2k$. Следовательно $\hat{t} = (2k + 1) + (-2k) = 1$. ■

Следовательно, обе операции имеют амортизированную стоимость $O(1)$.

56 Билет №56. Амортизированное время работы операции `splay` с помощью метода потенциалов

1. Что оцениваем

Операция `splay(x)` берёт вершину x в бинарном дереве поиска и поднимает её в корень последовательностью поворотов:

- ▷ **zig**: x — сын корня;
- ▷ **zig-zig**: x и его родитель p лежат *по одну сторону* от дедушки g (лево-лево или право-право);
- ▷ **zig-zag**: x и его родитель p лежат *по разные стороны* от дедушки g .

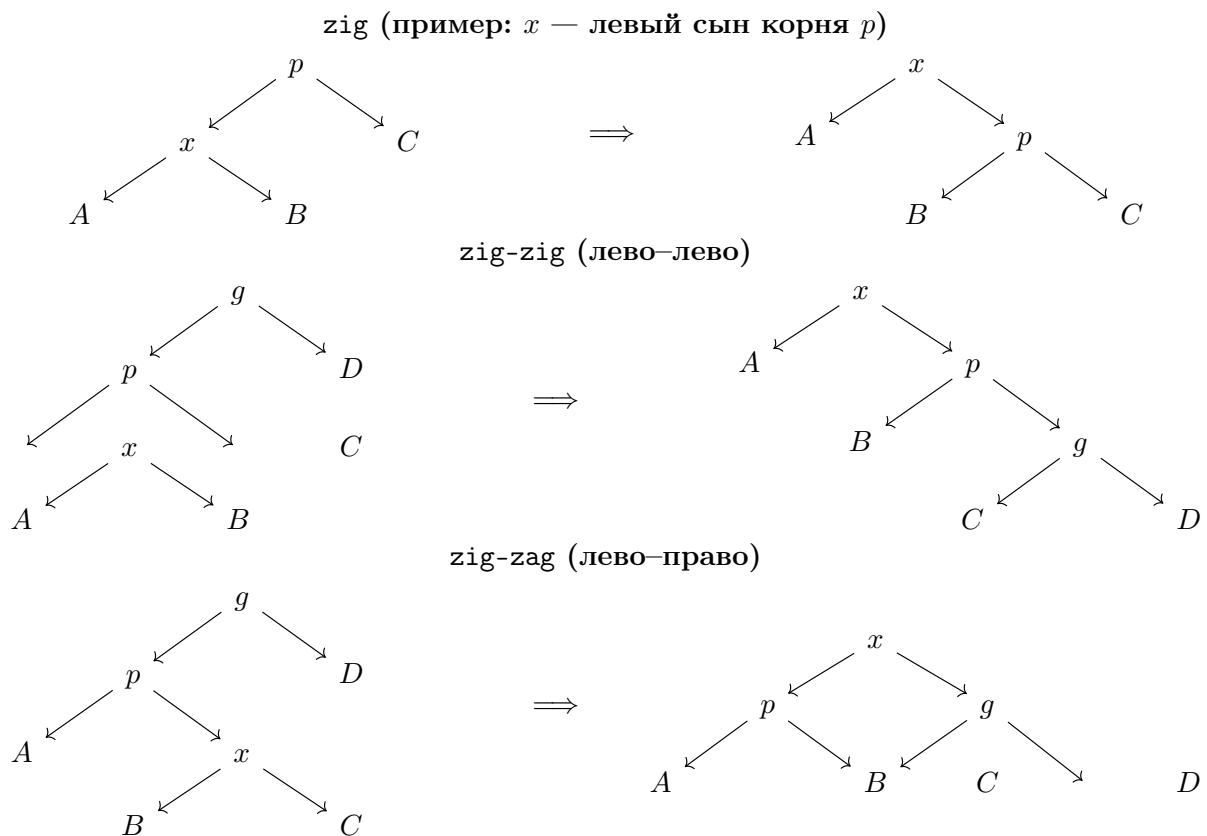


Рис. 1: Три типа `splay`-шагов (показаны «левые» варианты; правые симметричны).

Каждый **zig** — один поворот, каждый **zig-zig/zig-zag** — два поворота.

Модель стоимости: считаем, что один поворот выполняется за $O(1)$, поэтому реальная стоимость операции равна $t = \# \text{поворотов}$ (с точностью до константы).

2. Метод потенциалов (напоминание)

Пусть структура данных находится в состоянии S и после операции переходит в состояние S' . Выбираем *потенциал* $\Phi(S) \geq 0$ (обычно $\Phi(S_0) = 0$ в начале). Амортизированная

стоимость операции определяется как

$$\hat{t} = t + (\Phi(S') - \Phi(S)). \quad (56.1)$$

Тогда для любой последовательности операций $1, \dots, m$ выполняется $\sum_{i=1}^m t_i \leq \sum_{i=1}^m \hat{t}_i$, потому что сумма приращений потенциала телескопируется: $\sum (\Phi_i - \Phi_{i-1}) = \Phi_m - \Phi_0 \geq 0$.

3. Потенциал для splay

Для вершины v обозначим через $s(v)$ размер её поддерева (число вершин в поддереве v , включая v).

Определение 56.1. Ранг вершины v : $r(v) = \log_2 s(v)$. Потенциал дерева T :

$$\Phi(T) = \sum_{v \in T} r(v) = \sum_{v \in T} \log_2 s(v). \quad (56.2)$$

Очевидные факты:

- ▷ Если u — предок v , то $s(u) \geq s(v)$, значит $r(u) \geq r(v)$.
- ▷ Если в дереве n вершин, то $0 \leq r(v) \leq \log_2 n$, поэтому $0 \leq \Phi(T) \leq n \log_2 n$.

4. Лемма: амортизированная стоимость одного *splay-step*

Рассмотрим один шаг алгоритма **splay**, то есть один из случаев **zig**, **zig-zig**, **zig-zag**. Пусть $p = \text{parent}(x)$, $g = \text{parent}(p)$ (если существует). Штрихом ($'$) обозначим значения *после* шага.

Лемма 56.1 (о стоимости шага).

- ▷ (**zig**) $\hat{t} \leq 1 + 3(r'(x) - r(x))$.
- ▷ (**zig-zig** или **zig-zag**) $\hat{t} \leq 3(r'(x) - r(x))$.

$\triangle$ *Доказательство.* Во всех трёх случаях структура поддеревьев, “висящих” на стороне, меняется только у вершин x, p (и g , если есть): остальные поддерева не меняют ни состав, ни размеры, значит их вклад в Φ не меняется.

Случай zig. Реальная стоимость $t = 1$ (один поворот). Меняются ранги только x и p , поэтому

$$\Delta\Phi = \Phi' - \Phi = (r'(x) + r'(p)) - (r(x) + r(p)). \quad (56.3)$$

После поворота поддерево p *уменьшается* (оно теряет вершину x и часть её поддерева), значит $s'(p) \leq s(p)$ и $r'(p) \leq r(p)$. Тогда

$$\hat{t} = 1 + \Delta\Phi \leq 1 + (r'(x) - r(x)). \quad (56.4)$$

Так как $r'(x) \geq r(x)$ (поддерево x не уменьшается), то $1 + (r'(x) - r(x)) \leq 1 + 3(r'(x) - r(x))$.

Случаи zig-zig и zig-zag. Здесь $t = 2$ (два поворота). Меняются ранги только x, p, g . В обоих случаях после шага вершина x становится вершиной того поддерева, которое

раньше было укоренено в g , поэтому

$$s'(x) = s(g) \Rightarrow r'(x) = r(g). \quad (56.5)$$

Отсюда

$$\Delta\Phi = (r'(x) + r'(p) + r'(g)) - (r(x) + r(p) + r(g)) \quad (56.6)$$

$$= (r'(p) + r'(g)) - (r(x) + r(p)). \quad (56.7)$$

Оценим сумму $r'(p) + r'(g)$. После шага вершины p и g становятся *детьми* x , их поддеревья непересекаются, и

$$s'(p) + s'(g) \leq s'(x) - 1 \Rightarrow \frac{s'(p)}{s'(x)} + \frac{s'(g)}{s'(x)} \leq 1. \quad (56.8)$$

Положим $a = \frac{s'(p)}{s'(x)}$, $b = \frac{s'(g)}{s'(x)}$. Тогда $a, b > 0$ и $a + b \leq 1$. По неравенству между средним арифметическим и геометрическим:

$$\sqrt{ab} \leq \frac{a+b}{2} \leq \frac{1}{2} \Rightarrow ab \leq \frac{1}{4} \Rightarrow \log_2 a + \log_2 b = \log_2(ab) \leq -2. \quad (56.9)$$

Теперь

$$r'(p) + r'(g) = \log_2 s'(p) + \log_2 s'(g) = \log_2(as'(x)) + \log_2(bs'(x)) \quad (56.10)$$

$$= 2\log_2 s'(x) + \log_2 a + \log_2 b \leq 2r'(x) - 2. \quad (56.11)$$

Подставляем в амортизированную стоимость:

$$\hat{t} = 2 + \Delta\Phi = 2 + (r'(p) + r'(g)) - r(x) - r(p) \quad (56.12)$$

$$\leq 2 + (2r'(x) - 2) - r(x) - r(p) = 2r'(x) - r(x) - r(p). \quad (56.13)$$

Так как p — предок x , то $r(p) \geq r(x)$, следовательно

$$\hat{t} \leq 2r'(x) - 2r(x) = 2(r'(x) - r(x)) \leq 3(r'(x) - r(x)). \quad (56.14)$$

Лемма доказана. ■

5. Теорема (Access Lemma): амортизированная стоимость $\text{splay}(x)$

Теорема 56.1. Пусть в дереве T n вершин, и мы выполняем $\text{splay}(x)$. Тогда амортизированная стоимость

$$\hat{t}(\text{splay}(x)) \leq 1 + 3(r(\text{root}) - r(x)) = 1 + 3\log_2 \frac{n}{s(x)} \leq 1 + 3\log_2 n = O(\log n). \quad (56.15)$$

$\triangle$ *Доказательство.* Рассмотрим последовательность шагов splay , поднимающих x вверх. Пусть $r_0, r_1, \dots, r_k$ — ранги вершины x после каждого шага (то есть $r_0 = r(x)$ до начала, $r_k = r(\text{root})$ после завершения).

По лемме каждый шаг zig-zig / zig-zag имеет амортизированную стоимость не больше $3(r_i - r_{i-1})$. Шаг zig может произойти не более одного раза (только в самом конце,

когда x стал сыном корня), и его амортизированная стоимость не больше $1 + 3(r_k - r_{k-1})$. Складывая амортизированные стоимости всех шагов, получаем телескопирование:

$$\hat{t}(\text{splay}(x)) \leq 1 + 3 \sum_{i=1}^k (r_i - r_{i-1}) = 1 + 3(r_k - r_0) = 1 + 3(r(\text{root}) - r(x)). \quad (56.16)$$

Так как после `splay` вершина x становится корнем, $s(\text{root}) = n$, поэтому $r(\text{root}) = \log_2 n$, откуда следует оценка $O(\log n)$. ■

6. Следствие (для понимания общей картины)

Так как операции `find/insert/erase/merge/split` в сплей-дереве реализуются как “обычный спуск по BST” плюс один или несколько вызовов `splay`, то ключевой факт для их амортизированной оценки — именно $\hat{t}(\text{splay}) = O(\log n)$.

57 Билет №57. Splay-дерево: реализация insert, erase и find. Связь с splay (оценка времени — б/д)

57.1 Напоминание: что такое splay-дерево

Определение 57.1 (Бинарное дерево поиска (BST)). Бинарное дерево (каждая вершина имеет не более двух детей) называется *деревом поиска*, если для любой вершины v :

- ▷ все ключи в левом поддереве v строго меньше ключа v ;
- ▷ все ключи в правом поддереве v строго больше ключа v .

(Если разрешаем дубликаты, то нужно явно договориться, куда их класть: например, «равные уходят вправо»; в билете ниже считаем множество, т.е. ключи различны.)

Определение 57.2 (Операция splay). Пусть x — вершина BST. Операция $\text{splay}(x)$ — это последовательность поворотов (zig, zig-zig, zig-zag), которая поднимает x в корень, не нарушая BST-инвариант.

Определение 57.3 (Splay-дерево). *Splay-дерево* — это BST, в котором после каждой операции доступа (find, insert, erase) выполняется splay некоторой вершины:

- ▷ если ключ найден/вставлен — $\text{splay}(\text{вершина с этим ключом})$;
- ▷ если поиск завершился неудачей — $\text{splay}(\text{последняя посещённая вершина})$.

Это гарантирует, что «последняя затронутая» вершина становится корнем.

Замечание (Почему повороты сохраняют BST-инвариант). Один поворот меняет только взаимное положение двух вершин p и x и одного из поддеревьев между ними. Порядок ключей при этом не нарушается: все элементы из α остаются слева, из γ — справа, а поддерево β остаётся между ключами x и p . Поэтому любая последовательность поворотов сохраняет BST-инвариант.

57.2 Базовые примитивы: поворот и splay

Будем считать, что у каждой вершины хранятся указатели `left`, `right`, `parent`. Тогда удобнее реализовать splay «снизу вверх».

Псевдокод (идейно, близко к CLRS)

Условности. `NIL` — пустой указатель. У вершины есть поля `left`, `right`, `parent`.

Rotate(T, x): поднять x на один уровень вверх (один поворот вокруг родителя p).

1. $p \leftarrow x.\text{parent}$, $g \leftarrow p.\text{parent}$.
2. // Подвесить x вместо p к дедушке

```

3. if  $g = \text{NIL}$  then  $T.\text{root} \leftarrow x$  else if  $p = g.\text{left}$  then  $g.\text{left} \leftarrow x$  else  $g.\text{right} \leftarrow x$ .
4.  $x.\text{parent} \leftarrow g$ .
5. if  $x = p.\text{left}$  then
    (a)  $p.\text{left} \leftarrow x.\text{right}$ ; if  $x.\text{right} \neq \text{NIL}$  then  $x.\text{right}.\text{parent} \leftarrow p$ .
    (b)  $x.\text{right} \leftarrow p$ ;  $p.\text{parent} \leftarrow x$ .
else
    (a)  $p.\text{right} \leftarrow x.\text{left}$ ; if  $x.\text{left} \neq \text{NIL}$  then  $x.\text{left}.\text{parent} \leftarrow p$ .
    (b)  $x.\text{left} \leftarrow p$ ;  $p.\text{parent} \leftarrow x$ .

```

Splay(T, x): поднять x в корень.

```

1. while  $x.\text{parent} \neq \text{NIL}$  do
    (a)  $p \leftarrow x.\text{parent}$ ,  $g \leftarrow p.\text{parent}$ .
    (b) if  $g = \text{NIL}$  then (zig) Rotate( $T, x$ ).
    (c) else if  $(x = p.\text{left} \wedge p = g.\text{left})$  or  $(x = p.\text{right} \wedge p = g.\text{right})$  then
        i. (zig-zig) Rotate( $T, p$ ).
        ii. Rotate( $T, x$ ).
    (d) else
        i. (zig-zag) Rotate( $T, x$ ).
        ii. Rotate( $T, x$ ).
2. (после цикла)  $T.\text{root} \leftarrow x$ .

```

Замечание. Вместо **parent**-указателей можно хранить путь поиска в стек/вектор и делать **splay** по сохранённому пути, но в большинстве реализаций проще поддерживать **parent**.

а) Повороты выполняются за $O(1)$.

57.3 Операция find

Идея. Делаем обычный BST-поиск. Если нашли ключ — поднимаем найденную вершину в корень. Если не нашли — поднимаем в корень последнюю посещённую вершину (ту, откуда попытались уйти в **nullptr**).

Псевдокод (идейно, близко к CLRS)

Find(T, k):

```

1.  $v \leftarrow T.\text{root}$ ,  $\text{last} \leftarrow \text{NIL}$ .
2. while  $v \neq \text{NIL}$  do

```

```

(a)  $last \leftarrow v$ .
(b) if  $k < v.key$  then  $v \leftarrow v.left$ .
(c) else if  $k > v.key$  then  $v \leftarrow v.right$ .
(d) else break. // нашли
3. if  $last \neq \text{NIL}$  then  $\text{Splay}(T, last)$ .
4. if  $last \neq \text{NIL}$  and  $last.key = k$  then return  $last$ .
5. return  $\text{NIL}$ .

```

Утверждение 57.1 (Корректность `find`). Функция `find` возвращает указатель на вершину с ключом k , если она есть в дереве, и `nullptr` иначе; при этом BST-инвариант сохраняется.

Δ Доказательство. BST-поиск корректен стандартно: на каждом шаге мы выбираем единственное поддерево, в котором *может* лежать k . Если k найден, то найденная вершина действительно содержит k . Если поиск пришёл в `nullptr`, то в дереве нет места, где k мог бы находиться.

Далее вызывается `splay` от вершины `last`. Она состоит из поворотов, каждый из которых сохраняет BST-инвариант, значит инвариант сохраняется и после всего `splay`. Возврат `last` корректен по условию остановки цикла. ■

57.4 Операция `insert`

Идея. Вставляем ключ как в обычный BST (спускаемся до листа) и затем делаем `splay` вставленной вершины. Если ключ уже есть, вставку игнорируем, но всё равно полезно сделать `splay` найденной вершины (она становится «последней затронутой»).

Псевдокод (идейно, близко к CLRS)

Insert(T, k):

```

1. if  $T.root = \text{NIL}$  then создать корень с ключом  $k$  и return.
2.  $v \leftarrow T.root, p \leftarrow \text{NIL}$ .
3. while  $v \neq \text{NIL}$  do
    (a)  $p \leftarrow v$ .
    (b) if  $k < v.key$  then  $v \leftarrow v.left$ .
    (c) else if  $k > v.key$  then  $v \leftarrow v.right$ .
    (d) else // ключ уже есть
        i.  $\text{Splay}(T, v)$ .
        ii. return.
4. Создать новую вершину  $x$  с ключом  $k$ ,  $x.parent \leftarrow p$ .

```

```

5. if  $k < p.\text{key}$  then  $p.\text{left} \leftarrow x$  else  $p.\text{right} \leftarrow x$ .
6. Splay( $T, x$ ).

```

Утверждение 57.2 (Корректность insert). После $\text{insert}(\text{root}, k)$ дерево остаётся BST и содержит ключ k (если его не было ранее).

$\triangle$ Доказательство. Если ключ уже есть, структура не меняется, кроме **splay**, который сохраняет BST. Если ключа не было, алгоритм идёт по пути, определяемому BST-инвариантом, и прикрепляет новый лист в единственное корректное место (слева или справа от найденного родителя p). Значит, BST-инвариант выполнен. Дальнейший **splay** сохраняет BST, поэтому дерево корректно и содержит k . ■

57.5 Операция erase

Идея (вариант «splay-then-join»). Сначала $\text{find}(k)$ поднимает вершину с ключом k (если она существует) в корень. После этого у корня две независимые части:

- ▷ левое поддерево L содержит только ключи $< k$;
- ▷ правое поддерево R содержит только ключи $> k$.

Удаляем корень и *склеиваем* L и R .

Склейка (join). Если L пусто — ответ R . Иначе берём максимум в L (самый правый), делаем **splay** этого максимума внутри L . Тогда максимум становится корнем L и имеет пустого правого сына, значит можно подвесить R как правого ребёнка.

Псевдокод (идейно, близко к CLRS)

SubtreeMax(v):

1. while $v.\text{right} \neq \text{NIL}$ do $v \leftarrow v.\text{right}$.
2. return v .

Join(L, R): (предусловие: $\max(L) < \min(R)$).

1. if $L = \text{NIL}$ then return R .
2. $m \leftarrow \text{SubtreeMax}(L)$.
3. Создать временное дерево T с $T.\text{root} \leftarrow L$ и выполнить **Splay**(T, m).
4. // теперь m — корень и у него нет правого сына
5. $T.\text{root}.\text{right} \leftarrow R$; if $R \neq \text{NIL}$ then $R.\text{parent} \leftarrow T.\text{root}$.
6. return $T.\text{root}$.

Erase(T, k):

1. $x \leftarrow \text{Find}(T, k)$ (внутри уже выполнится **splay** последней посещённой вершины).
2. **if** $x = \text{NIL}$ **then return**.
3. // теперь $x = T.\text{root}$
4. $L \leftarrow x.\text{left}$, $R \leftarrow x.\text{right}$; отсоединить L и R (сделать их **parent**=NIL).
5. Удалить вершину x .
6. $T.\text{root} \leftarrow \text{Join}(L, R)$.

Утверждение 57.3 (Корректность **erase**). После **erase**(*root*, *k*) ключ *k* удалён (если он был), а структура остаётся корректным BST.

Δ Доказательство. Если *k* не найден, операция ничего не делает, корректность тривиальна. Пусть *k* найден. После **find** вершина с ключом *k* поднята в корень x . По BST-инварианту все ключи в $L = x \rightarrow \text{left}$ строго меньше *k*, а в $R = x \rightarrow \text{right}$ строго больше *k*.

Удаляя x , мы получаем два BST L и R . Если L пусто, то корнем становится R , BST-инвариант очевиден. Иначе берём максимальный ключ m в L и делаем **splay**(m) внутри L . Тогда m становится корнем дерева L и у него нет правого ребёнка (иначе существовал бы ключ больше m в L). Подвешивание R вправо корректно, потому что каждый ключ в R больше любого ключа в L , в частности больше m . Следовательно, итоговая структура — BST. Ключ *k* удалён. ■

57.6 Связь операций с **splay**

Важный паттерн:

- ▷ **find**: *поиск* $\rightarrow$ **splay**(*last*);
- ▷ **insert**: *вставка-листом* $\rightarrow$ **splay**(*new*);
- ▷ **erase**: **find** (с **splay**) $\rightarrow$ *удаление корня* $\rightarrow$ *ещё один splay* (максимума слева) при склейке.

То есть все операции выражаются через стандартный BST-спуск и константное число вызовов **splay**.

57.7 Оценка времени работы (б/д)

- ▷ Худший случай для одной операции может быть $\Theta(n)$ (дерево может быть сильно перекошенным).
- ▷ Однако **splay** имеет амортизированную стоимость $O(\log n)$.
- ▷ Поэтому **find**, **insert**, **erase** также работают за $O(\log n)$ *амортизированно*: каждая из них делает BST-спуск и $O(1)$ вызовов **splay**.

57.8 Короткие практические замечания

- ▷ Если разрешаете дубликаты, удобно хранить `cnt` в вершине (сколько одинаковых ключей) и удалять/вставлять, меняя `cnt`.
- ▷ При реализации поворотов обязательно обновляйте `parent`-указатели (и любые агрегаты: `size`, `sum`, ...), иначе дерево сломается.
- ▷ В `erase` важно *отцеплять* L и R от удаляемого корня (обнулять `parent`), чтобы `splay` работал корректно в новом дереве.

58 Билет №58. Splay-дерево: merge и split, связь с splay, оценка времени (б/д)

Контекст и постановка

Splay-дерево — это бинарное дерево поиска (BST), в котором после обращения к вершине v выполняется операция **splay**(v): вершина v поднимается в корень последовательностью вращений **zig**, **zig-zig**, **zig-zag**.

В этом билете нужны две «опциональные» операции над деревьями поиска:

- ▷ **merge**(A , B) — склеить два дерева поиска в одно.
- ▷ **split**(T , k) — разрезать дерево на два по ключу k .

Ключевой факт: обе операции удобно сводятся к **splay** (это и есть «связь с splay»).

Определения операций

Пусть ключ — значение, по которому устроено BST-сравнение.

Определение 58.1 (**split** по ключу). **split**(T , k) возвращает пару деревьев (L , R) такую, что

- ▷ L содержит все ключи из T , не превосходящие k ;
- ▷ R содержит все ключи из T , строго большие k ;
- ▷ L и R являются корректными BST, и множества их ключей дизъюнкты, а объединение равно множеству ключей T .

Определение 58.2 (**merge** (**join**)). **merge**(A , B) корректно и быстро реализуется в предположении

$$\max(A) < \min(B),$$

то есть *все ключи в A строго меньше всех ключей в B* .

Операция **merge**(A , B)

Идея: поднять максимум из A в корень, после чего справа у корня точно пусто, и можно подвесить B как правого сына.

Алгоритм

1. Если $A = \emptyset$, вернуть B ; если $B = \emptyset$, вернуть A .
2. Найти $m = \max(A)$ (идем по правым сыновьям до упора).
3. Выполнить **splay**(m) в дереве A .
4. Теперь m — корень, и у него нет правого сына. Присвоить $m.\text{right} \leftarrow B$ (и выставить $B.\text{parent} \leftarrow m$, если есть указатели на родителей).

Корректность

После `splay(m)` корень равен m , и все остальные элементы $A \setminus \{m\}$ лежат в левом поддереве (справа быть не может, так как m — максимум). Из предположения $\max(A) < \min(B)$ следует, что все ключи в B строго больше m , значит B можно сделать правым поддеревом корня, не нарушив BST-инвариант.

Время работы (б/д)

Основная стоимость — один `splay`. По известной оценке сплей-операции: `splay` работает за $O(\log n)$ амортизированно, где n — число вершин. Остальные действия (поиск максимума, подвешивание правого сына, локальные пересчеты полей) занимают $O(\text{height})$ или $O(1)$ и не меняют асимптотику. Итого: `merge` работает за $O(\log(|A| + |B|))$ амортизированно.

Операция `split(T, k)`

Идея: найти предшественника $x = \max\{v \in T \mid v \leq k\}$, поднять его в корень, и отрезать правое поддерево.

Как найти x (предшественника)

Идем из корня:

- ▷ если $\text{key}(v) > k$, идем влево;
- ▷ иначе ($\text{key}(v) \leq k$) запоминаем v как кандидата и идем вправо (там могут быть ключи ближе к k).

По итогу последний запомненный кандидат и есть x .

Алгоритм

1. Если $T = \emptyset$, вернуть $(\emptyset, \emptyset)$.
2. Найти $x = \max\{v \in T \mid v \leq k\}$.
 - ▷ Если такого x нет (то есть все ключи $> k$), вернуть $(\emptyset, T)$.
 - ▷ Если x существует, перейти к следующему шагу.
3. Выполнить `splay(x)` в дереве T .
4. Пусть после сплея x стал корнем. Обозначим $R \leftarrow x.\text{right}$.
5. Отрезать правое поддерево: присвоить $x.\text{right} \leftarrow \emptyset$ (и обнулить $R.\text{parent}$, если нужно).
6. Вернуть (L, R) , где L — дерево с корнем x .

Корректность

После `splay(x)` вершина x в корне. Тогда:

- ▷ в левом поддереве корня находятся только ключи $< x$, а значит $\leq x \leq k$;
- ▷ в правом поддереве находятся только ключи $> x$.

Но x выбран как *максимальный* ключ, не превосходящий k , значит ключей из диапазона $(x, k]$ в T нет. Следовательно, любой ключ из правого поддеревя строго больше k . Тогда отрезание ребра между корнем и правым сыном дает ровно требуемые деревья: слева все $\leq k$, справа все $> k$.

Время работы (б/д)

`split` делает один поиск предшественника и один `splay`; разрезание ребра занимает $O(1)$. Амортизированная стоимость доминируется `splay`, поэтому

$$\text{split} = O(\log n) \text{ амортизированно.}$$

Практические замечания по реализации

- ▷ Если в узлах хранятся дополнительные агрегаты (например, размер поддеревя, сумма, минимум), то при каждом вращении в `splay` нужно пересчитывать их локально для затронутых вершин. После `merge/split` достаточно обновить агрегаты в корне(ях) постоянным числом пересчетов.
- ▷ Граничные случаи (пустые деревья, $k < \min(T)$, $k \geq \max(T)$) удобнее обрабатывать отдельными ветками, чтобы не получить обращение к несуществующему предшественнику.

59 Билет №59. Дерево поиска по неявному ключу (динамический массив): вставка, удаление, изменение, сумма на отрезке

59.1 Постановка задачи

Нужно поддерживать **динамический массив** (последовательность) $a_0, a_1, \dots, a_{n-1}$, к которому онлайн поступают операции:

- ▷ `insert(pos, x)` — вставить элемент x *перед* позицией `pos` (то есть сделать x элементом с индексом `pos`);
- ▷ `erase(pos)` — удалить элемент с индексом `pos`;
- ▷ `set(pos, x)` — присвоить $a_{\text{pos}} := x$;
- ▷ `sum(l, r)` — найти сумму $\sum_{i=l}^{r-1} a_i$ на полуинтервале $[l, r)$.

Требуемая асимптотика — $O(\log n)$ на операцию.

59.2 Идея: BST по *неявному* ключу

Храним последовательность как бинарное дерево поиска так, что **симметричный обход** (inorder) даёт элементы массива слева направо. Тогда *ключом* вершины является её позиция в inorder, но **явно ключ не храним** — он восстанавливается по размерам поддеревьев.

Нужные поля в вершине:

- ▷ *val* — значение элемента массива в этой позиции;
- ▷ *prio* — приоритет (для балансировки; ниже будет декартово дерево / treap);
- ▷ *sz* — размер поддерева (число вершин);
- ▷ *sum* — сумма значений в поддереве.

Инварианты (для treap):

1. **Неявный порядок (inorder):** элементы левого поддерева идут *раньше* вершины, элементы правого — *позже*. Это и есть «массивный» порядок.
2. **Куча по приоритету:** $prio(v)$ меньше (или больше — в зависимости от соглашения) приоритетов детей.
3. **Агрегаты:**

$$sz(v) = 1 + sz(L) + sz(R), \quad sum(v) = val(v) + sum(L) + sum(R).$$

59.3 Базовые операции merge и split по позиции

Все операции над массивом сводятся к двум примитивам.

59.3.1 merge(A, B)

Предусловие: все элементы последовательности A должны идти *раньше* всех элементов последовательности B . В неявном ключе это означает: $\text{inorder}(A) + \text{inorder}(B)$ — корректная склейка.

Алгоритм (для treap):

- ▷ если $A = \emptyset$, вернуть B ; если $B = \emptyset$, вернуть A ;
- ▷ если $\text{prio}(A)$ «выше» $\text{prio}(B)$, то корень — A , а его правый сын равен $\text{merge}(A.\text{right}, B)$;
- ▷ иначе корень — B , а его левый сын равен $\text{merge}(A, B.\text{left})$;
- ▷ в конце пересчитать sz и sum .

Корректность: в каждом шаге выбирается корень с лучшим приоритетом, поэтому сохраняется кучевое свойство. Порядок элементов сохраняется, потому что рекурсивный вызов соединяет либо суффикс A с B , либо A с префиксом B ; элементы из A никогда не перемещаются вправо от элементов из B .

59.3.2 split(T, k)

Разбивает дерево T на пару (A, B) так, что A содержит **первые k элементов** последовательности (inorder), а B — остальные. То есть $\text{inorder}(T) = \text{inorder}(A) + \text{inorder}(B)$, причём $|A| = k$.

Пусть $lsz = sz(T.\text{left})$.

- ▷ Если $k \leq lsz$, то весь разрез находится в левом поддереве: разбиваем $T.\text{left}$ на (A, X) , кладём X обратно в $T.\text{left}$, получаем (A, T) .
- ▷ Если $k = lsz + 1$, то A — это левое поддерево плюс корень, а B — правое поддерево.
- ▷ Если $k > lsz + 1$, то разрез в правом поддереве: разбиваем $T.\text{right}$ по $k - lsz - 1$ на (X, B) , кладём X обратно в $T.\text{right}$, получаем (T, B) .

В конце каждого шага пересчитываем sz и sum текущей вершины.

Корректность (набросок): доказывается индукцией по размеру дерева. В каждом случае мы точно отделяем нужное количество вершин слева, потому что число вершин в левом поддереве известно (lsz), а inorder -порядок равен: $\text{inorder}(T.\text{left})$, затем корень, затем $\text{inorder}(T.\text{right})$.

59.4 Реализация операций динамического массива

Ниже индексация 0-based, границы в запросе суммы — полуинтервал $[l, r)$.

59.4.1 Вставка insert(pos, x)

1. $(A, B) \leftarrow \text{split}(\text{root}, pos)$.
2. $M \leftarrow$ новая вершина со значением x .
3. $\text{root} \leftarrow \text{merge}(\text{merge}(A, M), B)$.

59.4.2 Удаление `erase(pos)`

1. $(A, B) \leftarrow \text{split}(\text{root}, \text{pos})$.
2. $(M, C) \leftarrow \text{split}(B, 1)$ (вершина M — удаляемый элемент).
3. Удалить M (освободить память).
4. $\text{root} \leftarrow \text{merge}(A, C)$.

59.4.3 Изменение `set(pos, x)`

Изолируем элемент, меняем значение, пересчитываем агрегаты и склеиваем обратно:

1. $(A, B) \leftarrow \text{split}(\text{root}, \text{pos})$.
2. $(M, C) \leftarrow \text{split}(B, 1)$.
3. $\text{val}(M) \leftarrow x$, затем пересчитать sz, sum в M .
4. $\text{root} \leftarrow \text{merge}(A, \text{merge}(M, C))$.

59.4.4 Сумма на отрезке `sum(l, r)`

1. $(A, B) \leftarrow \text{split}(\text{root}, l)$.
2. $(M, C) \leftarrow \text{split}(B, r - l)$.
3. Ответ $ans = \text{sum}(M)$.
4. Восстановить дерево: $\text{root} \leftarrow \text{merge}(A, \text{merge}(M, C))$.

59.5 Асимптотика

Каждая операция использует константное число вызовов `split/merge`. Если в качестве балансировки используется **декартово дерево** с независимыми случайными приоритетами, то ожидаемая высота дерева $O(\log n)$, и каждый `split/merge` работает за $O(\log n)$ в среднем. Следовательно, все операции массива работают за $O(\log n)$ в среднем, память $O(n)$.

Замечание. Если вместо `treap` использовать `splay`-дерево, то те же формулы через `split/merge` работают, а оценка будет *амортизированная* $O(\log n)$.

59.6 Ключевые инварианты корректности

1. Inorder-обход дерева всегда совпадает с текущим массивом. Это верно в начале (пустое дерево) и сохраняется, потому что все операции — это разрезы на префикс/суффикс и обратная склейка.
2. Поля sz и sum корректны в каждой вершине, потому что после любого изменения указателей у затронутых вершин вызывается пересчёт: $\text{upd}(v) : sz(v) = 1 + sz(L) + sz(R), sum(v) = \text{val}(v) + sum(L) + sum(R)$.
3. Операция `split(T, k)` действительно выделяет первые k элементов, так как решение сравнивает k с размером левого поддерева (lsz) и рекурсивно переходит в ту часть, где лежит граница.

59.7 Псевдокод (treap по неявному ключу)

Псевдокод (идейно, близко к CLRS)

Вершина: поля val , $prio$, $left$, $right$ и агрегаты sz , sum .

Upd(v): пересчитать агрегаты.

1. $v.sz \leftarrow 1 + Sz(v.left) + Sz(v.right)$.
2. $v.sum \leftarrow v.val + Sum(v.left) + Sum(v.right)$.

(Здесь $Sz(NIL) = 0$, $Sum(NIL) = 0$.)

Merge(A, B): (все элементы A идут раньше всех элементов B).

1. **if** $A = NIL$ **then return** B ; **if** $B = NIL$ **then return** A .
2. **if** $A.prio < B.prio$ **then**
 - (a) $A.right \leftarrow Merge(A.right, B)$.
 - (b) **Upd**(A); **return** A .
- else**
 - (a) $B.left \leftarrow Merge(A, B.left)$.
 - (b) **Upd**(B); **return** B .

Split(T, k): разрез по первым k элементам (0-based): вернуть (A, B) , где A содержит индексы $[0..k-1]$, $B = [k..)$.

1. **if** $T = NIL$ **then return** (NIL, NIL) .
2. $lsz \leftarrow Sz(T.left)$.
3. **if** $k \leq lsz$ **then**
 - (a) $(A, X) \leftarrow Split(T.left, k)$.
 - (b) $T.left \leftarrow X$; **Upd**(T).
 - (c) **return** (A, T) .
- else**
 - (a) $(X, B) \leftarrow Split(T.right, k - lsz - 1)$.
 - (b) $T.right \leftarrow X$; **Upd**(T).
 - (c) **return** (T, B) .

Операции динамического массива (0-based):

1. **Insert**($root, pos, x$):
 - (a) $(A, B) \leftarrow Split(root, pos)$.

- (b) $M \leftarrow \text{NewNode}(x, \text{RandPrio}())$.
- (c) $\text{root} \leftarrow \text{Merge}(\text{Merge}(A, M), B)$.

2. **Erase**(root, pos):

- (a) $(A, B) \leftarrow \text{Split}(\text{root}, \text{pos})$.
- (b) $(M, C) \leftarrow \text{Split}(B, 1)$.
- (c) Удалить M .
- (d) $\text{root} \leftarrow \text{Merge}(A, C)$.

3. **Set**($\text{root}, \text{pos}, x$):

- (a) $(A, B) \leftarrow \text{Split}(\text{root}, \text{pos})$.
- (b) $(M, C) \leftarrow \text{Split}(B, 1)$.
- (c) $M.\text{val} \leftarrow x$; $\text{Upd}(M)$.
- (d) $\text{root} \leftarrow \text{Merge}(A, \text{Merge}(M, C))$.

4. **RangeSum**(root, l, r): сумма на $[l, r]$.

- (a) $(A, B) \leftarrow \text{Split}(\text{root}, l)$.
- (b) $(M, C) \leftarrow \text{Split}(B, r - l)$.
- (c) $\text{ans} \leftarrow \text{Sum}(M)$.
- (d) $\text{root} \leftarrow \text{Merge}(A, \text{Merge}(M, C))$.
- (e) **return** ans .

Замечание. В реальном коде приоритеты удобно генерировать через `std::mt19937`. Если боитесь переполнения рекурсии, используйте итеративные версии или увеличьте лимит стека (в контестах обычно хватает).

60 Билет №60. Декартово дерево (treap): определение, глубина, линейное построение при отсортированных ключах

60.1 Определение

Определение 60.1 (Декартово дерево / treap). Декартово дерево (часто: *treap* от *tree*+*heap*) — это бинарное дерево, в каждой вершине которого хранится пара

$$(\text{key}, \text{prior}),$$

и одновременно выполняются два свойства:

1. **Свойство бинарного дерева поиска по ключу (BST)**. Для любой вершины v все ключи в её левом поддереве строго меньше $\text{key}(v)$, а все ключи в правом поддереве строго больше $\text{key}(v)$.
2. **Свойство кучи по приоритету (heap)**. Будем фиксировать соглашение **max-heap по приоритету**: для любой вершины v выполнено

$$\text{prior}(v) \geq \text{prior}(u) \quad \text{для каждого ребёнка } u \text{ вершины } v.$$

То есть приоритеты не возрастают при спуске от корня к листьям.

Предполагаем, что ключи попарно различны (мы храним *множество*). Приоритеты удобно брать попарно различными: например, генерировать случайные prior из большого диапазона, либо хранить пару $(\text{prior}, \text{id})$ и сравнивать лексикографически.

Замечание (Эквивалентная формулировка). Треап — это **декартово дерево** над упорядоченными по ключу вершинами: in-order обход возвращает ключи в отсортированном порядке (BST), а корень содержит максимальный приоритет (heap).

60.2 Теорема о глубине (высоте)

Теорема 60.1 (О высоте treap, б/д). Пусть заданы n попарно различных ключей и каждому ключу независимо назначен случайный приоритет (из непрерывного распределения или из достаточно большого дискретного, чтобы совпадения пренебрежимо редки). Тогда:

1. (**Ожидаемая высота**) математическое ожидание высоты treap есть $O(\log n)$.
2. (**С высокой вероятностью**) высота treap равна $O(\log n)$ с вероятностью $1 - n^{-\Omega(1)}$.

Следовательно, основные операции поиска по ключу (например, *find* и *lower_bound*) в среднем работают за $O(\log n)$ сравнений.

Замечание (Почему это правдоподобно: связь со случайным BST). Если приоритеты попарно различны, то вершины treap однозначно упорядочены по приоритету. Если мысленно вставлять ключи в обычное (несбалансированное) BST в порядке *убывания приоритета*, то получится ровно то же дерево, что и treap: приоритет определяет «кто был вставлен раньше». При случайных независимых приоритетах этот порядок вставки — случайная перестановка ключей. Поэтому форма treap распределена так же, как у случайного BST, у которого высота $O(\log n)$ в среднем (это же распределение имеет и дерево рекурсии QuickSort).

60.3 Построение за линейное время при отсортированных ключах

Постановка. Даны ключи $k_1 < k_2 < \dots < k_n$ (уже отсортированы) и соответствующие им приоритеты $p_i = \text{prior}(k_i)$. Нужно построить treap за $O(n)$.

Идея. Так как ключи уже отсортированы, свойство BST означает: *in-order обход* дерева должен вернуть вершины ровно в порядке $1, 2, \dots, n$. Значит структура дерева определяется только приоритетами $p_1, \dots, p_n$ и условием max-heap: корнем становится элемент с максимальным приоритетом, левое поддерево строится по префиксу, правое — по суффиксу (классическая рекурсивная дефиниция декартова дерева).

Алгоритм (монотонный стек). Будем поддерживать стек вершин, образующий правый путь уже построенной части дерева, и при этом приоритеты в стеке будут **строго убывать** сверху вниз (т.е. от вершины стека к его дну).

Псевдокод (идейно, близко к CLRS)

BuildTreapSorted($k[1..n], p[1..n]$):

(ключи уже отсортированы)

1. $S \leftarrow$ пустой стек
2. **for** $i \leftarrow 1$ **to** n **do**
 - (a) $cur \leftarrow \text{NewNode}(k_i, p_i)$
 - (b) $last \leftarrow \text{NIL}$
 - (c) **while** $S \neq \emptyset$ **and** $\text{prior}(\text{Top}(S)) < \text{prior}(cur)$ **do**
 - i. $last \leftarrow \text{Top}(S)$
 - ii. $\text{Pop}(S)$
 - (d) $cur.\text{left} \leftarrow last$
 - (e) **if** $S \neq \emptyset$ **then** $\text{Top}(S).\text{right} \leftarrow cur$
 - (f) $\text{Push}(S, cur)$

3. **return** $\text{Bottom}(S)$

(корень — нижний элемент стека)

Почему корректно.

- ▷ **BST по ключу.** Мы обрабатываем ключи в порядке возрастания $k_1 < \dots < k_n$ и каждый новый узел cur ставим либо правым сыном некоторого узла слева, либо

делаем его корнем некоторого правого «хребта». При этом `cur.left` содержит только элементы с меньшими ключами, а правый сын всегда имеет больший ключ. Поэтому in-order обход возвращает ключи в порядке $k_1, \dots, k_n$.

- ▷ **Heap по приоритету (max-heap).** В цикле `while` мы выкидываем из стека все вершины с приоритетом меньше, чем у `cur`, значит `cur` становится родителем для снятого «хвоста» (он назначается в `cur.left`) и имеет приоритет не меньше приоритетов всех вершин этого хвоста. А если после цикла стек не пуст, то `stack.top.prior >= cur.prior`, поэтому связь `stack.top.right = cur` тоже удовлетворяет heap-свойству.

Почему $O(n)$. Каждая вершина ровно один раз кладётся в стек и ровно один раз из него извлекается, поэтому суммарное число итераций цикла `while` по всем i равно $O(n)$. Остальные операции внутри цикла — константные, значит общее время $O(n)$, память $O(n)$.

Замечание (Если в вашем коде heap-свойство наоборот (min-heap)). Если вы предпочитаете *min-heap по приоритету* (корень имеет минимальный приоритет), то во всех сравнениях в алгоритме нужно поменять знак: в `while` проверять `stack.top.prior > cur.prior`.

Замечание (Связь с «картезианским деревом массива»). Если рассмотреть последовательность значений $p_1, \dots, p_n$, то построенное дерево является картезианским деревом этой последовательности: in-order даёт индексы $1..n$, а heap-свойство выполняется по p_i .

61 Билет №61. Декартово дерево: операции merge и split

61.1 Постановка

Задача. Реализовать две базовые операции для *декартова дерева* (*treap*):

1. $merge(A, B)$ — слить два декартовых дерева A и B .
2. $split(T, x)$ — разбить декартово дерево T по ключу x на два дерева.

Требуется: алгоритмы, доказательства корректности, анализ асимптотики.

61.2 Напоминание: декартово дерево

Определение 61.1 (Treap). В каждой вершине хранятся **ключ** $key(v)$ и **приоритет** $prio(v)$. Дерево корректно, если одновременно:

1. **(BST)** для любой вершины v все ключи слева строго меньше $key(v)$, а справа строго больше;
2. **(Heap)** по приоритетам выполнено свойство кучи. В этом билете считаем **max-heap**:

$$prio(v) \geq prio(child) \quad \text{для каждого сына вершины } v.$$

Замечание. Обычно приоритеты выбирают случайно и независимо (например, $rng()$), тогда глубина treap в среднем $O(\log n)$ (теорема о глубине относится к билету 60 и может использоваться без доказательства).

61.3 Операция merge

61.3.1 Смысл и предусловие

Определение 61.2 (merge). Операция $merge(A, B)$ определена, если

$$\forall u \in A \forall v \in B : key(u) < key(v),$$

то есть **все ключи A меньше всех ключей B** . Результат — одно treap, содержащее все вершины из A и B .

61.3.2 Алгоритм

Корнем результата обязан стать корень с большим приоритетом (иначе нарушится heap-свойство). Если корень взят из A , то всё дерево B по ключам должно оказаться в правом поддереве этого корня (ведь $A < B$), и наоборот.

Псевдокод (идейно, близко к CLRS)

Merge(A, B) (предусловие: $\forall u \in A, \forall v \in B : key(u) < key(v)$):

1. if $A = \text{NIL}$ then return B .

```

2. if  $B = \text{NIL}$  then return  $A$ .
3. if  $A.\text{prio} > B.\text{prio}$  then
    (a)  $A.\text{right} \leftarrow \text{Merge}(A.\text{right}, B)$ .
    (b)  $\text{recalc}(A)$ .
    (c) return  $A$ .
else
    (a)  $B.\text{left} \leftarrow \text{Merge}(A, B.\text{left})$ .
    (b)  $\text{recalc}(B)$ .
    (c) return  $B$ .

```

61.3.3 Корректность

Лемма 61.1 (BST-свойство для `merge`). Пусть A и B — корректные *treap*, и все ключи A меньше всех ключей B . Тогда $\text{merge}(A, B)$ является корректным BST по ключам.

△ Доказательство. Индукция по $n = |A| + |B|$.

База: если одно дерево пусто, возвращаем другое; BST-свойство сохраняется.

Переход: оба дерева непусты.

- ▷ Если $\text{prio}(\text{root}(A)) > \text{prio}(\text{root}(B))$, то корнем результата становится $\text{root}(A)$. Левое поддерево не меняется. Новое правое поддерево равно $\text{merge}(A.\text{right}, B)$. По BST-свойству в A все ключи $A.\text{right}$ строго больше $\text{key}(\text{root}(A))$, а по предусловию все ключи B также больше любого ключа из A , значит $> \text{key}(\text{root}(A))$. Следовательно, все ключи в новом правом поддереве строго больше ключа корня. Корректность BST внутри правого поддерева следует из предположения индукции (мы рекурсивно мержим меньшие деревья, и предусловие $A.\text{right} < B$ сохраняется).
- ▷ Случай $\text{prio}(\text{root}(B)) \geq \text{prio}(\text{root}(A))$ симметричен: корнем становится $\text{root}(B)$, а левым поддеревом — $\text{merge}(A, B.\text{left})$; оно корректно встраивается слева, так как все его ключи $< \text{key}(\text{root}(B))$.

■

Лемма 61.2 (Heap-свойство для `merge`). При тех же условиях $\text{merge}(A, B)$ удовлетворяет heap-свойству по приоритетам.

△ Доказательство. Индукция по $|A| + |B|$.

База (пустое дерево) очевидна. В переходе корнем выбирается корень с максимальным приоритетом среди двух корней, значит приоритет корня результата не меньше приоритета корня “второго” дерева.

Далее мы рекурсивно мержим либо $(A.\text{right}, B)$, либо $(A, B.\text{left})$ и получившееся *treap* подвешиваем как одного из детей выбранного корня. Корень подвешиваемого дерева имеет приоритет не больше, чем:

- ▷ приоритет исходного корня соответствующего поддеревя (например, $A.right$), по hear-свойству внутри A или B ;
- ▷ и, тем более, не больше приоритета выбранного корня результата, потому что выбранный корень имел максимальный приоритет среди корней A и B .

Значит, приоритет родителя $\geq$ приоритета нового ребёнка. Внутри подвешенного treap hear-свойство верно по предположению индукции. ■

Утверждение 61.1 (Состав вершин). *$merge(A, B)$ содержит каждую вершину из A и B ровно один раз.*

$\triangle$ *Доказательство.* Алгоритм не создаёт и не удаляет вершины, а только переназначает один указатель на ребёнка на каждом шаге. Поэтому множество вершин сохраняется и не дублируется. ■

61.3.4 Время работы

Утверждение 61.2 (Асимптотика $merge$). *Пусть высота результата равна h . Тогда $merge(A, B)$ выполняется за $O(h)$. При случайных независимых приоритетах $\mathbb{E}[h] = O(\log(|A| + |B|))$, значит и математическое ожидание времени равно $O(\log(|A| + |B|))$.*

$\triangle$ *Доказательство.* Каждый рекурсивный шаг делает $O(1)$ действий и опускается на один уровень вниз (в $A.right$ или $B.left$). Число шагов не превосходит высоты итогового дерева h . ■

61.4 Операция split

61.4.1 Смысл

Определение 61.3 ($split$ по значению ключа). $split(T, x)$ возвращает пару (L, R) такую, что:

$$\forall v \in L: key(v) \leq x, \quad \forall v \in R: key(v) > x,$$

и при этом L и R — корректные treap, а их объединение по вершинам даёт исходное дерево T .

61.4.2 Алгоритм

$split$ устроен как поиск в BST: на каждом шаге мы идём либо влево, либо вправо и “разрезаем” соответствующее поддерево.

Псевдокод (идейно, близко к CLRS)

Split(T, x): вернуть (L, R) , где $key(L) \leq x$, $key(R) > x$.

1. if $T = \text{NIL}$ then return (NIL, NIL) .

2. if $T.key \leq x$ then

(a) $(A, B) \leftarrow \text{Split}(T.right, x)$.

```

(b)  $T.\text{right} \leftarrow A$ .
(c)  $\text{recalc}(T)$ .
(d) return  $(T, B)$ .

else

(a)  $(A, B) \leftarrow \text{Split}(T.\text{left}, x)$ .
(b)  $T.\text{left} \leftarrow B$ .
(c)  $\text{recalc}(T)$ .
(d) return  $(A, T)$ .

```

61.4.3 Корректность

Лемма 61.3 (Корректность `split`). $\text{split}(T, x)$ возвращает пару (L, R) , удовлетворяющую:

1. (разделение по ключу) все ключи $L \leq x$, все ключи $R > x$;
2. L и R корректны как *treap* ($BST + \text{Heap}$);
3. вершины T разбиваются на два непересекающихся множества вершин L и R .

△ *Доказательство.* Индукция по $|T|$. База $T = \emptyset$ тривиальна.

Пусть $T \neq \emptyset$, $v = \text{root}(T)$.

Случай 1: $\text{key}(v) \leq x$.

Тогда v и всё его левое поддерево целиком должно попасть в L , потому что в BST

$$\forall u \in v.\text{left} : \text{key}(u) < \text{key}(v) \leq x.$$

Правое поддерево $v.\text{right}$ может содержать как ключи $\leq x$, так и $> x$, поэтому рекурсивно делим $v.\text{right}$ на (A, B) . По предположению индукции в A все ключи $\leq x$, в B все ключи $> x$. Делаем $v.\text{right} \leftarrow A$ и возвращаем $(L, R) = (v, B)$.

Проверим свойства:

- ▷ Разделение по ключу выполнено по построению.
- ▷ BST в L : слева остаётся $v.\text{left}$ (ключи $< \text{key}(v)$), справа становится A (ключи $\geq \text{key}(v)$ и $\leq x$), всё корректно; $R = B$ корректно по индукции.
- ▷ Heap в L : мы заменили ребёнка $v.\text{right}$ на дерево A , которое является поддеревом исходного $v.\text{right}$. В max-heap любой потомок имеет приоритет $\leq$ приоритета корня поддерева $v.\text{right}$, а он, в свою очередь, $\leq \text{prio}(v)$. Значит $\text{prio}(v) \geq \text{prio}(\text{root}(A))$, и heap-свойство на ребре сохраняется. Внутри A и B heap-свойство верно по индукции.
- ▷ Вершины не копируются и не удаляются, мы лишь “разрезали” одно ребро и переназначили одно поддерево, значит множества вершин разбились без пересечений.

Случай 2: $\text{key}(v) > x$ симметричен: рекурсивно делим $v.\text{left}$ и присоединяем “правую часть” разбиения обратно как левое поддерево корня v , получая R . ■

61.4.4 Время работы

Утверждение 61.3 (Асимптотика `split`). Если высота T равна h , то `split( $T$ ,  $x$ )` работает за $O(h)$, поскольку на каждом шаге рекурсии уходит ровно в одного ребёнка. При случайных приоритетах ожидаемое время равно $O(\log |T|)$.

61.5 Практические замечания

- ▷ **Агрегация.** Если вы храните дополнительные поля (*size*, сумму, минимум и т.п.), то после изменения детей обязательно пересчитывайте их в `recalc(v)`.
- ▷ **Дубликаты ключей.** Если ключи могут повторяться, заранее фиксируйте правило (например, “равные уходят вправо”) и используйте его *везде*. Тогда в `split` сравнением управляется, попадёт ли $key = v$ в L или R .
- ▷ **Деструктивность.** Алгоритмы выше изменяют исходные деревья (перешивают указатели). Для персистентной версии нужно копировать вершины на пути рекурсии.

Замечание (Связь со следующим билетом). Операции `insert` и `erase` удобно выражаются через `split` и `merge` (билет 62).

62 Билет №62. Декартово дерево: выражение insert и erase через merge и split

62.1 Постановка задачи

Дано декартово дерево (*treap*) T , хранящее множество ключей S . Требуется реализовать операции `insert(x)` (добавить ключ x) и `erase(x)` (удалить ключ x) **только** через две базовые операции `merge` и `split`. Нужно описать алгоритмы, доказать корректность и оценить асимптотику.

62.2 Напоминание: инварианты treap

Треап хранит пары $(key, prio)$ и удовлетворяет двум условиям:

- ▷ **BST-инвариант по ключу:** для вершины v все ключи в левом поддереве строго меньше $key(v)$, а в правом — строго больше.
- ▷ **Неар-инвариант по приоритету:** приоритет в вершине не меньше приоритетов в детях (будем считать *max-heap*: $prio(v) \geq prio(child)$).

Обычно приоритеты выбирают случайно, что обеспечивает ожидаемую глубину $O(\log |S|)$.

62.3 Спецификация операций split и merge

В этом билете важны контракты операций. (Реализация `split/merge` разбирается в билете 61.)

Определение 62.1 (Split по ключу). `split(T, x)` возвращает пару деревьев (L, R) такую, что:

- ▷ L и R — корректные treap;
- ▷ все ключи в L удовлетворяют $key \leq x$;
- ▷ все ключи в R удовлетворяют $key > x$;
- ▷ множества ключей разъединены и $keys(T) = keys(L) \cup keys(R)$.

Определение 62.2 (Merge). `merge(L, R)` применим, если все ключи в L строго меньше всех ключей в R . Тогда результат $T = merge(L, R)$ удовлетворяет:

- ▷ T — корректный treap;
- ▷ $keys(T) = keys(L) \cup keys(R)$.

Замечание. Иногда удобна ещё одна разновидность разбиения: `split_less(T, x)` $\rightarrow (A, B)$, где $keys(A) < x$, $keys(B) \geq x$. Её легко получить, если в `split` поменять сравнение “ $\leq$ ” на “ $<$ ”. Для целочисленных ключей `split_less(T, x)` эквивалентен `split(T, x - 1)`.

62.4 Операция insert через split и merge

Пусть нужно добавить ключ x в структуру **множества** (дубликаты не храним).

Задача. Реализовать $insert(x)$ через $split$ и $merge$.

Алгоритм (без отдельного find). Идея: сначала **выделить** (если существует) поддерево с ключом x , а затем либо вернуть всё как было, либо вставить новую вершину.

1. $(A, B) = \text{split_less}(T, x)$, где $keys(A) < x$, $keys(B) \geq x$.
2. $(M, C) = \text{split}(B, x)$, где $keys(M) \leq x$, $keys(C) > x$. Так как $keys(B) \geq x$, получаем: M либо пусто, либо содержит ровно ключ x .
3. Если M непусто (ключ уже был), возвращаем

$$T' = \text{merge}(\text{merge}(A, M), C).$$

4. Иначе создаём новую вершину v с ключом x и случайным приоритетом и возвращаем

$$T' = \text{merge}(\text{merge}(A, v), C).$$

Почему корректно. После шага 1 все ключи в A строго меньше x , а в B — не меньше x . После шага 2 ключи в C строго больше x , а ключи в M не больше x ; но $M \subseteq B$, значит в M могут быть только ключи $= x$.

- ▷ Если $M \neq \emptyset$, то мы ничего не меняем по множеству ключей: $keys(T') = keys(A) \cup keys(M) \cup keys(C) = keys(T)$.
- ▷ Если $M = \emptyset$, то добавляем x ровно один раз: $keys(T') = keys(A) \cup \{x\} \cup keys(C) = keys(T) \cup \{x\}$.

Применимость **merge** выполнена, так как $keys(A) < x < keys(C)$. Корректность **treap** (BST+heap-инварианты) обеспечивается контрактом **split/merge**.

Псевдокод.

Псевдокод (идейно, близко к CLRS)

Insert(T, x): (множество, дубликаты не храним)

1. $(A, B) \leftarrow \text{SPLITLESS}(T, x)$ (A : $key < x$, B : $key \geq x$)
2. $(M, C) \leftarrow \text{SPLIT}(B, x)$ (M : $key \leq x$, C : $key > x$)
3. **if** $M \neq \emptyset$ **then return** $\text{MERGE}(\text{MERGE}(A, M), C)$
4. $v \leftarrow \text{NEWNODE}(x, \text{RANDOMPRIORITY}())$
5. **return** $\text{MERGE}(\text{MERGE}(A, v), C)$

62.5 Операция erase через split и merge

Пусть нужно удалить ключ x (если ключа нет, считаем, что операция ничего не меняет).

Задача. Реализовать $erase(x)$ через $split$ и $merge$.

Алгоритм (универсальный, без арифметики над ключами). Используем два разбиения, чтобы выделить поддерево с ключом x .

1. $(A, B) = \text{split_less}(T, x)$, где $keys(A) < x$, $keys(B) \geq x$.
2. $(M, C) = \text{split}(B, x)$, где $keys(M) \leq x$, $keys(C) > x$. Так как в B все ключи $\geq x$, получаем $keys(M) = \{x\}$ (для множества).
3. Удаляем M (освобождаем память, если нужно).
4. Склеиваем остаток: $T' = \text{merge}(A, C)$.

Алгоритм (для целочисленных ключей). Если ключи — целые числа, то $\text{split_less}(T, x)$ можно заменить на $\text{split}(T, x - 1)$:

1. $(A, B) = \text{split}(T, x - 1)$ ($A : \leq x - 1$, $B : > x - 1$, т.е. $\geq x$).
2. $(M, C) = \text{split}(B, x)$ ($M : \leq x$, $C : > x$). Тогда M содержит ровно ключ x .
3. Удаляем M , возвращаем $\text{merge}(A, C)$.

Почему корректно. После шага 1 ключи A строго меньше x , а ключи B не меньше x . После шага 2 ключи C строго больше x , а ключи M не больше x . Но так как $M \subseteq B$ и $keys(B) \geq x$, то в M могут остаться только ключи $= x$. Удаляя M и сливая A и C , мы:

- ▷ удаляем ключ x (и только его);
- ▷ сохраняем все остальные ключи;
- ▷ сохраняем BST-инвариант (так как $keys(A) < keys(C)$) и heap-инвариант (по контракту merge).

Псевдокод.

Псевдокод (идейно, близко к CLRS)

Erase(T, x):

1. $(A, B) \leftarrow \text{SPLITLESS}(T, x)$ $(A : key < x, B : key \geq x)$
2. $(M, C) \leftarrow \text{SPLIT}(B, x)$ $(M : key \leq x, C : key > x)$
3. $\text{DELETETREE}(M)$ (опционально: освобождение памяти)
4. **return** $\text{MERGE}(A, C)$

62.6 Асимптотика

Пусть высота `treap` равна h . `split` и `merge` работают за $O(h)$. `insert` использует один `split` и два `merge`, `erase` — два `split` и один `merge`. Следовательно,

$$\triangleright T_{insert} = O(h),$$

$$\triangleright T_{erase} = O(h).$$

При случайных независимых приоритетах ожидаемо $h = O(\log n)$, где $n = |S|$, поэтому `insert` и `erase` работают за $O(\log n)$ в среднем. В худшем случае (неудачные приоритеты) h может быть $\Theta(n)$.

63 Билет №63. В-дерево: определение и практическая значимость. Операция find

63.1 Интуиция и мотивация

В-дерево (*B-tree*) — это сбалансированное дерево поиска, в котором одна вершина хранит сразу *несколько* ключей и имеет *много* детей. Главная цель В-дерева — сделать **высоту маленькой** за счёт большого ветвления. Это особенно важно, когда данные лежат во **внешней памяти** (диск/SSD): стоимость чтения “страницы” (блока) велика, поэтому выгодно за одно обращение к памяти обрабатывать сразу много ключей.

63.2 Определение В-дерева (минимальная степень t)

В литературе встречаются разные параметры (*order m , minimum degree t*). В курсе дальше фигурирует именно t (см. следующий билет про глубину при фиксированном t), поэтому используем определение в стиле CLRS.

Определение 63.1 (В-дерево минимальной степени t). Пусть $t \geq 2$ — **минимальная степень**. В-деревом минимальной степени t называется корневое дерево, удовлетворяющее свойствам:

1. **Устройство вершины.** Каждая вершина v хранит отсортированный массив ключей

$$k_1 < k_2 < \dots < k_\ell, \quad \ell = \text{keyCount}(v).$$

Если v — внутренняя вершина, то у неё есть $\ell + 1$ указателей на детей

$$c_0, c_1, \dots, c_\ell.$$

2. **Разбиение по диапазонам (инвариант поиска).** Для всех ключей в поддеревьях выполняется:

- ▷ все ключи в поддереве c_0 строго меньше k_1 ;
- ▷ для $1 \leq i \leq \ell - 1$ все ключи в поддереве c_i лежат строго между k_i и k_{i+1} ;
- ▷ все ключи в поддереве c_ℓ строго больше k_ℓ .

(Если в задаче допускаются дубликаты, нужно заранее выбрать политику, например: “равные уводим вправо” или “храним счётчик/список значений у ключа”).

3. **Ограничения на число ключей.**

- ▷ Для любой вершины: $\ell \leq 2t - 1$.
- ▷ Для любой *некорневой* вершины: $\ell \geq t - 1$.
- ▷ Для корня: либо корень — лист (тогда может быть $\ell = 0$ в пустом дереве), либо $1 \leq \ell \leq 2t - 1$.

4. **Ограничения на число детей.** Для внутренней вершины число детей равно $\ell + 1$, поэтому:

- ▷ для любой внутренней вершины: $2 \leq \ell + 1 \leq 2t$ у корня (если корень не лист), и $t \leq \ell + 1 \leq 2t$ у некорневых внутренних вершин;
- ▷ листья детей не имеют.

5. **Балансировка.** Все листья находятся на одной и той же глубине.

Замечание. Часто встречается термин “*B-дерево порядка m* ”. В определении Кнута порядок m — это **максимальное число детей** у вершины; тогда $m = 2t$, а максимальное число ключей — $m - 1 = 2t - 1$.

63.3 Практическая значимость

- ▷ **Меньше обращений к диску/SSD.** Если вершина укладывается в одну “страницу” памяти, то на каждом уровне поиска нужно прочитать ровно одну страницу. Большая степень ветвления (до $2t$ детей) делает высоту порядка $\log_t n$, то есть уровней мало.
- ▷ **Используется в индексах и файловых системах.** На практике чаще применяют вариации (например, B^+ -деревья: все записи в листьях, внутренние вершины содержат только ключи-разделители, листья часто связаны в список для быстрых диапазонных запросов).

63.4 Операция find (поиск ключа)

Пусть мы ищем ключ x в B -дереве. Идея полностью аналогична поиску в BST : в каждой вершине мы выясняем, между какими ключами лежит x , и спускаемся *ровно в одного ребёнка*.

Поиск внутри одной вершины

В вершине v лежит отсортированный массив ключей $k_1 < \dots < k_\ell$. Найдём индекс

$$i = \min\{j \in \{1, \dots, \ell\} : x \leq k_j\} \quad (\text{или } i = \ell + 1, \text{ если } x > k_\ell).$$

Это можно сделать:

- ▷ линейным просмотром ($O(\ell)$ сравнений),
- ▷ бинарным поиском ($O(\log \ell)$ сравнений).

Переход в ребёнка

- ▷ Если $i \leq \ell$ и $x = k_i$, то ключ найден в вершине v .
- ▷ Иначе, если v — лист, то ключа x в дереве нет.
- ▷ Иначе спускаемся в ребёнка c_{i-1} (в 0-индексации удобнее писать c_i).

Псевдокод (0-индексация)

Пусть в вершине v : `keys[0..len-1]` — ключи, `child[0..len]` — дети (если `leaf=false`).

Псевдокод (идейно, близко к CLRS)**BTreeFind**(v, x):

1. $i \leftarrow \text{LowerBound}(v.\text{keys}, x)$ (первый индекс, где $\text{keys}[i] \geq x$)
2. **if** $i < v.\text{len}$ **and** $v.\text{keys}[i] = x$ **then return** FOUND(v, i)
3. **if** $v.\text{leaf} = \text{true}$ **then return** NOT_FOUND
4. **return** BTREEFIND($v.\text{child}[i], x$)

63.5 Доказательство корректности find

Докажем, что алгоритм возвращает FOUND тогда и только тогда, когда ключ x присутствует в дереве.

Утверждение 63.1. На каждом шаге алгоритма верен инвариант: если ключ x содержится в исходном дереве, то он содержится в поддереве текущей вершины v .

$\triangle$ *Доказательство.* База: в начале v — корень, инвариант тривиален.

Переход: пусть инвариант верен в вершине v . Мы находим i — первый индекс, где $x \leq k_i$ (или $i = \ell + 1$, если $x > k_\ell$). Если $x = k_i$, то алгоритм возвращает FOUND, и это корректно.

Иначе x строго лежит в одном из диапазонов, задаваемых ключами вершины. По свойству разбиения по диапазонам (п. 2 определения) все ключи из нужного диапазона находятся *ровно* в одном поддереве c_{i-1} , поэтому если x вообще присутствует в дереве, то он обязан лежать именно там. Значит, переход в этого ребёнка сохраняет инвариант.

Если же v — лист и x не равен ни одному ключу в v , то по тому же свойству разбиения x не может находиться ни в каких поддеревьях (их нет), значит x в дереве отсутствует. ■

63.6 Асимптотика find

Пусть h — высота дерева.

▷ За один уровень мы выполняем поиск позиции i среди $\ell \leq 2t - 1$ ключей. При бинарном поиске это $O(\log(2t)) = O(\log t)$ сравнений.

▷ Алгоритм посещает ровно одну вершину на каждом уровне: всего h вершин.

Итого:

$$T_{\text{find}} = O(h \cdot \log t).$$

При фиксированном t (как параметре структуры) это обычно записывают как $O(h)$. А так как для B-дерева $h = O(\log_t n)$ (см. следующий билет про оценку глубины), то получаем $T_{\text{find}} = O(\log n)$ в RAM-модели.

Замечание. Во внешней памяти стоимость часто считают в числе I/O-операций: один переход на уровень = одно чтение страницы. Тогда поиск требует $O(h) = O(\log_t n)$ обращений к памяти.

64 Билет №64. Оценка глубины B -дерева на n ключах при фиксированном параметре t

Определение 64.1 (B -дерево минимальной степени t). Пусть $t \in \mathbb{N}$, $t \geq 2$ — фиксированный параметр (минимальная степень). B -дерево (по *ключам*) минимальной степени t — это поисковое дерево, в котором:

- ▷ все листья находятся на одной глубине;
- ▷ ключи внутри каждой вершины отсортированы;
- ▷ если вершина (не лист) содержит k ключей, то она имеет $k + 1$ детей;
- ▷ для любой **не корневой** вершины число ключей лежит в диапазоне

$$t - 1 \leq k \leq 2t - 1,$$

а значит число детей (если вершина не лист) лежит в диапазоне $t \leq k + 1 \leq 2t$;

- ▷ корень содержит от 1 до $2t - 1$ ключей; если корень не является листом, то у него хотя бы 2 ребёнка.

Далее n — общее число ключей во всём дереве.

Соглашение о глубине. Пусть h — высота (глубина) дерева, равная числу рёбер на самом длинном пути от корня до листа (то есть корень на глубине 0, листья на глубине h). Тогда число уровней (вершин на пути) равно $h + 1$.

Теорема 64.1 (Оценка глубины через n и t). Пусть T — B -дерево минимальной степени $t \geq 2$, содержащее $n \geq 1$ ключей, а его высота равна h (по соглашению выше). Тогда выполнены оценки

$$\log_{2t}(n + 1) - 1 \leq h \leq \log_t \left(\frac{n + 1}{2} \right).$$

В частности, при фиксированном t имеем $h = \Theta(\log n)$.

△ *Доказательство.* Докажем две оценки отдельно.

1) Верхняя оценка (худший случай по высоте). Зафиксируем высоту $h \geq 0$ и минимизируем число ключей при этой высоте, соблюдая ограничения B -дерева. Чтобы дерево было как можно «тоньше»:

- ▷ корень (если не лист) берём с минимально возможным числом детей, то есть с 2 детьми (и 1 ключом);
- ▷ все остальные внутренние вершины берём с минимально возможным числом детей t (то есть с $t - 1$ ключами);
- ▷ все листья также берём с минимумом ключей $t - 1$.

Тогда число вершин на глубине i (корень на глубине 0) не меньше:

$$N_0 = 1, \quad N_i \geq 2t^{i-1} \quad \text{для } i = 1, 2, \dots, h,$$

потому что на первом уровне минимум 2 потомка у корня, а дальше каждый внутренний узел даёт минимум t детей.

Теперь оценим число ключей. В корне минимум 1 ключ, а в каждой вершине на глубине $i \geq 1$ минимум $t - 1$ ключей. Поэтому

$$n \geq 1 + (t - 1) \sum_{i=1}^h N_i \geq 1 + (t - 1) \sum_{i=1}^h 2t^{i-1} = 1 + 2(t^h - 1) = 2t^h - 1.$$

Отсюда $2t^h \leq n + 1$, значит $t^h \leq \frac{n+1}{2}$ и

$$h \leq \log_t \left(\frac{n+1}{2} \right).$$

2) Нижняя оценка (лучший случай по высоте). Теперь зафиксируем h и максимизируем число ключей при этой высоте. Чтобы дерево было как можно «шире», делаем все вершины *полными*: каждая вершина содержит $2t - 1$ ключей и (если не лист) имеет $2t$ детей. Тогда число вершин на глубине i не превосходит $(2t)^i$, и потому

$$n \leq (2t - 1) \sum_{i=0}^h (2t)^i = (2t - 1) \cdot \frac{(2t)^{h+1} - 1}{2t - 1} = (2t)^{h+1} - 1.$$

Следовательно, $n + 1 \leq (2t)^{h+1}$, то есть

$$h + 1 \geq \log_{2t}(n + 1) \implies h \geq \log_{2t}(n + 1) - 1.$$

Обе оценки доказаны. При фиксированном t основания логарифмов — константы, поэтому $h = \Theta(\log n)$. ■

Следствие (Связь с асимптотикой операций). Поскольку поиск/вставка/удаление в B -дереве выполняются спуском от корня к листу, они посещают $O(h)$ вершин. Следовательно, при фиксированном t число посещённых вершин (а в модели внешней памяти — число чтений страниц) равно $\Theta(\log n)$.

Замечание (Про «глубину» как число уровней). Если вместо h (число рёбер) считать число уровней $H = h + 1$ (то есть число вершин на самом длинном пути), то оценки переписываются как

$$\log_{2t}(n + 1) \leq H \leq 1 + \log_t \left(\frac{n + 1}{2} \right),$$

и снова $H = \Theta(\log n)$ при фиксированном t .

65 Билет №65. Реализация операции insert в В-дереве

65.1 Постановка

Дано В-дерево минимальной степени $t \geq 2$, хранящее множество ключей (без повторов). Требуется реализовать операцию `insert(x)`: добавить ключ x так, чтобы после вставки дерево осталось корректным В-деревом.

65.2 Инварианты В-дерева (минимальная степень t)

Определение 65.1. В-дерево минимальной степени t — дерево поиска, где каждая вершина v содержит:

- ▷ $k(v)$ отсортированных ключей $v.key[0], \dots, v.key[k(v) - 1]$;
- ▷ (если v не лист) $k(v) + 1$ указателей на детей $v.child[0], \dots, v.child[k(v)]$.

Требования (инварианты):

1. Границы на число ключей.

- ▷ Если v — не корень, то $t - 1 \leq k(v) \leq 2t - 1$.
- ▷ Если v — корень непустого дерева, то $1 \leq k(v) \leq 2t - 1$.

2. Число детей. Внутренняя вершина с k ключами имеет ровно $k + 1$ детей.

3. Отсортированность внутри вершины. $v.key[0] < v.key[1] < \dots < v.key[k(v)-1]$.

4. Свойство поиска (диапазоны детей). Пусть v имеет ключи $a_0 < a_1 < \dots < a_{k-1}$. Тогда:

- ▷ все ключи в поддереве $v.child[0]$ строго меньше a_0 ;
- ▷ для $1 \leq i \leq k - 1$ все ключи в поддереве $v.child[i]$ лежат строго между a_{i-1} и a_i ;
- ▷ все ключи в поддереве $v.child[k]$ строго больше a_{k-1} .

5. Одинаковая глубина листьев. Все листья находятся на одной глубине.

65.3 Идея вставки

Ключевая трудность: вершина может переполниться (стать содержать $2t - 1$ ключей). Стандартная техника: **никогда не спускаться в полный сын**. Если перед спуском видим, что нужный ребенок заполнен, *сначала* делаем **split** этого ребенка. Тогда до листа мы гарантированно дойдем в вершину, где есть место для нового ключа.

65.4 Вспомогательная процедура splitChild

Определение 65.2. `splitChild(parent, i)`: родитель $x = \text{parent}$ имеет полного ребенка $y = x.\text{child}[i]$ (то есть $k(y) = 2t - 1$). Процедура делит y на две вершины y и z по медианному ключу и поднимает медиану в x .

Что происходит при split:

- ▷ Медиана $m = y.\text{key}[t - 1]$ поднимается в родителя x .
- ▷ В y остаются $t - 1$ *наименьших* ключей $y.\text{key}[0..t - 2]$.
- ▷ Создается новая вершина z (того же типа: лист/не лист), в нее копируются $t - 1$ *наибольших* ключей $y.\text{key}[t..2t - 2]$.
- ▷ Если y не лист, то последние t детей $y.\text{child}[t..2t - 1]$ переезжают в $z.\text{child}[0..t - 1]$.
- ▷ В родителе x ключ m вставляется на позицию i , а указатель на z вставляется на позицию $i + 1$.

65.5 Алгоритм insert

Будем использовать две функции:

- ▷ `BTreeInsert(T, x)` — внешний интерфейс.
- ▷ `InsertNonFull(v, x)` — вставка в вершину v , которая гарантированно *не полная*.

Псевдокод

Псевдокод (идейно, близко к CLRS)

BTreeInsert(T, x):

1. $r \leftarrow T.\text{root}$
2. **if** $r.n = 2t - 1$ **then** (корень полон)
 - (a) $s \leftarrow \text{NewNode}(\text{leaf} = \text{false})$
 - (b) $T.\text{root} \leftarrow s$
 - (c) $s.n \leftarrow 0$
 - (d) $s.\text{child}[0] \leftarrow r$
 - (e) `SPLITCHILD`($s, 0$)
 - (f) `INSERTNONFULL`(s, x)
3. **else** `INSERTNONFULL`(r, x)

SplitChild(x, i): ($y = x.\text{child}[i]$ полон: $y.n = 2t - 1$)

1. $y \leftarrow x.\text{child}[i]$
2. $z \leftarrow \text{NewNode}(\text{leaf} = y.\text{leaf})$

```

3.  $z.n \leftarrow t - 1$ 
4.  $m \leftarrow y.key[t - 1]$  (медианный ключ)
5. for  $j \leftarrow 0$  to  $t - 2$  do  $z.key[j] \leftarrow y.key[j + t]$ 
6. if  $y.leaf = \text{false}$  then
    (a) for  $j \leftarrow 0$  to  $t - 1$  do  $z.child[j] \leftarrow y.child[j + t]$ 
7.  $y.n \leftarrow t - 1$  (в  $y$  оставили левую половину)
8. for  $j \leftarrow x.n$  downto  $i + 1$  do  $x.child[j + 1] \leftarrow x.child[j]$ 
9.  $x.child[i + 1] \leftarrow z$ 
10. for  $j \leftarrow x.n - 1$  downto  $i$  do  $x.key[j + 1] \leftarrow x.key[j]$ 
11.  $x.key[i] \leftarrow m$ 
12.  $x.n \leftarrow x.n + 1$ 

InsertNonFull( $x, k$ ): ( $x$  не полон)
1.  $i \leftarrow x.n - 1$ 
2. if  $x.leaf = \text{true}$  then
    (a) while  $i \geq 0$  and  $k < x.key[i]$  do
        i.  $x.key[i + 1] \leftarrow x.key[i]$ 
        ii.  $i \leftarrow i - 1$ 
    (b)  $x.key[i + 1] \leftarrow k$ 
    (c)  $x.n \leftarrow x.n + 1$ 
3. else
    (a) while  $i \geq 0$  and  $k < x.key[i]$  do  $i \leftarrow i - 1$ 
    (b)  $i \leftarrow i + 1$  (выбрали ребёнка  $x.child[i]$ )
    (c) if  $x.child[i].n = 2t - 1$  then
        i. SPLITCHILD( $x, i$ )
        ii. if  $k > x.key[i]$  then  $i \leftarrow i + 1$ 
    (d) INSERTNONFULL( $x.child[i], k$ )

```

65.6 Корректность

Докажем, что алгоритм всегда сохраняет инварианты В-дерева.

Лемма 65.1 (Корректность `splitChild`). Пусть x — корректная вершина B -дерева, а ее ребенок $y = x.child[i]$ полон: $k(y) = 2t - 1$. После выполнения `splitChild(x, i)` получаем:

1. y и z (новая вершина) содержат ровно $t - 1$ ключ, то есть удовлетворяют нижней и верхней границе для не корней.
2. В x число ключей увеличивается на 1 (и не превосходит $2t - 1$, т.к. `splitChild` вызывается только когда x не полон).
3. Сохраняется свойство поиска: ключи в поддереве y меньше поднятой медианы m , а ключи в поддереве z больше m ; остальные ключи/дети x не нарушают порядок.
4. Если y был листом, то z тоже лист, и глубина листьев не меняется. Если y был внутренней вершиной, то перераспределение детей сохраняет корректные диапазоны.

△ Доказательство. (1) По построению: в y оставляем $t - 1$ ключей, в z копируем $t - 1$ ключей. Если y не лист, в z переносим ровно t детей, соответствующих перенесенным ключам.

(2) Перед вызовом `splitChild` родитель x не полон, значит $k(x) \leq 2t - 2$; после вставки медианы получаем $k(x) \leq 2t - 1$.

(3) Медиана $m = y.key[t - 1]$ отделяет $t - 1$ меньших ключей и $t - 1$ больших ключей. По инварианту B -дерева до split все ключи в поддеревьях $y.child[0..t - 1]$ лежат слева от m , а ключи в $y.child[t..2t - 1]$ лежат справа от m . После переноса правой части в z эти диапазоны сохраняются. В родителе x ключ m вставляется между $x.child[i]$ и $x.child[i + 1]$, поэтому глобальный порядок не нарушается.

(4) Split не меняет глубины листьев внутри поддеревьев: мы только разделили вершину на две на том же уровне. ■

Лемма 65.2 (Инвариант “спускаемся только в неполную вершину”). В процессе выполнения `InsertNonFull(v, x)` функция рекурсивно вызывается только на вершинах, в которых $k(\cdot) \leq 2t - 2$ (то есть есть место для вставки).

△ Доказательство. По предположению входная вершина v неполна. Если v лист, рекурсии нет. Если v внутренняя, выбираем ребенка $c = v.child[i]$ для спуска. Если c полон, сначала выполняем `splitChild(v, i)`, после чего вместо одного полного ребенка получаем двух неполных ($t - 1$ ключей). Затем выбираем правильного ребенка (левого или правого) и спускаемся уже в него. ■

Теорема 65.1 (Корректность `insert`). После выполнения `BTreeInsert(T, x)` дерево T остается корректным B -деревом и содержит добавленный ключ x .

△ Доказательство. Рассмотрим два случая.

1) **Корень не полон.** Тогда вызывается `InsertNonFull(root, x)`. По предыдущей лемме, рекурсия всегда спускается в неполную вершину. В конце рекурсии мы попадаем в лист, который неполон, и вставляем ключ в отсортированном порядке. В листе число ключей увеличивается на 1 и не превышает $2t - 1$. Все остальные вершины либо не меняются, либо претерпевают `splitChild`, который корректен (первая лемма). Глубина листьев не

меняется.

2) Корень полон. Создается новый корень s с одним ребенком (старым корнем) и выполняется `splitChild(s, 0)`. Высота дерева увеличивается на 1, но все прежние листья становятся глубже ровно на 1, поэтому остаются на одной глубине. После `split` новый корень s неполон, и дальнейшая вставка сводится к случаю (1).

В обоих случаях после завершения процедуры все вершины удовлетворяют границам на число ключей, порядок ключей и диапазоны детей сохраняются, а ключ x действительно добавлен. ■

65.7 Асимптотика

Пусть в дереве n ключей. Высота В-дерева минимальной степени t равна $h = \Theta(\log_t n)$.

▷ На каждом уровне мы:

- находим индекс ребенка i (линейным поиском $O(t)$ или бинарным поиском $O(\log t)$);
- возможно делаем `splitChild`, который перемещает $O(t)$ ключей/детей.

▷ В худшем случае `split` происходит на каждом уровне (когда все вершины на пути полны).

Итого время вставки:

$$T_{\text{insert}} = O(t \cdot h) = O(t \log_t n).$$

Если t считается константой (типичный случай во внешней памяти, когда узел — это дисковая страница), то $T_{\text{insert}} = O(\log n)$ по числу уровней. По числу обращений к страницам памяти/диска: $O(h) = O(\log_t n)$.

66 Билет №66. Реализация операции erase в В-дереве

Определение 66.1 (В-дерево минимальной степени t). Пусть $t \geq 2$. В-дерево (*B-tree*) минимальной степени t — это сбалансированное дерево поиска, в каждой вершине которого хранится *отсортированный* массив ключей и (для внутренних вершин) массив указателей на детей.

Для вершины v пусть

$$\text{keys}(v) = (k_0 < k_1 < \dots < k_{m-1}), \quad m = |\text{keys}(v)|.$$

Тогда выполняются свойства:

▷ (Разделение ключей.) Если v не лист, то у него $m + 1$ детей $c_0, \dots, c_m$ и

$$\forall i \in [0, m]: \quad \text{keys}(c_i) \subset (k_{i-1}, k_i), \quad k_{-1} = -\infty, k_m = +\infty.$$

▷ (Заполненность.)

- если v — не корень, то $t - 1 \leq m \leq 2t - 1$;
- корень либо лист (тогда дерево из одной вершины), либо $1 \leq m \leq 2t - 1$.

▷ (Баланс.) Все листья лежат на одной глубине.

Замечание. В реализации удобно хранить в вершине два массива: `keys[0..m-1]` и `child[0..m]`. Любые локальные сдвиги ключей/детей требуют $O(m) = O(t)$ элементарных операций, поскольку $m \leq 2t - 1$.

Идея удаления

Опасность при удалении — *недозаполнение* (*underflow*): после удаления ключа вершина может получить меньше $t - 1$ ключей. Классическая стратегия (CLRS): **никогда не спускаться в ребёнка с $t - 1$ ключами**. Перед рекурсивным спуском “чинить” выбранного ребёнка так, чтобы у него стало **хотя бы t ключей**. Тогда после удаления одного ключа внизу он гарантированно останется с $\geq t - 1$ ключами.

Локальные операции (“взять в долг” и “слить”)

Рассмотрим внутреннюю вершину v с ключами $k_0, \dots, k_{m-1}$ и детьми $c_0, \dots, c_m$. Пусть нужно исправить ребёнка c_i , у которого сейчас $t - 1$ ключей.

1. **Заём у левого соседа (rotate right).** Если $i > 0$ и у левого брата c_{i-1} не меньше t ключей, то:
 - ▷ опускаем разделитель k_{i-1} из v в начало ключей c_i ;
 - ▷ поднимаем максимальный ключ из c_{i-1} на место k_{i-1} ;
 - ▷ если вершины не листья, переносим последнего ребёнка c_{i-1} в начало массива детей c_i .

В итоге c_i получает $+1$ ключ (становится t), а c_{i-1} теряет 1 ключ (остаётся $\geq t - 1$).

2. **Заём у правого соседа (rotate left).** Если $i < m$ и у правого брата c_{i+1} не меньше t ключей, то:

- ▷ опускаем разделитель k_i из v в конец ключей c_i ;
- ▷ поднимаем минимальный ключ из c_{i+1} на место k_i ;
- ▷ если вершины не листья, переносим первого ребёнка c_{i+1} в конец массива детей c_i .

3. **Слияние (merge).** Если занять нельзя (оба доступных брата имеют ровно $t - 1$ ключей), то сливаем c_i с одним из братьев. Самый удобный вариант, когда $i < m$: слить c_i и c_{i+1} через разделитель k_i :

- ▷ новый узел c_i получает ключи

$$\text{keys}(c_i) \leftarrow \text{keys}(c_i) \parallel [k_i] \parallel \text{keys}(c_{i+1}),$$

т.е. суммарно $(t - 1) + 1 + (t - 1) = 2t - 1$ ключей;

- ▷ если не листья — аналогично склеиваем массивы детей;
- ▷ из v удаляем ключ k_i и указатель на c_{i+1} (освобождаем c_{i+1}).

После merge у объединённого узла c_i ключей достаточно, а у родителя v на один ключ и одного ребёнка меньше.

Алгоритм erase

Пусть удаляем ключ x из В-дерева T . Опишем рекурсивную процедуру $\text{Delete}(v, x)$ для поддерева с корнем v .

Псевдокод (идейно, близко к CLRS)

Erase(T, x):

1. $\text{Delete}(T.\text{root}, x)$
2. **если** $T.\text{root}$ не лист и в нём стало 0 ключей, **то** сделать единственного ребёнка новым корнем (высота дерева уменьшается на 1).

Delete(v, x):

1. Найдём $i = \min\{j \mid j = m \text{ или } x \leq k_j\}$ (**lower_bound** в массиве ключей вершины).
2. **Случай А: ключ найден в вершине.** Если $i < m$ и $k_i = x$:
 - A1. Если v — лист, удалить k_i из массива ключей и завершить.
 - A2. v — внутренняя вершина. Пусть $y = c_i$ (левый ребёнок-разделяемое поддерево), $z = c_{i+1}$ (правый).

- i. Если в y не меньше t ключей, то берём *предшественника*

$$x^- = \max \text{keys}(y)$$

(идём из y всегда в правого ребёнка до листа), заменяем $k_i \leftarrow x^-$ и рекурсивно удаляем x^- из y .

- ii. Иначе, если в z не меньше t ключей, то берём *преемника*

$$x^+ = \min \text{keys}(z)$$

(идём из z всегда в левого ребёнка до листа), заменяем $k_i \leftarrow x^+$ и рекурсивно удаляем x^+ из z .

- iii. Иначе (и y , и z имеют ровно $t - 1$ ключей): **сливаем** y , ключ k_i и z в один узел y (получаем $2t - 1$ ключ), удаляя k_i и z из v . Затем рекурсивно вызываем $\text{Delete}(y, x)$.

3. **Случай В: ключа нет в вершине.** Если v — лист, то ключа в дереве нет (в задачах “удаление из множества” обычно можно просто завершить). Иначе x должен лежать в поддереве ребёнка c_i .

4. **Перед спуском обеспечим, что в c_i хотя бы t ключей.** Если в c_i ровно $t - 1$ ключей:

Б1. Если существует левый брат c_{i-1} и в нём $\geq t$ ключей — *занимаем слева*.

Б2. Иначе, если существует правый брат c_{i+1} и в нём $\geq t$ ключей — *занимаем справа*.

Б3. Иначе делаем *слияние*:

▷ если $i < m$, сливаем c_i с c_{i+1} через ключ k_i ;

▷ иначе ($i = m$), сливаем c_{i-1} с c_i через ключ k_{i-1} и далее считаем, что нужный ребёнок — это c_{i-1} .

5. Рекурсивно вызвать $\text{Delete}(\text{нужный_ребёнок}, x)$.

Корректность

Докажем, что алгоритм всегда заканчивается и сохраняет свойства В-дерева.

1. **Терминация.** Каждый рекурсивный вызов уходит строго на уровень ниже (в один из детей). В случае слияния (А2.3/Б3) мы также спускаемся в объединённого ребёнка, который лежит на уровень ниже текущего узла. Высота конечна, значит процесс завершится.
2. **Сохранение свойства порядка (дерево поиска).**

▷ При замене x на предшественника x^- мы берём *максимальный* ключ в левом поддереве c_i , значит $\forall u \in c_i : u \leq x^- \leq x$ и при этом $x^- <$ все ключи правее. Аналогично для преемника x^+ . Поэтому замена не нарушает разделение диапазонов ключей.

- ▷ Заём у брата и слияние двигают только *граничные* ключи: один ключ родителя опускается в ребёнка, а из брата поднимается крайний ключ (максимальный слева или минимальный справа). Эти ключи по определению лежат на границе диапазонов, поэтому после перестановки порядок “всё слева меньше разделителя меньше всё справа” сохраняется.

3. Сохранение ограничений на число ключей.

- ▷ В заёме ребёнок с $t - 1$ ключами получает один ключ и становится ровно t ; брат теряет один ключ, но у него было минимум t , значит остаётся $\geq t - 1$.
- ▷ При слиянии два узла с $t - 1$ ключами и один разделитель дают $2t - 1$ ключ — это ровно максимально допустимое значение. Родитель теряет один ключ и одного ребёнка; если родитель не корень, он мог стать на 1 меньше, но “провал” (меньше $t - 1$) будет обработан уровнем выше за счёт инварианта спуска.
- ▷ Единственная вершина, которой разрешено стать пустой, — корень. В конце мы отдельно обрабатываем случай, когда корень стал пустым: тогда либо дерево пусто (корень-лист), либо у корня один ребёнок, и мы понижаем высоту, делая этого ребёнка новым корнем.

4. **Баланс (все листья на одной глубине).** Заёмы и слияния не меняют глубину листьев: они лишь перераспределяют ключи и детей *между соседними узлами одного уровня*. Замена на предшественника/преемника тоже не меняет структуру уровней. Значит, все листья остаются на одной глубине.

Асимптотика

Пусть в дереве n ключей. Высота B-дерева с минимальной степенью t равна $h = \Theta(\log_t n)$. На каждом уровне мы выполняем:

- ▷ поиск позиции i в массиве ключей текущей вершины ($O(t)$, либо $O(\log t)$ при двоичном поиске, но обычно t считается константой страницы),
- ▷ возможно один заём или одно слияние (в худшем случае сдвиги массивов размера $O(t)$).

Следовательно, полная стоимость

$$T(n) = O(t \cdot h) = O(t \log_t n).$$

В модели внешней памяти (B-tree как страничная структура) это соответствует $O(\log_t n)$ обращений к страницам, так как на каждом уровне читается/правится $O(1)$ узлов.

67 Билет №67. Красно-чёрное дерево: определение

Что такое красно-чёрное дерево и зачем оно нужно

Красно-чёрное дерево (КЧД) — это **самобалансирующееся бинарное дерево поиска**. Оно хранит упорядоченное множество (или мультимножество) ключей и поддерживает стандартные операции `find/insert/erase/lower_bound` за $O(\log n)$ за счёт того, что дерево не становится «слишком высоким».

Базовое свойство бинарного дерева поиска

Во всех вершинах хранятся ключи из линейно упорядоченного множества.

Определение 67.1 (Бинарное дерево поиска (BST)). Дерево является **деревом поиска**, если для любой вершины v с ключом $key(v)$ выполнено:

- ▷ для любого ключа x в **левом** поддереве v верно $x < key(v)$;
- ▷ для любого ключа y в **правом** поддереве v верно $key(v) < y$.

Определение красно-чёрного дерева

Интуитивно: КЧД — это BST, где каждой вершине приписан цвет (красный/чёрный), а на цвета наложены инварианты, ограничивающие высоту.

Определение 67.2 (Красно-чёрное дерево). **Красно-чёрное дерево** — это BST, в каждой вершине которого хранится дополнительный бит **цвета** (RED/BLACK), и выполнены следующие **инварианты**:

1. Каждая вершина окрашена либо в **красный**, либо в **чёрный** цвет.
2. **Корень** дерева — чёрный.
3. Все **внешние листья** (фиктивные вершины NIL, отсутствующие дети) считаются **чёрными**.
4. У **красной** вершины **оба** ребёнка чёрные (*двух красных подряд быть не может*).
5. Для любой вершины v число **чёрных** вершин на любом пути из v до любого внешнего листа NIL (не считая саму v , но считая NIL) **одинаково**. Это число называют **чёрной высотой** вершины v .

Определение 67.3 (Чёрная высота). **Чёрная высота** вершины v (обозначение $bh(v)$) — это количество чёрных вершин на любом пути от v до внешнего листа NIL, **не включая** v и **включая** NIL. Корректность определения обеспечивается инвариантом 5: все такие пути содержат одинаковое число чёрных вершин.

Непосредственные следствия инвариантов

1. **Красные вершины изолированы**: у красной вершины родитель и дети чёрные.

2. На любом пути от корня до NIL красных вершин **не больше**, чем чёрных: между двумя чёрными вершинами может стоять **не более одной** красной.
3. Если h — высота дерева (число вершин на самом длинном пути от корня до NIL), то $h \leq 2bh(\text{root})$. (Идея: если бы красные шли максимально часто, путь бы чередовал цвета BLACK/RED/BLACK/...)

Подробная оценка $h = O(\log n)$ выводится в следующем билете (оценка глубины КЧД).

Замечания про реализацию

- ▷ Обычно вводят единственный объект-страж NIL (sentinel), который считается чёрным и используется вместо `nullptr` в полях `left/right`. Это упрощает код и формулировки инвариантов.
- ▷ Типичная структура узла: `key, color, left, right, parent`.
- ▷ Про **повторы ключей**: если требуется мультимножество, удобно хранить счётчик $cnt(v)$ в вершине (или договориться, что равные ключи всегда идут, например, вправо). В курсе часто подразумевается множество (все ключи различны).

Интуиция: связь с (2,3,4)-деревьями (необязательно, но полезно)

Если **склеивать** чёрную вершину с её красными детьми, то получаются «узлы» размера 2, 3 или 4, то есть КЧД можно рассматривать как бинарное представление (2,3,4)-дерева. Из этой интуиции видно, почему высота остаётся логарифмической.

68 Билет №68. Оценка глубины красно-чёрного дерева на n ключах

68.1 Инварианты красно-чёрного дерева

Красно-чёрное дерево (КЧ-дерево) — это бинарное дерево поиска, в котором каждая вершина помечена цветом **red** или **black** и выполняются следующие свойства:

1. Каждая вершина имеет цвет **red** или **black**.
2. Корень чёрный.
3. Все внешние листья (NIL-вершины, “пустые дети”) считаются чёрными.
4. У красной вершины оба ребёнка чёрные (двух красных подряд на пути нет).
5. Для любой вершины v число чёрных вершин на любом простом пути от v до NIL одинаково (*чёрная высота* определена корректно).

Далее n — число *внутренних* (не-NIL) вершин, то есть число хранимых ключей.

68.2 Чёрная высота

Определение 68.1. Чёрной высотой вершины v (обозначение $bh(v)$) назовём число чёрных вершин на любом пути от v **не включая** v до NIL **включая** NIL. Корректность определения обеспечивается свойством 5.

Заметим, что из свойства 2 (корень чёрный) и свойства 3 (NIL чёрные) следует:

- ▷ на любом пути от корня до NIL число чёрных *внутренних* вершин равно $bh(\text{root})$;
- ▷ высота дерева оценивается через $bh(\text{root})$.

Лемма 68.1. Пусть h — высота КЧ-дерева (число рёбер на самом длинном пути от корня до NIL). Тогда

$$h \leq 2 bh(\text{root}).$$

Δ *Доказательство.* Рассмотрим произвольный путь от корня до NIL. По свойству 4 после красной вершины обязательно идёт чёрная (красные не могут стоять подряд). Значит, между любыми двумя чёрными внутренними вершинами на пути может находиться не более одной красной.

На пути ровно $bh(\text{root})$ чёрных *внутренних* вершин. Следовательно, красных внутренних вершин на пути не больше, чем чёрных внутренних вершин, то есть число внутренних вершин на пути не превосходит $2 bh(\text{root})$.

Путь от корня до NIL содержит столько же рёбер, сколько на нём внутренних вершин: каждое ребро ведёт либо от внутренней вершины к внутренней, либо от последней внутренней вершины к NIL. Значит, число рёбер на пути (а это и есть высота h) не превосходит $2 bh(\text{root})$. ■

68.3 Нижняя оценка на размер через чёрную высоту

Лемма 68.2. *Поддерево, корнем которого является вершина v , содержит не менее*

$$2^{\text{bh}(v)} - 1$$

внутренних вершин.

$\triangle$ *Доказательство.* Докажем по индукции по $b = \text{bh}(v)$.

База: $b = 0$. Тогда v — NIL-вершина (путь от неё до NIL состоит из одной вершины, и чёрных вершин, не считая v , ровно 0). В её поддереве 0 внутренних вершин, и $0 = 2^0 - 1$.

Переход: пусть утверждение верно для всех вершин с чёрной высотой меньше b . Рассмотрим внутреннюю вершину v с $\text{bh}(v) = b > 0$ и её детей l и r .

Рассмотрим одного ребёнка $u \in \{l, r\}$. По определению $\text{bh}(v)$ на любом пути от v до NIL (не считая v) ровно b чёрных вершин. Тогда:

▷ если u чёрный, то u даёт одну чёрную вершину на пути от v до NIL, значит $\text{bh}(u) = b - 1$;

▷ если u красный, то он не добавляет чёрных вершин, значит $\text{bh}(u) = b$.

В первом случае (u чёрный) по предположению индукции

$$|T_u| \geq 2^{b-1} - 1.$$

Во втором случае (u красный) его дети чёрные по свойству 4. Пусть это u_1 и u_2 . Тогда для каждого из них $\text{bh}(u_i) = b - 1$ (так как чёрная вершина “съедает” один чёрный уровень), и по предположению индукции

$$|T_{u_i}| \geq 2^{b-1} - 1.$$

Следовательно,

$$|T_u| \geq 1 + |T_{u_1}| + |T_{u_2}| \geq 1 + 2(2^{b-1} - 1) = 2^b - 1 \geq 2^{b-1} - 1.$$

Итак, в **любом** случае каждый ребёнок u удовлетворяет $|T_u| \geq 2^{b-1} - 1$. Тогда число внутренних вершин в поддереве v не меньше

$$1 + (2^{b-1} - 1) + (2^{b-1} - 1) = 2^b - 1.$$

Переход доказан. ■

68.4 Итоговая оценка высоты

Теорема 68.1. *Пусть красно-чёрное дерево содержит n ключей (то есть n внутренних вершин). Тогда его высота удовлетворяет оценке*

$$h \leq 2 \log_2(n + 1) = O(\log n).$$

△ *Доказательство.* Пусть $b = \text{bh}(\text{root})$. По предыдущей лемме для корня получаем

$$n \geq 2^b - 1 \quad \Rightarrow \quad b \leq \log_2(n + 1).$$

По первой лемме $h \leq 2b$. Следовательно,

$$h \leq 2 \log_2(n + 1).$$



Замечание. Из $h = O(\log n)$ следует, что стандартные операции поиска в КЧ-дереве (`find/lower_bound`) выполняются за $O(\log n)$, а операции `insert/erase` выполняются за $O(\log n)$ поворотов/перекрасок, так как исправляют нарушения только на пути длины $O(h)$.

69 Билет №69. Реализация операции insert в красно-чёрном дереве

69.1 Что такое красно-чёрное дерево (инварианты)

Красно-чёрное дерево (КЧ-дерево, *red-black tree*) — это бинарное дерево поиска, в каждой вершине которого хранится дополнительный бит цвета. Будем использовать цвета **RED** (красный) и **BLACK** (чёрный).

Стандартные свойства КЧ-дерева (инварианты):

1. **(RB1)** Каждая вершина либо красная, либо чёрная.
2. **(RB2)** Корень чёрный.
3. **(RB3)** Все NIL-листья (фиктивные «пустые» дети) считаются чёрными.
4. **(RB4)** У красной вершины оба ребёнка чёрные (*нет двух красных подряд*).
5. **(RB5)** Для любой вершины число чёрных вершин на любом пути от неё до NIL-листа одинаково (это число называется *black-height*).

Замечание про реализацию: удобно хранить один общий NIL-узел (sentinel), у которого цвет **BLACK**, и вместо `nullptr` держать указатели на него. Тогда свойства RB3–RB5 формулируются и проверяются проще.

69.2 Вспомогательные операции: повороты

Вставка в КЧ-дерево использует **левый** и **правый** повороты. Они сохраняют свойство бинарного дерева поиска (порядок ключей) и работают за $O(1)$.

Левый поворот вокруг вершины x (обозначим `LEFT-ROTATE(x)`): пусть $y = x.\text{right}$. Тогда y поднимается на место x , а x становится левым ребёнком y . Поддерево $y.\text{left}$ переезжает в $x.\text{right}$.

Правый поворот `RIGHT-ROTATE(x)` — симметричен: пусть $y = x.\text{left}$, тогда y поднимается на место x , x становится правым ребёнком y , а поддерево $y.\text{right}$ переезжает в $x.\text{left}$.

69.3 Идея вставки: BST-вставка + исправление (fixup)

Операция `insert` состоит из двух частей:

1. **Обычная вставка в BST.** Вставляем новый ключ как лист в бинарное дерево поиска (как в наивном BST).
2. **Восстановление инвариантов КЧ-дерева** (процедура `RB-INSERT-FIXUP`).

69.3.1 Шаг 1: вставка как в BST

Пусть вставляем вершину z . Делаем стандартный спуск от корня как в `find/insert` BST и подвешиваем z листом. После этого:

- ▷ красим z в **RED**;
- ▷ $z.\text{left} = z.\text{right} = \text{NIL}$ (чёрные).

Почему новый узел красим в красный: так мы *не меняем* `black-height` на путях (RB5) — если бы покрасили в чёрный, `black-height` на пути через z вырос бы на 1.

Какие свойства могли нарушиться после такого шага:

- ▷ RB1 и RB3 выполнены по построению;
- ▷ RB5 сохраняется (почему — см. выше);
- ▷ RB2 может нарушиться только если z стал корнем;
- ▷ RB4 может нарушиться только если родитель z красный (получится два красных подряд).

Значит, дальше нужно чинить **только** возможное нарушение RB2/RB4.

69.3.2 Шаг 2: RB-INSERT-FIXUP — разбор случаев

Пока у z красный родитель, у нас нарушено RB4. Пусть:

- ▷ $p = z.\text{parent}$ — родитель,
- ▷ $g = p.\text{parent}$ — дедушка,
- ▷ u — **дядя** (*uncle*) — второй ребёнок g : если p левый сын g , то $u = g.\text{right}$, иначе $u = g.\text{left}$.

Далее есть три основные ситуации (и симметрия «лево/право»).

Случай 0: z — корень. Тогда просто красим z в **BLACK** и заканчиваем.

Случай 1: дядя u красный. Картинка: p и u красные, g чёрный (иначе RB4 нарушалось бы ещё до вставки). Делаем **перекраску**:

- ▷ $p.\text{color} \leftarrow \text{BLACK}$,
- ▷ $u.\text{color} \leftarrow \text{BLACK}$,
- ▷ $g.\text{color} \leftarrow \text{RED}$,
- ▷ $z \leftarrow g$ (поднимаем «проблему» на уровень выше).

Интуиция: мы устраняем «две красных подряд» внизу, но можем создать такую же проблему выше, если родитель g тоже красный. При этом RB5 не ломается: на путях через p или через u число чёрных вершин не меняется (каждый путь потерял одного чёрного g , но приобрёл чёрного p или u).

Случай 2: дядя u чёрный, а z — «внутренний» внук. То есть:

- ▷ p — левый сын g , а z — правый сын p (конфигурация LR),
- ▷ или p — правый сын g , а z — левый сын p (конфигурация RL).

Тогда делаем **поворот вокруг p** , чтобы перейти к случаю 3:

- ▷ если конфигурация LR, делаем LEFT-ROTATE(p);
- ▷ если RL, делаем RIGHT-ROTATE(p).

После этого вершина z «становится» внешним внуком относительно g .

Случай 3: дядя u чёрный, а z — «внешний» внук. То есть:

- ▷ p — левый сын g и z — левый сын p (LL),
- ▷ или p — правый сын g и z — правый сын p (RR).

Тогда делаем **перекраску + поворот вокруг g** :

- ▷ $p.color \leftarrow \text{BLACK}$,
- ▷ $g.color \leftarrow \text{RED}$,
- ▷ если LL, делаем RIGHT-ROTATE(g);
- ▷ если RR, делаем LEFT-ROTATE(g).

После этого RB4 восстановлено локально (красная вершина g становится ребёнком чёрной p), а RB5 не меняется (чёрная «масса» путей сохраняется). Дальше цикл заканчивается, потому что родитель текущего z стал чёрным.

Симметрия. Все случаи выше описаны для одной ориентации. Если p был правым сыном g , то во всех формулах надо поменять $\text{left} \leftrightarrow \text{right}$ и $\text{LEFT-ROTATE} \leftrightarrow \text{RIGHT-ROTATE}$.

69.3.3 Псевдокод (классический вариант)

Ниже ровно то же самое, но компактно (удобно заучивать):

Псевдокод (идейно, близко к CLRS)

RB-INSERT(T, z):

1. BST_INSERT(T, z) (как в обычном BST)
2. $z.color \leftarrow \text{RED}$
3. $z.left \leftarrow z.right \leftarrow T.NIL$
4. RB_INSERT_FIXUP(T, z)

RB-INSERT-FIXUP(T, z):


```

1. while  $z.parent.color = \text{RED}$  do
    (a) if  $z.parent = z.parent.parent.left$  then
        i.  $u \leftarrow z.parent.parent.right$  (дядя)
        ii. if  $u.color = \text{RED}$  then (Case 1)
            A.  $z.parent.color \leftarrow \text{BLACK}$ 
            B.  $u.color \leftarrow \text{BLACK}$ 
            C.  $z.parent.parent.color \leftarrow \text{RED}$ 
            D.  $z \leftarrow z.parent.parent$ 
        iii. else
            A. if  $z = z.parent.right$  then  $z \leftarrow z.parent$ ;  $\text{LEFT\_ROTATE}(T, z)$  (Case 2)
            B.  $z.parent.color \leftarrow \text{BLACK}$  (Case 3)
            C.  $z.parent.parent.color \leftarrow \text{RED}$ 
            D.  $\text{RIGHT\_ROTATE}(T, z.parent.parent)$ 
    (b) else (симметрично:  $left \leftrightarrow right$ )
        i.  $u \leftarrow z.parent.parent.left$ 
        ii. if  $u.color = \text{RED}$  then (Case 1)
            A.  $z.parent.color \leftarrow \text{BLACK}$ 
            B.  $u.color \leftarrow \text{BLACK}$ 
            C.  $z.parent.parent.color \leftarrow \text{RED}$ 
            D.  $z \leftarrow z.parent.parent$ 
        iii. else
            A. if  $z = z.parent.left$  then  $z \leftarrow z.parent$ ;  $\text{RIGHT\_ROTATE}(T, z)$  (Case 2)
            B.  $z.parent.color \leftarrow \text{BLACK}$  (Case 3)
            C.  $z.parent.parent.color \leftarrow \text{RED}$ 
            D.  $\text{LEFT\_ROTATE}(T, z.parent.parent)$ 

2.  $T.root.color \leftarrow \text{BLACK}$ 

```

69.4 Доказательство корректности

1) BST-свойство не нарушается. BST-вставка подвешивает z листом в единственно допустимую позицию по порядку ключей. Повороты LEFT_ROTATE / RIGHT_ROTATE не меняют inorder-порядок ключей (они лишь локально переставляют связи, сохраняя отношения $left < root < right$). Значит, после всех действий дерево остаётся деревом поиска.

2) Какие RB-свойства могли ломаться и что чинит fixup. После вставки красного листа могли нарушиться только:

- ▷ RB2 (если z — корень),
- ▷ RB4 (если z и его родитель p оба красные).

Свойство RB5 не нарушается при начальном окрашивании z в красный, а повороты не меняют количество чёрных вершин на путях (они только перестраивают связи).

3) Корректность случаев.

- ▷ **Case 1 (дядя красный):** перекраска делает p и u чёрными, значит локально RB4 восстановлено. Возможная новая проблема — что g стал красным и у него красный родитель. Поэтому мы поднимаем $z \leftarrow g$ и повторяем цикл.
- ▷ **Case 2 (дядя чёрный, внутренний внук):** один поворот вокруг p переводит конфигурацию в Case 3, не ухудшая RB5 и не создавая новых нарушений, кроме уже существующего.
- ▷ **Case 3 (дядя чёрный, внешний внук):** перекраска + поворот вокруг g устраняют нарушение RB4 окончательно. После поворота родитель z становится чёрным, поэтому цикл заканчивается.

4) Завершение. Цикл повторяется *только* в Case 1, где мы заменяем $z \leftarrow g$, то есть поднимаем рассматриваемую вершину на два уровня вверх. Значит, число итераций ограничено высотой дерева: $O(h)$. В конце принудительно красим корень в чёрный, восстанавливая RB2.

69.5 Асимптотика

- ▷ BST-вставка делает один спуск от корня до листа: $O(h)$.
- ▷ RB-INSERT-FIXUP выполняет $O(h)$ итераций, при этом поворотов не больше двух (один в Case 2 и один в Case 3), а остальные действия — перекраски и переходы по указателям за $O(1)$.

Итого `insert` работает за $O(h)$. Для красно-чёрного дерева $h = O(\log n)$, поэтому **вставка работает за $O(\log n)$.**

70 Билет №70. Реализация операции erase в красно-чёрном дереве

Контекст и соглашения

Красно-чёрное дерево (КЧ-дерево) — это бинарное дерево поиска, в котором поддерживаются инварианты, гарантирующие высоту $O(\log n)$. В реализации операции удаления удобно фиксировать стандартные соглашения:

- ▷ у каждой вершины есть поля `key`, `left`, `right`, `parent`, `color` (RED/BLACK);
- ▷ отсутствующие дети представлены одним общим сторожем NIL (sentinel), который считается **чёрным**;
- ▷ все проверки «ребёнок чёрный?» корректны и для NIL.

Идея удаления: сначала выполняем стандартное удаление в бинарном дереве поиска (BST), а затем, если удалённая (физически) вершина была чёрной, запускаем процедуру `fixup`, восстанавливающую КЧ-инварианты.

Шаг 1. BST-удаление и сведение к случаю ≤ 1 ребёнка

Пусть нужно удалить вершину z .

- ▷ Если у z нет левого ребёнка, то «заменяем» z её правым ребёнком.
- ▷ Если у z нет правого ребёнка, то «заменяем» z её левым ребёнком.
- ▷ Если у z два ребёнка, берём **следующего по порядку** $y = \min(z.\text{right})$ (inorder-successor), копируем/переставляем y на место z так, чтобы порядок BST сохранился, и далее **физически удаляем** y . Важно: у y по построению нет левого ребёнка, то есть физическое удаление сводится к одному из первых двух пунктов.

Технически удобно использовать подпрограмму `TRANSPLANT(T, u, v)`: заменить поддерево с корнем u на поддерево с корнем v , корректно переподвесив указатели `parent`.

Шаг 2. Что именно может сломаться после удаления

Если физически удалённая вершина была **красной**, то инварианты КЧ-дерева сохраняются автоматически: удаление красной вершины не уменьшает «чёрную высоту» (число чёрных вершин на путях к листьям).

Проблемы начинаются, когда удалена **чёрная** вершина: на всех путях, проходивших через неё, чёрная высота уменьшается на 1. Удобная модель: в узле x , который занял место удалённого узла (это может быть и NIL), появляется **дополнительная чёрнота** (*double-black*). Цель `RB_DELETE_FIXUP` — «протолкнуть» эту дополнительную чёрноту вверх или погасить локальными перекрасками/поворотами.

Псевдокод удаления (как в CLRS)

Псевдокод (идейно, близко к CLRS)

RB_Delete(T, z):

1. $y \leftarrow z$
2. $yColor \leftarrow y.color$
3. **if** $z.left = NIL$ **then**
 - (a) $x \leftarrow z.right$
 - (b) $TRANSPLANT(T, z, z.right)$
4. **else if** $z.right = NIL$ **then**
 - (a) $x \leftarrow z.left$
 - (b) $TRANSPLANT(T, z, z.left)$
5. **else**
 - (a) $y \leftarrow TREEMINIMUM(z.right)$ *(in-order successor)*
 - (b) $yColor \leftarrow y.color$
 - (c) $x \leftarrow y.right$
 - (d) **if** $y.parent = z$ **then**
 - i. $x.parent \leftarrow y$ *(важно даже если $x = NIL$)*
 - (e) **else**
 - i. $TRANSPLANT(T, y, y.right)$
 - ii. $y.right \leftarrow z.right$
 - iii. $y.right.parent \leftarrow y$
 - (f) $TRANSPLANT(T, z, y)$
 - (g) $y.left \leftarrow z.left$
 - (h) $y.left.parent \leftarrow y$
 - (i) $y.color \leftarrow z.color$
6. **if** $yColor = BLACK$ **then** $RB_DELETEFIXUP(T, x)$

Transplant(T, u, v):*(заменить поддереву u на поддереву v)*

1. **if** $u.parent = NIL$ **then** $T.root \leftarrow v$
2. **else if** $u = u.parent.left$ **then** $u.parent.left \leftarrow v$
3. **else** $u.parent.right \leftarrow v$
4. $v.parent \leftarrow u.parent$ *(даже если $v = NIL$)*

TreeMinimum(v):

1. **while** $v.\text{left} \neq \text{NIL}$ **do** $v \leftarrow v.\text{left}$
2. **return** v

RB_DeleteFixup(T, x):

1. **while** $x \neq T.\text{root}$ **and** $x.\text{color} = \text{BLACK}$ **do**
 - (a) **if** $x = x.\text{parent}.\text{left}$ **then**
 - i. $w \leftarrow x.\text{parent}.\text{right}$ (брат x)
 - ii. **if** $w.\text{color} = \text{RED}$ **then** (Case 1)
 - $w.\text{color} \leftarrow \text{BLACK}; x.\text{parent}.\text{color} \leftarrow \text{RED};$
 - $\text{LEFTROTATE}(T, x.\text{parent}); w \leftarrow x.\text{parent}.\text{right}.$
 - iii. **if** $w.\text{left}.\text{color} = \text{BLACK}$ **and** $w.\text{right}.\text{color} = \text{BLACK}$ **then** (Case 2)
 - $w.\text{color} \leftarrow \text{RED}; x \leftarrow x.\text{parent}.$
 - iv. **else** (Cases 3–4)
 - if** $w.\text{right}.\text{color} = \text{BLACK}$ **then** (Case 3)
 - $w.\text{left}.\text{color} \leftarrow \text{BLACK}; w.\text{color} \leftarrow \text{RED};$
 - $\text{RIGHTROTATE}(T, w); w \leftarrow x.\text{parent}.\text{right}.$
 - Case 4:
 $w.\text{color} \leftarrow x.\text{parent}.\text{color}; x.\text{parent}.\text{color} \leftarrow \text{BLACK};$
 $w.\text{right}.\text{color} \leftarrow \text{BLACK}; \text{LEFTROTATE}(T, x.\text{parent}); x \leftarrow T.\text{root}.$
 - (b) **else** (симметрично: x — правый ребёнок)
 - i. $w \leftarrow x.\text{parent}.\text{left}$
 - ii. **if** $w.\text{color} = \text{RED}$ **then** (Case 1')
 - $w.\text{color} \leftarrow \text{BLACK}; x.\text{parent}.\text{color} \leftarrow \text{RED};$
 - $\text{RIGHTROTATE}(T, x.\text{parent}); w \leftarrow x.\text{parent}.\text{left}.$
 - iii. **if** $w.\text{right}.\text{color} = \text{BLACK}$ **and** $w.\text{left}.\text{color} = \text{BLACK}$ **then** (Case 2')
 - $w.\text{color} \leftarrow \text{RED}; x \leftarrow x.\text{parent}.$
 - iv. **else** (Cases 3'–4')
 - if** $w.\text{left}.\text{color} = \text{BLACK}$ **then** (Case 3')
 - $w.\text{right}.\text{color} \leftarrow \text{BLACK}; w.\text{color} \leftarrow \text{RED};$
 - $\text{LEFTROTATE}(T, w); w \leftarrow x.\text{parent}.\text{left}.$
 - Case 4':
 $w.\text{color} \leftarrow x.\text{parent}.\text{color}; x.\text{parent}.\text{color} \leftarrow \text{BLACK};$
 $w.\text{left}.\text{color} \leftarrow \text{BLACK}; \text{RIGHTROTATE}(T, x.\text{parent}); x \leftarrow T.\text{root}.$
2. $x.\text{color} \leftarrow \text{BLACK}$

Разбор случаев RB_DELETE_FIXUP

Дальше считаем, что x — текущая вершина с «лишней чёрнотой», и w — её брат. Для определённости разберём ветку, когда x — левый ребёнок (вторая ветка симметрична).

1. **Case 1:** w **красный**. Тогда родитель p обязан быть чёрным (иначе было бы два

красных подряд). Делаем перекраску: w BLACK, p RED, затем поворот вокруг p в сторону x . После поворота у x появляется **чёрный** брат, и мы сводимся к Case 2–4.

2. **Case 2: w чёрный и оба ребёнка w чёрные.** Перекрашиваем w в красный: это как бы «передаёт» одну чёрноту из w на пути через него, и лишняя чёрнота у x поднимается на родителя: делаем $x \leftarrow p$ и продолжаем цикл.
3. **Case 3: w чёрный, ближний ребёнок w красный, дальний чёрный.** (Для x слева: ближний — $w.\text{left}$, дальний — $w.\text{right}$.) Перекрашиваем ближнего ребёнка в чёрный, w в красный и делаем поворот вокруг w в сторону, противоположную x (в примере — RIGHT_ROTATE). Это переводит конфигурацию в Case 4.
4. **Case 4: w чёрный и дальний ребёнок w красный.** Перекрашиваем w в цвет родителя p , p делаем чёрным, дальнего ребёнка w — чёрным, затем поворачиваем вокруг p в сторону x . В результате лишняя чёрнота «погашена», и цикл можно завершить (делаем $x \leftarrow$ корень).

После выхода из цикла красим x в чёрный (это корректно и для NIL).

Корректность (эскиз доказательства)

1. **BST-свойство** сохраняется: TRANSPLANT и замена удаляемой вершины на inorder-successor не меняют относительный порядок ключей.
2. Если физически удалена красная вершина, **чёрная высота** всех путей не меняется — КЧ-инварианты остаются верны.
3. Если физически удалена чёрная вершина, то единственная проблема — **лишняя чёрнота** в x . Цикл RB_DELETE_FIXUP поддерживает инвариант: «все КЧ-свойства верны, кроме, возможно, лишней чёрноты у текущего x ».
 - ▷ В Case 1 мы делаем поворот/перекраску, не меняя чёрной высоты путей, и переходим к ситуации с чёрным братом.
 - ▷ В Case 2 мы поднимаем лишнюю чёрноту на уровень выше (прогресс по высоте).
 - ▷ В Case 3 и 4 мы локально перестраиваем дерево так, что лишняя чёрнота устраняется, и алгоритм завершает работу.
4. Цикл завершится: в Case 2 x поднимается к корню (высота конечна), а в Case 4 происходит немедленное завершение.

Асимптотика

- ▷ BST-удаление делает $O(\text{высоты}) = O(\log n)$ шагов.
- ▷ RB_DELETE_FIXUP: каждая итерация цикла выполняет $O(1)$ действий, а x либо поднимается на уровень вверх (Case 2), либо алгоритм завершает работу (Case 4). Поэтому итераций $O(\log n)$.

Итого `erase` работает за $O(\log n)$ времени и $O(1)$ дополнительной памяти.

71 Билет №71. Сравнительный анализ различных реализаций дерева поиска: наивное, AVL-, splay-, B-, декартово и красно-чёрное дерево

71.1 Что такое «дерево поиска» и что мы от него хотим

Задача (динамическое упорядоченное множество). Храним множество ключей S в RAM-модели (обычно без дубликатов) и поддерживаем операции:

- ▷ `find( $x$ )`: проверить наличие ключа x (и/или вернуть указатель на вершину);
- ▷ `insert( $x$ )`: добавить ключ x ;
- ▷ `erase( $x$ )`: удалить ключ x ;
- ▷ `lower_bound( $x$ )`: минимальный ключ $\geq x$ (в деревьях поиска реализуется естественно);
- ▷ (иногда) `merge/split` для объединения/разделения деревьев.

Ключевой факт. Во всех вариантах (кроме B-дерева, где узел содержит много ключей) базовые операции работают за $\Theta(h)$ сравнений, где h — высота дерева (длина пути от корня до листа). Поэтому весь сравнительный анализ сводится к:

- ▷ какие гарантии/ожидания на h даёт структура;
- ▷ как дорого поддерживать эти гарантии при `insert/erase`;
- ▷ какие дополнительные «фишки» (локальность, `merge/split`, внешняя память) она даёт.

71.2 Критерии сравнения

Чтобы сравнивать реализации корректно, фиксируем типичные параметры:

1. **Гарантия времени:** худший случай (deterministic worst-case), амортизированное время, или ожидаемое (expected) по случайности.
2. **Высота:** верхняя граница на h в терминах $n = |S|$.
3. **Цена обновлений:** сколько поворотов/перестроек нужно на `insert/erase`.
4. **Память на вершину:** доп. поля (высота/баланс, цвет, приоритет, массив ключей).
5. **Практика:** константы, кэш-локальность, удобство `split/merge`, пригодность для внешней памяти.

71.3 Наивное бинарное дерево поиска (BST)

Инвариант. Для каждой вершины с ключом k : все ключи в левом поддереве строго меньше k , все в правом — строго больше k .

Операции.

- ▷ **find/lower_bound**: спуск от корня, каждый шаг выбираем лево/право по сравнению x и текущего ключа.
- ▷ **insert**: как **find**, но в первой “пустоте” подвешиваем новый лист.
- ▷ **erase**: стандартно 3 случая: (0) нет детей, (1) один ребёнок, (2) два ребёнка (замена на *следующий* по порядку ключ: минимум в правом поддереве, либо максимум в левом).

Асимптотика. Всё $\Theta(h)$.

- ▷ В худшем случае дерево вырождается в список: $h = \Theta(n)$ (например, вставки в отсортированном порядке).
- ▷ При “случайном” порядке вставок типично $h = \Theta(\log n)$, но это *не гарантия*.

Вывод. Наивное BST годится как учебный базис и как быстрый “в среднем” вариант при случайных данных, но не подходит, если нужен гарантированный худший случай.

71.4 AVL-дерево

Идея. Это BST с жёстким ограничением на баланс высот:

Определение 71.1. AVL-дерево — BST, где для любой вершины v выполняется

$$|h(\text{left}(v)) - h(\text{right}(v))| \leq 1.$$

Хранимое доп. поле. Обычно либо высота поддерева, либо баланс-фактор $\in \{-1, 0, +1\}$.

Поддержка инварианта.

- ▷ **insert**: после вставки на пути к корню обновляем высоты/баланс. При первом нарушении баланса исправляем *поворотами*:
 - LL/ RR (одинокный поворот),
 - LR/ RL (двойной поворот).
- ▷ **erase**: аналогично, но нарушения могут всплывать выше по пути к корню, поэтому поворотов в худшем случае может быть $\Theta(\log n)$.

Гарантия высоты и времени.

- ▷ Высота $h = \Theta(\log n)$, причём AVL балансирует “сильнее” красно-чёрного дерева.
- ▷ **find/lower_bound/insert/erase**: детерминированно $O(\log n)$.

Практический смысл. Очень хорош для сценариев “много поисков, мало изменений” (минимальная высота $\Rightarrow$ чуть меньше сравнений на поиск), но обновления (особенно **erase**) могут делать больше поворотов.

71.5 Красно-чёрное дерево (RB-tree)

Идея. BST, где баланс кодируется одним битом “цвета” и набором ограничений на раскраску.

Определение 71.2. Красно-чёрное дерево — BST, где каждая вершина имеет цвет (красный/чёрный) и выполнены свойства:

1. Корень чёрный.
2. Все NIL-листья (пустые дети) считаются чёрными.
3. У красной вершины оба ребёнка чёрные (нет двух красных подряд).
4. На любом пути от вершины до NIL-листа одинаковое число чёрных вершин (*black-height*).

Почему это даёт баланс. Из (3) следует, что красные вершины “разбавляют” путь, а из (4) следует, что число чёрных на всех путях одинаково. Поэтому самый длинный путь не более чем в 2 раза длиннее самого короткого, а высота $h = O(\log n)$.

Операции.

- ▷ `find/lower_bound`: как в BST.
- ▷ `insert/erase`: как в BST + исправление нарушений раскраски (перекраски + повороты). Случаи вставки/удаления стандартные.

Гарантии.

- ▷ Детеминированно: `find/insert/erase` за $O(\log n)$.
- ▷ Как правило, RB-дерево делает меньше поворотов на обновлениях, чем AVL, но допускает чуть большую высоту, чем AVL.

Практический смысл. Часто выбирают как “универсальный” сбалансированный BST для ассоциативных контейнеров (например, типичная реализация `std::map/std::set`).

71.6 Splay-дерево

Идея. Это BST, которое *не хранит явного баланса*, но после каждого доступа “подтягивает” затронутую вершину в корень операцией `splay`.

Splay-операция. Поднимаем вершину к корню серией поворотов:

- ▷ `zig` (вершина — ребёнок корня),
- ▷ `zig-zig` (два шага в одну сторону),
- ▷ `zig-zag` (ломаная).

Гарантии.

- ▷ Одна операция в худшем случае может стоить $\Theta(n)$ (дерево может быть “цепочкой”).

- ▷ Но **амортизированно** любая последовательность из m операций работает за $O(m \log n)$, то есть в среднем $O(\log n)$ на операцию.

Практический смысл.

- ▷ Очень хорош при локальности обращений: часто используемые элементы оказываются ближе к корню.
- ▷ Плохо подходит для “жёстких” real-time требований: отдельная операция может быть долгой.
- ▷ Даже **find** меняет структуру (так как делает **splay**), что усложняет параллельный доступ.

71.7 Декартово дерево (treap)

Идея. Совмещаем два инварианта:

- ▷ по ключу — BST (in-order даёт ключи по возрастанию),
- ▷ по приоритету — куча (min/max-heap по случайному приоритету).

Почему это балансируется. Если приоритеты независимы и равномерны (рандомизация), то форма дерева распределена как у случайного BST, и высота $O(\log n)$ в *ожидании* и “с высокой вероятностью”. В худшем случае (теоретически) может получиться $\Theta(n)$.

Главная фишка: merge/split.

- ▷ **split**(T, x): разбить treap T на (L, R) , где в L все ключи $< x$, в R все ключи $\geq x$.
- ▷ **merge**(L, R): слить два treap-а, если все ключи в L меньше всех ключей в R .

Эти операции особенно удобны, потому что:

- ▷ **insert/erase** выражаются через 1–2 **split** и 1–2 **merge**;
- ▷ легко делать **декартово дерево по неявному ключу** (структура “массив” с вставками/удалениями в середину и запросами на отрезках).

Гарантии. Обычно формулируются как ожидаемое $O(\log n)$ на все операции при случайных приоритетах.

71.8 В-дерево

Идея. Многопутевое дерево поиска, где один узел хранит *массив ключей* (и детей), а не один ключ. Проектировалось под внешнюю память.

Определение 71.3. В-дерево минимальной степени t (часто также говорят “порядка”) удовлетворяет:

- ▷ в каждом узле (кроме корня) от $t - 1$ до $2t - 1$ ключей (значит, от t до $2t$ детей);
- ▷ ключи внутри узла упорядочены;

▷ все листья на одной глубине.

Почему это важно для практики. Если узел подбирают по размеру страницы (диск/SSD) или кэш-линии, то один доступ к узлу приносит сразу много ключей, и высота становится маленькой: $h = \Theta(\log_t n)$.

Операции.

- ▷ **find**: на каждом уровне делаем поиск внутри узла (линейный или бинарный) и переходим в нужного ребёнка.
- ▷ **insert**: при спуске заранее “разрезаем” переполненные узлы (split) так, чтобы вставка попадала в неполный узел.
- ▷ **erase**: при спуске обеспечиваем, чтобы узлы не “обеднялись” (borrow/merge), поддерживая нижнюю границу $t - 1$ ключей.

Гарантии. Детерминированно $O(\log_t n)$ уровней (и, соответственно, $O(\log n)$ сравнений в RAM и $O(\log_t n)$ I/O-операций во внешней памяти).

71.9 Сводная таблица

Структура	Инвариант / идея	Гарантия высоты	Время операций и особенности
Наивное BST	BST-порядок (лево < корень < право)	Худший случай $\Theta(n)$; “в среднем” $\Theta(\log n)$ при удачных данных	find/insert/erase = $\Theta(h)$; очень простое, но без гарантий
AVL	$ h_L - h_R \leq 1$ для каждой вершины	Детерминированно $\Theta(\log n)$ (маленькая константа)	Поиск быстрый; insert обычно ≤ 2 поворота, erase может требовать $O(\log n)$ поворотов
RB-tree	Раскраска + black-height инвариант	Детерминированно $O(\log n)$ (высота до $\approx 2 \log_2 n$)	Меньше поворотов на обновлениях, чем AVL; стандарт для ассоциативных контейнеров
Splay	После доступа splay поднимает вершину в корень	Высота может быть $\Theta(n)$, но	Амортизированно $O(\log n)$ на операцию; отдельная операция может быть дорогой; хорошо при локальности
Treap	BST по ключу + heap по приоритету	Ожидаемо $O(\log n)$ при случайных приоритетах	Ожидаемо $O(\log n)$; очень удобные merge/split ; популярно для неявного ключа
В-дерево	Узел хранит много ключей (fan-out $\sim t$)	Детерминированно $\Theta(\log_t n)$	Лучшее для внешней памяти/кэша: мало уровней; сложнее реализация (split/merge узлов)

71.10 Итог: когда что выбирать

- ▷ **Нужен худший случай на каждую операцию (real-time):** выбирают AVL или RB-tree.
 - AVL — чуть лучше поиск (меньше высота), но потенциально дороже обновления.
 - RB-tree — компромисс: чуть выше дерево, но обычно проще и быстрее на смеси insert/erase.
- ▷ **Есть локальность обращений и важна “самоадаптация”:** splay (амортизированная гарантия и хорошие практические свойства на типичных паттернах).
- ▷ **Нужны split/merge как базовые операции и допустима рандомизация:** treap.
- ▷ **Данные во внешней памяти / нужен маленький уровень I/O:** B-дерево (или B⁺-дерево в СУБД).
- ▷ **Наивное BST** применяют, когда гарантии не важны, а код должен быть максимально простой, либо когда вход случайный/перемешанный.

72 Билет №72. Хеш-функции. Коллизии. Универсальные и k -независимые семейства хеш-функций. Примеры

72.1 Хеширование и коллизии

Определение 72.1. Пусть U — универсум ключей (все возможные значения ключа), а $m \in \mathbb{N}$ — число *бакетов* (корзин), обычно m равно размеру массива в хеш-таблице. *Хеш-функция* — это отображение

$$h : U \rightarrow [m] = \{0, 1, \dots, m-1\}.$$

Значение $h(x)$ называют *хеш-значением* ключа x .

Определение 72.2. Говорят, что для различных ключей $x \neq y$ произошла *коллизия*, если $h(x) = h(y)$.

Замечание. Коллизии неизбежны, если $|U| > m$ (принцип Дирихле): тогда существуют $x \neq y$ с $h(x) = h(y)$. Задача хеширования — сделать коллизии *редкими* и/или *хорошо контролируруемыми*.

72.2 Почему нужны семейства хеш-функций

Если хеш-функция фиксирована и известна, противник (или просто «неудачный» вход) может подобрать набор ключей, которые все попадают в один бакет, и тогда операции в хеш-таблице становятся медленными. Стандартная идея (Carter–Wegman): выбирать хеш-функцию *случайно* из некоторого хорошего семейства $\mathcal{H}$. Случайность — только в выборе $h \in \mathcal{H}$, а вход (набор ключей и запросов) считаем фиксированным.

72.3 Универсальные семейства

Определение 72.3. Семейство $\mathcal{H}$ функций $U \rightarrow [m]$ называется *универсальным* (или *2-универсальным* в «слабом» смысле), если для любых различных $x \neq y$ выполнено

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(y)] \leq \frac{1}{m}.$$

Эквивалентная (счётная) формулировка: число функций из $\mathcal{H}$, склеивающих x и y , не превосходит $|\mathcal{H}|/m$.

Утверждение 72.1 (ожидаемая длина цепочки). Пусть фиксировано множество ключей $S \subseteq U$, $|S| = n$, и выбрана случайная $h \sim \mathcal{H}$, где $\mathcal{H}$ универсально. Тогда для фиксированного ключа $x \in S$ математическое ожидание числа других ключей, попавших

в тот же бакет, не превосходит

$$\mathbb{E} |\{y \in S \setminus \{x\} : h(y) = h(x)\}| \leq \frac{n-1}{m}.$$

$\triangle$ *Доказательство.* Для каждого $y \in S \setminus \{x\}$ введём индикатор $I_y = [h(y) = h(x)]$. Тогда число коллизий с x равно $\sum_{y \neq x} I_y$. По линейности матожидания

$$\mathbb{E} \sum_{y \neq x} I_y = \sum_{y \neq x} \mathbb{E} I_y = \sum_{y \neq x} \Pr[h(y) = h(x)] \leq \sum_{y \neq x} \frac{1}{m} = \frac{n-1}{m}.$$

■

Замечание. Именно это лежит в основе оценки среднего времени операций в хеш-таблице с цепочками: при коэффициенте загрузки $\alpha = n/m$ ожидаемая длина цепочки $\Theta(\alpha)$, а значит ожидаемое время операций порядка $O(1 + \alpha)$. (Подробная реализация и rehash — в билетах про хеш-таблицы.)

72.4 Пример универсального семейства (Carter–Wegman для целых чисел)

Пусть ключи — целые числа, которые мы рассматриваем как элементы поля $\mathbb{Z}_p$ для простого p . Обычно берут простое p больше максимального возможного ключа (или больше $|U|$, если ключи заранее кодируются числами). Выберем случайно параметры

$$a \in \{1, 2, \dots, p-1\}, \quad b \in \{0, 1, \dots, p-1\}$$

и определим

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m.$$

Утверждение 72.2. Семейство $\mathcal{H} = \{h_{a,b}\}$ универсально: для любых $x \neq y$ выполнено $\Pr_{h \sim \mathcal{H}}[h(x) = h(y)] \leq 1/m$.

$\triangle$ *Доказательство.* Рассмотрим фиксированные $x \neq y$. Коллизия $h(x) = h(y)$ означает, что числа $u = (ax + b) \bmod p$ и $v = (ay + b) \bmod p$ дают одинаковый остаток по модулю m . Значит, существует целое t , такое что

$$u \equiv v + tm \pmod{p}.$$

Так как $u, v \in \{0, \dots, p-1\}$, достаточно рассматривать $t \in \{0, 1, \dots, \lfloor (p-1)/m \rfloor\}$. Подставляя u, v , получаем

$$a(x - y) \equiv tm \pmod{p}.$$

Поскольку p простое и $x \neq y$, число $(x - y)$ обратимо по модулю p , и для каждого фиксированного t это сравнение имеет не более одного решения по a . Итак, число «плохих» значений a не превосходит $\lfloor (p-1)/m \rfloor$, а выбор b на коллизию не влияет (он сократился). Следовательно,

$$\Pr[h(x) = h(y)] \leq \frac{\lfloor (p-1)/m \rfloor}{p-1} \leq \frac{1}{m}.$$

Замечание. Если дополнительно подобрать p кратным m (или использовать более аккуратное отображение из $\{0, \dots, p-1\}$ в $[m]$ без смещения), то вероятность коллизии становится близкой к $1/m$ «как у идеально случайной функции». На практике обычно достаточно брать $p \gg m$.

72.5 k -независимые (k -wise independent) семейства

Определение 72.4. Семейство $\mathcal{H}$ функций $U \rightarrow [m]$ называется k -независимым (k -wise independent), если для любых попарно различных $x_1, \dots, x_k \in U$ и любых $y_1, \dots, y_k \in [m]$ выполнено

$$\Pr_{h \sim \mathcal{H}}[h(x_1) = y_1 \wedge \dots \wedge h(x_k) = y_k] = \frac{1}{m^k}.$$

Иными словами, значения $h(x_1), \dots, h(x_k)$ ведут себя как *независимые равномерные* случайные величины на $[m]$.

Утверждение 72.3. Если семейство $\mathcal{H}$ k -независимо и $k \geq 2$, то оно универсально. а также: k -независимость $\Rightarrow t$ -независимость для всех $t \leq k$.

$\triangle$ *Доказательство.* Зафиксируем $x \neq y$ и положим $k = 2$ (или возьмём любые два из k точек). По определению 2-независимости

$$\Pr[h(x) = h(y)] = \sum_{z \in [m]} \Pr[h(x) = z \wedge h(y) = z] = \sum_{z \in [m]} \frac{1}{m^2} = \frac{1}{m}.$$

Тем самым выполнена универсальность (даже с равенством). Для вывода t -независимости из k -независимости достаточно «зафиксировать» оставшиеся $k-t$ точек и просуммировать по их значениям. ■

72.6 Пример k -независимого семейства: многочлены степени $k-1$ над полем

Пусть p — простое, и ключи рассматриваются как элементы $\mathbb{Z}_p$. Выберем случайные коэффициенты $a_0, \dots, a_{k-1} \in \mathbb{Z}_p$ и определим

$$h_{\mathbf{a}}(x) = a_0 + a_1x + a_2x^2 + \dots + a_{k-1}x^{k-1} \pmod{p}.$$

Это семейство отображает $\mathbb{Z}_p \rightarrow \mathbb{Z}_p$.

Утверждение 72.4. Семейство многочленов степени $< k$ над $\mathbb{Z}_p$ является k -независимым (на диапазоне $m = p$).

а именно: для любых попарно различных $x_1, \dots, x_k \in \mathbb{Z}_p$ и любых $y_1, \dots, y_k \in \mathbb{Z}_p$

$$\Pr_{\mathbf{a}}[h_{\mathbf{a}}(x_1) = y_1 \wedge \dots \wedge h_{\mathbf{a}}(x_k) = y_k] = \frac{1}{p^k}.$$

$\triangle$ *Доказательство.* Фиксируем попарно различные $x_1, \dots, x_k$ и произвольные $y_1, \dots, y_k$. Рассмотрим систему линейных уравнений на коэффициенты $a_0, \dots, a_{k-1}$:

$$\begin{cases} a_0 + a_1x_1 + \dots + a_{k-1}x_1^{k-1} \equiv y_1 \pmod{p} \\ \vdots \\ a_0 + a_1x_k + \dots + a_{k-1}x_k^{k-1} \equiv y_k \pmod{p} \end{cases}$$

Матрица этой системы — матрица Вандермонда $(x_i^{j-1})_{i,j}$, её определитель равен $\prod_{1 \leq i < j \leq k} (x_j - x_i)$ и ненулевой по модулю p , так как все x_i различны. Значит, система имеет *единственное* решение по $(a_0, \dots, a_{k-1})$. Всего возможных наборов коэффициентов p^k , и ровно один из них даёт заданные значения $y_1, \dots, y_k$. Следовательно, вероятность равна $1/p^k$. ■

Замечание. Если нужно хешировать в m бакетов, то «идеальная» k -независимость получается при $m = p$. Для произвольного $m < p$ часто делают дополнительное отображение $\mathbb{Z}_p \rightarrow [m]$ (например, взятием остатка по модулю m или взятием старших битов). Это может дать небольшую «смещённость» распределения, но при $p \gg m$ она контролируема и на практике обычно приемлема.

73 Билет №73. Хеш-таблицы с цепочками: find, insert, erase. Процедура rehash

73.1 Постановка и идея хеширования

Хотим поддерживать множество ключей S (без дубликатов) и обрабатывать запросы: $\text{find}(x)$ (принадлежность), $\text{insert}(x)$ (вставка), $\text{erase}(x)$ (удаление).

Определение 73.1. Хеш-таблица с цепочками (*separate chaining*) хранит:

- ▷ число бакетов (корзин) m ;
- ▷ хеш-функцию $h : U \rightarrow \{0, 1, \dots, m-1\}$, где U — универсум ключей;
- ▷ массив бакетов $L[0..m-1]$, где каждый $L[i]$ — контейнер (обычно список) ключей с хешем i .

Коллизия — ситуация, когда разные ключи $x \neq y$ имеют одинаковый хеш: $h(x) = h(y)$.

Инвариант. В любой момент времени для каждого ключа $x \in S$ он лежит ровно в одном бакете $L[h(x)]$. В остальных бакетах x не встречается.

Определение 73.2. Коэффициент заполнения (*load factor*)

$$\alpha = \frac{n}{m}, \quad n = |S|.$$

Интуитивно α — средняя длина цепочки (в среднем сколько элементов в бакете).

73.2 Реализация операций find, insert, erase

Будем считать, что сравнение ключей на равенство определено. Контейнер бакета можно реализовать как односвязный список (удобно удалять), либо как динамический массив/*vector* (лучше кэш, но удаление требует сдвигов).

Операция find

Псевдокод (идейно, близко к CLRS)

Find(x):

1. $i \leftarrow h(x)$
2. **for each** $y \in L[i]$ **do**
 - (a) **if** $y = x$ **then return true**
3. **return false**

Корректность. Если $x \in S$, то по инварианту x лежит в $L[h(x)]$, значит цикл обязательно его найдёт. Если же цикл не нашёл x в $L[h(x)]$, то по инварианту x не лежит ни в

одном бакете, следовательно $x \notin S$.

Операция insert

Псевдокод (идейно, близко к CLRS)

Insert(x):

1. **if** FIND(x) **then return** (множество: дубликаты игнорируем)
2. $i \leftarrow h(x)$
3. PUSHFRONT($L[i], x$) (или PushBack)
4. $n \leftarrow n + 1$
5. **if** $n > \alpha_{\max} \cdot m$ **then** REHASH(m_{new})

Корректность. Если x уже был, мы ничего не меняем, множество не меняется. Иначе добавляем x в бакет $L[h(x)]$; инвариант сохраняется, и x становится элементом множества.

Операция erase

Псевдокод (идейно, близко к CLRS)

Erase(x):

1. $i \leftarrow h(x)$
2. **if** REMOVEFROMBUCKET($L[i], x$) **then** $n \leftarrow n - 1$

где `remove_from_bucket` ищет x в контейнере $L[i]$ и, если находит, удаляет.

Корректность. Если $x \notin S$, то по инварианту x отсутствует в $L[h(x)]$, значит удаление ничего не меняет. Если $x \in S$, то x находится в $L[h(x)]$ и будет удалён; после этого x нигде не хранится, значит $x \notin S$.

73.3 Асимптотика операций (цепочки)

Пусть $\ell_i = |L[i]|$ — длина цепочки в бакете i . Тогда в худшем случае

$$\text{find, erase} = O(1 + \ell_{h(x)}), \quad \text{insert} = O(1 + \ell_{h(x)})$$

(если `insert` предварительно проверяет отсутствие ключа).

Худший случай. Если все ключи попали в один бакет, то $\ell_{h(x)} = \Theta(n)$ и операции могут деградировать до $\Theta(n)$.

Ожидаемое время. В стандартной вероятностной модели (*simple uniform hashing*) ожидаемая длина цепочки $\mathbb{E}[\ell_{h(x)}] = \alpha$, поэтому

$$\mathbb{E}[\text{find}] = \Theta(1 + \alpha), \quad \mathbb{E}[\text{erase}] = \Theta(1 + \alpha), \quad \mathbb{E}[\text{insert}] = \Theta(1 + \alpha).$$

Если поддерживать $\alpha = O(1)$, то все операции имеют ожидаемое время $\Theta(1)$.

73.4 Процедура rehash (перестройка таблицы)

Зачем нужна. При росте n растёт $\alpha = n/m$, цепочки удлиняются и запросы замедляются. Поэтому обычно поддерживают ограничение $\alpha \leq \alpha_{\max}$ (например, $\alpha_{\max} = 1$ или 2) и при нарушении делают перестройку.

Алгоритм rehash

Псевдокод (идейно, близко к CLRS)

Rehash(m_{new}):

1. создать $L_{\text{new}}[0..m_{\text{new}} - 1]$ (все бакеты пусты)
2. *опционально*: выбрать новую хеш-функцию h_{new}
3. **for** $i \leftarrow 0$ **to** $m - 1$ **do**
 - (a) **for each** $x \in L[i]$ **do**
 - i. $j \leftarrow h_{\text{new}}(x)$ (или $j \leftarrow h(x)$ с новым m_{new})
 - ii. **PUSHFRONT**($L_{\text{new}}[j], x$)
4. $L \leftarrow L_{\text{new}}, \quad m \leftarrow m_{\text{new}}$
5. *если перевыбирали*: $h \leftarrow h_{\text{new}}$

Корректность. Каждый старый элемент x переносится ровно в бакет $L_{\text{new}}[h_{\text{new}}(x)]$. Значит, после завершения процедуры инвариант выполнен для всех элементов множества, а само множество не изменилось (мы только переместили элементы между бакетами).

Стоимость rehash и амортизация

Перестройка проходит по всем элементам один раз, поэтому

$$T_{\text{rehash}} = \Theta(n + m_{\text{new}}).$$

Обычно выбирают $m_{\text{new}} = 2m$ (увеличение вдвое). Тогда между двумя соседними перестройками должно произойти $\Theta(m)$ вставок, потому что после увеличения вдвое коэффициент заполнения падает примерно вдвое, и чтобы снова достичь порога $\alpha_{\max}$, нужно добавить линейно много элементов. Следовательно, стоимость перестройки размазывается по этим вставкам, и вклад **rehash** в амортизированное время одной **insert** равен $O(1)$.

Практическое замечание. Часто вместе с **rehash** перевыбирают (рандомизируют) хеш-функцию из универсального семейства, чтобы исключить “плохие” входы, где все ключи попадают в один бакет.

74 Билет №74. Совершенное хеширование

Постановка задачи (static dictionary)

Дано фиксированное множество ключей $S \subseteq U$, $|S| = n$ (после построения *не меняется*). Нужно построить структуру данных, которая отвечает на запрос

$\text{find}(x)$: принадлежит ли x множеству S ,

за $O(1)$ в худшем случае по времени и использует $O(n)$ памяти.

Определения

Определение 74.1 (Совершенная (perfect) хеш-функция для множества S). Хеш-функция $h : U \rightarrow \{0, \dots, m-1\}$ называется *совершенной* для S , если она инъективна на S :

$$\forall x \neq y \in S \quad h(x) \neq h(y).$$

Определение 74.2 (Минимальная совершенная (minimal perfect) хеш-функция). Совершенная хеш-функция для S называется *минимальной*, если $m = n$.

Определение 74.3 (Универсальное семейство хеш-функций). Семейство $\mathcal{H}$ функций из U в $\{0, \dots, m-1\}$ называется *универсальным*, если для любых различных $x, y \in U$ выполнено

$$\Pr_{h \sim \mathcal{H}} [h(x) = h(y)] \leq \frac{1}{m}.$$

Замечание. Обычные хеш-таблицы дают $O(1)$ в среднем, но могут деградировать до $O(n)$ в худшем случае (длинные цепочки / длинные пробирования). Совершенное хеширование даёт $O(1)$ в худшем случае для find , но в базовой версии работает только для *статического* множества (без эффективных `insert/erase`).

Двухуровневая схема FKS (Фредман–Комлош–Семереди)

Классическая конструкция FKS решает статический словарь так: построение за $O(n)$ в матожидании, память $O(n)$, запрос find за $O(1)$ в худшем случае.

Высокоуровневая идея. Первым хешем раскладываем элементы по бакетам. Внутри каждого бакета строим отдельную вторичную таблицу и подбираем вторичный хеш *без коллизий* (то есть делаем совершенное хеширование *внутри бакета*).

Алгоритм построения

Пусть верхняя таблица имеет размер $m := n$. Выбираем универсальные семейства:

$$\mathcal{H}_1 : U \rightarrow \{0, \dots, m-1\}, \quad \mathcal{H}_2(M) : U \rightarrow \{0, \dots, M-1\} \quad (\text{для разных } M).$$

Шаг 1. Верхний уровень. Выбираем случайную функцию $h_1 \sim \mathcal{H}_1$ и разбиваем множество

$$S_i := \{x \in S : h_1(x) = i\}, \quad s_i := |S_i|.$$

Перевыбираем h_1 , пока не выполнится

$$\sum_{i=0}^{m-1} s_i^2 \leq 4n.$$

(ниже докажем, что это происходит за константное ожидаемое число попыток).

Шаг 2. Второй уровень (по бакетам). Для каждого i строим вторичную таблицу T_i размера

$$m_i := s_i^2.$$

- ▷ Если $s_i = 0$, то $m_i = 0$ и T_i пустая.
- ▷ Если $s_i > 0$, выбираем случайную $h_{2,i} \sim \mathcal{H}_2(m_i)$ (то есть $h_{2,i} : U \rightarrow \{0, \dots, m_i - 1\}$), пока $h_{2,i}$ не станет инъективной на S_i . После этого для каждого $x \in S_i$ кладем ключ в $T_i[h_{2,i}(x)]$.

Что именно храним.

- ▷ Верхний массив из $m = n$ ячеек: для каждой ячейки i храним размер m_i , параметры функции $h_{2,i}$ и указатель на массив T_i (или `nullptr`, если $m_i = 0$).
- ▷ В каждой вторичной таблице в каждой ячейке хранится либо ключ, либо метка `EMPTY`. Ключ хранить обязательно: если $x \notin S$, то он может попасть в «занятую» ячейку, и мы отличаем это проверкой равенства.

Запрос find

1. $i := h_1(x)$.
2. Если $m_i = 0$, вернуть `false`.
3. $j := h_{2,i}(x)$.
4. Вернуть `true` тогда и только тогда, когда $T_i[j] = x$.

Корректность

Утверждение 74.1. Алгоритм *find* возвращает *true* тогда и только тогда, когда $x \in S$.

$\triangle$ *Доказательство.* (Если) Пусть $x \in S$. Тогда $i = h_1(x)$ и $x \in S_i$. При построении $h_{2,i}$ выбиралась инъективной на S_i , значит ячейка $j = h_{2,i}(x)$ уникальна, и в неё при построении записали именно x . Следовательно, *find* вернёт `true`.

(Только если) Пусть $x \notin S$. Рассмотрим $i = h_1(x)$. Если $S_i = \emptyset$, то $m_i = 0$ и ответ `false`. Иначе $j = h_{2,i}(x)$: в ячейке $T_i[j]$ либо `EMPTY`, либо некоторый ключ $y \in S_i$. Так как $x \neq y$ для всех $y \in S$, проверка равенства $T_i[j] = x$ ложна, значит возвращается `false`. ■

Оценка памяти

Полная память:

$$m + \sum_{i=0}^{m-1} m_i = n + \sum_{i=0}^{m-1} s_i^2.$$

По условию принятия h_1 имеем $\sum s_i^2 \leq 4n$, значит суммарная память $\leq 5n = O(n)$.

Осталось обосновать, что хорошее h_1 находится быстро.

Лемма 74.1 (Ожидаемая сумма квадратов размеров бакетов). Пусть h_1 выбирается из универсального семейства в таблицу размера m . Тогда

$$\mathbb{E} \left[\sum_{i=0}^{m-1} s_i^2 \right] \leq n + \frac{n(n-1)}{m}.$$

В частности, при $m = n$ имеем $\mathbb{E}[\sum s_i^2] < 2n$.

△ Доказательство. Пусть

$$C := |\{ \{x, y\} \subset S : x \neq y, h_1(x) = h_1(y) \}|$$

— число неупорядоченных коллизий пар. Для бакета размера s величина s^2 равна числу упорядоченных пар (x, y) внутри бакета, включая (x, x) . Поэтому

$$\sum_{i=0}^{m-1} s_i^2 = n + 2C,$$

так как n — пары вида (x, x) , а каждая коллизия $\{x, y\}$ даёт пары (x, y) и (y, x) .

По линейности матожидания:

$$\mathbb{E}[C] = \sum_{x < y} \Pr[h_1(x) = h_1(y)] \leq \binom{n}{2} \cdot \frac{1}{m} = \frac{n(n-1)}{2m},$$

где использована универсальность. Тогда

$$\mathbb{E} \left[\sum s_i^2 \right] = n + 2\mathbb{E}[C] \leq n + \frac{n(n-1)}{m}.$$

■

Утверждение 74.2 (Ожидаемое число попыток выбора h_1 — константа). При $m = n$ вероятность того, что случайный h_1 удовлетворяет $\sum s_i^2 \leq 4n$, строго больше $1/2$.

△ Доказательство. По лемме $\mathbb{E}[\sum s_i^2] < 2n$. По неравенству Маркова:

$$\Pr \left[\sum s_i^2 > 4n \right] \leq \frac{\mathbb{E}[\sum s_i^2]}{4n} < \frac{2n}{4n} = \frac{1}{2}.$$

Значит, $\Pr[\sum s_i^2 \leq 4n] > 1/2$.

■

Оценка времени построения

Верхний уровень. Одна попытка выбора h_1 и подсчёта всех s_i занимает $O(n)$, а попыток в среднем константа, значит шаг 1 занимает $O(n)$ в матожидании.

Второй уровень. Рассмотрим бакет размера $k = s_i$ и таблицу размера $m_i = k^2$. Пусть C_i — число коллизий пар ключей внутри бакета при случайном $h_{2,i}$. Тогда

$$\mathbb{E}[C_i] \leq \binom{k}{2} \cdot \frac{1}{k^2} = \frac{k(k-1)}{2k^2} < \frac{1}{2}.$$

По Маркову $\Pr[C_i \geq 1] \leq \mathbb{E}[C_i] < 1/2$, значит $\Pr[C_i = 0] > 1/2$: вторичный хеш без коллизий находится за константное ожидаемое число попыток. Каждая попытка проверяется за $O(k)$ (прохождение по бакету и попытка расстановки), поэтому на бакет уходит $O(k)$ в матожидании, а суммарно:

$$\sum_i O(s_i) = O(n).$$

Итог

- ▷ память: $O(n)$ (конкретно $\leq 5n$ при $m = n$ и критерии $\sum s_i^2 \leq 4n$);
- ▷ построение: $O(n)$ в математическом ожидании;
- ▷ find: $O(1)$ в худшем случае (2 хеша + 2 обращения к массиву + сравнение ключей).

Как выбирать хеш-функции на практике (типовой пример)

Для целочисленных ключей удобно брать простое p и семейство

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m,$$

где $a \in \{1, \dots, p-1\}$, $b \in \{0, \dots, p-1\}$ выбираются случайно. Это (классический) пример универсального семейства (при корректных предположениях о представлении ключей). Одни и те же идеи применяются и на втором уровне: для каждого бакета выбираются свои параметры (a_i, b_i) и свой модуль $m_i = s_i^2$.

Замечание. Конструкция FKS — про *статическое* множество. Если нужно поддерживать вставки/удаления онлайн, обычно либо периодически перестраивают структуру, либо используют более сложное *dynamic perfect hashing* (это уже отдельная тема).

75 Билет №75. Хеш-таблицы с открытой адресацией. find/insert/erase. rehash. Теоремы о времени работы

75.1 Постановка задачи

Требуется поддерживать множество ключей $S \subseteq U$ (без дубликатов) и обрабатывать запросы:

- ▷ **find**(x): проверить, содержится ли $x \in S$;
- ▷ **insert**(x): добавить x в S (если его там ещё нет);
- ▷ **erase**(x): удалить x из S (если он там есть).

Хеш-таблица хранит элементы *неупорядоченно* (в отличие от деревьев поиска), поэтому запросы, завязанные на порядок (например, `lower_bound`), эффективно не поддерживаются.

75.2 Идея открытой адресации

В открытой адресации все элементы хранятся в *одном массиве* фиксированного размера

$$A[0..m-1].$$

Коллизии разрешаются тем, что для ключа x заранее фиксируется *последовательность проб* (probe sequence)

$$\text{run}_0(x), \text{run}_1(x), \text{run}_2(x), \dots$$

и мы пытаемся разместить/найти/удалить x именно по этим позициям.

Определение 75.1 (Состояния ячейки). Каждая ячейка $A[i]$ находится в одном из трёх состояний:

- ▷ **EMPTY** — ячейка никогда не была занята;
- ▷ **FILLED** — ячейка содержит «живой» ключ;
- ▷ **DELETED** — «могильная плита» (*tombstone*): когда-то здесь был ключ, но он удалён.

Замечание. Если при удалении превращать ячейку в **EMPTY**, то можно «сломать» поиск. Пример: пусть $h(x) = h(y) = h(z)$ и при линейном пробировании элементы легли подряд: x, y, z . Если удалить y и сделать его ячейку **EMPTY**, то **find**(z) остановится на этой пустой ячейке и ошибочно вернёт **false**. Поэтому удаление в открытой адресации обычно делают через **DELETED**.

75.3 Два важных варианта пробирования

Линейное пробирование.

$$\text{run}_j(x) = (h(x) + j) \bmod m.$$

То есть после коллизии мы идём вправо (по кругу) на 1.

Двойное хеширование. Задаём две функции $h_1, h_2 : U \rightarrow \{0, \dots, m-1\}$ и берём

$$\text{run}_j(x) = (h_1(x) + j \cdot h_2(x)) \bmod m.$$

Чтобы последовательность проб действительно «обошла» всю таблицу, нужно

$$\gcd(h_2(x), m) = 1,$$

например, часто берут m простым, а $h_2(x) \in \{1, \dots, m-1\}$.

75.4 Ключевой инвариант

Утверждение 75.1 (Инвариант открытой адресации). Пусть ключ x находится в таблице в позиции $\text{run}_t(x)$. Тогда для всех $j < t$ ячейка $A[\text{run}_j(x)]$ не является *EMPTY* (она может быть *FILLED* или *DELETED*).

$\triangle$ *Доказательство.* x при вставке помещается в первую «доступную» позицию своей последовательности проб (первую *EMPTY* или, при разрешении переиспользовать *DELETED*, первую подходящую *DELETED*). Следовательно, до позиции, куда x реально положили, *EMPTY* встретиться не могло. ■

Из инварианта следует корректный критерий остановки: **поиск можно прекращать на первой *EMPTY*.**

75.5 Операции find/insert/erase

Ниже во всех процедурах пробуем позиции $\text{run}_0(x), \text{run}_1(x), \dots$

find(x). Идём по пробам, пока не встретим *EMPTY*.

- ▷ Если встретили *FILLED* и ключ совпал с x , то **true**.
- ▷ Если встретили *EMPTY*, то **false**.
- ▷ Ячейки *DELETED* пропускаем (останавливаться на них нельзя).

insert(x) (без дубликатов). Идём по пробам, запоминая первую встреченную *DELETED* (если хотим переиспользовать могильные плиты).

- ▷ Если встретили *FILLED* с ключом x , ничего не делаем (дубликатов нет).
- ▷ Если встретили *EMPTY*, то вставляем x либо сюда, либо в самую раннюю *DELETED* (если она была).

erase(x). Находим x так же, как в **find**. Если нашли, помечаем его позицию как *DELETED*. Если дошли до *EMPTY*, значит x нет.

Утверждение 75.2 (Корректность `find`). *`find(x)` возвращает `true` тогда и только тогда, когда $x \in S$.*

$\triangle$ *Доказательство.* Если алгоритм вернул `true`, он явно нашёл ячейку `FILLED` со значением x , значит $x \in S$.

Пусть теперь $x \in S$. Тогда x лежит в некоторой позиции $\text{pin}_t(x)$. По инварианту для всех $j < t$ позиции $\text{pin}_j(x)$ не пусты (`EMPTY` нет). Значит `find` не может остановиться раньше t на `EMPTY` и дойдёт до позиции t , где обнаружит x и вернёт `true`.

Если же `find` дошёл до `EMPTY` и вернул `false`, то по инварианту никакой ключ x не мог быть вставлен «после» этой пустой позиции в своей последовательности проб, значит $x \notin S$. ■

75.6 Процедура `rehash`

Поскольку в таблице всего m ячеек, нужно следить за *коэффициентом заполнения*

$$\alpha = \frac{n}{m}, \quad n = \text{число FILLLED-ячеек}.$$

Также при `DELETED`-метках разумно отдельно хранить число «могильных плит» d (чтобы понимать, насколько таблица «засорилась»).

Определение 75.2 (`rehash`). `rehash` перестраивает таблицу:

1. выбираем новый размер m' , обычно $m' \approx 2m$ (часто берут ближайшее простое);
2. создаём новый массив $A'[0..m' - 1]$, все ячейки `EMPTY`;
3. (опционально) генерируем новую(ые) хеш-функции;
4. все «живые» ключи из старой таблицы заново вставляем в A' по правилам `insert`.

Когда вызывать `rehash`. Типичные практические условия:

- ▷ если α стала слишком большой (например, $\alpha > 1/2$ или $\alpha > 0.7$);
- ▷ если $(n + d)$ близко к m (много `DELETED`, пробирование стало длинным);
- ▷ если при `insert` сделали m проб и не нашли места (таблица переполнена).

Амортизационная стоимость `rehash`. Если каждый раз увеличивать размер таблицы в 2 раза, то суммарная стоимость всех `rehash` на длинной последовательности операций — $O(\text{число вставок})$. Стандартное объяснение: при удвоении m каждый элемент «переезжает» ограниченное число раз, а между двумя перестройками происходит $\Omega(m)$ успешных вставок, поэтому на одну операцию приходится $O(1)$ амортизированного «переноса».

75.7 Теоремы о времени работы (линейное пробирование и двойное хеширование)

Точное время работы операций в открытой адресации равно числу просмотренных ячеек (числу проб). Оно зависит от α и от того, насколько «хорошо» ведёт себя семейство хеш-функций.

Теорема 75.1 (Достаточные условия для $O(1)$ среднего времени). Пусть $\alpha \leq \alpha_0 < 1$ (то есть таблица не заполняется «почти до конца»), а хеш-функции выбираются случайно. Тогда:

1. **Линейное пробирование:** если h выбирается из 5-независимого семейства, то математическое ожидание числа проб для *find/insert/erase* равно $O(1)$ (при фиксированной α_0).
2. **Двойное хеширование:** если h_1, h_2 выбираются из 2-независимого семейства (и $\gcd(h_2(x), m) = 1$), то математическое ожидание числа проб также равно $O(1)$.

С учётом периодического *rehash* (удвоение m при превышении порога заполнения) операции становятся амортизированно $O(1)$ по математическому ожиданию.

Замечание. Интуитивно: *rehash* заставляет α оставаться ограниченной константой, а случайный выбор хеша «ломает» злонамеренные входы. Линейное пробирование хуже переживает зависимости и даёт *кластеризацию*, поэтому требует более сильной независимости (в лекции упоминалось 5-независимое семейство). Двойное хеширование лучше борется с кластеризацией и требует более слабых предположений.

Замечание (Практический вывод). Открытая адресация обычно быстрее цепочек по константам (лучше кэш-локальность, меньше указателей), но требует аккуратного *rehash* и корректного удаления (DELETED или более сложные схемы).

76 Билет №76. Фильтр Блума

76.1 Постановка задачи

Фильтр Блума (Bloom filter) — компактная вероятностная структура данных для задачи **приближённой проверки принадлежности** элемента множеству.

Поддерживаются операции:

- ▷ **insert(x)** — добавить элемент;
- ▷ **contains(x)** — проверить принадлежность.

Фильтр Блума допускает **ложноположительные** ответы: может сказать **yes** для элемента, которого на самом деле нет. Но **ложноотрицательных** ответов нет: если элемент был добавлен, то **contains** никогда не вернёт **no**.

Обычно рассматривают версию **без удаления** (вставки только добавляют информацию).

76.2 Структура данных

Параметры:

- ▷ m — длина битового массива (число битов);
- ▷ k — число хэш-функций;
- ▷ $h_1, \dots, h_k: U \rightarrow \{0, 1, \dots, m-1\}$ — хэш-функции.

Храним битовый массив $B[0 \dots m-1]$, изначально весь из нулей.

Требование к хэсам (модель): значения $h_i(x)$ равномерны на $\{0, \dots, m-1\}$ и (приближённо) независимы.

76.3 Алгоритм

Вставка insert(x). Для всех $i = 1 \dots k$ поставить

$$B[h_i(x)] \leftarrow 1.$$

Запрос contains(x).

- ▷ Если существует i такое, что $B[h_i(x)] = 0$, то вернуть **no** (**точно нет**).
- ▷ Иначе вернуть **yes** (**возможно, есть**).

76.4 Корректность

- ▷ **Отсутствие ложноотрицательных.** Если элемент x был вставлен, то при вставке все биты $B[h_i(x)]$ были установлены в 1. Так как биты никогда не сбрасываются в 0, в дальнейшем при запросе **contains(x)** ни один из этих битов не станет 0, значит **no** не вернётся.
- ▷ **Ложноположительные возможны.** Для элемента y , который не вставлялся, может случиться, что все позиции $h_i(y)$ были установлены в 1 другими элементами.

76.5 Вероятность ложноположительного ответа

Пусть в фильтр вставили n элементов.

Вероятность, что фиксированный бит равен 0. Один элемент устанавливает ровно k позиций. Для фиксированного бита вероятность *не* попасть в него одной хэш-функцией равна $1 - \frac{1}{m}$, значит после вставки одного элемента вероятность остаться нулём:

$$\left(1 - \frac{1}{m}\right)^k.$$

После n вставок:

$$\Pr[B[j] = 0] = \left(1 - \frac{1}{m}\right)^{kn} \approx e^{-kn/m}.$$

Следовательно,

$$\Pr[B[j] = 1] = 1 - \left(1 - \frac{1}{m}\right)^{kn} \approx 1 - e^{-kn/m}.$$

Ложноположительное срабатывание. Для элемента y , отсутствующего во множестве, `contains(y)` возвращает `yes` тогда и только тогда, когда *все* k битов $B[h_1(y)], \dots, B[h_k(y)]$ равны 1. В стандартной модели независимости получаем:

$$p_{\text{fp}} = \Pr[\text{false positive}] = \left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^k \approx (1 - e^{-kn/m})^k.$$

76.6 Оптимальный выбор параметров (k и m)

В этой части доказательства обычно не требуют (в программе стоит δ/∂), но формулы нужно знать.

Оптимальное k при фиксированных m и n . Минимизируя $p_{\text{fp}} \approx (1 - e^{-kn/m})^k$ по k , получаем

$$k_{\text{opt}} = \frac{m}{n} \ln 2.$$

В оптимуме половина битов в среднем равна 1: $e^{-k_{\text{opt}}n/m} = \frac{1}{2}$. Тогда минимальная вероятность ложноположительного ответа равна

$$p_{\text{min}} \approx \left(\frac{1}{2}\right)^{k_{\text{opt}}} = (0.6185\dots)^{m/n}.$$

Подбор m и k под заданную ошибку ε . Если требуется $p_{\text{fp}} \leq \varepsilon$, то выбирают

$$m = -\frac{n \ln \varepsilon}{(\ln 2)^2}, \quad k = \frac{m}{n} \ln 2 = \frac{\ln(1/\varepsilon)}{\ln 2}.$$

На практике берут $m = \lceil -\frac{n \ln \varepsilon}{(\ln 2)^2} \rceil$ и $k = \max\{1, \lfloor \frac{m}{n} \ln 2 + 0.5 \rfloor\}$.

76.7 Асимптотика и практические замечания

- ▷ Память: m бит.
- ▷ Время `insert` и `contains`: $O(k)$ вычислений хэшей и обращений к битам.
- ▷ Удаление в обычном Bloom filter **не поддерживается**: сброс бита может «сломать» другие элементы и привести к ложноотрицательным.
- ▷ Если нужно удаление, используют **counting Bloom filter**: вместо битов хранят маленькие счётчики (но тогда растёт память).
- ▷ В реализации часто применяют *double hashing*: вычисляют два базовых хэша h_a, h_b и задают $h_i(x) = (h_a(x) + i \cdot h_b(x)) \bmod m$, чтобы не считать k независимых хэшей.

77 Билет №77. Фильтр кукушки (б/д)

Задача и модель ответа

Фильтр кукушки (Cuckoo Filter) — вероятностная структура данных для **приближённой** проверки принадлежности:

$$\text{find}(x) \in \{\text{“definitely not”}, \text{“possibly yes”}\}.$$

Гарантии:

- ▷ **Ложноотрицательных** (“*definitely not*”) **нет**, если элемент действительно был успешно вставлен и затем не удалялся.
- ▷ **Ложноположительные** (“*possibly yes*”) возможны: два разных элемента могут иметь одинаковый *fingerprint*.

Фильтр поддерживает **динамические операции**: `insert`, `find`, `erase`. Это ключевое отличие от классического фильтра Блума (удаление там либо невозможно, либо требует более тяжёлых вариантов вроде counting Bloom filter).

Идея: “кукушкино хэширование” на отпечатках (fingerprints)

Вместо хранения исходных ключей (строк/объектов) фильтр хранит **короткие отпечатки**:

$$\text{fp}(x) \in \{0,1\}^f \quad (\text{обычно } f = 8\text{--}16 \text{ бит}).$$

Далее фильтр устроен как хэш-таблица из **бакетов** (“корзин”):

- ▷ число бакетов: m ;
- ▷ размер бакета: b (число слотов в бакете, типично $b = 4$);
- ▷ каждый слот хранит f -битный отпечаток (или 0 как маркер пустоты).

Две возможные позиции для одного элемента

Как и в cuckoo hashing, у каждого элемента есть **ровно два кандидата-места**.

Выбираем хэш-функции:

- ▷ $h(x)$ — хэш от ключа x ;
- ▷ $g(\text{fp})$ — хэш от отпечатка.

Определяем:

$$i_1 = h(x) \bmod m, \quad i_2 = i_1 \oplus g(\text{fp}(x)) \bmod m.$$

(На практике часто берут $m = 2^k$ и заменяют “ $\bmod m$ ” на маску “ $\& (m-1)$ ”).

Ключевой трюк: зная *одну* из позиций и сам fingerprint, можно восстановить вторую, *не зная исходный ключ*:

$$i_2 = i_1 \oplus g(\text{fp}), \quad i_1 = i_2 \oplus g(\text{fp}).$$

Это делает возможным “выталкивания” (kicks): мы можем перемещать отпечаток между двумя его допустимыми бакетами.

Операции

`find(x)`

1. $f \leftarrow \text{fp}(x)$, $i_1 \leftarrow h(x)$, $i_2 \leftarrow i_1 \oplus g(f)$.
2. Проверяем наличие f в бакете i_1 или i_2 .

Если f найден — отвечаем *possibly yes*, иначе — *definitely not*.

Время: $O(b)$ проверок в одном бакете, итого $O(2b) = O(1)$.

`erase(x)`

1. Вычисляем f, i_1, i_2 .
2. Если f встречается в бакете i_1 или i_2 , удаляем **одну** копию f (освобождаем слот).

Важно про дубликаты: разные ключи могут иметь одинаковый fingerprint и одинаковые два бакета. Поэтому корректная реализация должна позволять хранить *несколько одинаковых отпечатков* (как мультисет) — иначе при удалении возможны ложноотрицательные ответы.

Время: $O(1)$.

`insert(x)`

1. Вычисляем f, i_1, i_2 .
2. Если в одном из бакетов i_1 или i_2 есть свободный слот — кладём туда f и завершаем.
3. Иначе запускаем процедуру “кукушки”:
 - (a) Выбираем один из бакетов (например, случайно) $i \in \{i_1, i_2\}$.
 - (b) Повторяем не более **MaxKicks** раз:
 - i. Случайно выбираем слот в бакете i , вытесняем оттуда отпечаток f' , а на его место кладём текущий f .
 - ii. Теперь надо “пристроить” вытесненный f' : обновляем

$$i \leftarrow i \oplus g(f')$$

(это альтернативный бакет для f'), кладём $f \leftarrow f'$ и пытаемся вставить уже f в новый бакет i .

- iii. Если в бакете i нашёлся свободный слот — вставляем и завершаем.

Если после **MaxKicks** вставка не удалась, фильтр считается **переполненным**: обычно делают **rehash/resize** (увеличивают m и заново вставляют все fingerprints).

Асимптотика: при коэффициенте заполнения ниже порога вставка работает за ожидаемое $O(1)$; в худшем случае один **insert** может занять $O(\text{MaxKicks} \cdot b)$.

Вероятность ложноположительного срабатывания

Пусть fingerprint распределён равномерно по 2^f значениям (и f достаточно велико).

При запросе `find(x)` мы сравниваем $f = \text{fp}(x)$ с не более чем $2b$ отпечатками (два бакета по b слотов). Тогда

$$\Pr[\text{false positive}] = 1 - \left(1 - \frac{1}{2^f}\right)^{2b} \approx \frac{2b}{2^f} \quad (\text{если } 2b \ll 2^f).$$

Отсюда удобный подбор f под заданную ε :

$$\frac{2b}{2^f} \leq \varepsilon \Rightarrow f \geq \left\lceil \log_2 \frac{2b}{\varepsilon} \right\rceil.$$

Память, заполнение и практические параметры

- ▷ Всего слотов: $m \cdot b$. Обозначим **коэффициент заполнения** $\alpha = \frac{n}{mb}$.
- ▷ Память на элемент (в битах) примерно $\frac{f}{\alpha}$: мы храним f бит на отпечаток, но часть слотов свободна.
- ▷ На практике берут $b = 4$ и поддерживают α около 0.9–0.95 (выше вероятность провалов вставки резко растёт).
- ▷ **MaxKicks** выбирают константой порядка нескольких сотен (например, 500): это ограничивает худший случай вставки.

Сравнение с фильтром Блума (очень кратко)

- ▷ **Удаление:** у cuckoo filter — естественно поддерживается, у Bloom filter — нет (без модификаций).
- ▷ **Проверка:** Bloom делает k обращений к битам; Cuckoo — проверяет 2 бакета (до $2b$ сравнений отпечатков).
- ▷ **Вставка:** Bloom всегда успешна (цена — рост FPR); Cuckoo может потребовать вытеснения и иногда ресайза.

Мини-заметки по реализации (C++)

- ▷ Удобно резервировать fingerprint 0 как “пусто”, а реальный fingerprint получать как `fp = hash(x) & ((1<<f)-1); if(fp==0) fp=1;`
- ▷ Если $m = 2^k$, то индексы считаются быстрее: `i1 = h(x) & (m-1); i2 = (i1 ^ g(fp)) & (m-1);`
- ▷ Бакет — маленький массив фиксированного размера b ; поиск/удаление делаются линейным сканом по b элементам.

78 Билет №78. Skip List

Определение 78.1. Skip List (*скип-лист*) — вероятностная структура данных для хранения *упорядоченного* множества (или словаря). Она поддерживает операции **find**, **insert**, **erase** за ожидаемое $\Theta(\log n)$ (в худшем случае возможно $O(n)$).

78.1 Идея и устройство

Скип-лист можно воспринимать как несколько связанных между собой *отсортированных списков* (*уровней*), расположенных “один над другим”.

- ▷ Нижний уровень (уровень 0) содержит **все** элементы множества в порядке возрастания.
- ▷ Уровень 1 содержит часть элементов уровня 0, уровень 2 — часть элементов уровня 1 и т.д. Интуитивно: “примерно каждый второй” элемент поднимается на следующий уровень, поэтому уровни сверху становятся более разреженными.
- ▷ Если ключ присутствует на уровне i , то он присутствует и на всех уровнях $0, 1, \dots, i-1$. Набор копий ключа на уровнях $0..h-1$ образует его *башню* (tower).

Для удобства вводят фиктивные элементы (*сентинелы*) $-\infty$ и $+\infty$ (вместо $+\infty$ можно хранить `nullptr`):

- ▷ $-\infty$ присутствует на всех уровнях и является точкой старта поиска.
- ▷ $+\infty$ завершает каждый уровень и упрощает проверку “дошли до конца”.

Как хранить узел. Узел с ключом x имеет высоту $h(x) \geq 1$ и хранит массив “вперёд-указателей”

$$\text{next}[0], \text{next}[1], \dots, \text{next}[h(x) - 1],$$

где `next[i]` указывает на следующий элемент на уровне i .

Случайная высота (“монетка”). Фиксируем параметр $p \in (0, 1)$ (обычно $p = \frac{1}{2}$). При вставке нового элемента его башня строится снизу вверх: уровень 0 создаётся всегда, а дальше пока “выпадает орёл” (с вероятностью p) добавляем ещё один уровень. Тогда

$$\Pr(h(x) \geq k) = p^{k-1}, \quad \mathbb{E}[h(x)] = \frac{1}{1-p} \quad (\text{например, при } p = \frac{1}{2} \quad \mathbb{E}[h] = 2).$$

78.2 Инварианты корректности

Пусть текущая высота структуры равна H (есть уровни $0, 1, \dots, H-1$). Поддерживаются инварианты:

1. На каждом уровне элементы отсортированы по ключу.
2. Уровень $i+1$ является подпоследовательностью уровня i .
3. Каждый элемент встречается на уровнях $0, 1, \dots, h(x)-1$ и нигде выше.

78.3 Операция find

Поиск реализуется жадным спуском сверху вниз: на каждом уровне стараемся идти вправо, пока следующий ключ строго меньше искомого, и как только “вправо нельзя” — спускаемся на уровень ниже.

Псевдокод (идейно, близко к CLRS)

Find(x):

1. $v \leftarrow head$
2. **for** $i \leftarrow H - 1$ **downto** 0 **do**
 - (a) **while** $key(v.next[i]) < x$ **do** $v \leftarrow v.next[i]$
3. $v \leftarrow v.next[0]$
4. **if** $key(v) = x$ **then return** v
5. **else return** NIL

Корректность find. Поддерживаем инвариант: перед спуском с уровня i указатель v стоит на последнем элементе уровня i , который строго меньше x . Тогда $v.next[i]$ — первый элемент на уровне i , который $\geq x$. При переходе вниз мы не пропускаем ответ: на нижнем уровне между v и $v.next[i]$ могут появиться новые элементы, но все они всё равно лежат в интервале $(v, v.next[i])$. В конце на уровне 0 мы оказываемся у предшественника x , поэтому следующий элемент уровня 0 — минимальный $\geq x$, а значит либо это и есть x , либо элемента в множестве нет.

78.4 Операция insert

Вставка состоит из двух фаз.

Фаза 1: найти место вставки и запомнить предшественников. Во время поиска (как в find) запоминаем массив $update[i]$ — узел уровня i , после которого должен стоять x .

Фаза 2: вставить на уровне 0 и “достроить башню” вверх. Сначала вставляем x в нижний список (уровень 0). Далее подбрасываем монетку и, пока “орёл”, поднимаем башню на следующий уровень.

Псевдокод (идейно, близко к CLRS)

Insert(x):

1. $v \leftarrow head$
2. **for** $i \leftarrow H - 1$ **downto** 0 **do**
 - (a) **while** $key(v.next[i]) < x$ **do** $v \leftarrow v.next[i]$
 - (b) $update[i] \leftarrow v$

```

3. if  $\text{key}(\text{update}[0].\text{next}[0]) = x$  then return    (множество: дубликаты игнорируем)
4.  $\text{lvl} \leftarrow \text{RandomLevel}()$                     (1+ число подряд выпавших heads)
5. if  $\text{lvl} > H$  then
    (a) for  $i \leftarrow H$  to  $\text{lvl} - 1$  do  $\text{update}[i] \leftarrow \text{head}$ 
    (b)  $H \leftarrow \text{lvl}$ 
6.  $\text{node} \leftarrow \text{NewNode}(x, \text{lvl})$ 
7. for  $i \leftarrow 0$  to  $\text{lvl} - 1$  do
    (a)  $\text{node}.\text{next}[i] \leftarrow \text{update}[i].\text{next}[i]$ 
    (b)  $\text{update}[i].\text{next}[i] \leftarrow \text{node}$ 

```

Корректность insert. На каждом уровне $i < \text{lvl}$ мы “вклиниваем” новый узел строго между соседями $\text{update}[i]$ и $\text{update}[i].\text{next}[i]$. Так как $\text{update}[i].\text{key} < x \leq \text{update}[i].\text{next}[i].\text{key}$, порядок уровня сохраняется. Башня занимает уровни $0.. \text{lvl} - 1$ подряд, поэтому инвариант подпоследовательности (уровень $i + 1$ — часть уровня i) не нарушается.

78.5 Операция erase

Удаление также начинается с поиска и заполнения $\text{update}[i]$. Если элемент найден, то на каждом уровне, где он присутствует, перенаправляем одно ребро, “перепрыгивая” удаляемый узел.

Псевдокод (идейно, близко к CLRS)

Erase(x):

```

1.  $v \leftarrow \text{head}$ 
2. for  $i \leftarrow H - 1$  downto 0 do
    (a) while  $\text{key}(v.\text{next}[i]) < x$  do  $v \leftarrow v.\text{next}[i]$ 
    (b)  $\text{update}[i] \leftarrow v$ 
3.  $\text{target} \leftarrow \text{update}[0].\text{next}[0]$ 
4. if  $\text{key}(\text{target}) \neq x$  then return                (элемента нет)
5. for  $i \leftarrow 0$  to  $\text{target}.\text{level} - 1$  do  $\text{update}[i].\text{next}[i] \leftarrow \text{target}.\text{next}[i]$ 
6.  $\text{DELETE}(\text{target})$ 
7. while  $H > 1$  and  $\text{head}.\text{next}[H - 1] = \text{tail}$  do  $H \leftarrow H - 1$ 

```

Корректность erase. На каждом уровне, где присутствует x , мы удаляем ровно одно ребро из предшественника на этом уровне, сохраняя связность и отсортированность

списка. Элемент исчезает со всех уровней своей башни, остальные элементы не меняют относительный порядок. Если верхний уровень опустел, его можно “снять”, уменьшив H .

78.6 Асимптотика (б/д)

Пусть n — число элементов в нижнем уровне.

- ▷ Ожидаемое время работы `find/insert/erase`: $\Theta(\log n)$.
- ▷ Худший случай по времени: $O(n)$.
- ▷ Память: ожидаемо $\Theta(n)$, так как $\mathbb{E}[h] = \frac{1}{1-p}$ — константа. (В худшем случае возможно $O(n \log n)$.)

Доказательства этих оценок на экзамене не требуются (в программе отмечено «б/д»), но важно понимать источник логарифма: на каждом уровне в среднем остаётся p -доля элементов предыдущего уровня, поэтому число уровней порядка $\log_{1/p} n$.

78.7 Замечания

- ▷ Так как нижний уровень отсортирован, тривиально реализуются операции порядка (`lower_bound`, итерация по возрастанию и т.п.).
- ▷ По сравнению со строго сбалансированными деревьями поиска (AVL/RB) код операций обычно проще, но гарантий худшего случая нет: балансировка вероятностная.